UniSDK

发行版本 *main*

Telink Semiconductor

2026 年 10 月 14 日

入门指南

1 UniSDK 简介 **3**
- 1.1 泰凌微电子（Telink Semiconductor） . . . 3
- 1.2 UniSDK 核心功能与优势 . . . 3
- 1.3 架构概述 . . . 4
- 1.4 支持的芯片系列 . . . 4
- 1.5 许可证 . . . 4
- 1.6 社区资源 . . . 4
- 1.7 下一步 . . . 4

2 概述 **5**
- 2.1 关于本文档 . . . 5
- 2.2 适用范围 . . . 5
- 2.3 硬件和软件要求 . . . 5
- 2.4 下一步 . . . 6

3 搭建开发环境 **7**
- 3.1 VS Code 扩展（推荐） . . . 7
- 3.2 CLI 方式搭建（备选） . . . 12
- 3.3 下一步 . . . 12

4 获取和导入 SDK **13**
- 4.1 获取 SDK . . . 13
- 4.2 SDK 目录结构 . . . 13
- 4.3 下一步 . . . 14

5 Compile, Flash and Run Your First Example (VS Code) **15**
- 5.1 选择第一个示例 . . . 15
- 5.2 构建项目 . . . 15
- 5.3 烧录固件 . . . 18
- 5.4 验证结果 . . . 19
- 5.5 下一步 . . . 19

6 故障排除 **21**
- 6.1 环境问题 . . . 21
- 6.2 构建问题 . . . 22
- 6.3 Menuconfig 问题 . . . 22
- 6.4 获取帮助 . . . 23

7 开发指南 **25**
7.1 内容导航 . . . 25
7.2 新项目快速入门 . . . 25
7.3 开发工作流程概览 . . . 26
7.4 下一步 . . . 26

8 Linux 环境搭建 **27**
8.1 系统要求 . . . 27
8.2 安装系统依赖 . . . 27
8.3 搭建 Python 虚拟环境 . . . 27
8.4 初始化 West . . . 28
8.5 配置工具链 . . . 28
8.6 配置 BDT 工具（可选） . . . 29
8.7 验证环境 . . . 29
8.8 下一步 . . . 29

9 macOS 环境搭建 **31**
9.1 系统要求 . . . 31
9.2 安装系统依赖 . . . 31
9.3 搭建 Python 虚拟环境 . . . 32
9.4 初始化 West . . . 32
9.5 配置工具链 . . . 32
9.6 验证环境 . . . 32
9.7 下一步 . . . 32

10 Windows 环境搭建 **33**
10.1 系统要求 . . . 33
10.2 安装方法 . . . 33
10.3 配置 BDT 工具（可选） . . . 35
10.4 验证环境 . . . 36
10.5 常见问题 . . . 36
10.6 注意事项 . . . 36
10.7 下一步 . . . 37

11 First Example: Blinky (CLI) **39**
11.1 前提条件 . . . 39
11.2 示例概述 . . . 39
11.3 步骤 1：配置芯片类型 . . . 39
11.4 步骤 2：编译 . . . 40
11.5 步骤 3：烧录 . . . 40
11.6 步骤 4：验证 . . . 40
11.7 自定义修改 . . . 40
11.8 下一步 . . . 41

12 SDK 目录结构 **43**
12.1 整体结构 . . . 43
12.2 关键目录职责 . . . 44
12.3 代码组织原则 . . . 45

13 项目创建与管理 **47**
13.1 应用程序项目位置 . . . 47
13.2 创建新项目 . . . 47
13.3 构建项目 . . . 48
13.4 配置管理 . . . 48
13.5 清理 . . . 49

13.6 开发工作流程建议 49

14 核心架构 **51**
14.1 系统启动流程 51
14.2 初始化系统 51
14.3 中断管理 52
14.4 低功耗管理 52
14.5 上下文切换 (tl_context.S) 53
14.6 时钟系统 53

15 初始化钩子文档 **55**
15.1 概述 55
15.2 概念 55
15.3 级别与优先级 56
15.4 API 参考 56
15.5 补充说明 58

16 API 文档 **59**
16.1 概述 59
16.2 API 参考 59

17 调试与测试 **63**
17.1 硬件调试 63
17.2 软件调试 63
17.3 性能分析 64
17.4 使用 GDB 调试 64
17.5 常用调试技巧 64
17.6 下一步 64

18 IDE 集成 **65**
18.1 VS Code 配置（推荐） 65
18.2 Eclipse 配置 67
18.3 Segger Embedded Studio 67
18.4 下一步 67

19 VSCode 的 clangd 设置指南 **69**
19.1 什么是 clangd? 69
19.2 为什么使用 clangd? 69
19.3 关键文件 70
19.4 安装与设置 70
19.5 故障排除 71

20 芯片与开发板 **73**
20.1 芯片系列概览 73
20.2 选择芯片 73
20.3 开发板列表 74
20.4 芯片与开发板配对验证 74
20.5 构建时指定 74

21 TL321X 芯片系列 **75**
21.1 概述 75
21.2 核心参数 75
21.3 存储器 75
21.4 外设列表 76
21.5 支持的开发板 77

22 TL721X 芯片系列 **79**
22.1 概述 . . . 79
22.2 核心参数 . . . 79
22.3 存储器 . . . 79
22.4 外设列表 . . . 80
22.5 支持的开发板 . . . 81

23 TLSR922X 芯片系列 **83**
23.1 概述 . . . 83
23.2 核心参数 . . . 83
23.3 存储器 . . . 83
23.4 外设列表 . . . 84
23.5 支持的开发板 . . . 84

24 TLSR952X 芯片系列 **85**
24.1 概述 . . . 85
24.2 核心参数 . . . 85
24.3 存储器 . . . 85
24.4 外设列表 . . . 86
24.5 支持的开发板 . . . 86

25 TL3218X_EVK 开发板 **87**
25.1 概述 . . . 87
25.2 构建 . . . 87

26 TL7218X_EVK 开发板 **89**
26.1 概述 . . . 89
26.2 构建 . . . 89

27 TLSR9228A_EVK 开发板 **91**
27.1 概述 . . . 91
27.2 构建 . . . 91

28 TLSR9528A_EVK 开发板 **93**
28.1 概述 . . . 93
28.2 构建 . . . 93

29 TLSR9528A_DONGLE **95**
29.1 概述 . . . 95
29.2 构建 . . . 95

30 外设与驱动 **97**
30.1 驱动概览 . . . 97
30.2 驱动编程模型 . . . 97
30.3 Kconfig 配置 . . . 98

31 GPIO 驱动 **99**
31.1 概述 . . . 99
31.2 数据类型与枚举 . . . 99
31.3 驱动 API . . . 101
31.4 Kconfig 配置 . . . 104
31.5 相关资源 . . . 104

32 UART 驱动 **105**
32.1 概述 . . . 105
32.2 数据类型与枚举 . . . 105

32.3 驱动 API . . . 108
32.4 传输模式 . . . 110
32.5 Kconfig 配置 . . . 110
32.6 相关资源 . . . 111

33 Power Management (PM) Driver 113
33.1 Overview . . . 113
33.2 Sleep Modes . . . 113
33.3 Wakeup Sources . . . 114
33.4 GPIO Wakeup Level . . . 114
33.5 Driver API . . . 114
33.6 Typical Usage . . . 115
33.7 Kconfig Configuration . . . 116
33.8 Related Resources . . . 116

34 模拟驱动 117
34.1 概述 . . . 117
34.2 API 参考 . . . 117
34.3 寄存器定义 . . . 118

35 PLIC 中断控制器 119
35.1 概述 . . . 119
35.2 数据类型和枚举 . . . 119
35.3 API 参考 . . . 120
35.4 PLIC 配置 . . . 121
35.5 中断处理模式 . . . 122
35.6 中断嵌套 . . . 122
35.7 与 GPIO 中断的集成 . . . 122

36 I2C 驱动 123
36.1 概述 . . . 123
36.2 API 参考 . . . 123
36.3 示例 . . . 123

37 SPI 驱动 125
37.1 概述 . . . 125
37.2 API 参考 . . . 125
37.3 示例 . . . 125

38 ADC 驱动 127
38.1 概述 . . . 127
38.2 示例 . . . 127

39 DMA 驱动 129
39.1 概述 . . . 129
39.2 示例 . . . 129

40 看门狗驱动 131
40.1 概述 . . . 131
40.2 示例 . . . 131

41 定时器驱动 133
41.1 概述 . . . 133
41.2 示例 . . . 133

42 USB 驱动 **135**
42.1 概述 . . . 135
42.2 示例 . . . 135

43 PMP 驱动 **137**
43.1 概述 . . . 137
43.2 地址匹配模式 . . . 137
43.3 权限 . . . 138
43.4 锁定 . . . 138
43.5 API . . . 138
43.6 配置 . . . 138

44 连接性 **139**
44.1 概述 . . . 139
44.2 代码位置 . . . 139
44.3 编码规范 . . . 139
44.4 Kconfig 配置 . . . 140
44.5 示例 . . . 140
44.6 规划中 . . . 140

45 BLE 控制器编码规范 **141**
45.1 概述 . . . 141
45.2 文件命名 . . . 141
45.3 函数命名 . . . 141
45.4 变量命名 . . . 142
45.5 中断函数 . . . 142

46 系统服务 **143**
46.1 当前可用组件 . . . 143

47 构建与配置 **145**
47.1 核心组件 . . . 145
47.2 三种构建方式 . . . 145
47.3 文档导航 . . . 146
47.4 构建系统架构 . . . 146

48 CMake 构建系统 **147**
48.1 入口模式 . . . 147
48.2 模块加载顺序 . . . 148
48.3 核心模块详情 . . . 148
48.4 配置流程 . . . 149
48.5 包含保护机制 . . . 149
48.6 构建目录结构 . . . 149

49 Kconfig 配置系统 **151**
49.1 架构 . . . 151
49.2 配置生成流程 . . . 151
49.3 关键脚本 . . . 152
49.4 Menuconfig 触发方式 . . . 152
49.5 配置选项示例 . . . 152

50 West 命令 **155**
50.1 命令注册 . . . 155
50.2 `west tl-build` —构建 . . . 156
50.3 `west tl-config` —配置 . . . 157

50.4 west tl-bdt —烧录与调试 . . . 157
50.5 west tl-boards —板卡信息 . . . 158
50.6 west tl-socs —芯片信息 . . . 158

51 Make 支持 159
51.1 Makefile 目标 . . . 159
51.2 Makefile 变量 . . . 159
51.3 使用示例 . . . 160
51.4 构建流程（make build） . . . 160
51.5 SOC/BOARD 参数转发 . . . 160

52 构建流程 161
52.1 概述 . . . 161
52.2 完整构建流程 . . . 161
52.3 入门指南 . . . 161
52.4 六个阶段 . . . 163
52.5 配置命令 . . . 164
52.6 基本构建命令 . . . 165
52.7 配置文件生成逻辑 . . . 165
52.8 构建结果 . . . 166
52.9 不使用 West 构建 . . . 166
52.10故障排除 . . . 167

53 UniSDK 构建系统架构文档 169
53.1 1. Overall Architecture Overview . . . 169
53.2 2. CMake Build System . . . 171
53.3 3. Kconfig Configuration System . . . 187
53.4 4. West Compilation Support . . . 188
53.5 5. Make Support . . . 199
53.6 6. Compilation Generation Flow . . . 203
53.7 7. Files and Scripts Composition . . . 204
53.8 8. Build System Working Principles . . . 207
53.9 9. Extension and Customization . . . 208

54 扩展与定制 211
54.1 添加新的 CMake 模块 . . . 211
54.2 扩展 West 命令 . . . 211
54.3 添加 Kconfig 配置 . . . 212
54.4 添加新的应用 . . . 212
54.5 West 命令行为说明 . . . 213

55 文件与脚本参考 215
55.1 核心配置文件 . . . 215
55.2 构建产物（build/ 目录） . . . 215
55.3 CMake 文件 . . . 216
55.4 构建脚本（scripts/） . . . 216
55.5 Kconfig 文件 . . . 217

56 开发工具 219
56.1 工具概览 . . . 219

57 BDT 烧录和调试工具 221
57.1 支持的操作 . . . 221
57.2 快速开始 . . . 221
57.3 Flash 操作 . . . 222

57.4 BDT 路径配置 . . . 222
57.5 跨平台说明 . . . 223
57.6 常见问题 . . . 223

58 Pinmux 引脚配置工具 225
58.1 概述 . . . 225
58.2 启动方式 . . . 225
58.3 主视图 . . . 226
58.4 选择功能 . . . 226
58.5 快捷键 . . . 226
58.6 保存 . . . 227
58.7 文件格式 . . . 227
58.8 覆盖机制 . . . 228
58.9 内部执行流程 . . . 228
58.10 配置文件位置 . . . 229
58.11 构建集成 . . . 229
58.12 CI . . . 230

59 Menuconfig 配置界面 231
59.1 概述 . . . 231
59.2 启动方式 . . . 231
59.3 导航与操作 . . . 232
59.4 双阶段配置 . . . 232
59.5 结果 . . . 233
59.6 内部执行流程 . . . 233
59.7 常见问题 . . . 234

60 安全 235

61 生产指南 237

62 示例与演示 239
62.1 示例列表 . . . 239
62.2 构建示例 . . . 240

63 GPIO 示例 243
63.1 概述 . . . 243
63.2 功能演示 . . . 243
63.3 工作原理 . . . 243
63.4 配置选项 . . . 244
63.5 构建与运行 . . . 244

64 UART 示例 245
64.1 概述 . . . 245
64.2 功能演示 . . . 245
64.3 工作原理 . . . 245
64.4 配置选项 . . . 246
64.5 构建与运行 . . . 246

65 PM 电源管理示例 247
65.1 概述 . . . 247
65.2 功能演示 . . . 247
65.3 工作原理 . . . 247
65.4 配置选项 . . . 248
65.5 构建与运行 . . . 248

66 PMP 驱动示例 **249**
66.1 概述 . . . 249
66.2 Features Demonstrated . . . 249
66.3 工作原理 . . . 249
66.4 配置 . . . 250
66.5 Build & Run . . . 250

67 U-mode 驱动示例 **251**
67.1 概述 . . . 251
67.2 Features Demonstrated . . . 251
67.3 工作原理 . . . 251
67.4 配置 . . . 252
67.5 Build & Run . . . 252

68 ADC 示例 **253**
68.1 Overview . . . 253
68.2 Features Demonstrated . . . 253
68.3 How It Works . . . 253
68.4 Configuration Options . . . 255
68.5 构建与运行 . . . 255

69 DMA 示例 **257**
69.1 Overview . . . 257
69.2 Features Demonstrated . . . 257
69.3 How It Works . . . 258
69.4 Configuration Options . . . 259
69.5 构建与运行 . . . 259

70 I2C 示例 **261**
70.1 Overview . . . 261
70.2 Features Demonstrated . . . 261
70.3 How It Works . . . 262
70.4 Configuration Options . . . 263
70.5 构建与运行 . . . 263

71 SPI 示例 **265**
71.1 Overview . . . 265
71.2 Features Demonstrated . . . 265
71.3 How It Works . . . 265
71.4 Configuration Options . . . 267
71.5 构建与运行 . . . 267

72 I2S Sample **269**
72.1 Overview . . . 269
72.2 Features Demonstrated . . . 269
72.3 How It Works . . . 269
72.4 Configuration Options . . . 270
72.5 Build & Run . . . 270

73 BLE 示例 **273**
73.1 Overview . . . 273
73.2 Features Demonstrated . . . 273
73.3 How It Works . . . 274
73.4 Configuration Options . . . 275
73.5 构建与运行 . . . 275

73.6 相关资源 . . . 275

74 BLE Host Sample **277**
74.1 Overview . . . 277
74.2 Features Demonstrated . . . 277
74.3 How It Works . . . 278
74.4 Configuration Options . . . 279
74.5 Build & Run . . . 280

75 RF Sample **281**
75.1 Overview . . . 281
75.2 Features Demonstrated . . . 281
75.3 How It Works . . . 282
75.4 Configuration Options . . . 284
75.5 Build & Run . . . 284

76 IRQ 嵌套示例 **285**
76.1 Overview . . . 285
76.2 Features Demonstrated . . . 285
76.3 How It Works . . . 286
76.4 Configuration Options . . . 287
76.5 构建与运行 . . . 287

77 STIMER 示例 **289**
77.1 Overview . . . 289
77.2 Features Demonstrated . . . 289
77.3 How It Works . . . 289
77.4 Configuration Options . . . 291
77.5 构建与运行 . . . 291

78 看门狗示例 **293**
78.1 Overview . . . 293
78.2 Features Demonstrated . . . 293
78.3 How It Works . . . 293
78.4 Configuration Options . . . 294
78.5 构建与运行 . . . 294

79 USB CDC 示例 **295**
79.1 Overview . . . 295
79.2 Features Demonstrated . . . 295
79.3 How It Works . . . 296
79.4 Configuration Options . . . 297
79.5 构建与运行 . . . 297

80 Random Sample **299**
80.1 Overview . . . 299
80.2 Features Demonstrated . . . 299
80.3 How It Works . . . 299
80.4 Configuration Options . . . 300
80.5 Build & Run . . . 300

81 NV Storage Sample **301**
81.1 Overview . . . 301
81.2 Features Demonstrated . . . 301
81.3 How It Works . . . 301

81.4 Configuration Options . . . 302
81.5 Build & Run . . . 302

82 System Log Sample **305**
82.1 Overview . . . 305
82.2 Features Demonstrated . . . 305
82.3 How It Works . . . 305
82.4 Configuration Options . . . 307
82.5 Build & Run . . . 307

83 Task Planner Sample **309**
83.1 Overview . . . 309
83.2 Features Demonstrated . . . 309
83.3 How It Works . . . 309
83.4 Configuration Options . . . 311
83.5 Build & Run . . . 311

84 Audio Sample **313**
84.1 Overview . . . 313
84.2 Features Demonstrated . . . 313
84.3 How It Works . . . 314
84.4 Configuration Options . . . 315
84.5 Build & Run . . . 315

85 mbedTLS Sample **317**
85.1 Overview . . . 317
85.2 Features Demonstrated . . . 317
85.3 How It Works . . . 317
85.4 Configuration Options . . . 318
85.5 Build & Run . . . 319

86 MCUboot Bootloader Sample **321**
86.1 Overview . . . 321
86.2 Features Demonstrated . . . 321
86.3 How It Works . . . 321
86.4 Configuration Options . . . 323
86.5 Build & Run . . . 324

87 API 参考 **325**
87.1 自动生成说明 . . . 325

88 Kconfig 参考 **327**
88.1 计划内容 . . . 327
88.2 实现计划 . . . 327

89 YAML 描述参考 **329**
89.1 计划内容 . . . 329
89.2 实现计划 . . . 329

90 发布版本 **331**
90.1 版本命名规范 . . . 331
90.2 当前版本 . . . 331
90.3 版本兼容性 . . . 331
90.4 版本历史 . . . 332
90.5 升级指南 . . . 332

91 V0.2.0(FR) 333
91.1 版本 . . . 333
91.2 代码大小 . . . 335

92 V0.3.0(FR) 337
92.1 Version . . . 337
92.2 Features . . . 337
92.3 Bug Fixes . . . 338
92.4 BREAKING CHANGES . . . 340
92.5 Note . . . 340
92.6 Royalty fee for certain Audio Codec: . . . 340
92.7 CodeSize . . . 340

93 V0.4.0(FR) 343
93.1 Version . . . 343
93.2 Features . . . 344
93.3 Bug Fixes . . . 345
93.4 BREAKING CHANGES . . . 346
93.5 Note . . . 347
93.6 CodeSize . . . 347

94 贡献指南 359
94.1 文档仓库：unisdk-doc . . . 359
94.2 文档版本与 SDK 版本 . . . 360
94.3 发布流程 . . . 360
94.4 多版本维护 . . . 361
94.5 CI/CD 行为 . . . 361
94.6 文档结构标准 . . . 361
94.7 提交 Issue . . . 363
94.8 代码规范（SDK 贡献） . . . 363

95 技术支持 365
95.1 联系我们 . . . 365
95.2 提交问题 . . . 365
95.3 社区资源 . . . 365
95.4 常见问题 . . . 366
95.5 资源链接 . . . 366

96 UniSDK 术语表 367
96.1 构建系统 . . . 367
96.2 West 工具 . . . 367
96.3 硬件抽象 . . . 368
96.4 Kconfig 配置 . . . 368
96.5 外设与驱动 . . . 368
96.6 文档状态标记 . . . 368

97 关于 Telink Semiconductor 369
97.1 我们的使命 . . . 369
97.2 技术领先 . . . 369
97.3 全球布局 . . . 370
97.4 为什么选择 Telink? . . . 370
97.5 了解更多 . . . 370

小技巧

文档状态：`STABLE` 表示内容已完成并可供使用；`DRAFT` 表示内容正在开发中。欢迎提交 Issue 和 PR 以帮助改进文档。

- **多芯片统一平台**—一套代码库，一个构建系统，覆盖 TL321X / TL721X / TLSR922X / TLSR952X 全系列芯片
- **模块化构建系统**—基于 CMake + Kconfig 的灵活配置，支持三种构建入口：West / Make / CMake
- **丰富的外设驱动**—统一 API 层提供全系列外设驱动：GPIO、UART、I2C、SPI、ADC、DMA、USB 等
- **BLE 协议栈**—集成 BLE Controller，支持灵活的无线连接应用
- **低功耗管理**—全面的电源管理框架，支持多种低功耗模式：Suspend / Retention 等
- **Pinmux 可视化工具**—图形化引脚复用配置，简化硬件设计
- **BDT 烧录与调试**—通过 `west tl-bdt` 一键 Flash 烧录、读取、擦除

</details>

CHAPTER 1

UniSDK 简介

1.1 泰凌微电子（Telink Semiconductor）

泰凌微电子是一家专注于设计低功耗无线连接芯片的公司，提供涵盖 BLE、Zigbee、Thread 和 Matter 等协议的完整芯片解决方案。泰凌芯片广泛应用于智能家居、可穿戴设备、物联网传感器等场景。

1.2 UniSDK 核心功能与优势

UniSDK（统一软件开发套件）为泰凌全系列芯片提供统一的软件开发平台。其核心优势包括：

1.2.1 统一的开发体验

- **一套代码，多芯片适配**：基于统一的 API 层和 HAL 抽象层，应用程序代码可在不同泰凌芯片之间无缝迁移
- **统一构建系统**：所有芯片系列共用同一套 CMake + Kconfig 构建基础设施

1.2.2 现代化构建系统

UniSDK 集成了业界成熟的构建工具链：

组件	用途	描述
CMake	构建系统核心	项目配置、依赖管理、构建规则生成
Ninja	构建执行引擎	高性能并行编译
West	项目管理工具	Zephyr 社区标准的项目管理工作流
Kconfig	配置管理系统	分层管理编译时配置选项
Make	传统接口	兼容传统 Make 使用习惯的前端接口

1.2.3 丰富的外设驱动

提供 GPIO、UART、I2C、SPI、ADC、DMA、Watchdog、Timer 等全系列外设驱动，通过统一 API 封装底层硬件差异。

1.2.4 低功耗设计

全面的电源管理框架，支持多种低功耗模式，为电池供电设备提供卓越的能效表现。

1.3 架构概述

1.4 支持的芯片系列

芯片系列	内核	主要特性	典型应用
TL321X	RISC-V	BLE、GPIO、UART、I2C、SPI、ADC、DMA	通用物联网
TL721X	RISC-V	BLE、GPIO、UART、I2C、SPI、ADC、DMA、USB	通用物联网
TLSR922X	RISC-V	BLE、GPIO、UART、I2C、SPI、ADC	通用物联网
TLSR952X	RISC-V	BLE、GPIO、UART、I2C、SPI、ADC、DMA	通用物联网

1.5 许可证

UniSDK 采用 Apache License 2.0 许可证。

1.6 社区资源

- 官方网站：https://www.telink-semi.com
- 技术论坛：泰凌开发者论坛

1.7 下一步

- 快速开始 —搭建开发环境
- 开发指南 —深入了解 SDK 开发工作流
- 示例和演示 —直接运行代码示例

CHAPTER 2

概述

2.1 关于本文档

本文档将引导您完成 Telink UniSDK（统一软件开发套件）的完整入门流程——从搭建开发环境、获取 SDK、编译和烧录第一个项目，到成功运行第一个示例。本指南面向 UniSDK 的新手嵌入式软件开发人员，要求具备 C 语言编程基础和嵌入式开发的基本知识。

2.2 适用范围

Telink UniSDK 支持低功耗蓝牙（BLE）协议栈，适用于以下芯片系列：

芯片系列	内核
TL321X	RISC-V
TL721X	RISC-V
TLSR922X	RISC-V
TLSR952X	RISC-V

有关支持的芯片和开发板的详细信息，请参阅发布说明。

2.3 硬件和软件要求

2.3.1 硬件要求

- 一块泰凌官方评估套件（任选其一）：
  - TL3218X_EVK
  - TL7218X_EVK
  - TLSR9528A_EVK

  - TLSR9228A_EVK
  - TLSR9528A_DONGLE
- 一个用于烧录固件的泰凌编程器（任选其一）：
  - 编程器 V1.0 ~ V3.0
  - 编程器 V5.0

2.3.2 软件要求

组件	说明
操作系统	Linux（Ubuntu 20.04+）、Windows 10/11、macOS 12+
Git	版本控制系统
CMake	构建系统生成器（>= 3.20.0）
Ninja	构建系统执行器（>= 1.10）
Python	脚本和 west 工具运行时（>= 3.10）
West	Zephyr 项目管理工具（>= 0.14.0）
Telink RISC-V 工具链	交叉编译工具链（RISC-V GCC V5.4.1+）
BDT	泰凌烧录与调试工具（用于固件烧录）

2.3.3 快速入门流程

请按顺序执行以下步骤：

1. **搭建开发环境** —安装 VS Code 扩展和依赖项
2. **获取和导入 *SDK*** —克隆仓库并了解目录结构
3. **编译、烧录和运行第一个示例** —构建并运行 gpio_demo

2.4 下一步

- 开发指南 —深入了解 SDK 开发工作流
- 构建和配置 —了解 CMake 和 Kconfig 构建系统
- 示例和演示 —探索更多示例项目
- 外设和驱动 —学习使用 GPIO、UART、I2C 等外设驱动

CHAPTER 3

搭建开发环境

本章介绍如何安装和配置 UniSDK 开发环境。推荐使用 Telink VS Code 扩展，它提供了图形界面来管理依赖项。另外，您也可以通过命令行来搭建环境。

3.1 VS Code 扩展（推荐）

Telink Development Tool VS Code 扩展提供图形界面，可一键安装依赖项、管理工具链和初始化工作区。

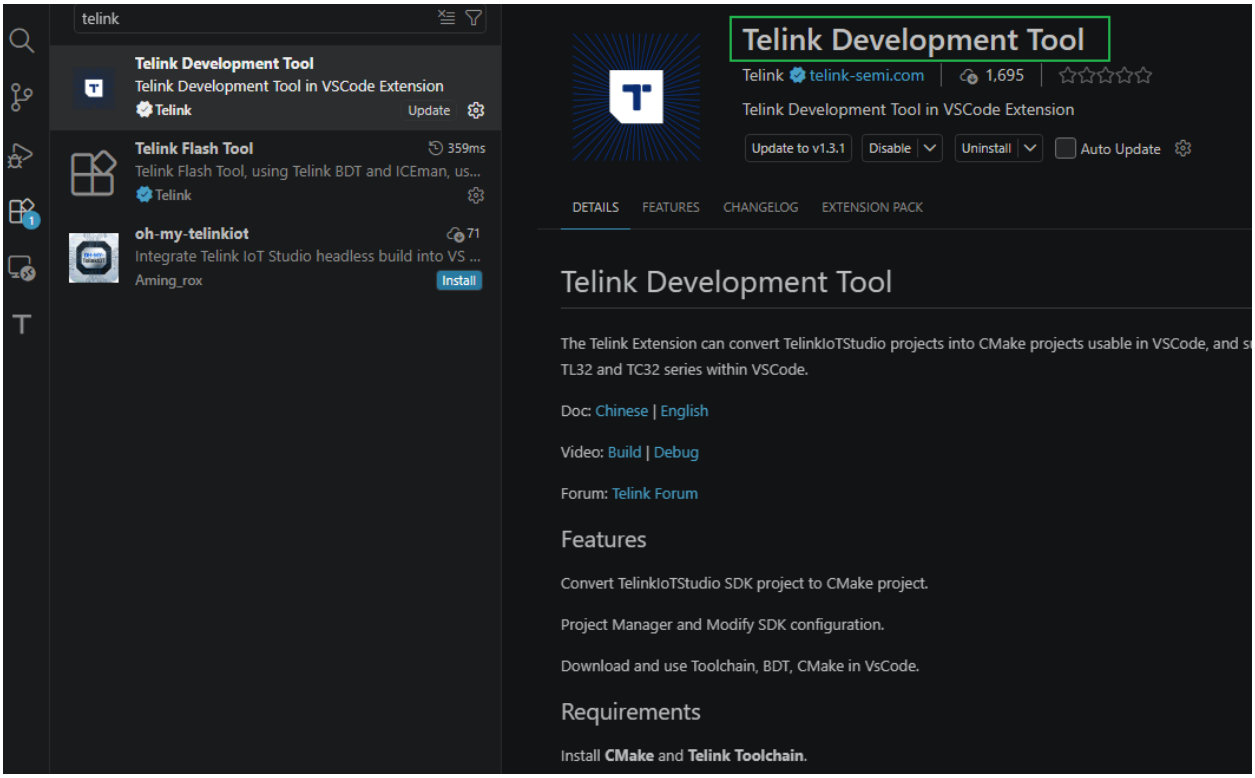
3.1.1 前提条件

- **VS Code**：安装最新版本的 Visual Studio Code。

  如果您的 VS Code 不是最新版本，扩展可能无法正常工作，或者某些功能可能不符合预期。

3.1.2 第 1 步：安装 Telink VS Code 扩展

1. 启动 VS Code。
2. 单击左侧活动栏中的 **Extensions** 图标（或按 **Ctrl+Shift+X**）。
3. 搜索 **Telink**。
4. 在结果中找到 **Telink Development Tool**，然后单击 **Install**。

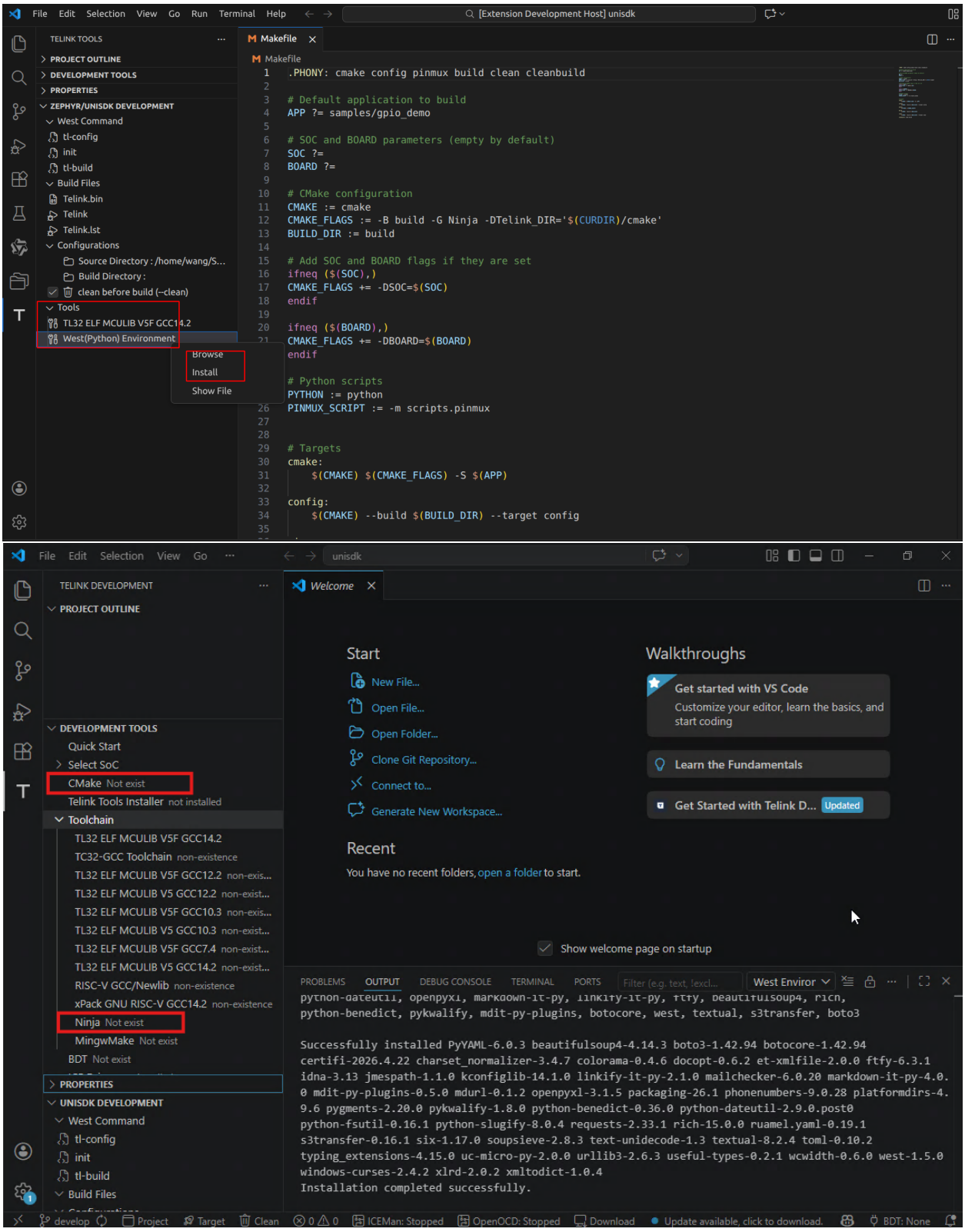


安装完成后，左侧边栏将出现 **Telink DEVELOPMENT** 图标。

> 为方便后续工具自动识别 Unified SDK，请在安装完成后在 VS Code 中打开 Unified SDK 文件夹。

3.1.3 第 2 步：通过扩展安装依赖项

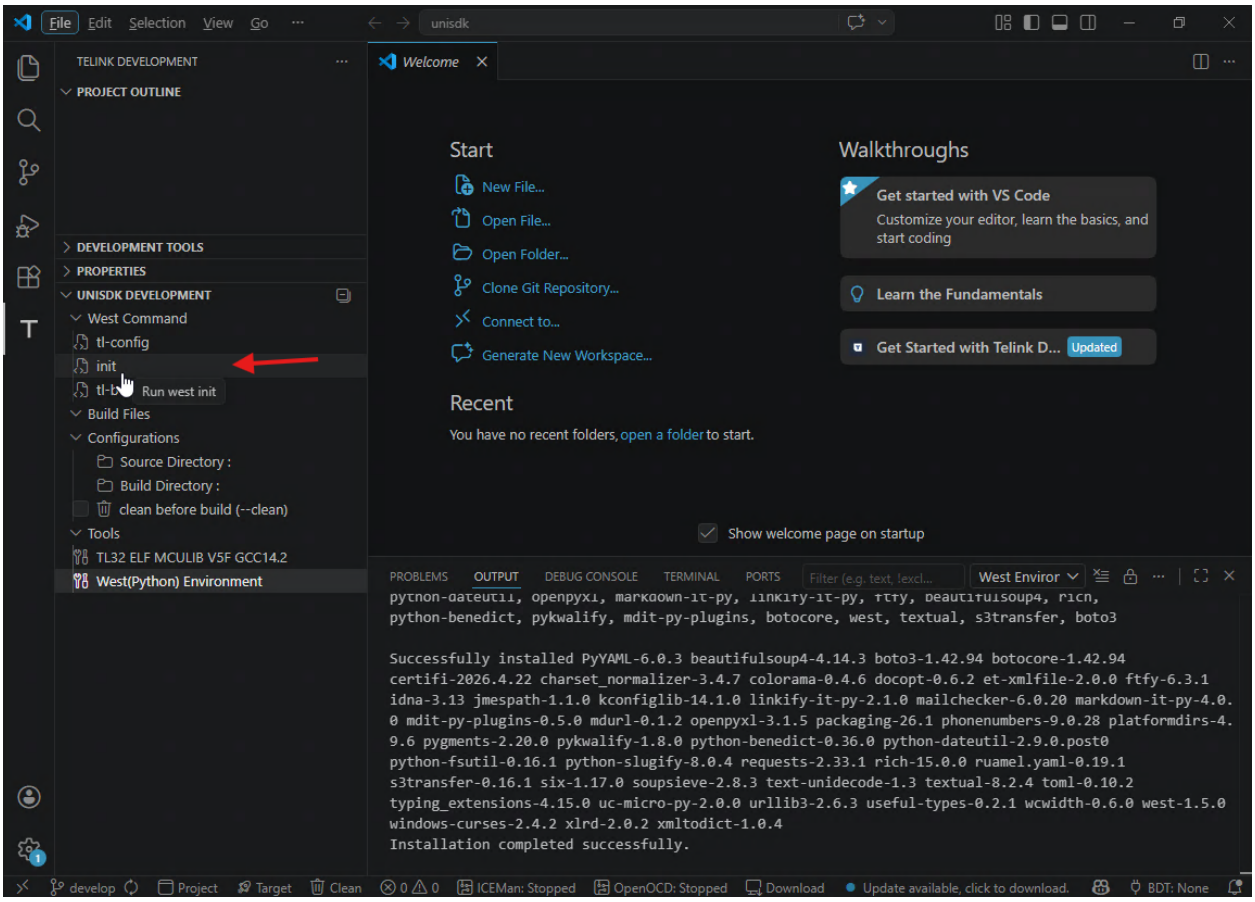
1. 单击左侧边栏中的 **Telink DEVELOPMENT** 按钮。
2. 在 **DEVELOPMENT TOOLS** 树中，您将看到所需的组件：**Toolchain**、**West**、**CMake** 和 **Ninja**。
3. 安装每个组件：
   - **Toolchain**：单击工具链条目并选择 **Install Toolchain**。
   - **West**：右键单击 **West** 并选择 **Install**。建议为 West 使用 Python 虚拟环境。
   - **CMake**：右键单击 **CMake** 并选择 **Install**。
   - **Ninja**：右键单击 **Ninja** 并选择 **Install**。



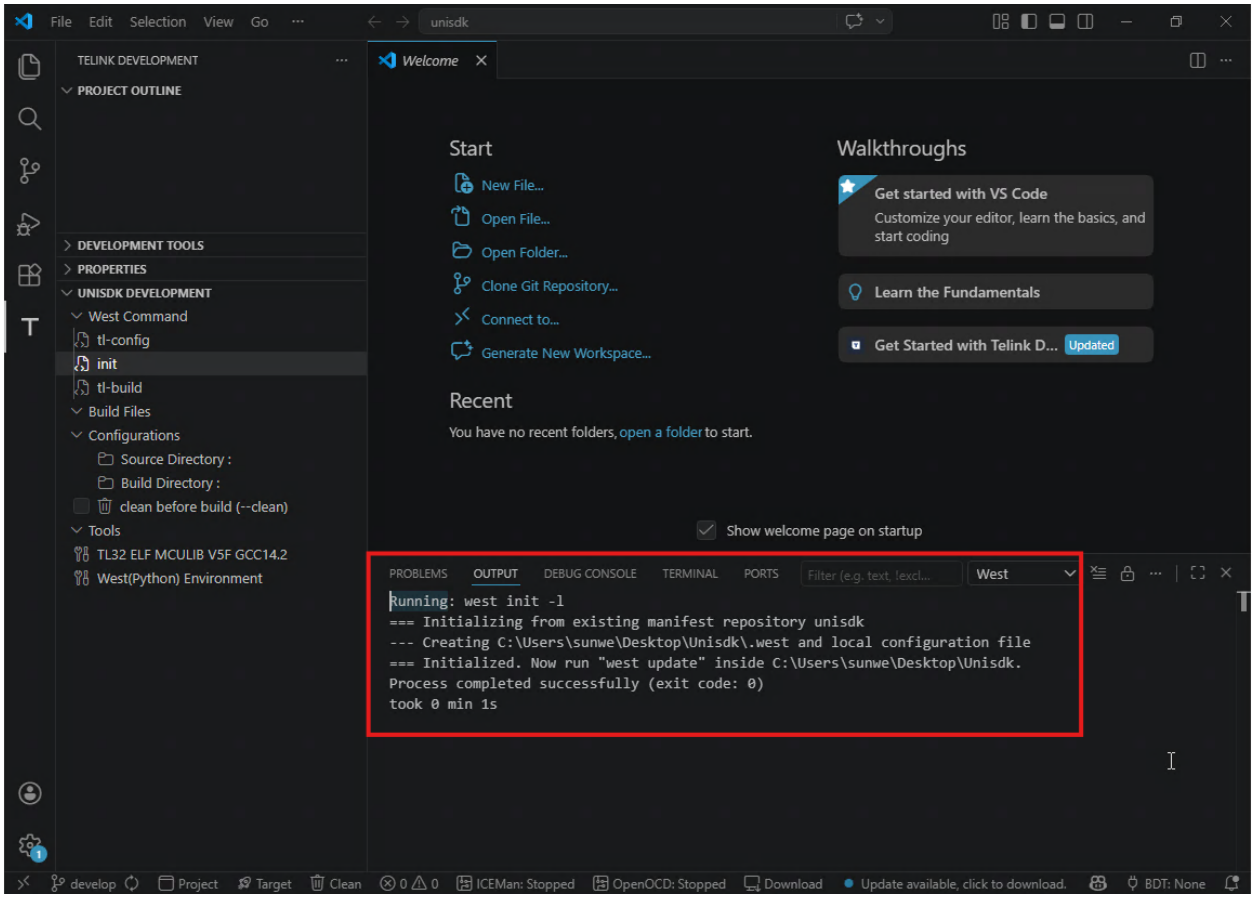
等待每个组件安装完成。安装完成后，右下角会出现通知提示。

3.1.4 第 3 步：初始化 West 工作区

1. 在 **Telink DEVELOPMENT** 视图中，找到 **West Command** 部分。
2. 单击 **init** 按钮。

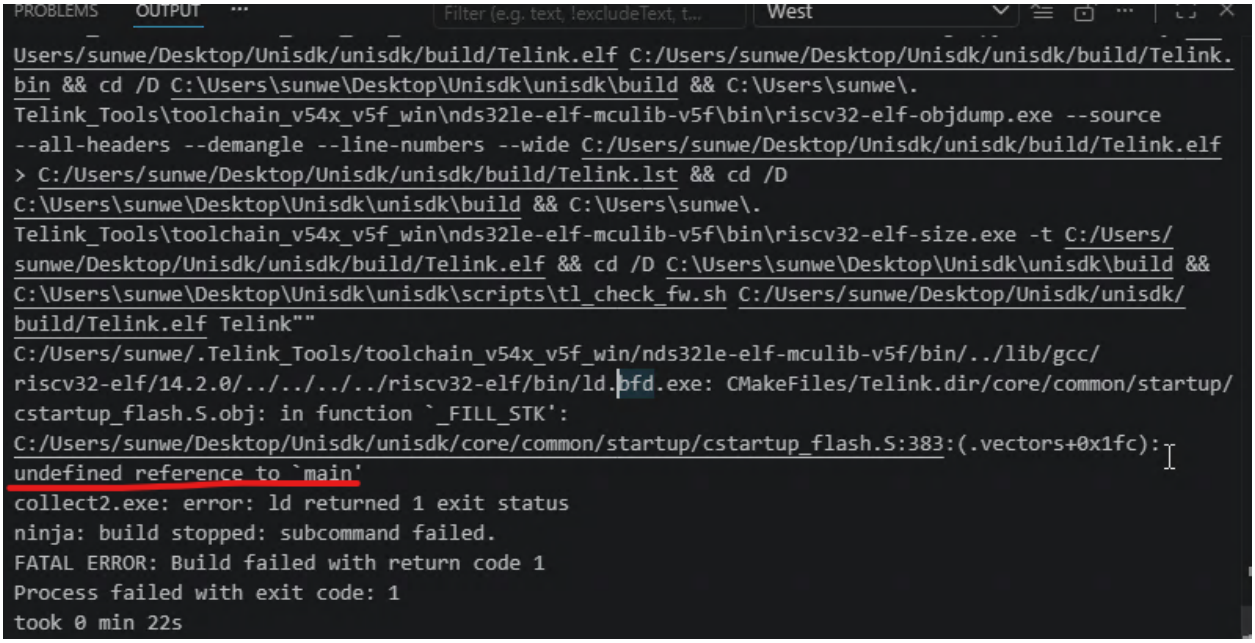


3. 查看 **OUTPUT** 终端面板。初始化成功完成后将显示确认消息。



3.1.5 第 4 步：验证环境

1. 在 **West Command** 部分，单击 **tl-build** 按钮。
2. 出现链接器错误是预期的行为，因为尚未配置源目录。这表示工具链、构建系统和 West 环境已正确配置。



3.2 CLI 方式搭建（备选）

如果您更倾向于通过命令行搭建开发环境，请参阅各平台指南：

- *Linux* 环境搭建
- *Windows* 环境搭建
- *macOS* 环境搭建

3.3 下一步

完成环境搭建后，请继续获取和导入 *SDK*。

CHAPTER 4

获取和导入 SDK

4.1 获取 SDK

使用 Git 获取 UniSDK:

```
git clone https://github.com/telink-semi/unisdk.git
cd unisdk
```

4.2 SDK 目录结构

获取 SDK 后，顶层目录结构如下所示:

SDK 顶层目录结构

```
unisdk/
├── api/              # Public API layer
├── core/             # Core peripheral drivers
├── soc/              # SoC-specific features and drivers
├── boards/           # Board configurations (pinmux, etc.)
├── common/           # Common libraries
├── subsystem/        # Functional subsystems (e.g., BLE Controller)
├── modules/          # Third-party module integrations (e.g., TinyUSB)
├── samples/          # Example projects
├── cmake/            # CMake build system modules
└── scripts/          # Build scripts and tools
```

有关每个目录的详细说明，请参阅*SDK 目录结构*。

4.3 下一步

获取 SDK 后，请继续执行编译、烧录和运行第一个示例。

CHAPTER 5

Compile, Flash and Run Your First Example (VS Code)

This guide walks you through compiling, flashing, and running the first UniSDK example program using the **VS Code Extension GUI**. If you prefer the command-line interface, see the *CLI-based Blinky Guide*.

5.1 选择第一个示例

推荐的第一个示例是 **gpio_demo**（GPIO LED 闪烁）。它的逻辑最简单（仅含外设初始化和基本循环），依赖项最少，且不涉及复杂的无线协议栈。这提供了最短路径来验证您的工具链、SDK 配置和硬件连接（编程器到目标板）是否全部正常工作。

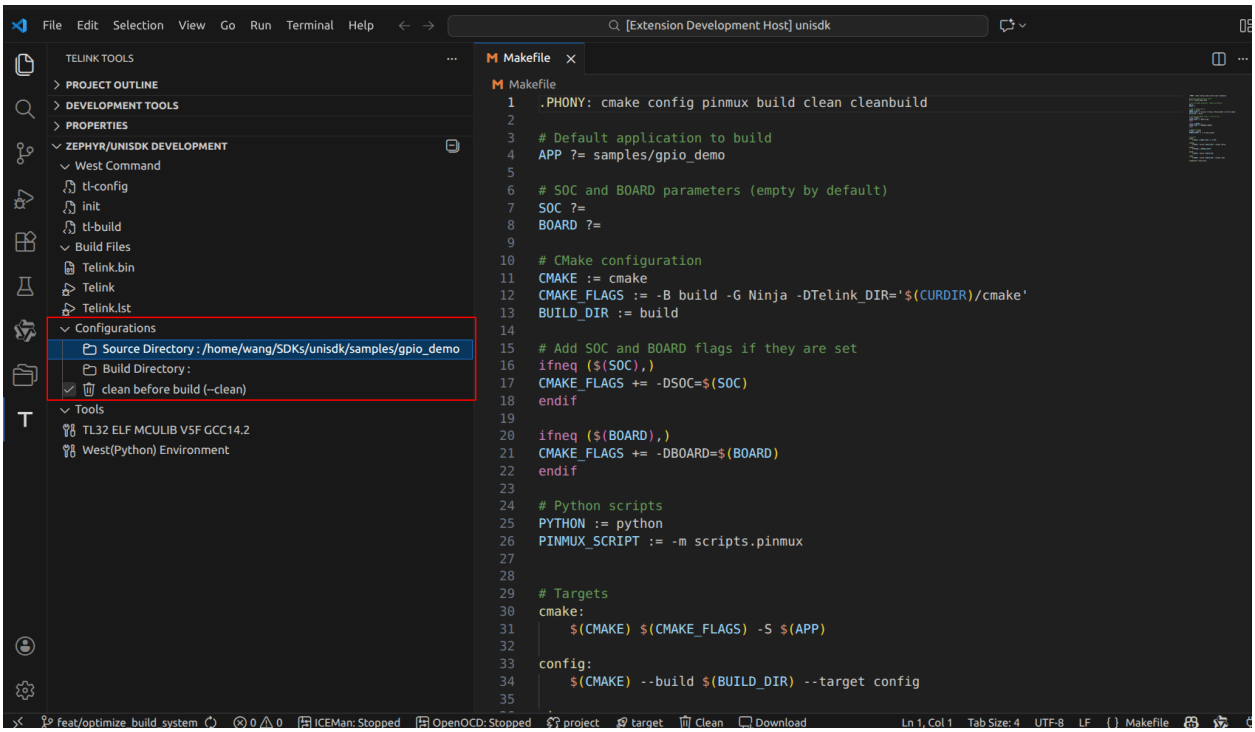
源代码位置：`samples/gpio_demo/main.c`

5.2 构建项目

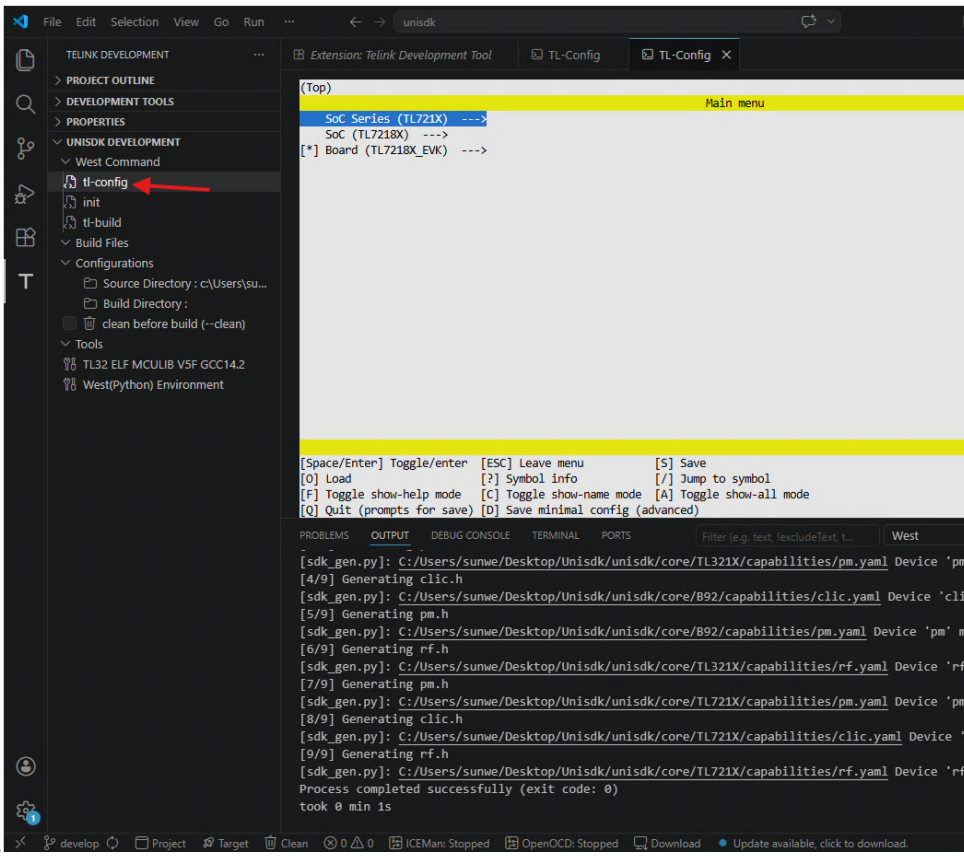
5.2.1 VS Code 扩展（推荐）

第 1 步：选择源目录

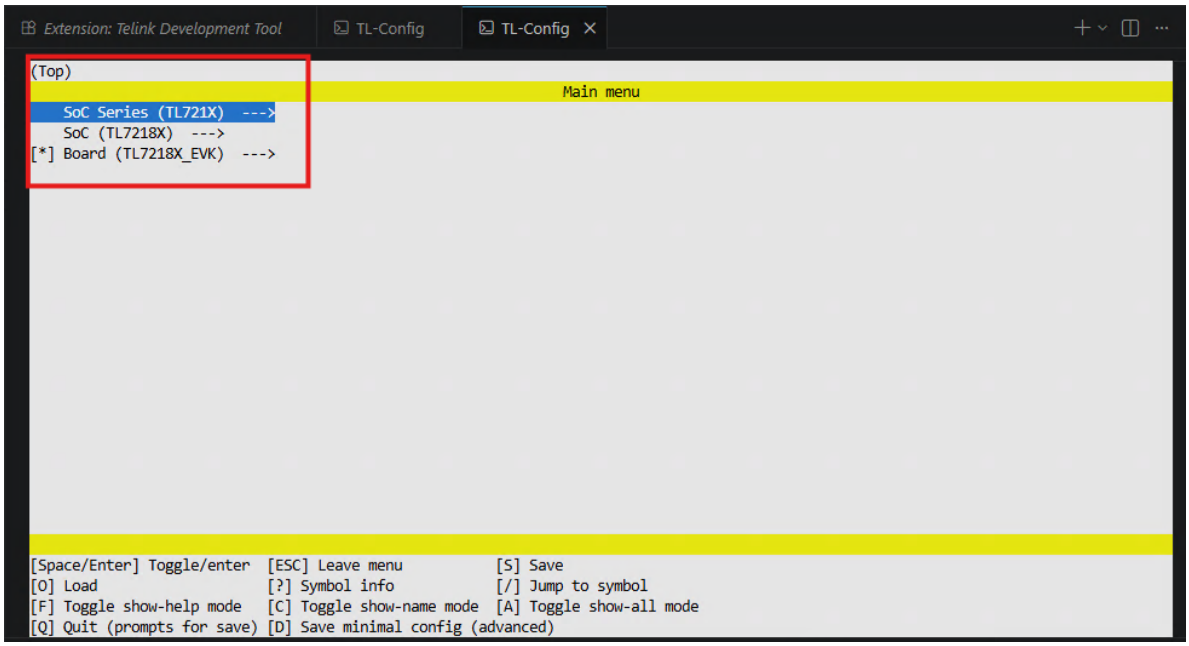
在 **Telink DEVELOPMENT** 视图中，单击 **Source Directory** 按钮（或文件夹图标），然后选择 `samples/gpio_demo` 目录。



第 2 步：配置芯片和开发板



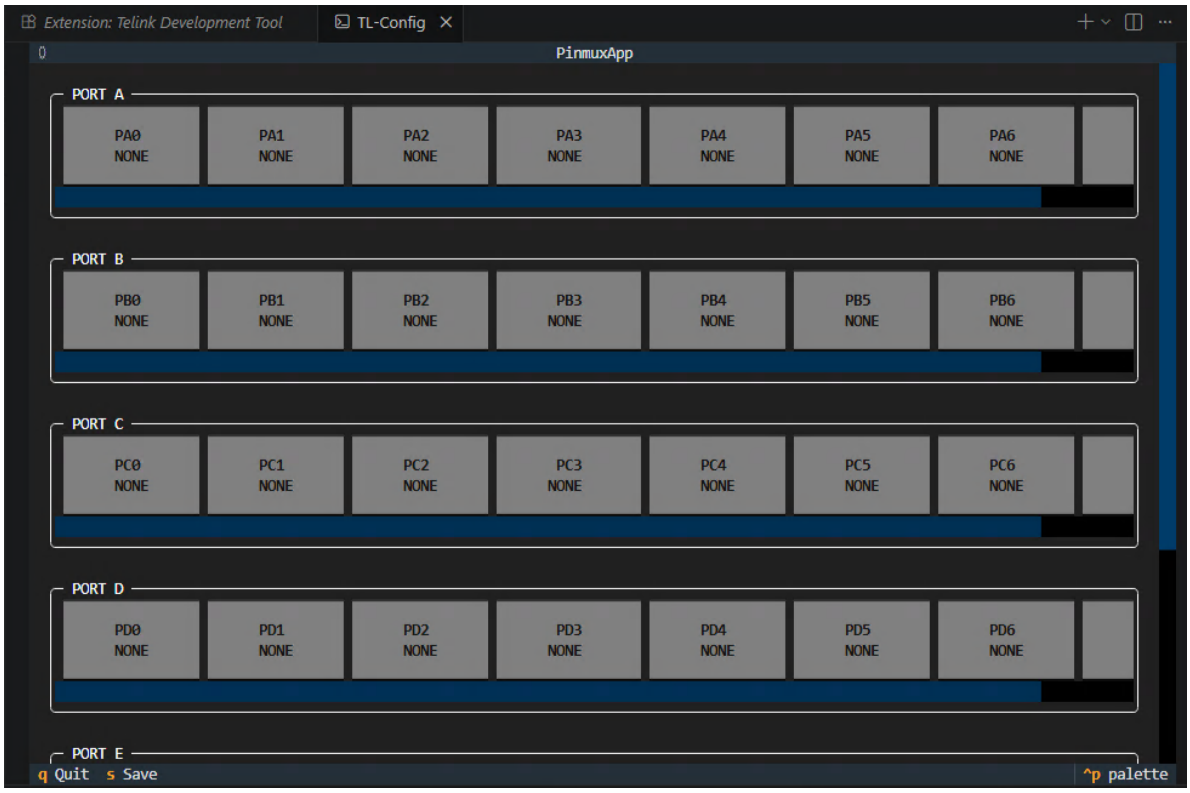
1. 单击 **tl-config** 按钮打开配置界面。
2. 选择目标芯片和开发板（例如 TLSR9528A / TLSR9528A_EVK）。





3. 配置构建选项。

4. 配置 pinmux（引脚功能）。在每个步骤中按 **Q** 保存并退出。



第 3 步：构建

单击 **tl-build** 开始构建。构建输出将显示在终端面板中。

构建成功后，输出文件将生成在 `build/` 目录中：

构建输出文件

```
build/
├── Telink.bin       # Firmware binary file
├── Telink.elf       # ELF executable (contains debug information)
└── Telink.map       # Memory map file
```

5.2.2 CLI 方式（备选）

For command-line users, refer to the *CLI-based Blinky Guide* for detailed CLI build and flash instructions.

5.3 烧录固件

5.3.1 前提条件：连接硬件

1. 使用 USB 线将 PC 连接到泰凌编程器。
2. 使用杜邦线将编程器连接到目标板：
   - **Power**: VCC <-> VCC, GND <-> GND

- **Data**: SWM (Programmer) <-> SWS (Target board)

5.3.2 通过 BDT（命令行）烧录

```
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

指定烧录地址：

```
west tl-bdt download --chip TLSR9528A -i build/Telink.bin -a 0x00
```

有关基于 GUI 的烧录说明，请参阅*BDT 工具指南*。

5.3.3 通过 JTAG 烧录

JTAG 烧录需要 ICEman 后端服务。有关详细说明，请参阅开发指南。

5.4 验证结果

烧录完成后：

1. 开发板会自动复位（或者手动按下 **Reset** 按钮）。
2. 观察板载 LED——它应该以固定间隔闪烁。
3. （可选）连接串口终端查看调试日志输出。

预期结果：板载 LED 周期性闪烁，表明应用程序已在目标芯片上成功运行。

5.5 下一步

恭喜您成功运行了第一个 UniSDK 程序！继续探索：

- 开发指南 —深入了解 SDK 开发工作流
- *Blinky* 详细指南 —示例的逐步分解说明
- 示例和演示 —探索更多示例项目
- 外设和驱动 —学习使用 GPIO、UART、I2C 等外设

CHAPTER 6

故障排除

6.1 环境问题

6.1.1 未找到 TELINK_TOOLCHAIN_PATH

错误：错误：未设置 TELINK_TOOLCHAIN_PATH 环境变量

解决方法：

```
# Linux/macOS
export TELINK_TOOLCHAIN_PATH=/path/to/toolchain

# Windows CMD
set TELINK_TOOLCHAIN_PATH=path\to\toolchain

# Windows PowerShell
$env:TELINK_TOOLCHAIN_PATH="path\to\toolchain"
```

6.1.2 west init -l 报告” 已初始化”

原因：ZEPHYR_BASE 环境变量指向包含 .west 目录的 Zephyr 工作区。

解决方法：

```
# Linux/macOS
unset ZEPHYR_BASE && west init -l

# Windows PowerShell
Remove-Item Env:ZEPHYR_BASE; west init -l
```

6.1.3 CMake 配置失败

解决方法:

1. 验证 CMake 版本（需要 >= 3.20.0）。
2. 清除旧构建缓存并重试:

```
west tl-build --clean
```

3. 验证所有必需的依赖项是否正确安装。

6.1.4 Python 依赖项安装失败

解决方法:

- 检查您的网络连接。
- 验证虚拟环境是否已激活（终端提示符应有 `(.venv)` 前缀）。
- 如有需要，配置代理:

```
pip install --proxy http://proxy:port -r requirements.txt
```

6.2 构建问题

6.2.1 编译错误

解决方法:

1. 检查源代码中是否存在语法错误。
2. 验证芯片和开发板的配置是否正确。
3. 执行完全清理并重试:

```
west tl-build --pristine always
```

6.2.2 找不到 pinmux.h

解决方法:

```
# Manually generate pinmux configuration
make pinmux
# or
python -m scripts.pinmux
```

6.3 Menuconfig 问题

6.3.1 在 VS Code 中 menuconfig 方向键无法使用（Windows）

解决方法:

- 使用字母键代替方向键。
- 使用 Windows 11。

- 使用命令提示符（CMD）代替 PowerShell 进行 menuconfig 操作。

6.4 获取帮助

- 技术支持
- 提交问题

CHAPTER 7

开发指南

本章面向已开始使用 UniSDK 的开发者，介绍完整的开发工作流程和最佳实践。

7.1 内容导航

章节	描述	状态
环境搭建	各平台的搭建指南（Linux / Windows / macOS）	稳定
首个示例：*Blinky*	gpio_demo 示例的详细分步说明	稳定
SDK 目录结构	代码组织规范及各目录职责	稳定
项目创建与管理	创建新项目、CMakeLists.txt 配置	稳定
核心架构	启动流程、异常处理、中断管理、低功耗机制	稳定
调试与测试	硬件调试、日志系统、Shell 交互	稳定
IDE 集成	VS Code clangd 配置、Eclipse、Segger Embedded Studio	稳定

7.2 新项目快速入门

```
# 1. Create project directory (can be anywhere)
mkdir my_app && cd my_app

# 2. Write CMakeLists.txt (see project_management.md)
cat > CMakeLists.txt << 'EOF'
cmake_minimum_required(VERSION 3.20.0)
find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})
project(MyApp LANGUAGES C)
file(GLOB C_SOURCES "*.c")
target_sources(Telink PRIVATE ${C_SOURCES})
EOF

# 3. Write main.c
```

(续下页)

(接上页)

```
cat > main.c << 'EOF'
#include <tlk_api.h>

int main(void) {
    while (1) {
        tlk_sleep_ms(1000);
    }
    return 0;
}
EOF

# 4. Build
west tl-build . --board TLSR9528A_EVK
```

7.3 开发工作流程概览

7.4 下一步

- 构建与配置 —深入了解 CMake 和 Kconfig 构建系统
- 外设与驱动 —使用 GPIO、UART、I2C 等外设驱动
- 示例与演示 —参考可直接运行的示例代码

CHAPTER 8

Linux 环境搭建

本文档详细介绍了在 Linux 系统（以 Ubuntu 为例）上搭建 UniSDK 开发环境的完整步骤。

8.1 系统要求

- Ubuntu 20.04 LTS 或更高版本
- 至少 4 GB RAM，建议 8 GB 以上
- 充足的磁盘空间（SDK + 工具链约需 2 GB）

8.2 安装系统依赖

```
sudo apt update
sudo apt install --no-install-recommends git cmake ninja-build \
    python3-dev python3-venv python3-tk make
```

验证安装：

```
cmake --version     # Should be >= 3.20.0
python3 --version   # Should be >= 3.10
git --version
ninja --version
```

8.3 搭建 Python 虚拟环境

```
# Enter the UniSDK directory
cd /path/to/unisdk

# Create virtual environment
```

(续下页)

(接上页)

```
python3 -m venv .venv

# Activate virtual environment
source .venv/bin/activate

# Upgrade pip and install dependencies
pip install --upgrade pip
pip install -r requirements.txt
```

小技巧

Recommendation Remember to activate the virtual environment before each development session:

```
source .venv/bin/activate
```

8.4 初始化 West

```
# Verify west installation
west --version

# Initialize workspace
west init -l
```

警告

Common Issue: already initialized If you encounter the `already initialized in {path}, aborting.` error:

```
unset ZEPHYR_BASE
west init -l
```

8.5 配置工具链

1. 下载 Telink RISC-V 工具链（请联系 Telink 团队获取）
2. 将工具链解压到指定目录：

   ```
   mkdir -p ~/telink/toolchain
   tar -xzf telink_toolchain_*.tar.gz -C ~/telink/toolchain
   ```

3. 设置环境变量（建议添加到 `~/.bashrc` 或 `~/.zshrc`）：

   ```
   export TELINK_TOOLCHAIN_PATH=$HOME/telink/toolchain
   ```

8.6 配置 BDT 工具（可选）

如果需要使用 `west tl-bdt` 命令进行烧录和调试：

```
# Set BDT tool path
export BDT_PATH=/path/to/BDT

# Or specify temporarily at runtime
west tl-bdt download --chip TLSR9528A --bdt-path /path/to/BDT -i build/firmware.bin
```

8.7 验证环境

```
# Test building an example
west tl-build samples/gpio_demo

# If the build succeeds, you will see output similar to:
# [100%] Built target Telink
```

8.8 下一步

- 首个示例：*Blinky*
- 入门指南

CHAPTER 9

macOS 环境搭建

本文档详细介绍了在 macOS 上搭建 UniSDK 开发环境的完整步骤。

9.1 系统要求

- macOS 12 (Monterey) 或更高版本
- Apple Silicon (M1/M2/M3) 或 Intel 处理器
- 至少 4 GB RAM

9.2 安装系统依赖

我们推荐使用 Homebrew 包管理器:

```
# Install Homebrew (if not already installed)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/
↪install.sh)"

# Install UniSDK dependencies
brew install cmake ninja python3 python-tk git
```

验证安装:

```
cmake --version     # Should be >= 3.20.0
python3 --version   # Should be >= 3.10
git --version
ninja --version
```

9.3 搭建 Python 虚拟环境

```
# Enter the UniSDK directory
cd /path/to/unisdk

# Create virtual environment
python3 -m venv .venv

# Activate virtual environment
source .venv/bin/activate

# Install Python dependencies
pip install --upgrade pip
pip install -r requirements.txt
```

9.4 初始化 West

```
west --version
west init -l
```

9.5 配置工具链

由于 UniSDK 使用 RISC-V 交叉编译工具链，您需要从 Telink 获取对应 macOS 版本的工具链。

1. 联系 Telink 获取 macOS 工具链
2. 解压到指定目录：

   ```
   mkdir -p ~/telink/toolchain
   tar -xzf telink_toolchain_macos_*.tar.gz -C ~/telink/toolchain
   ```

3. 设置环境变量：

   ```
   export TELINK_TOOLCHAIN_PATH=$HOME/telink/toolchain
   ```

   建议将上述命令添加到 `~/.zshrc`（macOS 的默认 shell）。

9.6 验证环境

```
# Test build
west tl-build samples/gpio_demo
```

9.7 下一步

- 首个示例：*Blinky*
- *Linux* 环境搭建
- 入门指南

CHAPTER 10

Windows 环境搭建

本文档详细介绍了在 Windows 10/11 上搭建 UniSDK 开发环境的完整步骤。

10.1 系统要求

- Windows 10 (1709+) 或 Windows 11
- 至少 4 GB RAM，建议 8 GB 以上
- 管理员权限（推荐）

10.2 安装方法

10.2.1 方法 1：一键安装脚本（推荐）

UniSDK 提供了一个自动化部署脚本（`windows_setup.ps1`），可简化 Windows 系统上的环境配置过程。

1. 以管理员身份打开 PowerShell
2. 导航至脚本目录并运行：

```
cd <path_to_unisdk_repository>\scripts
.\windows_setup.ps1
```

脚本功能详解

该脚本自动执行以下任务：

1. **PowerShell 执行策略配置**
   - 检查并将执行策略设置为 `RemoteSigned`，以允许运行本地脚本
2. **工具安装**

- 检测是否安装了 `winget`
- 如果 winget 可用：自动下载并安装 CMake、Ninja、Python 和 Git
- 如果 winget 不可用：提供官方下载链接以引导手动安装

3. **Python 环境配置**
   - 在 UniSDK 根目录下创建 `.venv` 虚拟环境
   - 将 pip 升级到最新版本
   - 从 `requirements.txt` 安装所有 Python 依赖
   - 确保 west 工具已正确安装
4. **环境变量设置**
   - 设置 `TELINK_TOOLCHAIN_PATH` 指向工具链目录
   - 将 CMake 和 Ninja 路径添加到系统 PATH
5. **Telink 工具链设置**
   - 创建 `toolchain` 目录
   - 设置 `TELINK_TOOLCHAIN_PATH` 环境变量
   - 由于工具链下载需要特殊权限，用户需要手动下载并解压工具链
6. **West 工作空间初始化**
   - 通过执行 `west init -l` 初始化 west 工作空间

日志记录

脚本会在脚本目录下自动生成 `setup_log.txt` 文件，记录整个配置过程的详细日志，以便故障排查。

备注

Toolchain Requires Manual Download Since toolchain download requires special permissions, the script only creates the `toolchain` directory and sets the environment variable. Please manually download the toolchain and extract it to the `<unisdk>\toolchain` directory. Download link: https://drive.weixin.qq.com/s?k=AKwA0AfNAA8Ox1r0ab

10.2.2 方法 2：手动安装

1. Install System Dependencies

使用 winget 一键安装:

```
winget install Kitware.CMake Ninja-build.Ninja Python Git.Git
```

如果不支持 winget，请手动下载并安装:

- CMake —勾选”Add CMake to PATH”
- Ninja —手动添加到 PATH
- Python —勾选”Add Python.exe to PATH”
- Git

验证安装:

```
cmake --version
python --version
git --version
ninja --version
```

2. Set Up Python Virtual Environment

```
# Enter the UniSDK directory
cd <path_to_unisdk>

# Create virtual environment
python -m venv .venv

# Activate virtual environment
.venv\Scripts\activate.ps1

# If you encounter a script execution permission error, run this first:
# Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

# Install dependencies
pip install -r requirements.txt
```

3. Initialize West

```
west init -l
```

如果遇到错误 `already initialized in {where}, aborting.`, 请临时取消设置 ZEPHYR_BASE:

```
Remove-Item Env:ZEPHYR_BASE; west init -l
```

4. Configure Toolchain

```
# PowerShell
$env:TELINK_TOOLCHAIN_PATH="path\to\toolchain"

# or CMD
set TELINK_TOOLCHAIN_PATH=path\to\toolchain
```

10.3 配置 BDT 工具（可选）

```
# Set BDT path
$env:BDT_PATH="C:\BDT"

# Or specify temporarily at runtime
west tl-bdt download --chip TLSR9528A --bdt-path C:\BDT -i build\firmware.bin
```

10.4 验证环境

```
# Test build
west tl-build samples\gpio_demo
```

10.5 常见问题

10.5.1 PowerShell 脚本无法执行

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser -Force
```

10.5.2 环境变量未生效

- 重启终端、VS Code 或系统
- 检查系统环境变量设置
- 如果上一步未解决问题，请手动检查 PATH 是否包含正确的路径，并将其添加到用户 PATH 中

10.5.3 在 VS Code 的 menuconfig 中方向键无法使用

- 使用字母键代替方向键
- 使用 Windows 11
- 使用 CMD 代替 PowerShell 进行 menuconfig 操作

10.5.4 winget 安装失败

- 检查 Windows 版本（需要 Windows 10 1709 或更高版本）
- 尝试从 Microsoft Store 安装 App Installer

10.5.5 Python 依赖安装失败

- 确保网络连接正常
- 可能需要配置代理服务器设置

10.5.6 找不到 CMake 或 Ninja

- 手动检查安装路径
- 脚本会尝试自动查找常见的安装位置，但可能需要手动添加到 PATH

10.6 注意事项

1. **管理员权限**：虽然不是必须的，但建议以管理员身份运行脚本，以确保环境变量设置正确
2. **环境变量生效**：脚本完成后，打开一个新的 PowerShell 窗口以使所有环境变量更改生效
3. **虚拟环境激活**：使用以下命令激活虚拟环境：

```
<path_to_unisdk>\.venv\Scripts\Activate.ps1
```

10.7 下一步

- 首个示例：*Blinky*
- *Linux* 环境搭建
- 入门指南

CHAPTER 11

First Example: Blinky (CLI)

> **Tip**: If you prefer using the VS Code Extension GUI, see the *VS Code quick-start guide* instead.

本文档详细介绍如何编译、烧录并运行第一个 UniSDK 示例程序——GPIO 演示（Blinky LED 闪烁）。

11.1 前提条件

- 环境搭建已完成
- 工具链 `TELINK_TOOLCHAIN_PATH` 已配置
- 手头有一块 Telink 开发板（本示例使用 TLSR9528A_EVK）

11.2 示例概述

`gpio_demo` 示例演示了基本的 GPIO 输入/输出功能和中断处理：

- 将 GPIO 引脚配置为输出模式，周期性翻转以实现 LED 闪烁
- 配置中断处理以响应按键输入
- 演示睡眠 API 的使用

源代码位置：`samples/gpio_demo/main.c`

11.3 步骤 1：配置芯片类型

```
# Method 1: Interactive configuration (recommended for first use)
west tl-config

# Method 2: Using Make (with SOC/BOARD preset)
make config SOC=TLSR9528A BOARD=TLSR9528A_EVK
```

小技巧

Available Chips and Boards List all supported options:

```
west tl-socs      # View all chips
west tl-boards    # View all boards
```

11.4 步骤 2：编译

```
# West command (recommended)
west tl-build samples/gpio_demo --board TLSR9528A_EVK

# Or using Make
make build APP=samples/gpio_demo BOARD=TLSR9528A_EVK
```

构建成功后，输出文件位于 `build/` 目录中：

构建输出文件

```
build/
├── Telink.bin     # Firmware binary file
├── Telink.elf     # ELF executable (contains debug information)
└── Telink.map     # Memory map file
```

11.5 步骤 3：烧录

```
# Flash using BDT tool
west tl-bdt download --chip TLSR9528A -i build/Telink.bin

# Specify flash address
west tl-bdt download --chip TLSR9528A -i build/Telink.bin -a 0x00
```

11.6 步骤 4：验证

烧录完成后，开发板上的 LED 将以固定频率闪烁，表示程序正常运行。

11.7 自定义修改

打开 `samples/gpio_demo/main.c` 修改参数，例如 LED 闪烁频率：

```
// Modify the delay time to change the blinking frequency
tlk_sleep_ms(500);  // 500ms = blinks once per second
```

修改后，重新编译并烧录：

```
west tl-build samples/gpio_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

11.8 下一步

- *GPIO* 驱动文档
- 构建系统详情
- 更多示例

CHAPTER 12

SDK 目录结构

本文档详细介绍 UniSDK 代码仓库的目录组织方式，帮助开发者快速定位各类资源。

12.1 整体结构

UniSDK 目录树

```
unisdk/
├── CMakeLists.txt              # CMake main entry point (root directory mode)
├── Makefile                    # Make command interface
├── west.yml                    # West workspace configuration
├── Kconfig.chip               # Chip Kconfig entry point
├── Kconfig.build              # Build Kconfig entry point
├── readme.md                   # Project overview
│
├── api/                        # Public API layer
│   ├── include/                # API header files
│   ├── src/                    # API implementations
│   ├── CMakeLists.txt
│   └── Kconfig
│
├── core/                       # Chip core drivers
│   ├── B92/
│   ├── TL321X/
│   ├── TL721X/
│   ├── common/
│   ├── configs/
│   ├── cpp_wrappers/
│   ├── include/
│   └── CMakeLists.txt
│
├── soc/                        # SoC layer
```

(续下页)

(接上页)

```
│   ├── TL321X/
│   ├── TL721X/
│   ├── TLSR922X/
│   └── TLSR952X/
│
├── boards/                      # Board configurations
│   ├── TL3218X_EVK/
│   ├── TL7218X_EVK/
│   ├── TLSR9228A_EVK/
│   ├── TLSR9528A_EVK/
│   └── TLSR9528A_DONGLE/
│
├── common/
│   ├── include/
│   └── src/
│
├── subsystem/
│   └── ble/
│       └── controller/
│
├── modules/
│   └── tinyUSB/
│
├── samples/
│   ├── gpio_demo/
│   ├── uart_demo/
│   ├── pm_demo/
│   ├── adc_demo/
│   ├── dma_demo/
│   ├── i2c_demo/
│   ├── spi_demo/
│   ├── adv_demo/
│   └── ...
│
├── cmake/
│   ├── modules/
│   └── TelinkConfig.cmake
│
└── scripts/
    ├── west_commands/
    ├── pinmux/
    ├── kconfig.py
    ├── menuconfig.py
    └── ...
```

12.2 关键目录职责

12.2.1 api/ —公共 API 层

提供应用程序开发中最常用的统一 API 接口。所有示例应用程序都应使用 `api/` 中的 API 进行开发。主要内容包括:

- `tlk_sleep.h` / `tlk_time.h` —睡眠延时和时间管理

12.2.2 `core/` —核心驱动层

按芯片系列组织，包括：

- **drivers/** —外设驱动实现（GPIO、UART、I2C、SPI、ADC、DMA 等）
- **properties/** —YAML 格式的设备属性描述
- **registers/** —YAML 格式的寄存器定义
- **configs/** —Kconfig 配置选项

12.2.3 `soc/` —SoC 特定层

包含芯片特定的功能：

- **drivers/** —平台特定驱动（MSPI、OTP 等）
- **pinmux/** —引脚复用定义（每个封装变体的 yaml 文件）
- **registers/** —芯片特定的寄存器定义

12.2.4 `boards/` —开发板配置

仅包含开发板特定的引脚映射配置（`board_pinout.yaml`）。

12.3 代码组织原则

1. **公共 API 放在 `api/`** —所有芯片共享的接口
2. **核心驱动放在 `core/`** —每个芯片系列的外设驱动
3. **芯片特定代码放在 `soc/`** —特定芯片独有的功能
4. **应用代码可放在任何位置**—不限于 `samples/` 目录
5. **使用 Kconfig 进行配置**—通过 Kconfig 管理编译时选项

CHAPTER 13

项目创建与管理

13.1 应用程序项目位置

UniSDK 应用程序项目**可以放在任何目录中**，不限于 `samples/` 目录。每个应用程序项目需要提供自己的 `CMakeLists.txt` 和源代码。

13.2 创建新项目

13.2.1 最小项目结构

最小项目结构

```
my_project/
├── CMakeLists.txt    # Build entry point
├── main.c            # Main program
└── Kconfig           # Optional: project-level Kconfig configuration
```

13.2.2 CMakeLists.txt 模板

```
# SPDX-License-Identifier: Apache-2.0

cmake_minimum_required(VERSION 3.20.0)

# Load Telink SDK
find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})

# Define project
project(MyProject LANGUAGES C CXX)

# Add C source files
file(GLOB C_SOURCES "*.c")
```

(续下页)

(接上页)

```
target_sources(Telink PRIVATE ${C_SOURCES})

# Optional: conditionally add C++ source files
if(CONFIG_TLK_ALLOW_CPP_WRAPPERS)
    file(GLOB CPP_SOURCES "*.cpp")
    target_sources(Telink PRIVATE ${CPP_SOURCES})
endif()
```

13.2.3 关键说明

1. **find_package(Telink)** —通过 TELINK_BASE 环境变量定位 SDK 路径
2. **project()** —项目名称将用作输出的固件名称
3. **target_sources(Telink ...)** —应用程序源文件被添加到 SDK 的主目标 Telink 中
4. **CONFIG_TLK_ALLOW_CPP_WRAPPERS** —检查是否启用了 C++ 包装器

13.3 构建项目

13.3.1 West 命令（推荐）

```
# Run in the project directory (must be within a west workspace when using west)
west tl-build . --board TLSR9528A_EVK

# Specify build directory
west tl-build -d build_custom . --board TLSR9528A_EVK
```

13.3.2 CMake 命令

```
# Configure
cmake -B build -G Ninja -S . -DBOARD=TLSR9528A_EVK

# Build
cmake --build build
```

13.3.3 Make 命令

```
make cmake build APP=. BOARD=TLSR9528A_EVK
```

13.4 配置管理

13.4.1 使用项目 Kconfig

在项目目录中创建 Kconfig 文件，定义项目特定的配置选项:

```
menu "My Project Configuration"

config MY_FEATURE_ENABLED
```

(续下页)

(接上页)

```
    bool "Enable my feature"
    default y
    help
        Enable custom project feature.

endmenu
```

Kconfig 文件通过 kconfig.cmake 中的 KCONFIG_APPLICATION_DIR 自动集成到构建系统中。

13.4.2 交互式配置

```
west tl-config .
```

13.5 清理

```
# Clean build artifacts
west tl-build . --clean

# Or manually delete
rm -rf build/
```

13.6 开发工作流程建议

1. **从示例开始**—复制最相似的 samples/ 示例作为起点
2. **Kconfig 隔离**—将项目特定的配置放在项目级别的 Kconfig 中
3. **版本管理**—将应用程序项目和 SDK 分开管理，通过 TELINK_BASE 链接它们

CHAPTER 14

核心架构

本文档介绍 UniSDK 的核心运行机制，包括启动流程、中断管理、异常处理和低功耗框架。

14.1 系统启动流程

UniSDK 的启动过程从硬件复位开始，经过以下阶段：

14.1.1 阶段说明

1. **`cstartup_flash.S`** —汇编启动代码，设置堆栈、复制数据段、清零 BSS、跳转到 C 语言入口点
2. **`tlk_init()`** —SDK 初始化函数，按顺序初始化各个子系统
3. **`tlk_loop()`** —主事件循环，处理 BLE 事件、定时器等

14.2 初始化系统

14.2.1 init 框架

UniSDK 使用模块化初始化框架，通过 `tlk_init.h` 中定义的宏实现：

```
// Each subsystem registers an initialization callback via TLK_INIT_ENTRY
// Initialization is performed in priority order
```

初始化优先级层次：

优先级	典型模块	描述
最早	时钟、内存	系统基础能力
早期	GPIO、PLIC	外设基础设施
中期	UART、I2C、SPI	通信外设
后期	BLE	协议栈
最晚	应用回调	用户应用程序初始化

14.3 中断管理

14.3.1 PLIC（平台级中断控制器）

RISC-V 平台使用 PLIC 管理外设中断：

- **中断源**—每个外设（GPIO、UART、I2C 等）都有专用的 PLIC 中断号
- **优先级**—PLIC 支持多个中断优先级
- **中断嵌套**—通过抢占支持中断嵌套

```
// Enable global interrupts
tlk_core_interrupt_enable();

// Disable global interrupts
tlk_core_interrupt_disable();
```

14.3.2 GPIO 中断示例

```
// Register GPIO interrupt callback
struct tlk_gpio_irq_callback cb = {
    .handler = my_gpio_handler,
    .pin = GPIO_PIN_0
};
tlk_gpio_irq_add_callback(GPIO_PORT_A, &cb);
tlk_gpio_irq_configure(GPIO_PORT_A, GPIO_PIN_0, TLK_GPIO_INTR_RISING_EDGE);
tlk_core_interrupt_enable();
```

14.4 低功耗管理

UniSDK 支持多种低功耗模式：

模式	功耗	唤醒速度	保留内容	使用场景
挂起	中等	快速（< 1ms）	大部分上下文	短暂空闲
深度保留	低	中等	部分 SRAM 保留	长时间空闲，需保持状态
深度睡眠	极低	慢速	SRAM 不保留	长时间休眠

14.4.1 典型用法

```c
#include <tlk_api.h>

void deep_sleep_example(void) {
    // Set GPIO wake-up source
    tlk_pm_set_gpio_wakeup(GPIO_PORT_A, GPIO_PIN_0,
                           TLK_PM_GPIO_WAKEUP_LEVEL_HIGH);

    // Enter Deep Sleep for 10 seconds
    tlk_pm_sleep(TLK_PM_SLEEP_MODE_DEEP_SLEEP, 10000000);

    // Check wake-up reason
    enum tlk_pm_wakeup_source reason = tlk_pm_get_wakeup_reason();
}
```

14.5 上下文切换 (tl_context.S)

`tl_context.S` 实现轻量级的上下文保存和恢复，用于:

- 中断进入/退出时的寄存器保存
- 从低功耗模式唤醒后的状态恢复

14.6 时钟系统

- **系统定时器 (STIMER)** —高精度系统定时器
- **机器定时器 (MTIMER)** —RISC-V 标准定时器，用于调度和延时
- **外设时钟**—每个外设的独立时钟源配置

CHAPTER 15

初始化钩子文档

15.1 概述

本文档描述了由 `common/include/tlk_init.h` 和 `common/tlk_init.c` 提供的初始化和电源状态钩子机制。

系统公开了几种组件可以注册的钩子类型。钩子通过注册宏收集到专用的链接器段中，并由运行时使用 `tlk_init.c` 中实现的函数进行调用。

主要目标：

- 为系统、驱动和应用代码提供有序的初始化钩子（pre-init）。
- 为挂起/睡眠转换提供生命周期钩子（之前/之后）。
- 允许组件通过布尔回调阻止挂起/睡眠。

15.2 概念

- **通过链接器段注册：** 注册宏将函数指针放入命名的链接器段中（例如 `.pre_init_array_<level>_<priority>`）。构建/链接器应按级别和优先级的顺序放置这些段，以便运行时按预期的顺序处理回调。
- **调用：** `tlk_init.c` 中的每个公共 `tlk_*` 函数从链接器提供的 `__*_start` 到 `__*_end` 范围内迭代，并调用找到的每个非 NULL 函数指针。
- **排序约定：** 数值越小表示优先级越早/越高。头文件定义了命名的级别和优先级常量（例如 `TLK_INIT_LEVEL_SYSTEM`/`TLK_INIT_PRIORITY_HIGH`）。最终顺序取决于排列段的链接器脚本，但在注册回调时应使用数值约定。

15.3 级别与优先级

- **级别（示例）：**
  - TLK_INIT_LEVEL_SYSTEM = 2
  - TLK_INIT_LEVEL_DRIVER = 5
  - TLK_INIT_LEVEL_APPLICATION = 8
- **优先级（示例）：**
  - TLK_INIT_PRIORITY_HIGH = 2
  - TLK_INIT_PRIORITY_NORMAL = 5
  - TLK_INIT_PRIORITY_LOW = 8

使用级别按大阶段（系统/驱动/应用）对回调进行分组，并使用优先级在该阶段内微调顺序。数值越小优先级越高。未来预计会添加更多级别或优先级。

15.4 API 参考

15.4.1 初始化钩子（pre-init）

- **类型：** typedef void (*tlk_pre_init_func_t)(void)
- **注册宏：** TLK_REGISTER_PRE_INIT(fn, level, priority) — 将函数指针放入 .pre_init_array_<level>_<priority> 中，并标记为 used，以便链接器不会丢弃它。
- **调用器：** void tlk_pre_init(void) —从 __pre_init_start 到 __pre_init_end 迭代，并调用每个注册的函数。

使用示例：

```
void my_driver_init(void) {
    /* driver init work */
}

TLK_REGISTER_PRE_INIT(my_driver_init, TLK_INIT_LEVEL_DRIVER, TLK_INIT_PRIORITY_HIGH);
```

15.4.2 挂起钩子（之前/之后）

- **类型：**
  - typedef void (*tlk_before_suspend_func_t)(void)
  - typedef void (*tlk_after_suspend_func_t)(void)
- **注册宏：**
  - TLK_REGISTER_BEFORE_SUSPEND(fn, level, priority) → 段 .before_suspend_array_<level>_<priority>
  - TLK_REGISTER_AFTER_SUSPEND(fn, level, priority) → 段 .after_suspend_array_<level>_<priority>
- **调用器：**
  - void tlk_before_suspend(void) —调用所有已注册的挂起前回调。
  - void tlk_after_suspend(void) —调用所有已注册的挂起后回调。

使用示例：

```
void save_peripherals(void) { /* ... */ }

TLK_REGISTER_BEFORE_SUSPEND(save_peripherals, TLK_INIT_LEVEL_DRIVER, TLK_INIT_
↪PRIORITY_NORMAL);
```

15.4.3 睡眠钩子（之前/之后）

- **类型：**
  - typedef void (*tlk_before_sleep_func_t)(void)
  - typedef void (*tlk_after_sleep_func_t)(void)
- **注册宏：**
  - TLK_REGISTER_BEFORE_SLEEP(fn, level, priority) → 段 .before_sleep_array_<level>_<priority>
  - TLK_REGISTER_AFTER_SLEEP(fn, level, priority) → 段 .after_sleep_array_<level>_<priority>
- **调用器：**
  - void tlk_before_sleep(void)
  - void tlk_after_sleep(void)

使用示例：

```
void board_prepare_sleep(void) { /* disable sensors, clocks, ... */ }

TLK_REGISTER_BEFORE_SLEEP(board_prepare_sleep, TLK_INIT_LEVEL_SYSTEM, TLK_INIT_
↪PRIORITY_HIGH);
```

15.4.4 阻止挂起/阻止睡眠回调

- **类型：**
  - typedef bool (*tlk_prevent_suspend_func_t)(void)
  - typedef bool (*tlk_prevent_sleep_func_t)(void)
- **注册宏：**
  - TLK_REGISTER_PREVENT_SUSPEND(fn) → 段 .prevent_suspend_array
  - TLK_REGISTER_PREVENT_SLEEP(fn) → 段 .prevent_sleep_array
- **调用器与语义：**
  - bool tlk_prevent_suspend(void) —当任何已注册的 tlk_prevent_suspend_func_t 返回 true 时立即返回 true；否则返回 false。
  - bool tlk_prevent_sleep(void) —当任何已注册的 tlk_prevent_sleep_func_t 返回 true 时立即返回 true；否则返回 false。

使用示例：

```
bool usb_active(void) {
    return tlk_usb_is_busy();
}

TLK_REGISTER_PREVENT_SLEEP(usb_active);
```

15.5 补充说明

此 API 提供的函数**不应**由用户调用。它们会在相应的生命周期阶段被自动调用。

CHAPTER 16

API 文档

16.1 概述

API 模块为基本的系统工具提供了高级抽象层。它将系统计时、电源管理（睡眠模式）和调试打印整合到统一的接口中。Print 模块作为一个轻量级的格式化输出工具，将调试消息定向到已配置的 UART 实例。

主要特性：

- **系统计时：**微秒级精度的时间戳生成。
- **电源管理：**统一的睡眠功能，支持可配置的电源模式（挂起、深度睡眠、保留）。
- **调试输出：**类 `printf` 的格式化能力。
- **UART 集成：**将调试输出定向到用户可配置的特定 UART 硬件实例。

16.2 API 参考

本节描述 API 和 Print 模块中可用的函数。

16.2.1 1. tlk_api_micros

返回当前系统运行时间的微秒数。该函数使用系统定时器（`stimer`）计算计时器计数值。

原型：

```
uint32_t tlk_api_micros(void);
```

参数：无

返回值： `uint32_t`：自系统启动以来经过的微秒数。

16.2.2 2. tlk_api_sleep

此 *API* 仅在启用 *CONFIG_TLK_API_SLEEP* 时可用。

检查延时时间，如果为零则直接返回。如果启用了 CONFIG_TLK_PM，则调用参数为 TLK_PM_SLEEP_MAX_LEVEL-1 的 tlk_pm_sleep。如果 CONFIG_TLK_PM 未启用，或 tlk_pm_sleep 返回错误，则编程定时器以请求延时，使能定时器中断，并通过 WFI 指令等待。

原型：

```
void tlk_api_sleep(uint32_t duration_ms);
```

参数：

参数	类型	描述
duration_ms	uint32_t	睡眠持续时间，单位为毫秒。

返回值： 无

16.2.3 3. tlk_api_delay

阻塞延时函数

原型：

```
void tlk_api_delay(uint32_t duration_us);
```

参数：

参数	类型	描述
duration_us	uint32_t	等待持续时间，单位为微秒。

返回值： 无

16.2.4 4. tlk_api_print

此 *API* 仅在启用 *CONFIG_TLK_DEBUG_PRINT* 时可用。

一个类似于标准 C printf 的格式化输出函数。它格式化字符串并通过已配置的 UART 接口传输。

原型：

```
int tlk_api_print(const char *restrict format, ...);
```

参数：

参数	类型	描述
format	const char *	格式字符串（例如："Value: %d\n"）。
...	varargs	与格式说明符对应的可变参数。

返回值： int：写入输出缓冲区的字符数。

注意： 此函数使用由 CONFIG_TLK_DEBUG_PRINT_BUFFER_SIZE 定义的内部静态缓冲区。请确保格式化的消息不超过此限制。此函数由 SDK 内部使用，可能在 API 头文件中没有公开声明。

CHAPTER 17

调试与测试

17.1 硬件调试

17.1.1 调试器支持

UniSDK 支持以下调试接口：

调试器	接口	支持平台
J-Link	JTAG/SWD	所有平台
Telink 调试器	SWS（单线）	所有平台
BDT	烧录/调试	所有平台

17.1.2 调试连接

1. 将调试器连接到开发板的 JTAG/SWS 接口
2. 使用 GDB 或 IDE 进行源码级调试

17.2 软件调试

17.2.1 日志系统（调试打印）

UniSDK 内置了调试日志系统，可通过 Kconfig 启用：

```
# Enable debug logging
CONFIG_TLK_DEBUG_PRINT=y
CONFIG_TLK_DEBUG_PRINT_UART=0
CONFIG_TLK_DEBUG_PRINT_BUFFER_SIZE=256
```

日志通过 UART 输出，可在串口终端中查看。

17.2.2 固件验证脚本

```
# Verify firmware integrity
bash scripts/tl_check_fw.sh build/Telink.elf
```

17.3 性能分析

17.3.1 内存使用报告

编译后会自动输出内存使用概况:

```
Memory region         Used Size  Region Size  %age Used
          RAM:        1234 B        64 KB      1.88%
        FLASH:        5678 B       512 KB      1.08%
```

17.3.2 ELF 分析

```
# Generate disassembly for analysis
riscv32-elf-objdump -d build/Telink.elf > disasm.txt

# View section sizes
riscv32-elf-size build/Telink.elf
```

17.4 使用 GDB 调试

```
# Start GDB server (using J-Link or Telink Debugger)
# Then connect the GDB client

riscv32-elf-gdb build/Telink.elf
(gdb) target remote :2331
(gdb) break main
(gdb) continue
```

17.5 常用调试技巧

1. **LED 调试**—在关键代码路径中切换 LED 状态
2. **UART 输出**—通过 TLK_DEBUG_PRINT 输出变量值
3. **断言检查**—使用断言捕获异常状态
4. **内存检查**—查看 .map 文件以验证内存布局

17.6 下一步

- *IDE 集成* —VS Code 开发环境配置
- 构建与配置 —详细的构建系统文档

CHAPTER 18

IDE 集成

18.1 VS Code 配置（推荐）

18.1.1 clangd 语言服务器

clangd 是一个基于 LLVM/Clang 的 C/C++ 语言服务器，能够理解 UniSDK 的复杂宏定义和配置系统，提供：

- 智能代码补全
- 跳转到定义
- 查找引用
- 语义高亮
- 代码导航
- 内联诊断
- 重命名重构

为什么选择 clangd？

UniSDK 大量使用 Kconfig 生成的配置宏（`autoconf.h`、`capabilities.h`）和交叉编译。普通语法高亮无法正确解析这些宏，会导致错误误报和导航失败。clangd 通过读取 `compile_commands.json` 解决了这个问题。

安装步骤

1. 从 VS Code 扩展市场安装 **clangd**（发布者：LLVM）
2. 禁用 Microsoft C/C++ IntelliSense 引擎以避免冲突：
    - `Ctrl+Shift+P` → `Preferences: Open Settings (UI)` → Extensions → C/C++ → IntelliSense

- 将 `IntelliSenseEngine` 设置为 `disabled`

关键文件

`compile_commands.json`

由 CMake 在构建过程中自动生成在 `build/` 目录中。clangd 读取此文件以了解项目的编译选项。

```
[
  {
    "file": "main.c",
    "command": "riscv32-elf-gcc -I/path/to/include -DCONFIG_DEBUG ...",
    "directory": "/project/build"
  }
]
```

`.clangd`（项目根目录的可选配置文件）

```
CompileFlags:
  Add:
    - -Wall
    - -Wextra

Diagnostics:
  Suppress:
    - unused-parameter
```

常见问题

切换到不同芯片后文件显示错误

clangd 基于当前构建的 `compile_commands.json` 提供语义分析。如果项目之前是为 TL321X 构建的，则 TL721X 的文件将无法被正确解析。

解决方法：使用目标芯片重新构建项目。

重新构建后仍显示警告

文件索引是延迟执行的，需要手动触发更新。

解决方法：编辑并保存目标文件以强制更新。

18.1.2 推荐扩展

扩展	用途
clangd	C/C++ 语言服务器
CMake Tools	CMake 项目支持
Cortex-Debug	ARM/RISC-V 调试
GitLens	Git 增强
Error Lens	内联错误显示

18.2 Eclipse 配置

1. 安装 Eclipse CDT 和 RISC-V GCC 插件
2. 以”Existing CMake Project” 方式导入项目
3. 配置交叉编译工具链路径

18.3 Segger Embedded Studio

1. 通过 File → Open Solution 导入项目
2. 将调试器配置为 J-Link
3. 将目标设备设置为 Telink RISC-V 芯片

18.4 下一步

- 项目创建与管理 —创建新项目
- 调试与测试 —调试技巧

CHAPTER 19

VSCode 的 clangd 设置指南

19.1 什么是 clangd?

clangd 是一个基于 LLVM/Clang 工具生态的 C/C++ 语言服务器。它为编辑器提供类似 IDE 的功能，例如：

- 复杂宏展开
- 智能自动补全
- 跳转到定义
- 查找引用
- 语义高亮
- 内联诊断
- 代码导航
- 重命名重构
- 悬停信息

19.2 为什么使用 clangd?

与基本的语法高亮不同，clangd 实际理解：

- 编译器定义
- 包含目录
- 生成的头文件
- 特定标志
- 警告配置

这在使用了 Kconfig、生成配置头文件和交叉编译工具链的项目中非常重要。没有正确的编译信息，编辑器经常显示错误的错误提示和失效的导航。clangd 通过读取项目编译数据库解决了这一问题。

19.3 关键文件

19.3.1 compile_commands.json

这是 clangd 最重要的文件。它由 CMake 生成，包含每个源文件使用的确切编译命令。clangd 读取此文件以正确理解您的项目。

示例：

```
[
 {
    "file": "src/main.c",
    "command": "arm-none-eabi-gcc -Iinclude -DDEBUG ...",
    "directory": "/project/build"
 }
]
```

19.3.2 .clangd

可选的 clangd 项目配置文件。

典型用途：

- 添加额外的编译器标志
- 抑制警告
- 配置索引行为
- 修复生成头文件的包含问题

示例：

```
CompileFlags:
  Add:
 - -Wall
 - -Wextra

Diagnostics:
  Suppress:
 - unused-parameter
```

19.4 安装与设置

1. 打开扩展（Ctrl+Shift+X）并安装 clangd（发布者 - LLVM）。
2. 按照下载提示操作。
3. 建议禁用 Microsoft C/C++ IntelliSense 引擎以避免冲突。(Ctrl+Shift+P) -> Preferences: Open Settings (UI) -> Extension -> C/C++ -> IntelliSense
4. 然后将 IntelliSenseEngine 设置为 disabled。

19.5 故障排除

19.5.1 扩展正常工作，直到我查看另一个核的文件时出现问题

解决方法：以显示错误的文件为目标重新构建项目。**说明**：clangd 基于当前构建的 `compile_commands.json` 提供信息，因此如果您之前以 TL321X 核为目标构建了项目，它将无法正确解析属于 TL721X 核的文件，因为这些文件未包含在构建中。

19.5.2 我刚刚重新构建了项目，但打开的文件中仍然显示警告

解决方法：通过编辑/保存文件来强制更新。**说明**：文件索引是延迟执行的，由于无法确定重建后需要更新什么，您可能需要强制触发更新。

CHAPTER 20

芯片与开发板

20.1 芯片系列概览

UniSDK 支持以下 Telink 芯片系列：

芯片系列	内核	Flash	SRAM	关键外设	典型应用
TL321X	RISC-V	—	—	BLE、GPIO、UART、I2C、SPI、ADC、DMA	通用物联网
TL721X	RISC-V	—	—	BLE、GPIO、UART、I2C、SPI、ADC、DMA、USB	通用物联网
TLSR922X	RISC-V	—	—	BLE、GPIO、UART、I2C、SPI、ADC	通用物联网
TLSR952X	RISC-V	—	—	BLE、GPIO、UART、I2C、SPI、ADC、DMA	通用物联网

20.2 选择芯片

```
# List all supported chips
west tl-socs

# List detailed chip information
west tl-socs -v
```

20.3 开发板列表

开发板	芯片	描述
TL3218X_EVK	TL321X	TL321X 评估套件
TL7218X_EVK	TL721X	TL721X 评估套件
TLSR9228A_EVK	TLSR922X	TLSR922X 评估套件
TLSR9528A_EVK	TLSR952X	TLSR952X 评估套件
TLSR9528A_DONGLE	TLSR952X	TLSR952X USB 适配器

```
# List all supported boards
west tl-boards

# Filter boards by chip
west tl-boards --soc TLSR9528A

# List detailed board information
west tl-boards -v
```

20.4 芯片与开发板配对验证

```
# Validate combination
west tl-config --validate TLSR9528A TLSR9528A_EVK
```

20.5 构建时指定

```
# Specify chip via command line arguments
west tl-build samples/gpio_demo --soc TLSR9528A
# Specify board via command line arguments
west tl-build samples/gpio_demo --board TLSR9528A_EVK

# Specify chip via CMake arguments
cmake -B build -DSOC=TLSR9528A -S .
# Specify board via CMake arguments
cmake -B build -DBOARD=TLSR9528A_EVK -S .
```

CHAPTER 21

TL321X 芯片系列

备注

Document Status: DRAFT —Detailed specifications to be added. The information below is based on SDK metadata and is subject to update.

21.1 概述

TL321X 是 Telink 推出的 RISC-V 微控制器系列，专为通用物联网应用场景而设计。

21.2 核心参数

参数	值
内核	RISC-V（32 位）
支持的协议	BLE
工具链	`andes-riscv32-v5-gcc14.2`（TL32 ELF MCULIB V5 GCC14.2）

21.3 存储器

区域	大小	起始地址
ROM	2048 KB	0x20000000
IRAM	96 KB	0x00068000
DRAM	32 KB	0x00080000

21.4 外设列表

21.4.1 标准外设

- GPIO
- UART
- I2C
- SPI
- ADC
- DMA
- USB

21.4.2 系统与控制

- PLIC（平台级中断控制器）
- PM（电源管理：挂起、深度睡眠、深度保持）
- PMP（物理内存保护）
- STIMER（系统定时器）
- MTIMER（机器定时器）
- PWM
- QDEC（正交解码器）
- MSPI

21.4.3 安全与加密

- PKE（公钥引擎）
- SKE（对称密钥引擎）
- HASH
- IR（红外）

21.4.4 调试与杂项

- SWIRE（单线接口）
- JTAG
- TRAP

21.4.5 支持的封装选项

- QFN28、QFN32、QFN38、QFN48、QFN52、QFN64
- LGA42

21.5 支持的开发板

- *TL3218X_EVK*

CHAPTER 22

TL721X 芯片系列

备注

Document Status: DRAFT —Detailed specifications to be added. The information below is based on SDK metadata and is subject to update.

22.1 概述

TL721X 是 Telink 推出的 RISC-V 微控制器系列，在 TL321X 的基础上增加了 USB 支持。

22.2 核心参数

参数	值
内核	RISC-V（32 位）
支持的协议	BLE
工具链	`andes-riscv32-v5f-gcc14.2`（TL32 ELF MCULIB V5F GCC14.2）

22.3 存储器

区域	大小	起始地址
ROM	2048 KB	0x20000000
IRAM	256 KB	0x00040000
DRAM	256 KB	0x00080000

22.4 外设列表

22.4.1 标准外设

- GPIO
- UART
- I2C
- SPI
- ADC
- DMA
- USB

22.4.2 系统与控制

- PLIC（平台级中断控制器）
- PM（电源管理：挂起、深度睡眠、深度保持）
- PMP（物理内存保护）
- STIMER（系统定时器）
- MTIMER（机器定时器）
- PWM
- QDEC（正交解码器）
- MSPI

22.4.3 安全与加密

- PKE（公钥引擎）
- SKE（对称密钥引擎）
- HASH
- TRNG（真随机数生成器）
- ChaCha20/Poly1305

22.4.4 调试与杂项

- SWIRE（单线接口）
- IR（红外）
- OTP（一次性可编程）
- TRAP

22.4.5 支持的封装选项

- QFN38、QFN68、QFN88
- BGA56、BGA94

22.5 支持的开发板

- *TL7218X_EVK*

CHAPTER 23

TLSR922X 芯片系列

备注

Document Status: DRAFT —Detailed specifications to be added. The information below is based on SDK metadata and is subject to update.

23.1 概述

TLSR922X 是 Telink 推出的 RISC-V 微控制器系列，专为无线音频应用场景而设计。

23.2 核心参数

参数	值
内核	RISC-V（32 位）
支持的协议	BLE
工具链	`andes-riscv32-v5f-gcc14.2`（TL32 ELF MCULIB V5F GCC14.2）

23.3 存储器

区域	大小	起始地址
ROM	—	—
IRAM	256 KB	0x00000000
DRAM	256 KB	0x00080000

23.4 外设列表

23.4.1 标准外设

- GPIO
- UART
- I2C
- SPI
- ADC
- DMA

23.4.2 音频

- 音频编解码器

23.4.3 系统与控制

- PLIC（平台级中断控制器）
- PM（电源管理：挂起、深度睡眠、深度保持）
- PMP（物理内存保护）
- STIMER（系统定时器）
- MTIMER（机器定时器）
- MSPI

23.4.4 安全与加密

- PKE（公钥引擎）
- SKE（对称密钥引擎）
- HASH
- TRNG（真随机数生成器）

23.4.5 支持的封装选项

- QFN48、QFN88

23.5 支持的开发板

- *TLSR9228A_EVK*

CHAPTER 24

TLSR952X 芯片系列

备注

Document Status: DRAFT —Detailed specifications to be added. The information below is based on SDK metadata and is subject to update.

24.1 概述

TLSR952X 是 Telink 推出的 RISC-V 微控制器系列，专为通用物联网应用场景而设计。

24.2 核心参数

参数	值
内核	RISC-V（32 位）
支持的协议	BLE
工具链	`andes-riscv32-v5f-gcc14.2`（TL32 ELF MCULIB V5F GCC14.2）

24.3 存储器

区域	大小	起始地址
ROM	2048 KB	0x20000000
IRAM	256 KB	0x00000000
DRAM	256 KB	0x00080000

24.4 外设列表

24.4.1 标准外设

- GPIO
- UART
- I2C
- SPI
- ADC
- DMA

24.4.2 系统与控制

- PLIC（平台级中断控制器）
- PM（电源管理：挂起、深度睡眠、深度保持）
- PMP（物理内存保护）
- STIMER（系统定时器）
- MTIMER（机器定时器）
- PWM
- QDEC（正交解码器）
- MSPI

24.4.3 安全与加密

- PKE（公钥引擎）
- AES
- TRNG（真随机数生成器）

24.4.4 支持的封装选项

- QFN40、QFN56、QFN88

24.5 支持的开发板

- *TLSR9528A_EVK*
- *TLSR9528A_DONGLE*

CHAPTER 25

TL3218X_EVK 开发板

备注

Document Status: DRAFT —Detailed specifications to be added

25.1 概述

TL3218X_EVK 是 TL321X 芯片系列的官方评估套件。

25.2 构建

```
west tl-build samples/gpio_demo --board TL3218X_EVK
```

CHAPTER 26

TL7218X_EVK 开发板

备注

Document Status: DRAFT —Detailed specifications to be added

26.1 概述

TL7218X_EVK 是 TL721X 芯片系列的官方评估套件。

26.2 构建

```
west tl-build samples/gpio_demo --board TL7218X_EVK
```

CHAPTER 27

TLSR9228A_EVK 开发板

备注

Document Status: DRAFT —Detailed specifications to be added

27.1 概述

TLSR9228A_EVK 是 TLSR922X 芯片系列的官方评估套件。

27.2 构建

```
west tl-build samples/gpio_demo --board TLSR9228A_EVK
```

CHAPTER 28

TLSR9528A_EVK 开发板

备注

Document Status: DRAFT —Detailed specifications to be added

28.1 概述

TLSR9528A_EVK 是 TLSR952X 芯片系列的官方评估套件。

28.2 构建

```
west tl-build samples/gpio_demo --board TLSR9528A_EVK
```

CHAPTER 29

TLSR9528A_DONGLE

备注

Document Status: DRAFT —Detailed specifications to be added

29.1 概述

TLSR9528A_DONGLE 是 TLSR952X 芯片系列的 USB 适配器形态。

29.2 构建

```
west tl-build samples/gpio_demo --board TLSR9528A_DONGLE
```

CHAPTER 30

外设与驱动

UniSDK 为整个 Telink 芯片系列提供了统一的外设驱动 API。所有驱动都遵循一致的编程模型和命名约定。

30.1 驱动概览

驱动	API 前缀	传输模式	描述
GPIO	`tlk_gpio_*`	直接寄存器	通用输入/输出，中断处理
UART	`tlk_uart_*`	阻塞 / PLIC / DMA	通用异步收发传输器, 流控制
电源管理 *PM*	`tlk_pm_*`	—	低功耗模式管理
模拟	`tlk_analog_*`	直接寄存器	模拟寄存器访问
PLIC	`tlk_plic_*`	—	平台级中断控制器
I2C	`tlk_i2c_*`	阻塞 / DMA	I2C 主/从通信
SPI	`tlk_spi_*`	阻塞 / DMA	SPI 主/从通信
ADC	`tlk_adc_*`	阻塞	模数转换
DMA	`tlk_dma_*`	DMA	直接内存访问
看门狗	`tlk_wdt_*`	—	看门狗定时器
定时器	`tlk_stimer_*` / `tlk_mtimer_*`	—	系统/机器定时器
USB	`tlk_usb_*` + TinyUSB	—	USB CDC 等

30.2 驱动编程模型

所有 UniSDK 驱动都遵循统一的编程模型：

30.2.1 配置 → 使用 → 反初始化

```
// 1. Configure (via menuconfig or code)
tlk_gpio_configure(GPIO_PORT_A, GPIO_PIN_0, TLK_GPIO_OUTPUT);

// 2. Use
tlk_gpio_pin_write(GPIO_PORT_A, GPIO_PIN_0, true);

// 3. Optional: Disable
tlk_gpio_disable(GPIO_PORT_A, GPIO_PIN_0);
```

30.2.2 传输模式

某些通信外设支持多种传输模式：

- **阻塞模式**—同步等待传输完成，适用于简单场景
- **PLIC（中断）**—基于中断的非阻塞传输，无需 CPU 轮询
- **DMA** —最高效的批量数据传输，CPU 零负载

通过 Kconfig 为每个外设实例独立配置传输模式。

30.3 Kconfig 配置

所有外设驱动均在编译时通过 Kconfig 进行配置。所有配置选项请参见*Kconfig 参考*。

快速配置入口：

```
# Interactive configuration
west tl-config

# or directly use
make config
```

CHAPTER 31

GPIO 驱动

31.1 概述

GPIO（通用输入/输出）驱动提供了配置和使用通用微控制器引脚的接口。它支持引脚模式配置（输入、输出、上拉、下拉），引脚复用（Pinmux），以及直接引脚状态操作（写、读、翻转）。它还支持可配置触发条件和回调注册的中断处理。

主要特性：

- GPIO 模式配置（输入、输出、上拉、下拉）
- 引脚复用（Mux）
- 直接引脚控制（写、读、翻转）
- 中断处理（边沿/电平触发 + 回调）
- 禁用引脚功能的能力

31.2 数据类型与枚举

31.2.1 端口标识符

`enum tlk_gpio_port` — 在编译时根据 `UNISDK_GPIO_PORT_COUNT` 和 `CONFIG_TLK_GPIO_PORT_##num##_ENABLED` 生成。

值	描述
`GPIO_PORT_x`	每个已启用端口的生成标识符

31.2.2 引脚定义

enum tlk_gpio_pin —定义端口内引脚的位掩码。

值	描述
TLK_GPIO_PIN_NONE	未选择引脚（值为 0）
GPIO_PIN_x	引脚索引位掩码（x 为 0-7），定义为 TLK_BIT(x)

31.2.3 引脚模式

enum tlk_gpio_mode

值	描述
TLK_GPIO_OUTPUT	数字输出（值为 -1）
TLK_GPIO_INPUT_NO_PULL	浮空数字输入
TLK_GPIO_INPUT_PULL_UP	带上拉电阻的数字输入
TLK_GPIO_INPUT_PULL_DOWN	带下拉电阻的数字输入
TLK_GPIO_INPUT_PULL_UP_{na	基于配置 *UNISDK_GPIO_PULL_DOWN_OPTIONS_COUNT* 生成具有特定上拉值的输入。
TLK_GPIO_INPUT_PULL_DOWN_	基于配置 *UNISDK_GPIO_PULL_DOWN_OPTIONS_COUNT* 生成具有特定下拉值的输入。

31.2.4 中断触发类型

enum tlk_gpio_intr_trigger

值	描述
TLK_GPIO_INTR_RISING_EDGE	上升沿触发
TLK_GPIO_INTR_FALLING_EDGE	下降沿触发
TLK_GPIO_INTR_HIGH_LEVEL	高电平触发
TLK_GPIO_INTR_LOW_LEVEL	低电平触发
TLK_GPIO_INTR_BOTH_EDGE	双沿触发

31.2.5 中断回调类型

用于通过用户注册的回调处理 GPIO 中断的类型。

回调处理函数类型

GPIO 中断回调的函数指针类型。

```
typedef void (*tlk_gpio_irq_callback_handler)(enum tlk_gpio_port port,
                                              enum tlk_gpio_pin pin);
```

参数	描述
port	发生中断的 GPIO 端口。
pin	触发中断的 GPIO 引脚。

回调注册结构体

用于在内部回调列表中注册 GPIO 中断回调的结构体。

```
struct tlk_gpio_irq_callback {
    struct tlk_list_node node;
    tlk_gpio_irq_callback_handler handler;
    enum tlk_gpio_pin pin;
};
```

字段	描述
node	用于链接回调的内部列表节点。
handler	用户定义的中断回调函数。
pin	与此回调关联的 GPIO 引脚。

31.2.6 端口引脚结构体

```
struct tlk_gpio_port_pin {
    enum tlk_gpio_port port;
    enum tlk_gpio_pin pin;
};
```

31.3 驱动 API

31.3.1 tlk_gpio_disable

禁用引脚功能并重置配置。

```
void tlk_gpio_disable(enum tlk_gpio_port port, enum tlk_gpio_pin pin);
```

参数:

参数	类型	描述
port	enum tlk_gpio_port	GPIO 端口。
pin	enum tlk_gpio_pin	GPIO 引脚。

返回值: 无

31.3.2 tlk_gpio_configure

设置 GPIO 引脚模式（输入/输出，可选上拉/下拉）。

```
void tlk_gpio_configure(enum tlk_gpio_port port, enum tlk_gpio_pin pin,
                        enum tlk_gpio_mode mode);
```

参数:

参数	类型	描述
port	enum tlk_gpio_port	GPIO 端口。
pin	enum tlk_gpio_pin	GPIO 引脚。
mode	enum tlk_gpio_mode	GPIO 模式（输出/输入上拉/下拉）。

返回值： 无

31.3.3 tlk_gpio_pin_write

设置引脚输出电平。

```
void tlk_gpio_pin_write(enum tlk_gpio_port port, enum tlk_gpio_pin pin, bool value);
```

参数：

参数	类型	描述
port	enum tlk_gpio_port	GPIO 端口。
pin	enum tlk_gpio_pin	GPIO 引脚。
value	bool	逻辑电平（true = 高，false = 低）。

返回值： 无

31.3.4 tlk_gpio_pin_toggle

翻转引脚的当前输出值。

```
void tlk_gpio_pin_toggle(enum tlk_gpio_port port, enum tlk_gpio_pin pin);
```

参数：

参数	类型	描述
port	enum tlk_gpio_port	GPIO 端口。
pin	enum tlk_gpio_pin	GPIO 引脚。

返回值： 无

31.3.5 tlk_gpio_pin_read

读取引脚的当前逻辑电平。

```
bool tlk_gpio_pin_read(enum tlk_gpio_port port, enum tlk_gpio_pin pin);
```

参数：

参数	类型	描述
port	enum tlk_gpio_port	GPIO 端口。
pin	enum tlk_gpio_pin	GPIO 引脚。

返回值： 如果引脚电平为高则返回 `true`，为低则返回 `false`。

31.3.6 tlk_gpio_set_mux

配置引脚复用功能。

```
void tlk_gpio_set_mux(enum tlk_gpio_port port, enum tlk_gpio_pin pin, uint8_t mux);
```

参数：

参数	类型	描述
port	enum tlk_gpio_port	GPIO 端口。
pin	enum tlk_gpio_pin	GPIO 引脚。
mux	uint8_t	所选功能的复用器索引。

返回值： 无

31.3.7 tlk_gpio_irq_configure

设置中断触发类型。

```
void tlk_gpio_irq_configure(enum tlk_gpio_port port, enum tlk_gpio_pin pin,
                            enum tlk_gpio_intr_trigger trigger);
```

参数：

参数	类型	描述
port	enum tlk_gpio_port	GPIO 端口。
pin	enum tlk_gpio_pin	GPIO 引脚。
trigger	enum tlk_gpio_intr_trigger	中断触发模式（边沿或电平）。

返回值： 无

31.3.8 tlk_gpio_irq_add_callback

注册中断回调函数。

```
void tlk_gpio_irq_add_callback(enum tlk_gpio_port port,
                               struct tlk_gpio_irq_callback *callback);
```

参数：

参数	类型	描述
port	enum tlk_gpio_port	GPIO 端口。
callback	struct tlk_gpio_irq_callback *	指向 GPIO 中断回调结构体的指针。

返回值： 无

31.4 Kconfig 配置

```
GPIO_PORT_{name}_ENABLED
|   Enables the specific GPIO port instance (where {name} is the port name, e.g., A,␣
↪B, etc.).
|   Automatically selects the corresponding internal port index.
|   Default: `y`

TLK_GPIO_PREVENT_SLEEP
|   Indicates that the system should not enter sleep mode if any GPIO configured as␣
↪an output is enabled.
|   Default: `y` if `PM` is enabled, otherwise `n`.

TLK_GPIO_PM_DEVICE
|   Indicates that Power Management support is enabled for the GPIO driver.
|   Default: `y` if `PM` is enabled, otherwise `n`.
```

31.4.1 在 menuconfig 中查看

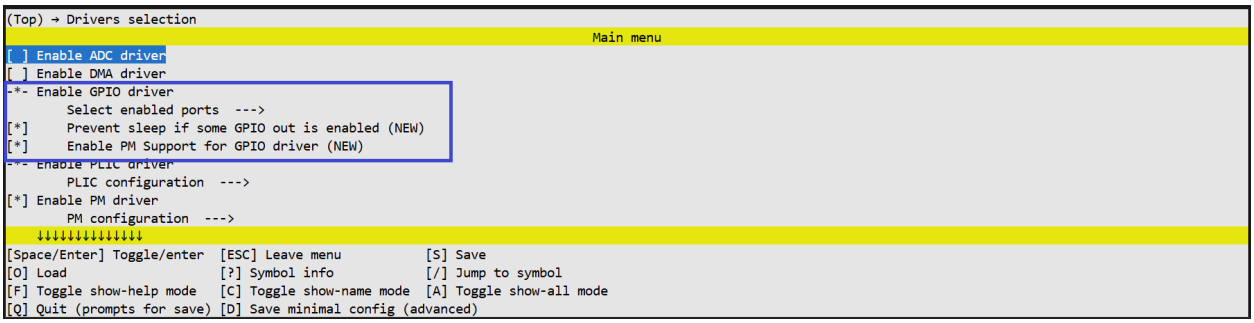


图 *1. GPIO* 驱动配置

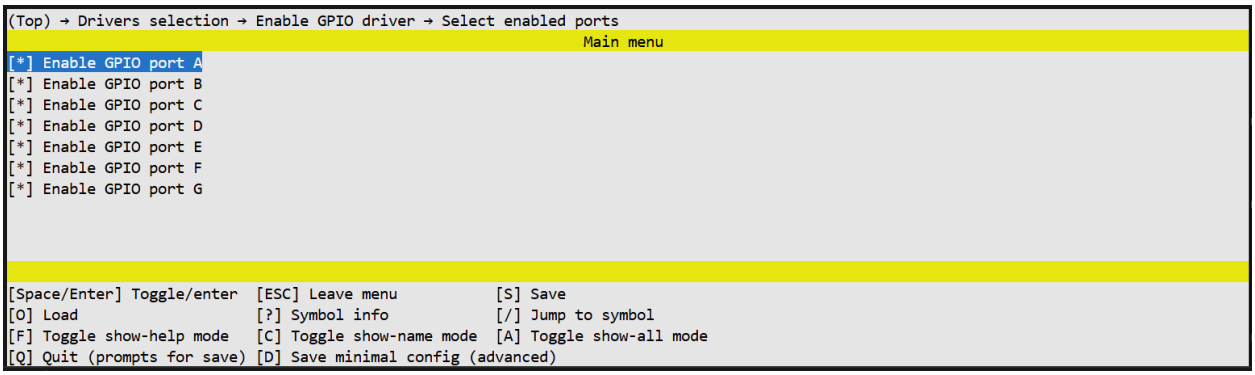


图 *2. GPIO* 驱动端口配置

31.5 相关资源

- *GPIO 示例* —可直接运行的 GPIO 示例代码

CHAPTER 32

UART 驱动

32.1 概述

UART 驱动提供了配置和使用异步串口的接口。它支持通信参数配置（波特率、校验位、停止位），引脚复用（Pinmux），数据发送（阻塞模式）和数据接收（阻塞和非阻塞中断模式）。它还支持硬件流控制（RTS/CTS）。

主要特性：

- UART 参数配置（波特率、校验位、停止位）
- TX/RX 和 RTS/CTS 引脚复用
- 数据发送（阻塞 / PLIC / DMA 模式）
- 数据接收（阻塞 / PLIC / DMA 模式 + 回调）
- 硬件流控制（RTS/CTS）

32.2 数据类型与枚举

32.2.1 UART 模块标识符

`enum tlk_uart_module` —根据 `UNISDK_UART_COUNT` 和已启用的实例自动生成。值包括 `UART0`、`UART1` 等。

32.2.2 校验位

`enum tlk_uart_parity`

值	描述
`TLK_UART_PARITY_NONE`	无校验
`TLK_UART_PARITY_EVEN`	偶校验
`TLK_UART_PARITY_ODD`	奇校验

32.2.3 停止位

enum tlk_uart_stop_bit

值	描述
TLK_UART_STOP_BIT_ONE	1 个停止位
TLK_UART_STOP_BIT_ONE_DOT_FIVE	1.5 个停止位
TLK_UART_STOP_BIT_TWO	2 个停止位

32.2.4 RX 超时倍数

enum tlk_uart_rx_timeout_mul

RX 超时时长的倍数。总 RX 超时时间 =（一个 UART 帧时长）× 倍数。

值	描述
TLK_UART_RX_TIMEOUT_MUL_1	1 倍帧时长
TLK_UART_RX_TIMEOUT_MUL_2	2 倍帧时长
TLK_UART_RX_TIMEOUT_MUL_3	3 倍帧时长
TLK_UART_RX_TIMEOUT_MUL_4	4 倍帧时长

32.2.5 UART 配置结构体

struct tlk_uart_config

包含初始化 UART 模块所需所有配置参数的结构体。与 tlk_uart_configure 和 tlk_uart_configure_pinmux 配合使用。

字段	类型	描述
baudrate	uint32_t	通信速率，单位为波特
parity	enum tlk_uart_parity	校验模式
stop_bit	enum tlk_uart_stop_bit	停止位数
rx_timeout_mul	enum tlk_uart_rx_timeout_mul	RX 超时倍数
tx_port_pin	struct tlk_gpio_port_pin	TX 端口和引脚
rx_port_pin	struct tlk_gpio_port_pin	RX 端口和引脚

32.2.6 操作状态

enum tlk_uart_status

驱动函数用于指示操作结果的返回码。

值	描述
TLK_UART_OK	操作成功完成。
TLK_UART_TIMEOUT	操作超时。
TLK_UART_ERROR	常规执行错误。

32.2.7 流控制模式

enum tlk_uart_flow_control

值	描述
TLK_UART_FLOW_NONE	流控制已禁用。
TLK_UART_FLOW_RTS	仅 RTS 信号。
TLK_UART_FLOW_CTS	仅 CTS 信号。
TLK_UART_FLOW_RTS_CTS	完整 RTS/CTS 流控制。

32.2.8 流控制极性

enum tlk_uart_flow_polarity

定义 RTS 和 CTS 信号的有效电平逻辑。

值	描述
TLK_UART_ACTIVE_LOW	**低电平有效**。当 CTS 为低电平时 UART 停止发送，并将 RTS 置为低电平以停止接收。
TLK_UART_ACTIVE_HIG	**高电平有效**。当 CTS 为高电平时 UART 停止发送，并将 RTS 置为高电平以停止接收。

32.2.9 流控制配置结构体

struct tlk_uart_flow_control_config

包含 UART 硬件流控制所有配置参数的结构体。与 tlk_uart_configure_flow_control 和 tlk_uart_configure_flow_control_pinmux 配合使用。

字段	类型	描述
flow_control	enum tlk_uart_flow_control	流控制模式（RTS/CTS）
flow_polarity	enum tlk_uart_flow_polarity	RTS/CTS 信号的有效电平
rts_port_pin	struct tlk_gpio_port_pin	RTS 端口和引脚
cts_port_pin	struct tlk_gpio_port_pin	CTS 端口和引脚

32.2.10 发送处理函数类型

tlk_uart_tx_handler_t

函数指针类型，用于中断和 DMA 模式下数据发送完成时调用的回调函数。

```
typedef void (*tlk_uart_tx_handler_t)(void);
```

32.2.11 接收处理函数类型

tlk_uart_rx_handler_t

函数指针类型，用于中断和 DMA 模式下数据接收完成时调用的回调函数。

```
typedef void (*tlk_uart_rx_handler_t)(uint32_t rx_count);
```

32.3 驱动 API

32.3.1 tlk_uart_configure

使用提供的配置初始化 UART 模块。

```
void tlk_uart_configure(enum tlk_uart_module uart_num,
                        struct tlk_uart_config *config);
```

参数:

参数	类型	描述
uart_num	enum tlk_uart_module	UART 模块索引。
config	struct tlk_uart_config *	指向包含波特率、校验位、停止位和引脚分配的配置结构体的指针。

返回值: 无

32.3.2 tlk_uart_configure_pinmux

配置 UART TX/RX 引脚复用。

```
void tlk_uart_configure_pinmux(enum tlk_uart_module uart_num,
                               struct tlk_uart_config *config);
```

参数:

参数	类型	描述
uart_num	enum tlk_uart_module	UART 模块索引。
config	struct tlk_uart_config *	指向包含 TX 和 RX 引脚分配的配置结构体的指针。

返回值: 无

32.3.3 tlk_uart_send_bytes

发送数据缓冲区。

```
enum tlk_uart_status tlk_uart_send_bytes(enum tlk_uart_module uart_num,
    const uint8_t *buff, uint32_t buff_size,
    tlk_uart_tx_handler_t tx_handler, uint32_t timeout_us);
```

32.3.4 tlk_uart_receive_bytes

接收数据缓冲区。接收完成时会调用提供的处理函数。

```
enum tlk_uart_status tlk_uart_receive_bytes(enum tlk_uart_module uart_num,
    uint8_t *buff, uint32_t buff_size,
    tlk_uart_rx_handler_t rx_handler);
```

参数：

参数	类型	描述
uart_num	enum tlk_uart_module	UART 模块索引。
buff	uint8_t *	指向接收缓冲区的指针。
buff_size	uint32_t	要接收的字节数。
rx_handler	tlk_uart_rx_handler_t	接收完成后调用的回调函数。

返回值： enum tlk_uart_status —返回 TLK_UART_OK。

32.3.5 tlk_uart_configure_flow_control

配置指定 UART 模块的流控制。默认禁用。应在 tlk_uart_configure 之后调用。

```
void tlk_uart_configure_flow_control(enum tlk_uart_module uart_num,
                        struct tlk_uart_flow_control_config *config);
```

参数：

参数	类型	描述
uart_num	enum tlk_uart_module	UART 模块索引。
config	struct tlk_uart_flow_control_config *	指向流控制配置结构体的指针。

返回值： 无

32.3.6 tlk_uart_configure_flow_control_pinmux

配置 UART RTS 和 CTS 引脚复用。

```
void tlk_uart_configure_flow_control_pinmux(enum tlk_uart_module uart_num,
                                struct tlk_uart_flow_control_config *config);
```

参数：

参数	类型	描述
uart_nur	enum tlk_uart_module	UART 模块索引。
config	struct tlk_uart_flow_control_co *	指向包含 RTS 和 CTS 引脚分配的流控制配置结构体的指针。

返回值： 无

32.4 传输模式

每个 UART 实例可以独立配置其传输模式：

模式	Kconfig 值	描述
阻塞	TLK_UART_XFER_BLOCKING	同步等待，带超时
中断	TLK_UART_XFER_PLIC	中断驱动，自动填充 FIFO
DMA	TLK_UART_XFER_DMA	DMA 驱动，CPU 零负载

32.5 Kconfig 配置

32.5.1 全局选项

TLK_UART_SELECTED 表示系统中至少启用了一个 UART 实例。当任何 UART{i}_ENABLED' 选项启用时，此选项会自动被选中。

TLK_UART_USED_AUTO_CONFIG 表示系统中启用了自动 UART 配置。当任何 UART 实例启用了 UART{i}_AUTO_CONFIG 时，此选项会自动被选中。

TLK_UART_USED_IRQ 表示至少一个 UART 实例需要 PLIC 功能。当任何 UART 实例启用了 UART{i}_USED_IRQ 或 UART{i}_USED_DMA 时，此选项会自动被选中。

TLK_UART_USED_DMA 表示至少一个 UART 实例需要 DMA 功能。当任何 UART 实例启用了 UART{i}_USED_DMA 时，此选项会自动被选中。

TLK_UART_USED_PM_DEVICE 表示系统中启用了 UART 电源管理支持。当任何 UART 实例启用了 UART{i}_PM_DEVICE 时，此选项会自动被选中。

TLK_UART_USED_FLOW_CONTROL 表示系统中启用了硬件流控制（RTS/CTS）。当任何 UART 实例启用了 UART{i}_FLOW_CONTROL 时，此选项会自动被选中。

32.5.2 逐实例配置

每个 UART 实例（i = 0, 1, 2...）支持以下分层配置：

32.5.3 在 menuconfig 中查看

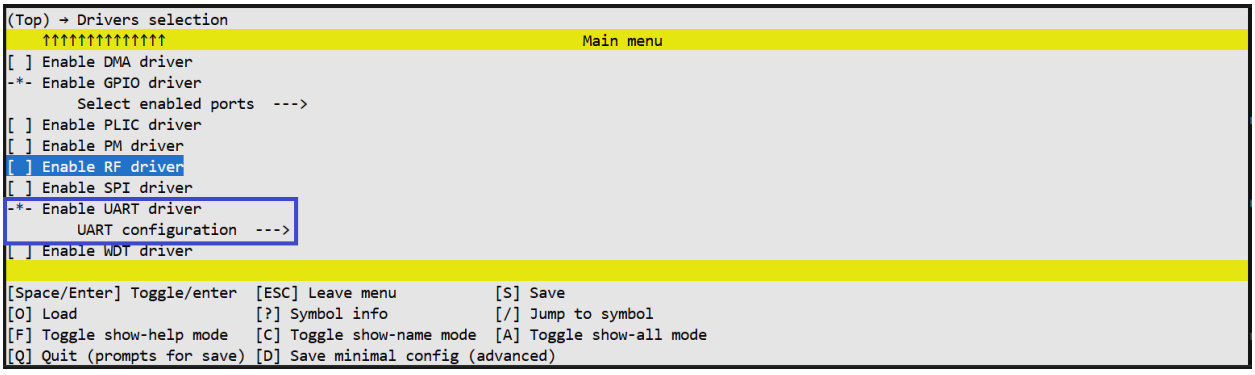


图 *1. UART* 驱动配置

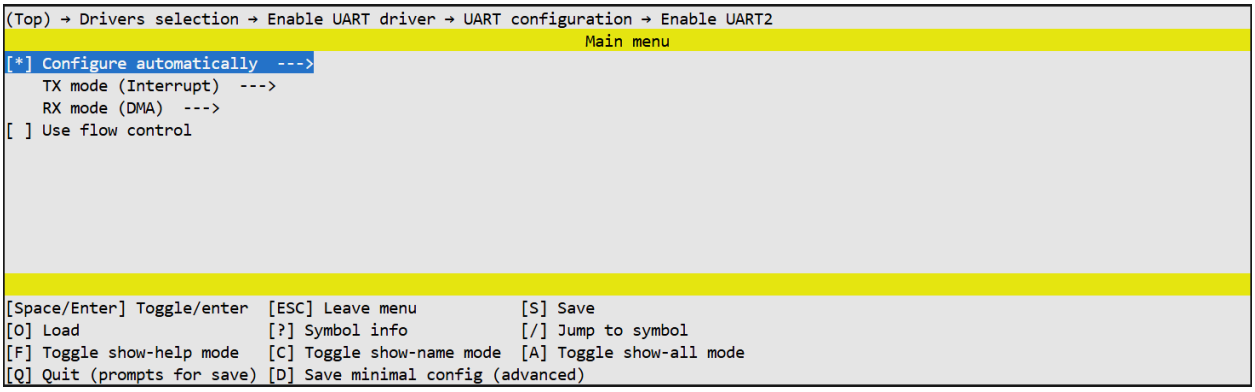


图 2. *UART* 实例配置

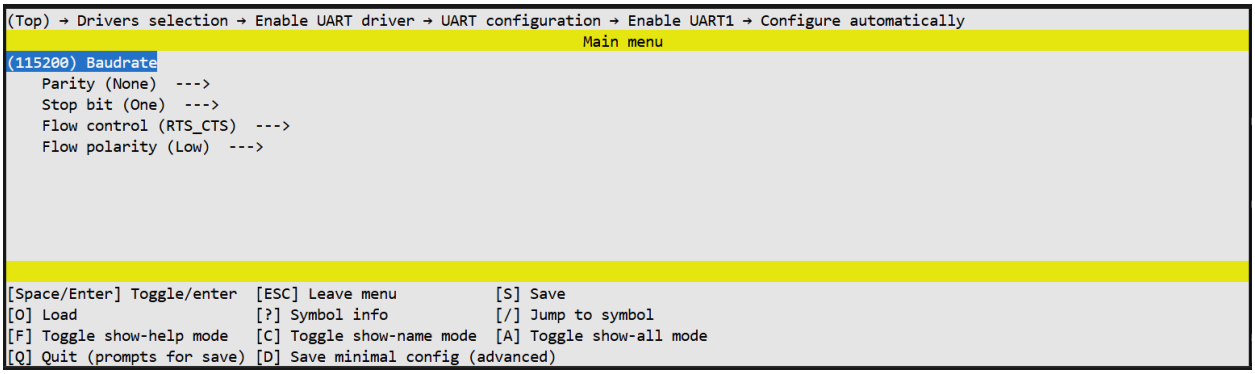


图 3. *UART* 实例自动配置选项

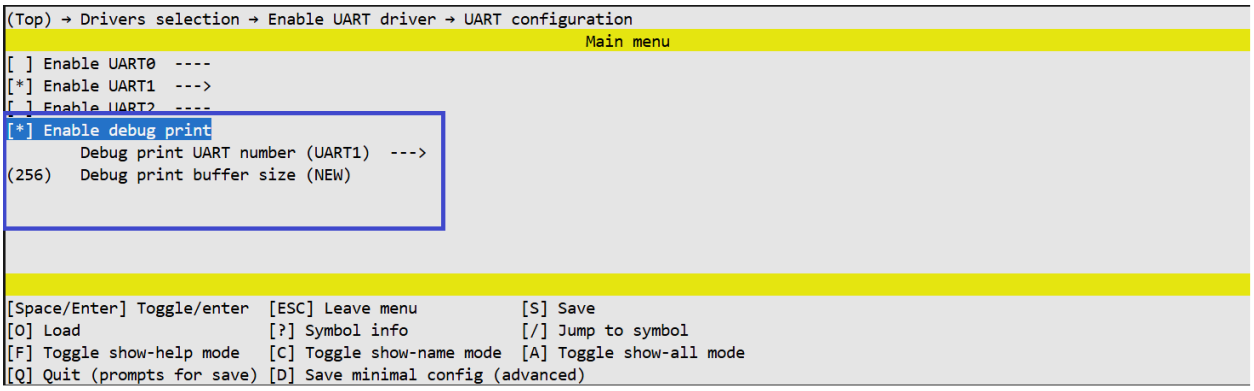


图 4. *UART* 调试配置

32.6 相关资源

- *UART* 示例

CHAPTER 33

Power Management (PM) Driver

33.1 Overview

The PM (Power Management) driver provides an interface for controlling microcontroller power states. It supports the system entering various low-power modes to save energy, and defines system wakeup mechanisms. The driver also manages retention memory and sleep duration validation.

Key Features:

- Multiple sleep modes (Suspend, Deep Sleep, Deep Retention)
- Wakeup source management (GPIO Pad, Timer, Core, Comparator)
- Configurable sleep duration limits (minimum time)
- Retention memory size configuration
- Error handling for invalid sleep requests

33.2 Sleep Modes

`enum tlk_pm_sleep_mode`

Mode	Power	Wakeup Speed	Description
`TLK_PM_SLEEP_MODE_SUSPEND`	Mediun	Fast	SRAM retained, peripherals partially powered
`TLK_PM_SLEEP_MODE_DEEP_SL`	Very low	Slow	SRAM not retained, most peripherals powered off
`TLK_PM_SLEEP_MODE_DEEP_RE`	Low	Medium	SRAM partially retained

33.3 Wakeup Sources

`enum tlk_pm_wakeup_source` (bitmask)

Value	Description
`TLK_PM_WAKEUP_SOURCE_PAD`	GPIO pin wakeup
`TLK_PM_WAKEUP_SOURCE_TIMER`	System timer wakeup
`TLK_PM_WAKEUP_SOURCE_CORE`	Core event wakeup
`TLK_PM_WAKEUP_SOURCE_COMPARATOR`	Comparator wakeup

33.4 GPIO Wakeup Level

`enum tlk_pm_gpio_wakeup_level`

Defines the logic level required on a GPIO pin to trigger a wakeup.

Value	Description
`TLK_PM_GPIO_WAKEUP_LEVEL_LOW`	Wake up when the pin is Low (0)
`TLK_PM_GPIO_WAKEUP_LEVEL_HIGH`	Wake up when the pin is High (1)

33.5 Driver API

33.5.1 tlk_pm_sleep

Requests the system to enter a low-power mode.

```
enum tlk_pm_sleep_status tlk_pm_sleep(enum tlk_pm_sleep_mode mode,
                                      uint32_t duration_us);
```

Return values:

Value	Description
`TLK_PM_SLEEP_OK`	Successfully entered and exited sleep
`TLK_PM_SLEEP_UNSUPPORTED`	The requested sleep mode is not supported by the hardware
`TLK_PM_SLEEP_TOO_SHORT`	Requested duration is less than the configured minimum
`TLK_PM_SLEEP_TOO_LONG`	The requested duration exceeded the hardware timer limits
`TLK_PM_SLEEP_NO_WAKEUP_SOUR(`	Sleep was aborted because no wakeup sources were configured
`TLK_PM_SLEEP_DENIED`	Sleep denied (e.g., blocked by a peripheral)

33.5.2 tlk_pm_set_gpio_wakeup

This API is available only if `CONFIG_TLK_GPIO` is enabled.

Configures a GPIO pin as a wakeup source. The system will exit sleep mode when the selected pin detects the configured wakeup polarity.

```
enum tlk_pm_sleep_status tlk_pm_set_gpio_wakeup(
    enum tlk_gpio_port port, enum tlk_gpio_pin pin,
    enum tlk_pm_gpio_wakeup_level polarity);
```

Parameters:

Parameter	Type	Description
`port`	`enum tlk_gpio_port`	GPIO port
`pin`	`enum tlk_gpio_pin`	GPIO pin
`polarity`	`enum tlk_pm_gpio_wakeup_level`	The logic level that triggers wakeup

Return Value:

`enum tlk_pm_sleep_status`: `TLK_PM_SLEEP_OK` if successful, or `TLK_PM_SLEEP_UNSUPPORTED` if the pin cannot be used as a wakeup source.

33.5.3 tlk_pm_get_wakeup_reason

Gets the last wakeup reason.

```
enum tlk_pm_wakeup_source tlk_pm_get_wakeup_reason(void);
```

33.5.4 tlk_pm_clear_wakeup_sources

Clears wakeup source flags in preparation for the next sleep.

```
void tlk_pm_clear_wakeup_sources(void);
```

33.6 Typical Usage

```
#include <tlk_api.h>

void sleep_example(void) {
    // Set GPIO A0 high level wakeup
    tlk_pm_set_gpio_wakeup(GPIO_PORT_A, GPIO_PIN_0,
                           TLK_PM_GPIO_WAKEUP_LEVEL_HIGH);

    // Enter Deep Sleep for 5 seconds
    enum tlk_pm_sleep_status status =
        tlk_pm_sleep(TLK_PM_SLEEP_MODE_DEEP_SLEEP, 5000000);

    if (status == TLK_PM_SLEEP_OK) {
        // Check wakeup reason
        enum tlk_pm_wakeup_source src = tlk_pm_get_wakeup_reason();
        if (src & TLK_PM_WAKEUP_SOURCE_PAD) {
            // GPIO wakeup
        }
        tlk_pm_clear_wakeup_sources();
    }
}
```

33.7 Kconfig Configuration

PM Kconfig Options

```
TLK_PM_SUSPEND_MIN_DURATION_MS
|   Defines the minimum allowed duration for Suspend mode (in milliseconds).
|   If `tlk_pm_sleep` is called with a time shorter than this, it returns `TLK_PM_
↪SLEEP_TOO_SHORT`.
|   Default: 2 ms
|
TLK_PM_DEEP_SLEEP_MIN_DURATION_MS
|   Defines the minimum allowed duration for Deep Sleep mode (in milliseconds).
|   Default: 3 ms
|
TLK_PM_DEEP_RETENTION_MIN_DURATION_MS
|   Defines the minimum allowed duration for Deep Retention mode (in milliseconds).
|   Default: 3 ms
|
TLK_PM_RETENTION_MEMORY_SIZE
|   Selects the size of the RAM to be retained during Deep Retention mode.
|   The available options (e.g., 16K, 32K) depend on the specific SoC capabilities.
```

33.7.1 Menuconfig View

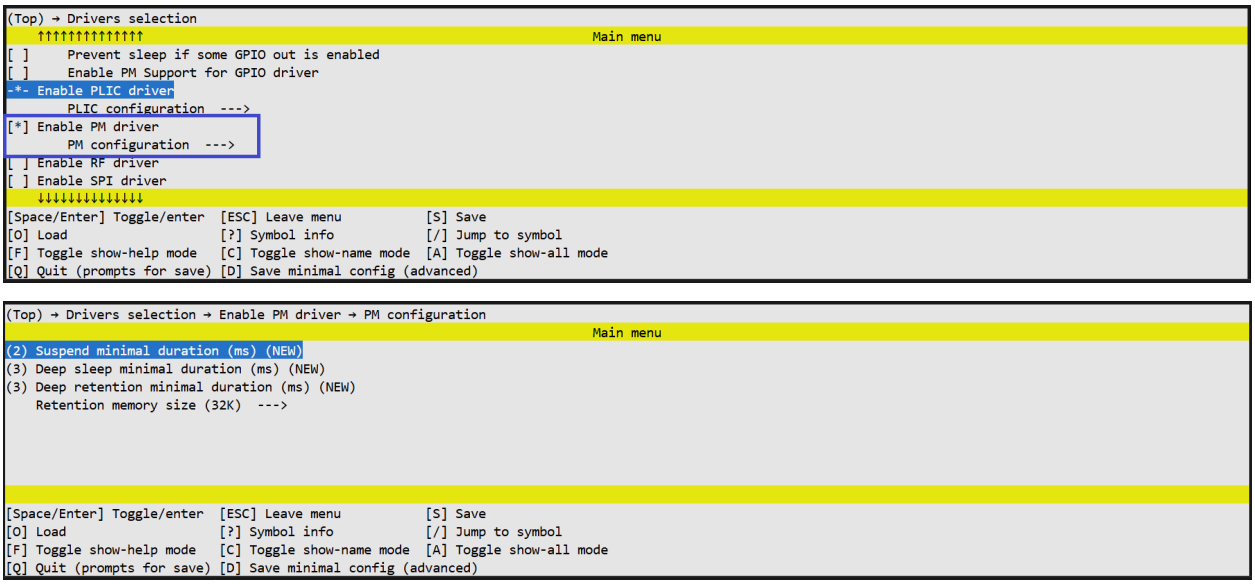


Figure 1-2. PM driver configurations

33.8 Related Resources

- *PM Samples*

CHAPTER 34

模拟驱动

34.1 概述

模拟驱动提供对芯片模拟寄存器的原始访问以及类型安全的寄存器字段读/写宏定义。

主要特性：

- 按字节、半字和字读写模拟寄存器
- YAML 寄存器描述转换为 C 类型定义
- 类型安全的寄存器读/写/修改宏

34.2 API 参考

34.2.1 C 函数

```
// Analog register read/write
uint8_t  tlk_analog_read_reg8(uint8_t addr);
void     tlk_analog_write_reg8(uint8_t addr, uint8_t data);
uint16_t tlk_analog_read_reg16(uint8_t addr);
void     tlk_analog_write_reg16(uint8_t addr, uint16_t data);
uint32_t tlk_analog_read_reg32(uint8_t addr);
void     tlk_analog_write_reg32(uint8_t addr, uint32_t data);
```

这些函数构成底层传输层，主要由类型安全宏内部使用。

34.2.2 类型安全宏

> **重要提示**：为充分利用此功能，您需要使用能够展开深层宏的语言服务器，例如 clangd。然后您将能够完整地使用自动补全、类型提示、高亮等功能。

要使用这些宏，请包含带有完整模拟寄存器描述的 `registers/tlk_analog.h`。其中包含带有相应地址的命名宏（例如 `TLK_AREG_BG_CTRL0 -> 0x00`），应优先使用这些宏而非原始地址。

如果寄存器包含位字段，则会生成联合体类型：

```
typedef union tlk_areg_bg_ctrl0 {
    struct {
        <bitfields>
    } bit;
    uint8_t raw;
} tlk_areg_0x00;
```

宏	描述
TLK_AREG_TYPE(reg)	展开为指定模拟寄存器的 C 类型
TLK_AREG_SIZE(reg)	展开为生成的寄存器大小常量（以位为单位）
TLK_ANALOG_READ(reg)	读取寄存器，并将返回值强制转换为寄存器值类型
TLK_ANALOG_WRITE(reg, var)	将类型化的寄存器值写回寄存器
TLK_ANALOG_MODIFY(reg, statement)	读取、修改并写回类型化的寄存器值
TLK_ANALOG_MODIFY_RAW8(reg, statement)	使用 uint8_t 修改原始 8 位寄存器数据
TLK_ANALOG_MODIFY_RAW16(reg, statement)	使用 uint16_t 修改原始 16 位寄存器数据
TLK_ANALOG_MODIFY_RAW32(reg, statement)	使用 uint32_t 修改原始 32 位寄存器数据

注意：头文件中的内部辅助宏以下划线开头，不应用于用户代码。

34.3 寄存器定义

寄存器定义来自 core/*/registers/analog.yaml，类型安全的 C 访问宏通过 registers_gen.py 自动生成。

CHAPTER 35

PLIC 中断控制器

35.1 概述

PLIC（平台级中断控制器）驱动提供系统中断管理接口。它允许开发者配置中断优先级、启用或禁用特定中断源，以及管理中断抢占（嵌套）。

中断处理模式（常规模式 vs. 向量模式）在编译时通过系统配置（`.config`）确定。

主要特性：

- 编译时选择全局中断模式（常规 / 向量）
- 中断优先级和阈值管理
- 中断源启用/禁用
- 中断嵌套支持
- 可选的软件中断支持（触发、启用、禁用）

35.2 数据类型和枚举

35.2.1 中断优先级

`enum tlk_irq_priority`

值	描述
`TLK_IRQ_PRI_LEV0`	优先级 0 —不产生中断（实际禁用）
`TLK_IRQ_PRI_LEV1`	优先级级别 1（最低）
`TLK_IRQ_PRI_LEV2`	优先级级别 2
`TLK_IRQ_PRI_LEV3`	优先级级别 3（最高）

35.3 API 参考

35.3.1 TLK_PLIC_ISR_REGISTER（宏）

一个辅助宏，用于为特定中断号定义和注册中断服务例程（ISR）。它会自动为向量模式或常规模式生成正确的函数属性。

```
TLK_PLIC_ISR_REGISTER(my_uart_handler, IRQ_UART0);
```

参数	描述
isr	作为处理函数调用的 C 函数名称
irq_num	中断号（例如 IRQ_UART0）

35.3.2 tlk_plic_interrupt_enable

启用特定的中断源。

```
void tlk_plic_interrupt_enable(uint32_t src);
```

35.3.3 tlk_plic_interrupt_disable

禁用特定的中断源。

```
void tlk_plic_interrupt_disable(uint32_t src);
```

35.3.4 tlk_plic_set_priority

设置特定中断源的优先级级别。

```
void tlk_plic_set_priority(uint32_t src, enum tlk_irq_priority priority);
```

35.3.5 tlk_plic_set_threshold

设置全局中断阈值。只有优先级严格高于此阈值的中断才会被处理。

```
void tlk_plic_set_threshold(enum tlk_irq_priority threshold);
```

35.3.6 tlk_plic_set_pending

手动将中断源设置为挂起状态。

```
void tlk_plic_set_pending(uint32_t src);
```

35.3.7 tlk_plic_clr_all_request

清除所有挂起的中断请求。

```
int32_t tlk_plic_clr_all_request(void);
```

35.3.8 tlk_plic_preempt_enable

为高于指定级别的优先级启用中断抢占（嵌套中断）。

```
void tlk_plic_preempt_enable(tlk_core_preempt_pri_e preempt_pri);
```

35.3.9 tlk_plic_preempt_disable

禁用中断抢占。

```
void tlk_plic_preempt_disable(void);
```

35.3.10 tlk_plic_irqs_preprocess_for_wfi

在系统进入”等待中断”（WFI）或休眠状态前准备 PLIC 控制器。

```
void tlk_plic_irqs_preprocess_for_wfi(uint8_t flag, tlk_mie_e mie);
```

35.3.11 tlk_plic_irqs_postprocess_for_wfi

在系统从 WFI 或休眠状态唤醒后恢复 PLIC 状态。

```
void tlk_plic_irqs_postprocess_for_wfi(void);
```

35.3.12 软件中断 API

函数	描述
`tlk_plic_sw_set_pending()`	手动触发软件中断
`tlk_plic_sw_interrupt_enable()`	启用软件中断源
`tlk_plic_sw_interrupt_disable()`	禁用软件中断源
`tlk_plic_sw_interrupt_claim()`	获取软件中断源（通常在 ISR 内部使用）
`tlk_plic_sw_interrupt_complete()`	完成软件中断，允许处理新的请求

35.4 PLIC 配置

本节描述 Kconfig 中可用的 PLIC 配置选项。

配置	默认值	描述
TLK_PLIC_VECTOR_M	y	选择全局中断处理模式。启用时使用向量模式（中断直接跳转到其向量地址）。禁用时使用常规模式（所有中断共享一个公共处理函数）。
PLIC_SW（或 CONFIG_TLK_PLIC_S	n	启用软件中断支持。启用后，`tlk_plic_sw_set_pending()` 等 API 将可用。

35.5 中断处理模式

模式	描述
常规模式	所有中断共享一个入口点；在 ISR 中查询中断源
向量模式	每个中断源具有独立的向量地址；直接跳转

35.6 中断嵌套

PLIC 支持基于优先级的中断抢占：

- 高优先级中断可以抢占低优先级中断
- 通过 `tlk_plic_set_threshold` 控制最小响应优先级

35.7 与 GPIO 中断的集成

GPIO 中断通过 PLIC 路由。配置步骤：

1. 配置 PLIC 中断优先级
2. 配置 GPIO 中断触发条件（`tlk_gpio_irq_configure`）
3. 注册 GPIO 中断回调函数（`tlk_gpio_irq_add_callback`）
4. 启用全局中断（`tlk_core_interrupt_enable`）

```
// Complete example
tlk_plic_set_priority(IRQ_GPIO, 3);
tlk_gpio_irq_configure(GPIO_PORT_A, GPIO_PIN_0, TLK_GPIO_INTR_RISING_EDGE);
tlk_gpio_irq_add_callback(GPIO_PORT_A, &my_callback);
tlk_core_interrupt_enable();
```

CHAPTER 36

I2C 驱动

备注

Document Status: DRAFT —Content is being finalized

36.1 概述

I2C 驱动支持主/从模式通信，提供阻塞和 DMA 传输模式。

36.2 API 参考

```
// I2C initialization
void tlk_i2c_configure(enum tlk_i2c_module i2c_num, uint32_t speed);

// I2C read/write
enum tlk_i2c_status tlk_i2c_write(enum tlk_i2c_module i2c_num, ...);
enum tlk_i2c_status tlk_i2c_read(enum tlk_i2c_module i2c_num, ...);
```

36.3 示例

请参见*I2C 示例*。

CHAPTER 37

SPI 驱动

备注

Document Status: DRAFT —Content is being finalized

37.1 概述

SPI 驱动支持主/从模式通信，提供阻塞和 DMA 传输模式。

37.2 API 参考

```
// SPI initialization
void tlk_spi_configure(enum tlk_spi_module spi_num, uint32_t speed);

// SPI read/write
enum tlk_spi_status tlk_spi_write(enum tlk_spi_module spi_num, ...);
enum tlk_spi_status tlk_spi_read(enum tlk_spi_module spi_num, ...);
```

37.3 示例

请参见*SPI 示例*。

CHAPTER 38

ADC 驱动

备注

Document Status: DRAFT —Content is being finalized

38.1 概述

ADC 驱动提供模数转换功能。

38.2 示例

请参见*ADC 示例*。

CHAPTER 39

DMA 驱动

备注

Document Status: DRAFT —Content is being finalized

39.1 概述

DMA 驱动提供直接内存访问功能，支持从外设到内存以及从内存到外设的高效数据传输。

39.2 示例

请参见*DMA 示例*。

CHAPTER 40

看门狗驱动

备注

Document Status: DRAFT —Content is being finalized

40.1 概述

看门狗驱动提供看门狗定时器功能，用于系统错误恢复。

40.2 示例

请参见看门狗示例。

CHAPTER 41

定时器驱动

备注

Document Status: DRAFT —Content is being finalized

41.1 概述

定时器驱动包括 STIMER（系统定时器）和 MTIMER（机器定时器），提供高精度计时和调度功能。

41.2 示例

请参见定时器示例。

CHAPTER 42

USB 驱动

备注

Document Status: DRAFT —Content is being finalized

42.1 概述

USB 驱动基于 TinyUSB 框架，提供 USB CDC 和其他设备类支持。

42.2 示例

请参见*USB CDC 示例*。

CHAPTER 43

PMP 驱动

提供用于配置 RISC-V 内核上物理内存保护（PMP）单元的 API。PMP 允许 M 模式软件限制每个已定义区域的物理内存访问权限（读、写、执行），在硬件上对 U 模式强制执行，并且在锁定时可选择对 M 模式强制执行。

43.1 概述

PMP 单元支持最多 **8 个条目**。每个条目定义一个受保护的内存区域和一组访问权限。

任何违规都会在处理器处精确捕获。可以通过 `mcause`、`mdcause` 和 `mtval` CSR 检查异常原因。

43.2 地址匹配模式

每个条目在以下两种地址匹配模式之一中运行：

模式	API	描述
TOR	`tlk_pmp_tor_c`	范围顶部。保护区域为 `[pmpaddr[entry-1], address)`。对于条目 0，下限隐式为 `0x00000000`。
NAPOT	`tlk_pmp_napot`	自然对齐的 2 的幂。区域为 `[address, address + size)`。基地址必须与 `size` 对齐；`size` 必须是 2 的幂，最小为 8 字节。

要使用带有显式下限的 TOR，请使用所需的下限地址在 `entry - 1` 上调用 `tlk_pmp_entry_disable`。这会将地址写入 `pmpaddr[entry-1]`，而不会在该条目上激活保护。

43.3 权限

每个条目的权限通过 `struct tlk_pmp_config` 定义：

字段	描述
r	允许读取访问。
w	允许写入访问。
x	允许指令执行。
l	锁定条目。请参见下面的锁定。

43.4 锁定

当在条目上设置 `l = 1` 时：

- 对 `PMPiCFG` 和 `PMPADDRi` 的写入在系统复位前将被硬件**忽略**。
- 对于 TOR 条目，`pmpaddr[entry-1]` 也会被冻结。
- 权限适用于**所有特权模式**，包括 M 模式。

  锁定的条目只能通过系统复位清除。仅在保护必须承受任何软件故障（包括 M 模式代码中的错误）时设置 `l = 1`。

43.5 API

```
void tlk_pmp_tor_config(enum tlk_pmp_entry entry, void *address,
                        struct tlk_pmp_config *config);
```

在 TOR 模式下配置 `entry`。`address` 是保护区域的独占上限。

```
void tlk_pmp_napot_config(enum tlk_pmp_entry entry, void *address, uint32_t size,
                          struct tlk_pmp_config *config);
```

在 NAPOT 模式下配置 `entry`。`address` 必须与 `size` 对齐。`size` 必须是 2 的幂且至少为 8 字节。

```
void tlk_pmp_entry_disable(enum tlk_pmp_entry entry, void *address);
```

将条目设置为 OFF 模式，移除任何活动的保护。`address` 的值保留在 `pmpaddr[entry]` 中，可作为后续 TOR 条目的下限。如果地址不相关，请传递 `NULL`。如果条目已锁定（`l = 1`），则无效果。

43.6 配置

所有选项均在 Kconfig 的 `PMP` 下可用。

符号	默认值	描述
TLK_PMP_PM_DEVIC	n	启用电源管理支持。启用后，所有 PMP 条目寄存器在休眠前保存，在唤醒后恢复。

CHAPTER 44

连接性

44.1 概述

UniSDK 集成了 BLE（低功耗蓝牙）控制器（链路层），为 BLE 应用开发提供基础的无线连接能力。

44.2 代码位置

BLE 控制器目录

```
subsystem/ble/controller/
├── acl/                    # ACL Link Layer
├── CMakeLists.txt
├── Kconfig
├── README.md               # Coding standards
└── README_zh.md
```

44.3 编码规范

BLE 控制器遵循严格的编码规范，以确保性能和可靠性：

44.3.1 文件命名

- 源文件使用小写字母和下划线，例如 `tlk_ble_ll_acl.c`

44.3.2 函数命名

- 公共函数使用 `tlk_ble_` 前缀
- 内部函数使用 `_` 前缀

44.3.3 中断函数

- 中断处理函数有特定的命名和修饰符要求
- 中断函数应尽可能简短，避免复杂操作

44.4 Kconfig 配置

```
# BLE related configuration options (configured interactively via menuconfig)
```

44.5 示例

- *BLE 示例* —BLE 广播和连接演示

44.6 规划中

以下协议栈功能正在规划中：

- BLE 主机层（GAP、GATT、SM）
- Zigbee
- Thread
- Matter

  如需了解更多详情，请联系 Telink 团队获取最新信息。

CHAPTER 45

BLE 控制器编码规范

45.1 概述

本文档描述了 BLE 控制器 ROM 的编码规范，适用于 `subsystem/ble/controller/` 下的所有代码。

45.2 文件命名

- 源文件使用小写字母，以下划线分隔
- 示例：`tlk_ble_ll_acl.c`

45.3 函数命名

45.3.1 公共函数

使用 `tlk_ble_` 前缀：

```
void tlk_ble_ll_init(void);
int  tlk_ble_ll_send_data(uint8_t *data, uint16_t len);
```

45.3.2 内部函数

使用 `_` 前缀标记模块内部函数：

```
static void _internal_process(void);
```

45.4 变量命名

- 全局变量使用 `g_` 前缀
- 静态变量使用 `s_` 前缀
- 局部变量使用小写加下划线风格

45.5 中断函数

中断处理函数应满足以下要求:

- 名称必须包含 `_irq_` 或 `_isr_` 标识符
- 使用特定的修饰符以确保正确的调用约定
- 避免在中断函数中进行复杂操作和阻塞调用
- 尽可能使用标志位加主循环处理的模式

CHAPTER 46

系统服务

备注

Document Status: PLANNED —Content for this module has not yet been written

系统服务模块将涵盖 UniSDK 提供的高级系统服务组件，包括:

- 任务调度和事件处理
- 内存管理
- 日志系统（Debug Print）
- 系统初始化框架

46.1 当前可用组件

组件	API	描述
系统初始化	`tlk_init.h`	模块化初始化框架
主循环	`tlk_loop.h`	事件驱动的主循环
延时/睡眠	`tlk_sleep.h`	`tlk_sleep_ms()` / `tlk_sleep_us()`
时间管理	`tlk_time.h`	时间戳和定时器

详细文档将在后续添加。欢迎提交 PR 贡献。

CHAPTER 47

构建与配置

UniSDK 采用模块化、可扩展的构建系统架构，集成了 CMake、Ninja、West、Kconfig 和 Make 等工具，形成了完整的编译构建流程。

47.1 核心组件

组件	用途	描述
CMake	构建系统核心	项目配置、依赖管理、构建规则生成
Ninja	构建执行引擎	高性能并行编译，CMake 的默认后端
West	项目管理工具	统一命令行入口，简化构建工作流
Kconfig	配置管理系统	分层管理编译时配置选项
Make	传统接口	兼容传统 Make 习惯的前端接口

47.2 三种构建方式

UniSDK 支持三种构建入口，适应不同场景：

```
# Method 1: West command (recommended, simplest)
west tl-build samples/gpio_demo --board TLSR9528A_EVK

# Method 2: Make command (compatible with traditional conventions)
make build APP=samples/gpio_demo BOARD=TLSR9528A_EVK

# Method 3: Direct CMake invocation (most flexible)
cmake -B build -G Ninja -S samples/gpio_demo -DBOARD=TLSR9528A_EVK
cmake --build build
```

47.3 文档导航

章节	描述	状态
CMake 构建系统	CMake 核心流程、模块加载、配置阶段详解	稳定
Kconfig 配置系统	配置层次结构、处理流程、关键脚本	稳定
West 命令	tl-build / tl-config / tl-bdt 等命令详解	稳定
Make 支持	Makefile 前端接口、目标和变量说明	稳定
构建流程	环境验证、配置生成、编译链接完整流程	稳定
扩展与定制	添加新模块、扩展 West 命令、Kconfig 定制	稳定
文件与脚本参考	配置文件、CMake 模块、脚本清单	稳定

47.4 构建系统架构

CHAPTER 48

CMake 构建系统

CMake 是 UniSDK 构建系统的核心，负责项目配置、依赖解析和构建规则生成。

48.1 入口模式

UniSDK 支持两种 CMake 入口模式：

模式	入口	用途	命令示例
根入口	CMakeLists.txt（仓库根目录）	SDK 开发/测试	`cmake -B build -G Ninja .`
应用入口	应用的 CMakeLists.txt	构建特定应用	`cmake -B build -G Ninja samples/gpio_demo`

48.1.1 应用级 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})
project(GPIO_Demo LANGUAGES C CXX)

file(GLOB C_SOURCES "*.c")
target_sources(Telink PRIVATE ${C_SOURCES})

if(CONFIG_TLK_ALLOW_CPP_WRAPPERS)
    file(GLOB CPP_SOURCES "*.cpp")
    target_sources(Telink PRIVATE ${CPP_SOURCES})
endif()
```

48.2 模块加载顺序

`telink_default.cmake` 按以下顺序加载 CMake 模块:

1. **`python.cmake`** —Python 环境检测（Python 3.10+）
2. **`extensions.cmake`** —CMake 扩展函数（条件变量、JSON 处理）
3. **`version.cmake`** —版本信息处理
4. **`west.cmake`** —West 工具集成（版本 >= 0.14.0）
5. **`compiler.cmake`** —RISC-V 编译器配置
6. **`kconfig.cmake`** —Kconfig 配置集成
7. **`pinmux.cmake`** —引脚复用配置生成
8. **`linker.cmake`** —链接脚本处理

48.3 核心模块详情

48.3.1 compiler.cmake

- 检测 `TELINK_TOOLCHAIN_PATH` 环境变量
- 配置 RISC-V GCC 交叉编译工具链
- 提供 `telink_apply_compiler_options()` 函数
- 从 `build_settings.json` 动态加载编译选项

48.3.2 linker.cmake

- 设置 RISC-V 链接命令模板
- 从 `build_settings.json` 读取链接器选项
- 预处理链接脚本（`flash_boot.link` → `flash_boot.ld`）
- 后处理: 生成 `.bin` + `.map` + 固件验证

48.3.3 kconfig.cmake

- 管理 `chip.config` → `build.config` → `.config` 的生成流程
- 生成 `capabilities.h` 和 `autoconf.h`
- 定义 CMake 目标，如 `config` / `parse_config`
- SOC/BOARD 参数处理与组合验证

48.3.4 pinmux.cmake

- 生成 `.pinmux` 和 `pinmux.h`
- 依赖于 `.config` 文件
- 构建目录会自动添加到包含路径中

48.3.5 extensions.cmake

```
# Conditional variable setting
set_ifndef(variable value [PARENT_SCOPE])

# Conditional source file addition
add_sources_ifdef(config src)

# JSON array to CMake list
json_to_list(JSON_ARRAY_STRING OUTPUT_LIST)
```

48.4 配置流程

48.5 包含保护机制

保护标志	设置位置	用途
TELINK_ROOT_INCLUDED	根 CMakeLists.txt	防止重复包含根目录
TELINK_PACKAGE_INCLUDED	TelinkConfig.cmake	防止重复加载包

48.6 构建目录结构

构建目录结构

```
build/
├── .config              # Merged final configuration
├── chip.config          # Chip configuration
├── build.config         # Build configuration
├── autoconf.h           # C configuration macros
├── capabilities.h       # Chip capability declarations
├── pinmux.h             # Pin multiplexing configuration
├── flash_boot.ld        # Linker script
├── build.ninja          # Ninja build file
├── CMakeCache.txt       # CMake cache
└── Kconfig/             # Chip-specific Kconfig fragments
```

CHAPTER 49

Kconfig 配置系统

Kconfig 是 UniSDK 的配置管理核心，采用分层结构来管理编译时配置选项。

49.1 架构

49.1.1 配置入口

UniSDK 采用双入口 Kconfig 设计：

```
Kconfig.chip     → Chip-related configuration (SoC selection, peripheral enabling)
Kconfig.build    → Build-related configuration (feature options, parameter settings)
```

49.1.2 层次结构

49.1.3 应用层 Kconfig 集成

通过 KCONFIG_APPLICATION_DIR 变量，应用层可以定义自己的 Kconfig 选项，与 SDK 配置无缝集成。

49.2 配置生成流程

49.2.1 关键配置文件

文件	来源	描述
chip.config	Kconfig.chip	芯片相关配置（SoC、外设使能）
build.config	Kconfig.build	构建选项配置
.config	chip.config + build.config	最终合并配置
capabilities.h	chip.config → 预处理	芯片能力声明头文件
autoconf.h	.config → 解析	C 预处理器宏定义

49.2.2 SOC/BOARD 参数处理

```
# Specify SOC during build
cmake -B build -DSOC=TLSR9528A -S .
# Specify BOARD during build
cmake -B build -DBOARD=TLSR9528A_EVK -S .

# West command (specify SOC)
west tl-build --soc TLSR9528A

# West command (specify BOARD)
west tl-build --board TLSR9528A_EVK
```

SOC/BOARD passed this way only take effect the **first time** `chip.config` is generated for a build directory (as of UniSDK v0.3.0) —CMake seeds the choice into the non-interactive Kconfig pass so it comes out selected, instead of requiring the interactive `menuconfig` UI. If `chip.config` already exists (e.g. a build directory reused across configure runs), these flags are ignored; delete `chip.config` or edit it directly to change SOC/BOARD.

`kconfig_parser.py`/`soc_board_setter.py` provide standalone SOC/BOARD dependency resolution and combination validation, but are not currently invoked automatically as part of this flow.

49.3 关键脚本

脚本	用途
`kconfig.py`	非交互式 Kconfig 处理、配置合并、有效性验证
`menuconfig.py`	交互式终端配置界面
`preprocess_kconfig.py`	配置预处理、YAML 变量展开
`kconfig_parser.py`	Kconfig 解析与 SOC/BOARD 验证
`soc_board_setter.py`	SOC/BOARD 配置与依赖解析

49.4 Menuconfig 触发方式

1. **构建时自动触发**—配置文件缺失时自动打开交互界面
2. **手动触发**—`python scripts/menuconfig.py Kconfig.chip`
3. **非交互式生成**—CMake 配置阶段使用 `kconfig.py` 自动生成默认值

49.5 配置选项示例

```
# core/configs/Kconfig.gpio

config TLK_GPIO_PORT_A_ENABLED
    bool "Enable GPIO Port A"
    default y
    help
        Enable GPIO Port A instance.

config TLK_GPIO_PREVENT_SLEEP
```

(续下页)

(接上页)

```
    bool "Prevent sleep when GPIO is active"
    depends on TLK_PM
    default y
```

CHAPTER 50

West 命令

West 是 UniSDK 的统一命令行入口。UniSDK 通过 `scripts/west-commands.yml` 注册了多个自定义命令扩展。

50.1 命令注册

```yaml
west-commands:
  - file: scripts/west_commands/build.py
    commands:
      - name: tl-build
        class: Build
        help: compile a Telink application
  - file: scripts/west_commands/boards.py
    commands:
      - name: tl-boards
        class: Boards
        help: display information about supported Telink boards
  - file: scripts/west_commands/socs.py
    commands:
      - name: tl-socs
        class: Socs
        help: display information about supported Telink socs
  - file: scripts/west_commands/config.py
    commands:
      - name: tl-config
        class: Config
        help: configure a Telink application
  - file: scripts/west_commands/bdt.py
    commands:
      - name: tl-bdt
        class: Bdt
        help: use BDT tool to configure and flash a Telink chip
```

50.2 west tl-build —构建

构建 Telink 应用。构建目录默认为当前工作目录下的 build/。

```
# Basic usage
west tl-build [source_dir] [-d build_dir] [options]

# Specify source directory
west tl-build samples/gpio_demo

# Specify build directory
west tl-build -d build_tl3218

# Specify SOC during build
west tl-build --soc TLSR9528A

# Specify BOARD during build
west tl-build --board TLSR9528A_EVK

# Build after interactive configuration
west tl-build --kconfig

# Clean build
west tl-build --clean
west tl-build --pristine always

# Build and flash
west tl-build --flash

# Build specific target
west tl-build -t flash

# CMake configuration only, no compilation
west tl-build --cmake-only

# Pass additional CMake options
west tl-build -- -DCONFIG_DEBUG=y
```

50.2.1 执行流程

1. 检查 TELINK_TOOLCHAIN_PATH 环境变量
2. 验证 SOC/BOARD 组合
3. 设置源代码和构建目录
4. 如需则运行 Kconfig 配置
5. 调用 CMake 生成构建系统
6. 执行编译和链接
7. 输出构建结果

50.3 west tl-config —配置

独立配置 Telink 应用。

```
# Basic usage
west tl-config [source_dir] [-d build_dir]

# CMake-only configuration (skip interactive GUI)
west tl-config -c

# List available SOCs
west tl-config --list-socs

# List available boards
west tl-config --list-boards

# Validate SOC/BOARD combination
west tl-config --validate TLSR9528A TLSR9528A_EVK
```

50.4 west tl-bdt —烧录与调试

BDT（烧录与调试工具）命令接口。

50.4.1 子命令

子命令	描述
download	下载固件到 Flash
read	读取 Flash 数据
write	写入 Flash 数据
erase	擦除 Flash
lock	锁定 Flash 区域
unlock	解锁 Flash
reset	复位设备

50.4.2 常见示例

```
# Download firmware
west tl-bdt download --chip TLSR9528A -i build/Telink.bin

# Read 16 bytes from Flash
west tl-bdt read --chip TLSR9528A -a 0x00 -s 16

# Erase Flash
west tl-bdt erase --chip TLSR9528A -a 0x00 -s 4k

# Lock Flash
west tl-bdt lock --chip TLSR9528A -a 0x00 -s 512k

# Reset device
west tl-bdt reset --chip TLSR9528A
```

(续下页)

(接上页)

```
# Specify BDT tool path
west tl-bdt download --bdt-path /path/to/BDT --chip TLSR9528A -i build/firmware.bin
```

50.4.3 BDT 路径配置（优先级从高到低）

1. 命令行：`--bdt-path /path/to/bdt`
2. 环境变量：`BDT_PATH=/path/to/bdt`
3. 系统 PATH
4. SDK 目录：`${TELINK_BASE}/tools/`

50.4.4 跨平台

平台	可执行文件
Windows	`Cmd_download_tool.exe`
Linux	`bdt`

50.5 west tl-boards —板卡信息

```
west tl-boards              # List all supported boards
west tl-boards -v           # Show detailed information
west tl-boards --soc TLSR9528A  # Filter by chip
```

50.6 west tl-socs —芯片信息

```
west tl-socs                # List all supported chips
west tl-socs -v             # Show detailed information
```

CHAPTER 51

Make 支持

UniSDK 的 `Makefile` 提供了传统的 make 命令接口，作为 CMake 的前端来简化构建命令。**只能在项目根目录下使用**。

51.1 Makefile 目标

目标	描述	命令
`cmake`	运行 CMake 配置	`make cmake`
`config`	生成项目配置	`make config`
`pinmux`	运行 Pinmux 配置工具	`make pinmux`
`build`	完整构建（cmake + config + build）	`make build`
`clean`	清理构建产物	`make clean`
`cleanbuild`	清理后重新构建	`make cleanbuild`

51.2 Makefile 变量

变量	默认值	描述
`APP`	`samples/gpio_demo`	应用路径
`SOC`	（空）	目标芯片
`BOARD`	（空）	目标开发板
`BUILD_DIR`	`build`	构建目录

51.3 使用示例

```
# Default build
make build

# Specify application
make cmake build APP=samples/gpio_demo

# Specify chip and board
make cmake build SOC=TLSR9528A BOARD=TLSR9528A_EVK

# Specify application, chip, and board simultaneously
make cmake build APP=samples/gpio_demo SOC=TLSR9528A BOARD=TLSR9528A_EVK

# Clean and rebuild
make cleanbuild

# Configure only
make config
make pinmux
```

51.4 构建流程（make build）

1. **cmake** —为指定应用初始化 CMake 构建系统
2. **config** —生成配置文件（.config 等）
3. **预构建检查**—4 项自动检查和恢复步骤:
   - 检查构建目录是否存在
   - 检查 .config 文件是否存在
   - 检查 pinmux.h 是否存在
   - 检查 pinmux.h 是否是最新的
4. **cmake --build** —执行 Ninja 编译

51.5 SOC/BOARD 参数转发

Makefile 自动将 SOC 和 BOARD 变量传递给 CMake:

```
ifneq ($(SOC),)
CMAKE_FLAGS += -DSOC=$(SOC)
endif
ifneq ($(BOARD),)
CMAKE_FLAGS += -DBOARD=$(BOARD)
endif
```

CHAPTER 52

构建流程

52.1 概述

本文档详细介绍了从配置到固件生成的完整构建流程。

52.2 完整构建流程

52.3 入门指南

52.3.1 安装依赖

您需要使用包管理器安装一些主机依赖。

Ubuntu

使用 apt 安装所需依赖：

```
sudo apt install --no-install-recommends git cmake ninja-build python3-dev python3-
↪venv python3-tk make
```

验证版本：

```
cmake --version
python3 --version
git --version
ninja --version
```

如果显示"command not found"，则表示相应工具未安装。

MacOS

使用 Homebrew 安装所需依赖:

```
brew install cmake ninja python3 python-tk
```

验证版本:

```
cmake --version
python3 --version
git --version
ninja --version
```

Windows

您可以通过以下两种方式安装:

1. 对于支持 winget 的系统:

```
winget install Kitware.CMake Ninja-build.Ninja Python Git.Git
```

2. 对于不支持 winget 的系统，请从官方网站下载并安装:
   - CMake（勾选"Add CMake to the PATH"）
   - Ninja（手动添加到 PATH）
   - Python（勾选"Add Python.exe to PATH"）
   - Git

安装后验证:

```
cmake --version
python --version
git --version
ninja --version
```

52.3.2 安装 Python 依赖

Python 虚拟环境

```
# Create virtual environment
python -m venv .venv          # Windows
python3 -m venv .venv         # Linux/MacOS
```

```
# Activate virtual environment
source .venv/bin/activate           # Linux/Mac
.venv\Scripts\activate.bat          # Windows Batchfile
.venv\Scripts\activate.ps1          # Windows PowerShell
```

使用 PowerShell 时，可能会遇到:

```
Activate.ps1 cannot be loaded because running scripts is disabled on this system.
```

解决方法:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

然后安装 Python 依赖:

```
pip install -r requirements.txt
```

West 工具设置

```
# Verify west installation
west --version

# Initialize west workspace
west init -l
```

west init -l 的潜在问题:

如果遇到错误 `already initialized in {where}, aborting.`，这很可能是由于 `ZEPHYR_BASE` 环境变量指向了已存在 `.west` 目录的 Zephyr 工作空间。

解决方法: 临时取消设置 `ZEPHYR_BASE`:

- **Windows (PowerShell):**

  ```
  Remove-Item Env:ZEPHYR_BASE; west init -l
  ```

- **Linux/macOS:**

  ```
  unset ZEPHYR_BASE; west init -l
  ```

工具链配置

下载并解压工具链（仅 Telink 可用）: Toolchain_V5.4.1

设置 `TELINK_TOOLCHAIN_PATH` 环境变量，并确保工具链可执行文件可在 PATH 中访问。

52.4 六个阶段

52.4.1 1. Environment Validation

- 验证 Python 虚拟环境已激活
- 验证 West 可用且版本 >= 0.14.0
- 验证 `TELINK_TOOLCHAIN_PATH` 已设置
- 验证工具链可执行文件可访问

52.4.2 2. CMake Configuration

```
cmake -B build -G Ninja -S samples/gpio_demo
```

加载并执行所有 CMake 模块（python -> extensions -> west -> compiler -> kconfig -> pinmux -> linker）。

52.4.3 3. Configuration Generation (Config Target)

```
cmake --build build --target config
```

按顺序执行:

1. 清理旧配置 -> 2. 生成 `chip.config` -> 3. 生成 `capabilities.h` -> 4. 生成 `build.config` -> 5. 合并 `.config` -> 6. 生成 pinmux 配置

52.4.4 4. Source File Compilation

Ninja 根据 `build.ninja` 规则并发编译所有源文件。

52.4.5 5. Linking

将目标文件链接为 ELF 可执行文件，输出 `.elf`、`.bin`、`.map`。

52.4.6 6. Firmware Generation

- 使用 `objcopy` 将 ELF 转换为二进制 `.bin` 文件
- 使用 `objdump` 生成反汇编代码用于调试
- 运行 `tl_check_fw.sh` 验证固件

52.5 配置命令

52.5.1 基本配置

```
west tl-config
```

此命令使用默认设置配置当前目录中的应用。

流程:

1. 设置环境变量（TELINK_BASE）
2. 按需创建构建目录
3. 按需运行初始 CMake 配置
4. 执行统一的 `config` 目标，该目标:
    - 从 `Kconfig.chip` 生成 `chip.config`
    - 从芯片配置生成 `capabilities.h`
    - 从 `Kconfig.build` 生成 `build.config`
    - 将配置合并为最终的 `.config`
    - 生成 pinmux 配置文件

52.5.2 常见配置示例

```
# Specify build directory
west tl-config -d build_custom

# Using different source directory
west tl-config path/to/application -d build_custom

# Configure and build in one command
west tl-build -k
```

52.6 基本构建命令

```
# Simple build (current directory)
west tl-build
```

此命令构建当前目录中的应用。如果是首次构建，将自动进行配置和构建。

预构建检查：

- 检查是否存在 `.config`、`build.config`、`chip.config`、`pinmux.h` 和 `capabilities.h` 文件
- 调用 CMake 生成构建系统
- 使用选定的工具链编译源代码
- 在 `build/` 目录中生成固件二进制文件

52.6.1 常见构建命令示例

```
# Specify build directory
west tl-build -d build_tl3218

# Specify source directory
west tl-build path/to/application

# Run CMake after configuration
west tl-build -c

# Clean build directory and rebuild
west tl-build --clean

# Pristine build
west tl-build --pristine=always
```

52.7 配置文件生成逻辑

52.7.1 文件层次结构

配置文件按优先级递增的顺序加载：

1. **chip.config** -- 芯片默认配置（最低优先级）
2. **build.config** -- 用户定义的构建选项
3. **.config** -- 最终合并配置（最高优先级）

52.7.2 配置生成流程

执行 `west tl-config` 时，流程如下：

1. **参数处理阶段**：解析源目录和构建目录选项
2. **CMake 配置阶段**：检查现有的 `CMakeCache.txt`，使用 Ninja 生成器运行 CMake 配置
3. **配置目标执行**：执行 `cmake --build <build_dir> --target config`

执行 `west tl-build`（带 `-k/--kconfig`）时，流程如下：

1. **参数处理**：检查 `--board` 或 `--soc` 参数
2. **默认配置生成**：运行 `scripts/kconfig.py` 生成 `chip.config` 和 `build.config`，合并为 `.config`
3. **CMake 配置**：将 `BOARD` 和 `SOC` 参数传递给 CMake

52.7.3 依赖关系

- `.config` 依赖于 `chip.config` 和 `build.config`（自动生成时）
- CMake 构建系统依赖 `.config` 来确定编译选项
- CMake 监视 `.config` 的变化，并在文件被修改时重新配置项目

52.8 构建结果

构建结果

```
build/
├── Telink.bin          # Firmware binary (for flashing)
├── Telink.elf          # ELF executable (for debugging)
├── Telink.map          # Memory map file
└── Telink.lst          # Disassembly file
```

52.9 不使用 West 构建

本节说明如何在不使用 west 工具的情况下进行构建。

52.9.1 CMake 配置

```
cmake -B path/to/build -G Ninja -DTelink_DIR='path/to/telink-package' -S path/to/
↪source
```

参数：

- **`-B path/to/build`**：指定构建目录
- **`-G Ninja`**：指定 Ninja 作为生成器
- **`-DTelink_DIR='path/to/telink-package'`**：指向 TelinkConfig.cmake 文件（如果已设置 `TELINK_BASE` 则无需指定）
- **`-S path/to/source`**：指定源代码目录

52.9.2 CMake 构建

```
# Build the project
cmake --build path/to/build

# Or use Ninja directly
ninja -C path/to/build

# Clean the build directory
cmake --build path/to/build --target clean
```

52.10 故障排除

52.10.1 环境问题

问题：未找到 TELINK_TOOLCHAIN_PATH

```
# Windows
set TELINK_TOOLCHAIN_PATH=path/to/toolchain           # CMD
$env:TELINK_TOOLCHAIN_PATH=path/to/toolchain           # PowerShell

# Linux/macOS
export TELINK_TOOLCHAIN_PATH=path/to/toolchain
```

问题：CMake 配置失败

- 确保所有依赖已安装
- 检查 CMake 版本兼容性
- 使用 `--clean` 清除旧的构建缓存

52.10.2 构建问题

问题：编译错误

- 检查应用代码是否存在语法错误
- 确保使用了正确的板卡和芯片配置
- 尝试使用 `--pristine=always` 进行干净构建

52.10.3 menuconfig 问题

问题：VSCode PowerShell 中箭头图标无法使用

- 回滚到较早的 VSCode 版本或使用测试版 ConPty 库
- 使用 Windows 11
- 使用字母键代替箭头图标
- 使用命令提示符代替 PowerShell

52.10.4 环境变量问题

问题：环境变量未生效

- 重启终端、VS Code 或系统
- 手动检查 PATH 是否包含正确的路径并添加到用户 PATH 中

52.10.5 其他问题

1. 确保您使用的是最新版本的 UniSDK
2. 清除构建目录并重新构建:

```
west tl-build --clean
```

3. 检查系统日志以获取更详细的错误信息

CHAPTER 53

UniSDK 构建系统架构文档

53.1 1. Overall Architecture Overview

UniSDK 构建系统采用模块化、可扩展的架构设计，集成了 Make、CMake、Ninja、West 和 Kconfig 等多种工具，形成了完整的编译构建工作流。系统支持多种编译方式，包括 West 命令、直接 CMake 调用和 Make 命令，为开发者提供灵活的构建选项。

53.1.1 1.1 核心组件

- **CMake**：主构建系统，负责项目配置、依赖管理和构建流程定义
- **Ninja**：高性能构建工具，作为 CMake 的后端执行实际的编译任务
- **West**：Zephyr 项目管理工具，在 UniSDK 中扩展用于管理项目和简化构建工作流
- **Kconfig**：配置管理系统，用于处理编译时配置选项
- **Make**：提供传统的 make 命令接口，作为 CMake 的前端

53.1.2 1.2 目录结构

构建系统相关文件主要分布在以下目录:

SDK 目录结构（构建系统）

```
unisdk/
├── CMakeLists.txt         # Main CMake configuration file
├── Makefile               # Make command interface
├── west.yml               # West workspace configuration
├── Kconfig.chip           # Chip Kconfig entry
├── Kconfig.build          # Build Kconfig entry
├── cmake/                 # CMake modules
│   ├── modules/
```

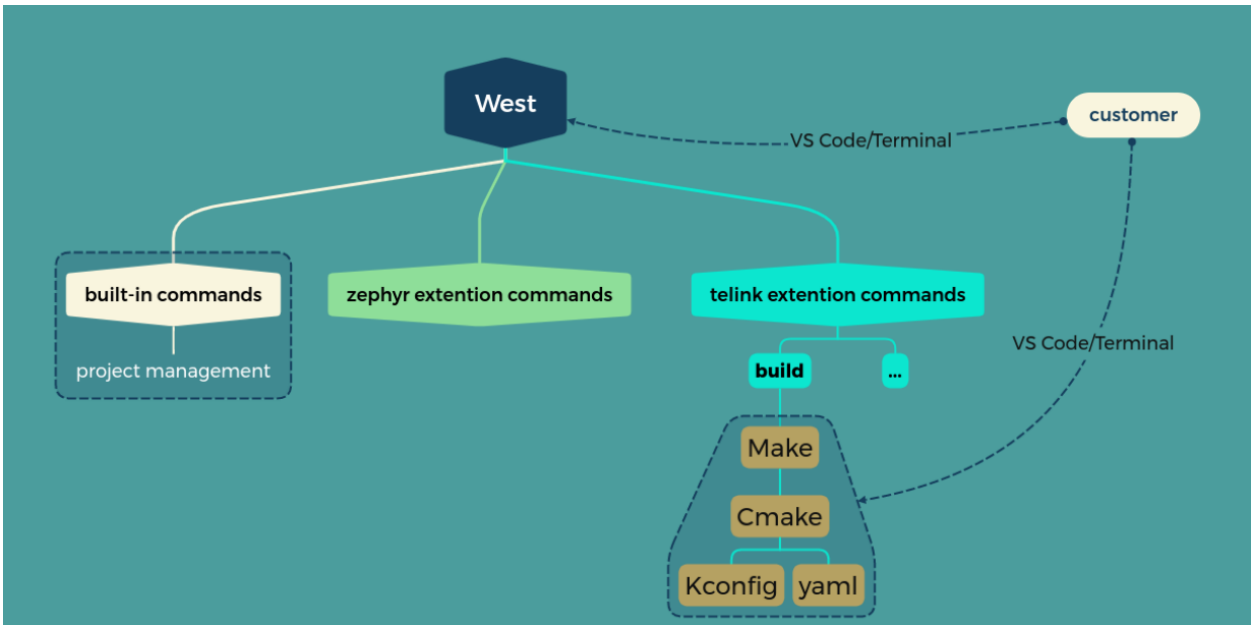
(续下页)

(接上页)

```
│   └── TelinkConfig.cmake
├── scripts/                # Build scripts
│   ├── ci/
│   ├── pinmux/
│   ├── pre_build/
│   ├── west_commands/
│   ├── kconfig.py
│   ├── menuconfig.py
│   ├── preprocess_kconfig.py
│   ├── set_telink_base.sh
│   ├── tlk_check_fw.py
│   ├── west-commands.yml
│   └── windows_setup.ps1
├── core/
├── soc/
├── boards/
└── docs/
```

53.1.3 1.3 构建系统

UniSDK 构建系统是一个综合框架，结合了多种工具以提供无缝的开发体验。下图说明了整体架构和工具链关系：



构建系统围绕以下关键组件设计：

- **West**：面向开发者的命令行接口，为各种构建操作提供统一入口
- **Make**：处理高级构建编排和预构建检查
- **CMake**：核心构建系统，负责项目配置、依赖管理和构建规则生成
- **Kconfig**：配置管理系统，用于处理编译时选项
- **YAML**：用于各种系统配置（包括引脚复用）的配置格式

该架构实现了灵活且模块化的构建过程，使开发者能够轻松配置、构建和管理项目，同时在不同目标平台之间保持一致性。

53.2 2. CMake Build System

53.2.1 2.1 CMake 核心流程

CMake 构建系统是 UniSDK 编译过程的核心，负责项目配置、依赖解析和构建规则生成。

2.1.1 初始化阶段

1. **入口**：构建系统支持**两种入口模式**：
   - **根目录入口**：使用根目录的 `CMakeLists.txt` 作为入口来构建整个 SDK
   - **应用入口**：使用应用级 `CMakeLists.txt`（通常位于 `samples/` 目录中）作为入口来构建特定应用
2. **版本要求**：设置最低 CMake 版本（3.20.0）
3. **SDK 集成**：通过 `find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})` 定位并加载 Telink 包配置
4. **项目设置**：配置项目特定设置，包括项目名称、支持的语言和目标创建

关键包含保护：构建系统使用两个重要的包含保护来管理包含并防止冗余处理：

1. **TELINK_ROOT_INCLUDED**：
   - 在根目录的 `CMakeLists.txt` 中设置为 `TRUE`
   - 由 `telink_default.cmake` 用于判断根目录是否已经被包含
   - 如果未定义，`telink_default.cmake` 将包含根目录的 `CMakeLists.txt`
   - 在未来的版本中，当应用级 CMakeLists.txt 作为入口得到支持时，此机制将被移除。
2. **TELINK_PACKAGE_INCLUDED**：
   - 在加载 Telink 包时，在 `TelinkConfig.cmake` 中设置为 `TRUE`
   - 防止多次加载 Telink 包配置
   - 在根目录的 `CMakeLists.txt` 中进行检查，以避免冗余的包加载

入口模式对比：

模式	入口	用法	输出
根目录入口	`CMakeLists.txt`（根目录）	`cmake -B build -G Ninja .`	将整个 SDK 构建为库
应用入口	`CMakeLists.txt`（samples/）	`cmake -B build -G Ninja samples/gpio_demo`	构建特定应用的可执行文件

应用级入口实现： `samples/` 目录中的所有应用都必须提供自己的 `CMakeLists.txt` 文件，该文件在构建特定应用时作为构建过程的入口。此文件应：

1. 设置最低 CMake 版本
2. 定义项目名称和默认语言（C）
3. 使用通配符添加 C 源文件

4. 创建可执行目标
5. 如果启用了 CONFIG_TLK_ALLOW_CPP_WRAPPERS，则条件性地添加 C++ 支持

应用级 CMakeLists.txt 由构建系统（通过 Make 或 West 命令）自动处理，该系统负责处理与核心 SDK 组件的集成。

2.1.2 模块加载阶段

`telink_default.cmake` 按特定顺序加载所有必要的 CMake 模块：

```
list(APPEND telink_cmake_modules python)
list(APPEND telink_cmake_modules extensions)
list(APPEND telink_cmake_modules west)
list(APPEND telink_cmake_modules compiler)
list(APPEND telink_cmake_modules kconfig)
list(APPEND telink_cmake_modules pinmux)
list(APPEND telink_cmake_modules linker)
```

每个模块负责特定功能的配置和实现，共同构成完整的构建系统。

53.2.2 2.2 核心 CMake 模块

2.2.1 python.cmake

此模块来自 Zephyr 项目。

`python.cmake` 模块负责配置和验证 UniSDK 构建系统所需的 Python 环境。作为构建过程中加载的第一个模块，它确保所有依赖 Python 的组件都拥有一致且兼容的 Python 环境。

主要特性和功能：

1. **UTF-8 编码配置**
   - 在 Windows 系统上将 `PYTHONIOENCODING` 设置为”utf-8”
   - 确保 Python 脚本由 CMake 执行时输出编码一致
   - 防止 Python 未连接到终端时可能出现的编码问题
2. **Python 版本管理**
   - 定义所需的最低 Python 版本（3.10）
   - 实现了健壮的 Python 可执行文件发现机制
   - 优先使用与 West 相同的 Python 解释器（如果可用）
3. **Python 发现流程**
   - 首先检查现有的 `Python3_EXECUTABLE` 变量
   - 回退到使用 `WEST_PYTHON`（如果可用）
   - 在系统路径和虚拟环境中搜索”python” 和”python3” 可执行文件
   - 验证发现的 Python 版本满足最低要求
   - 明确拒绝 Python 2.x 安装
4. **与 CMake 的集成**
   - 使用 `find_package(Python3)` 完成 Python 配置
   - 设置旧版 `PYTHON_EXECUTABLE` 变量，以兼容期望此变量的脚本

- 使用 `include_guard(GLOBAL)` 防止多次包含

此模块确保整个构建过程中使用的所有 Python 脚本（如 Kconfig 处理、pinmux 生成和构建工具）都能访问兼容的 Python 解释器，在不同平台上提供一致的构建环境。

2.2.2 compiler.cmake

`compiler.cmake` 模块是 UniSDK 构建系统的核心模块之一，负责配置编译器工具链、编译选项和编译标志。此模块提供了完整的 RISC-V 编译器配置和管理机制。

主要特性包括：

1. **编译器环境检测和配置**
   - 自动检测 TELINK_TOOLCHAIN_PATH 环境变量设置
   - 支持跨平台路径处理
   - 验证 RISC-V GCC 编译器的存在和有效性
   - 在未找到编译器时提供清晰的错误信息和配置指导
2. **工具链组件设置**
   - 配置主要编译工具：C/C++ 编译器、汇编器
   - 设置辅助工具：objdump、objcopy 等
   - 统一配置 RISC-V 目标架构和处理器类型
   - 将系统名称设置为 Generic（用于裸机开发）
3. **编译选项应用函数**
   - 提供 `telink_apply_compiler_options()` 函数，将编译选项应用到目标
   - 自动应用预处理器宏定义（autoconf.h、capabilities.h、pinmux.h）
   - 支持从 JSON 配置文件动态加载编译选项
   - 根据不同的语言（ASM、C、C++）应用特定的编译标志

此模块通过统一的编译器配置机制确保整个项目在不同平台上的编译一致性，同时提供灵活的选项配置方法以支持项目定制需求。

2.2.3 linker.cmake

`linker.cmake` 模块负责配置 UniSDK 构建系统中的链接过程。它处理链接脚本、链接选项和构建后处理命令。以下是其主要功能：

2.2.3.1 基本链接器配置

- 为 RISC-V 交叉编译设置 CMAKE_C_LINK_EXECUTABLE 命令模板
- 配置详细的 makefile 输出以获得更好的构建可见性
- 禁用 RPATH 以确保不嵌入运行时路径依赖
- 移除默认库后缀以匹配嵌入式系统需求

2.2.3.2 链接器选项管理

- 从 build_settings.json 配置文件中读取链接器选项
- 将 JSON 定义的链接器选项转换为 CMake 兼容的列表格式
- 添加必要的链接器标志

build_settings.json 文件是 UniSDK 构建系统使用的中心配置文件，用于存储各种构建相关设置。它采用 JSON 格式，包含特定于平台的配置选项，包括：

```
{
    "linker_options": [
      "--gc-sections",
      "--print-memory-usage"
    ]
    "c_compile_options": [
        "-O2",
        "-Wall"
    ],
}
```

此文件提供了一种灵活的方式来配置不同平台和项目的构建设置，允许轻松定制而无需直接修改 CMake 脚本。

2.2.3.3 链接脚本预处理

- 使用 C 编译器实现链接脚本的预处理
- 支持在链接脚本中使用 capabilities.h 和 autoconf.h 宏进行条件编译
- 创建自定义目标 preprocess_linker，确保链接脚本始终是最新的
- 自动处理基础的 flash_boot.link 脚本，并将最终的链接脚本输出到构建目录

2.2.3.4 构建后处理

- 提供 telink_apply_linker_options(target) 函数，将链接器设置应用到特定目标
- 配置 objcopy 从 ELF 可执行文件生成二进制固件文件
- 设置 objdump 创建汇编清单用于调试和分析
- 添加大小报告以监控编译固件的内存使用情况
- 通过 tlk_check_fw.py 脚本集成固件验证

2.2.4 kconfig.cmake

Kconfig 集成模块负责在 CMake 构建过程中管理整个配置系统：

- **配置文件管理**：
  - 定义关键配置文件路径（.config、chip.config、build.config、autoconf.h、capabilities.h）
  - 设置构建目录属性以确保正确清理配置文件
  - 管理配置文件的重新生成和依赖关系
- **配置生成流程**：

  - 当未定义 SOC 和 BOARD 时，处理从 Kconfig.chip 自动生成 chip.config
  - 根据芯片配置使用预处理脚本生成 capabilities.h
  - 从 Kconfig.build 创建 build.config，依赖于 capabilities
  - 将 chip.config 和 build.config 合并为最终的 .config 文件
- **预处理器集成**:
  - 集成 Python 脚本（menuconfig.py、preprocess_kconfig.py、kconfig.py）
  - 解析 .config 文件以生成带有相应 C 定义的 autoconf.h
  - 处理配置中的不同值类型（布尔、字符串、数字）
- **自定义目标**:
  - **parse_config**：确保构建期间的配置解析
  - **config**：统一目标，按顺序执行整个配置流程
  - **generate_chip_config**：使用 menuconfig 或 kconfig.py 生成 chip.config
  - **generate_capabilities**：从芯片配置生成 capabilities.h
  - **generate_build_config**：使用 menuconfig 或 kconfig.py 生成 build.config
  - **update_dotconfig**：将 chip.config 和 build.config 合并为 .config
- **依赖跟踪**:
  - 设置显式目标依赖关系：generate_chip_config → generate_capabilities → generate_build_config → update_dotconfig
  - 为配置文件操作创建跨平台的 CMake 命令
  - 通过正确的依赖关系管理配置生成顺序

2.2.4.1 SOC 和 BOARD 参数处理

kconfig.cmake 模块包含全面的 SOC 和 BOARD 参数处理:

参数检测和处理:

```
# Process SOC and BOARD parameters if provided
if(DEFINED SOC)
  message(STATUS "SOC parameter detected: ${SOC}")
  set(SOC_CONFIG_ARG "SOC=${SOC}")
else()
  set(SOC_CONFIG_ARG "")
endif()

if(DEFINED BOARD)
  message(STATUS "BOARD parameter detected: ${BOARD}")
  set(BOARD_CONFIG_ARG "BOARD=${BOARD}")
else()
  set(BOARD_CONFIG_ARG "")
endif()
```

SOC/BOARD 组合验证:

- 使用动态 Kconfig 解析验证 SOC 和 BOARD 组合
- 从命令行参数中提取 SOC 和 BOARD 名称

- 调用带 `--validate` 选项的 `kconfig_parser.py` 来验证兼容性
- 为无效组合提供清晰的错误信息

自动配置应用：

- 使用 `soc_board_setter.py` 脚本应用 SOC 和 BOARD 配置
- 自动处理依赖解析（BOARD 依赖触发 SOC 配置）
- 支持单独的 SOC/BOARD 设置以及组合设置
- 与现有的配置生成流程集成

集成点：

- **命令行**：通过 `-DSOC=` 和 `-DBOARD=` CMake 标志传递参数
- **Make 集成**：Makefile 自动将 SOC 和 BOARD 变量传递给 CMake
- **West 集成**：West 命令可以通过命令行选项指定 SOC/BOARD
- **验证**：实时验证防止不兼容的硬件配置

2.2.5 west.cmake

此模块来自 Zephyr 项目。

`west.cmake` 模块负责将 West 构建系统工具与 UniSDK 构建基础设施集成。West 是基于 Zephyr 的项目的官方元工具，此模块确保在 UniSDK 环境中的正确配置和版本兼容性。

2.2.5.1 West 安装检测

- 通过尝试导入 west.version 模块来检查 West 安装情况
- 通过 WEST_PYTHON 环境变量的存在来验证安装状态
- 优雅地处理已安装和未安装 West 的场景
- 在需要 West 但未找到时提供清晰的错误信息和安装说明

2.2.5.2 Python 环境一致性

- 确保 West 使用的 Python 解释器与 CMake 使用的匹配
- 解析符号链接以识别实际的 Python 可执行文件路径
- 检测并报告 West 和 CMake 之间的 Python 解释器不匹配
- 在 Python 环境不同步时添加详细的诊断信息

2.2.5.3 版本兼容性验证

- 强制要求最低 West 版本（0.14.0）
- 将检测到的版本与最低要求进行比较
- 在版本要求不满足时提供升级说明
- 与 scripts/requirements-base.txt 中指定的要求保持版本一致性

2.2.5.4 West 命令配置

- 将 WEST 变量设置为调用 West 的命令前缀
- 配置 WEST 变量在内部缓存以便 IDE 集成
- 将命令格式化为 `${PYTHON_EXECUTABLE} -m west` 以确保正确的模块执行
- 输出有关检测到的 West 版本的状态信息

2.2.5.5 工作空间检测

- 使用 `west topdir` 命令确定 West 工作空间顶级目录
- 配置 WEST_TOPDIR 变量以供其他构建脚本使用
- 通过将 WEST 设置为 WEST-NOTFOUND 来优雅地处理非 West 项目
- 保持与 West 管理和自定义项目结构的兼容性

2.2.6 pinmux.cmake

引脚复用配置模块，负责生成和管理项目的引脚映射配置文件：

Pinmux 配置文件

- 定义生成的 pinmux 头文件的 `PINMUX_H` 变量（${CMAKE_BINARY_DIR}/pinmux.h）
- 定义中间配置文件的 `DOTPINMUX` 变量（${CMAKE_BINARY_DIR}/.pinmux）

自定义目标

- **`generate_dotpinmux`**：使用 `scripts.pinmux` Python 模块生成 `.pinmux` 配置文件，可选的 `--skip-gui` 参数
- **`generate_pinmux_h`**：使用 `pinmux_gen.py` 预处理脚本将 `.pinmux` 转换为 `pinmux.h`
- **`generate_pinmux`**：主目标，依赖于 `generate_pinmux_h` 以确保正确的执行顺序

构建目标和依赖管理

- `generate_dotpinmux` 依赖于 `.config` 文件
- `generate_pinmux_h` 依赖于 `.pinmux` 文件
- `generate_pinmux` 仅依赖于 `generate_pinmux_h` 以避免重复执行
- 使用 `USES_TERMINAL` 选项在生成期间获取实时输出

构建集成

- 将构建目录添加到包含路径，确保 `pinmux.h` 可以正确包含在源文件中
- 主构建目标依赖于 `generate_pinmux`，以确保引脚配置在编译前已就绪

Python 脚本增强

- `scripts.pinmux` 模块现在支持 `--skip-gui` 参数，可以在不启动交互式 GUI 的情况下生成默认配置
- `pinmux_gen.py` 预处理脚本在构建过程中将 `.pinmux` 转换为 `pinmux.h`
- 两个脚本都依赖于 `python.cmake` 模块提供的 Python 环境

清理目标集成

- 将 `pinmux.h` 和 `.pinmux` 添加到项目清理目标要清理的文件列表中

2.2.7 extensions.cmake

`extensions.cmake` 模块提供了必要的 CMake 工具函数，通过条件逻辑、源文件管理和数据处理能力扩展了核心 CMake 功能以支持 UniSDK 构建系统。

主要函数和特性：

1. **条件变量管理**
   - `set_ifndef(variable value [PARENT_SCOPE])`：仅在变量未定义时将其设置为指定值。此函数有助于建立默认值，同时允许被覆盖。可选的 `PARENT_SCOPE` 参数使变量在父作用域中可用。
2. **条件源文件管理**
   - `add_sources_ifdef(config src)`：仅在特定配置选项启用时将源文件添加到目标。这允许基于构建配置实现特定功能的代码包含。
   - `add_sources(src)`：`target_sources()` 的简化包装器，无条件地将源文件添加到当前目标（`${TARGET_NAME}`）。
3. **JSON 数组处理**
   - `json_to_list(JSON_ARRAY_STRING OUTPUT_LIST [OUTPUT_STRING])`：将 JSON 数组字符串转换为 CMake 列表。此函数执行类型验证以确保输入是有效的 JSON 数组，处理每个元素，并处理元素内的空格分隔值。它可以输出 CMake 列表和可选的空格连接字符串表示。
4. **实现细节**
   - 该模块使用 `include_guard(GLOBAL)` 防止多次包含
   - 函数在适当的情况下设计了父作用域传播
   - 对无效的 JSON 输入实现了错误处理

此模块作为构建系统的基础组件，提供可重用的功能，简化了 CMake 脚本中的条件构建、功能切换和数据处理。

53.2.3 2.3 CMake 配置系统流程

UniSDK CMake 配置系统遵循结构化的流程，协调初始化、模块加载和各个构建组件的集成。本节从头到尾详细介绍了配置过程。

2.3.1 初始化阶段

配置过程从在主 `CMakeLists.txt` 文件上调用 CMake 开始。此阶段通过与 UniSDK 基础设施集成来为构建系统建立基本环境。

2.3.1.1 CMakeLists.txt 入口

UniSDK 构建系统支持两种不同的入口模式，每种模式服务于不同的目的：

1. 根目录入口

根目录的 `CMakeLists.txt` 文件主要用于构建核心 SDK 功能或测试 SDK 组件。它为构建系统提供了基础，但**不会自动包含 `samples` 目录中的应用代码**。主要特点：

1. **构建系统初始化**：
   - 设置基本变量，如 `TELINK_BASE`、`AUTOCONF_H`、`CAPABILITIES_H` 和 `PINMUX_H`
   - 将 `cmake/modules` 添加到 CMake 模块路径以发现自定义模块

- 将 TELINK_ROOT_INCLUDED 设置为 TRUE 以防止冗余包含
- 如果尚未定义 TELINK_PACKAGE_INCLUDED，则加载 Telink 包配置

2. **项目配置**:
   - 定义名称为"Telink" 的主项目
   - 配置支持的语言（ASM、C、CXX）
   - 使用核心启动文件创建主可执行目标
3. **组件集成**:
   - 将关键的 SDK 子目录（api、common、core、soc）添加到构建中
   - **不包含 samples 目录**
   - 设置目标之间的依赖关系（parse_config、generate_pinmux、preprocess_linker）
   - 将编译器和链接器选项应用于主目标
4. **限制**:
   - 缺少 main 函数 - 会产生链接器错误 undefined reference to 'main'
   - 无法从 samples 目录编译完整的应用
   - 用于 SDK 开发/测试而非应用构建

2. 应用入口

推荐的应用构建方式是使用应用级 CMakeLists.txt 文件。应用**可以位于文件系统的任何位置**，不仅限于 samples/ 目录。构建系统使用 TELINK_BASE 环境变量或路径来定位 Telink SDK 包。典型的应用级 CMakeLists.txt 包括:

```
# SPDX-License-Identifier: Apache-2.0

cmake_minimum_required(VERSION 3.20.0)
find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})
project(GPIO_Demo LANGUAGES C CXX)

# Add C source files
file(GLOB C_SOURCES "*.c")
target_sources(Telink PRIVATE ${C_SOURCES})

# Add C++ files only if CONFIG_TLK_ALLOW_CPP_WRAPPERS is enabled
if(CONFIG_TLK_ALLOW_CPP_WRAPPERS)
    file(GLOB CPP_SOURCES "*.cpp")
    target_sources(Telink PRIVATE ${CPP_SOURCES})
endif()
```

入口之间的主要区别:

特性	根目录入口	应用入口
主要用途	SDK 开发/测试	构建完整应用
应用位置	无	文件系统上的任何位置（不仅限于 `samples/` 目录）。使用 west 时，必须与 `.west` 文件位于同一目录或其子目录中
包含 main 函数	❌ 否	✅ 是（在应用代码中）
构建命令	`cmake -B build -G Ninja .`	`cmake -B build -G Ninja /path/to/application`
输出	核心 SDK 组件（没有 `main` 会失败）	完整的可执行固件

正确使用示例:

```
# To build an application (recommended)
cmake -B build -G Ninja samples/gpio_demo
cmake --build build

# To build SDK core (for SDK development/testing)
cmake -B build -G Ninja .
cmake --build build  # Will fail without manual main function addition
```

当前实现状态和限制：虽然应用级 CMakeLists.txt 被设计为入口，但由于演示名称、应用层和整体 SDK Kconfig 系统之间的紧密耦合，目前实现中存在一些限制：

1. **Kconfig 系统耦合**：演示应用的配置选项与 SDK 的全局 Kconfig 系统深度集成，使得完全隔离应用特定配置变得具有挑战性。
2. **演示名称依赖**：一些构建系统组件依赖于特定的演示命名约定，这在应用名称和内部 SDK 构建逻辑之间创建了依赖关系。
3. **应用层集成**：应用层与核心 SDK 组件的交互尚未完全模块化，需要仔细管理包含路径和配置设置。

这些限制正在通过持续的开发工作来解决，以创建一个更加模块化和灵活的构建系统，充分实现应用级入口设计。

2.3.1.2 包加载：find_package(Telink)

如果未定义 `TELINK_PACKAGE_INCLUDED`，系统将加载 Telink 包:

```
if(NOT DEFINED TELINK_PACKAGE_INCLUDED)
    # Load Telink default modules
    find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})
endif()
```

这将触发从 `cmake` 目录加载 `TelinkConfig.cmake`。

2.3.1.3 TelinkConfig.cmake 处理

`TelinkConfig.cmake` 文件执行以下任务:

1. 将 `TELINK_PACKAGE_INCLUDED` 设置为 `TRUE` 以防止重新加载
2. 定义 `include_boilerplate()` 宏，该宏:
   - 如果尚未设置 `TELINK_BASE`，则进行设置（使用环境变量或相对路径）

- 配置 Telink_DIR 缓存变量
- 将 CMake 模块目录添加到模块路径
- 如果未定义，则设置应用源码和二进制目录
- 包含 telink_default.cmake

3. 调用 include_boilerplate() 执行样板代码

2.3.2 模块加载阶段

在初始环境设置完成后，配置系统会加载实现构建系统功能的核心 CMake 模块。

2.3.2.1 telink_default.cmake 处理

telink_default.cmake 文件负责按特定顺序加载所有必要的 CMake 模块：

1. 再次设置最低 CMake 版本（用于验证）
2. 输出应用目录和 CMake 版本信息
3. 对已知的 CMake 3.22.1/3.22.2 版本错误进行检查
4. 按顺序定义所需的模块列表：
   - python - Python 解释器检测和配置
   - extensions - 自定义 CMake 函数和宏
   - version - 版本信息处理
   - west - West 工具集成
   - compiler - 编译器配置
   - generated_file_directories - 生成文件的目录
   - kconfig - Kconfig 集成（配置的关键）
   - pinmux - 引脚复用配置
   - linker - 链接脚本处理
5. 根据可用模块验证任何请求的子组件
6. 按顺序包含每个模块
7. 从 SUB_COMPONENTS 列表中移除已加载的模块
8. 如果未定义 TELINK_ROOT_INCLUDED，则将根目录添加为子目录

```
# Load core CMake modules in sequence
include(python)
include(extensions)
include(version)
include(west)
include(compiler)
include(generated_file_directories)
include(kconfig)
include(pinmux)
include(linker)
```

这种顺序加载确保每个模块都能访问先前加载的模块提供的必要基础设施。顺序被精心维护以管理模块之间的依赖关系。

2.3.3 Kconfig 集成阶段

Kconfig 系统集成是一个关键阶段，管理构建系统和目标固件的配置选项。

2.3.3.1 kconfig.cmake 处理

`kconfig.cmake` 模块负责配置文件的生成和处理：

1. 定义关键文件路径：
   - `CHIP_CONFIG_FILE` - 芯片特定配置
   - `BUILD_CONFIG_FILE` - 构建特定配置
   - `DOTCONFIG` - 合并的配置文件（.config）
   - `CAPABILITIES_H` - 生成的能力头文件
   - `AUTOCONF_H` - 自动生成的配置头文件
   - `MENUCONFIG_SCRIPT` - 交互式 menuconfig 脚本路径
   - `PARSE_SCRIPT` - 配置解析脚本路径
   - `PREPROCESS_SCRIPT` - Kconfig 预处理脚本路径
   - `KCONFIG_BINARY_DIR` - Kconfig 源包含的目录
2. 配置清理目标以移除生成的文件：
   - 为所有生成的配置文件设置 `ADDITIONAL_CLEAN_FILES` 属性，包括：
     - `CHIP_CONFIG_FILE`
     - `BUILD_CONFIG_FILE`
     - `DOTCONFIG`
     - `AUTOCONF_H`
     - `CAPABILITIES_H`
     - `PINMUX_H`
     - `KCONFIG_BINARY_DIR`
3. 芯片配置生成：
   - 创建自定义命令，使用 `menuconfig.py` 和 `Kconfig.chip` 生成 `chip.config`
   - **当 `chip.config` 缺失时**：自定义命令使用 Kconfig.chip 文件触发 `menuconfig.py`，该脚本将：
     - 如果没有现有配置，则自动启动交互式 menuconfig 界面
     - 允许用户选择 SoC 和相关配置选项
     - 在用户退出菜单后将选择保存到 `chip.config`
4. 能力头文件生成：
   - 创建自定义命令，使用 `preprocess_kconfig.py` 生成 `capabilities.h`
   - 依赖于 `chip.config`
   - **当 `capabilities.h` 缺失时**：自定义命令将在其依赖项（`chip.config`）可用时自动执行，基于所选芯片的能力生成头文件
5. 构建配置生成：

- 创建自定义命令，使用 menuconfig.py 和 Kconfig.build 生成 build.config
- 依赖于 capabilities.h
- **当 build.config 缺失时**：自定义命令使用 Kconfig.build 文件触发 menuconfig.py，该脚本将：
  - 如果没有现有配置，则自动启动交互式 menuconfig 界面
  - 允许用户选择构建时的选项和功能
  - 考虑 capabilities.h 中定义的能力，以过滤可用选项
  - 在用户退出菜单后将选择保存到 build.config

6. .config 文件创建（配置阶段）：
   - 如果在 CMake 配置阶段 .config 不存在，系统执行以下步骤：
     1. 检查 chip.config 是否存在；如果不存在，则执行 kconfig.py 以非交互方式生成它
     2. 执行 preprocess_kconfig.py 生成 capabilities.h
     3. 检查 build.config 是否存在；如果不存在，则执行 kconfig.py 以非交互方式生成它
     4. 创建一个带有自动生成头部注释的新 .config 文件
     5. 将 chip.config 和 build.config 的内容合并到新的 .config 文件中
7. 配置依赖跟踪：
   - 将 CMAKE_CONFIGURE_DEPENDS 设置为 .config，以便在其更改时触发重新配置
8. 合并脚本创建：
   - 写入 CMake 脚本（merge_config_files.cmake）以合并配置文件
   - 此脚本从 build.config 和 chip.config 读取内容并将其追加到 .config
9. 构建期间更新.config：
   - 创建自定义命令，在依赖项（build.config 或 chip.config）更改时更新 .config
   - 移除旧的 .config 文件，创建新的文件，然后运行合并脚本
10. 解析配置目标：
    - 创建依赖于 .config 的自定义目标 parse_config
    - 此目标确保配置解析在主构建之前发生
11. 配置解析（构建期间）：
    - 从 .config 读取行
    - 为每个配置选项生成带有 #define 语句的 autoconf.h
    - 处理不同的值类型（布尔、字符串、数字）
    - **当 autoconf.h 缺失时**：文件在构建过程中根据 .config 的内容自动生成

2.3.3.2 Menuconfig 触发机制

menuconfig 界面可以通过以下几种方式触发：

1. **构建期间自动触发（交互式）**：当配置文件（chip.config 或 build.config）缺失且启动构建时，系统将通过 menuconfig.py 自动启动交互式 menuconfig 界面
2. **手动执行**：用户可以针对特定的配置类型手动调用 menuconfig：

- 芯片配置：`python scripts/menuconfig.py Kconfig.chip`
- 构建配置：`python scripts/menuconfig.py Kconfig.build`

3. **非交互式生成**：在 CMake 配置阶段，如果 `.config` 缺失，系统使用 `kconfig.py` 以非交互方式使用默认值生成 `chip.config` 和 `build.config`
4. **自定义命令执行**：`kconfig.cmake` 中定义的自定义命令将在其各自的输出文件（如 `chip.config`、`build.config`）需要但缺失时触发 menuconfig

2.3.4 Pinmux 配置生成

`pinmux.cmake` 模块负责生成包含引脚复用配置的 `pinmux.h` 头文件：

1. 将生成的 `pinmux.h` 文件路径设置为 `${CMAKE_BINARY_DIR}/pinmux.h`
2. 创建自定义命令以生成 `pinmux.h`：
   - 使用 Python 脚本 `scripts.pinmux` 模块
   - 将 `CMAKE_BINARY_DIR` 作为参数
   - 将工作目录设置为 `TELINK_BASE`
   - 为构建输出添加描述性注释
   - 使用终端执行命令
   - 依赖于 `.config` 文件
3. 创建自定义目标 `generate_pinmux`，该目标：
   - 始终构建（ALL）
   - 依赖于 `pinmux.h` 和 `.config`
4. 二进制目录被添加到包含路径，以便 `pinmux.h` 可以被包含在源文件中

```
# Example of pinmux configuration generation
add_custom_command(
  OUTPUT "${PINMUX_H}"
  COMMAND ${PYTHON_EXECUTABLE} -m scripts.pinmux "${CMAKE_BINARY_DIR}"
  WORKING_DIRECTORY "${TELINK_BASE}"
  COMMENT "Generating pinmux.h..."
  USES_TERMINAL
  DEPENDS "${DOTCONFIG}"
)

add_custom_target(generate_pinmux ALL
  DEPENDS "${PINMUX_H}" "${DOTCONFIG}"
)

# Make main target depend on pinmux configuration
add_dependencies(${TARGET_NAME} generate_pinmux)
```

2.3.5 构建配置阶段

在加载所有模块并集成 Kconfig 系统后，配置系统继续为特定目标设置构建环境。

2.3.5.1 目标配置

配置系统使用适当的设置配置主目标:

```
# Define the main target
add_executable(${TARGET_NAME} ${SOURCES})

# Configure target properties
target_include_directories(${TARGET_NAME} PUBLIC
  "${CMAKE_CURRENT_BINARY_DIR}"  # For autoconf.h
  "${PROJECT_INCLUDE_DIRS}"
  "${SDK_INCLUDE_DIRS}"
)

# Set target linker script
target_link_options(${TARGET_NAME} PRIVATE
  -T${LINKER_SCRIPT_PATH}
  --gc-sections
)

# Add libraries to link
target_link_libraries(${TARGET_NAME} PRIVATE
  ${SDK_LIBS}
  ${PROJECT_LIBS}
)
```

2.3.5.2 构建目录结构

配置系统设置结构化的构建目录以组织构建输出:

构建目录结构

```
build/
├── .config               # Current configuration
├── ${APP_NAME}           # Final executable output
├── autoconf.h            # Generated C header
├── build.config          # Build configuration
├── build.config.old      # Previous build configuration
├── build.ninja           # Ninja build file
├── capabilities.h        # SoC capabilities header
├── chip.config           # Chip-specific configuration
├── chip.config.old       # Previous chip configuration
├── CMakeCache.txt        # CMake cache
├── CMakeFiles/           # CMake build files
├── cmake_install.cmake   # CMake install script
├── flash_boot.ld         # Linker script
├── Kconfig/              # Chip-specific Kconfig files
│   ├── Kconfig.dma
│   ├── Kconfig.gpio
│   ├── Kconfig.pm
│   ├── Kconfig.rf
│   └── ...
├── merge_config_files.cmake
├── pinmux.h              # Generated pinmux configuration
└── telink/               # SDK build files
```

(续下页)

(接上页)

```
    ├── api/
    ├── common/
    ├── core/
    ├── samples/
    └── soc/
```

2.3.6 完整配置流程图

下图说明了从开始到结束的完整配置流程:

2.3.7 配置系统集成

2.3.7.1 Python 集成

配置系统严重依赖 Python 脚本进行 Kconfig 操作:

- `menuconfig.py` - 交互式 menuconfig 界面
- `kconfig.py` - 非交互式 Kconfig 处理
- `preprocess_kconfig.py` - 预处理 Kconfig 文件
- `kconfig_parser.py` - Kconfig 解析和验证工具
- `soc_board_setter.py` - SOC/BOARD 配置和验证

`python.cmake` 模块确保兼容的 Python 解释器可用。

2.3.8 CMake Config 目标执行流程

`config` 目标按顺序执行一系列配置任务:

1. **芯片配置**: `generate_chip_config` 使用 Kconfig 生成 `chip.config`
2. **能力生成**: `generate_capabilities` 根据所选芯片创建 `capabilities.h`
3. **构建配置**: `generate_build_config` 使用 Kconfig 生成 `build.config`
4. **配置合并**: `update_dotconfig` 将 `chip.config` 和 `build.config` 合并为 `.config`
5. **Pinmux 配置**: `generate_pinmux` 处理引脚复用配置:
   - 首先 `generate_dotpinmux` 使用 `scripts.pinmux` 和 `--skip-gui` 从 Kconfig 创建 `.pinmux`
   - 然后 `generate_pinmux_h` 使用 `pinmux_gen.py` 将 `.pinmux` 转换为 `pinmux.h`

此目标确保所有配置文件按正确顺序生成，并在任务之间具有正确的依赖关系。

2.3.9 关键配置文件及其用途

- **Kconfig.chip**: 定义可用的 SoC 选项及其配置
- **Kconfig.build**: 定义构建时的选项和功能
- **chip.config**: 生成的 SoC 特定配置
- **build.config**: 生成的构建特定配置
- **.config**: 用于构建的合并配置文件

- **capabilities.h**：基于所选芯片生成的包含 SoC 能力的头文件
- **autoconf.h**：包含所有配置选项作为 C 定义的头文件
- **pinmux.h**：包含引脚复用配置的生成头文件

2.3.10 配置问题故障排除

常见问题及其解决方案：

- **配置未生效**：确保 `parse_config` 目标在主目标之前正确构建
- **menuconfig 中缺少选项**：检查 Kconfig 文件是否正确包含以及依赖是否满足
- **Python 脚本错误**：验证 Python 3.10+ 已安装且所有依赖可用
- **未找到 CMake 模块**：确保 `TELINK_BASE` 正确设置并指向 SDK 根目录
- **生成的文件未被清理**：系统配置为使用 `make clean` 清理生成的文件，但您也可以手动删除构建目录

53.3 3. Kconfig Configuration System

53.3.1 3.1 Kconfig 架构

Kconfig 系统采用层次结构，通过 `rsource` 指令组织配置文件：

3.1.1 配置文件层次结构

- **顶层配置**：`Kconfig.chip` 和 `Kconfig.build` 作为配置入口
- **功能配置**：子目录中的 `Kconfig` 文件定义特定功能配置
- **配置工具**：`kconfig.py`、`menuconfig.py` 和 `preprocess_kconfig.py` 提供配置处理

3.1.2 Kconfig 文件组织

Kconfig 文件形成层次结构，其中高级别的 Kconfig 文件使用 `rsource` 指令包含低级别的文件。组织方式如下：

3.1.3 应用层 Kconfig 集成

除了 SDK 内置的 Kconfig 文件外，应用层也可以提供自己的 Kconfig 配置。这是通过 `kconfig.cmake` 中定义的 `KCONFIG_APPLICATION_DIR` 变量实现的：

```
set(KCONFIG_APPLICATION_DIR ${CMAKE_CURRENT_SOURCE_DIR})
```

此变量被设置为调用 CMake 的当前源目录。当运行 Kconfig 命令（如 `menuconfig`）时，系统会自动在此目录中查找 Kconfig 文件，允许应用特定配置与 SDK 内置配置无缝集成。

应用层 Kconfig 集成过程：

1. `kconfig.cmake` 将 `KCONFIG_APPLICATION_DIR` 设置为当前 CMake 源目录
2. 此变量作为环境变量传递给 Python 脚本（menuconfig.py、kconfig.py 等）
3. 脚本使用此变量查找并包含应用特定的 Kconfig 文件
4. 在配置生成过程中，应用配置与 SDK 配置合并

这种设计允许应用定义特定于其功能的配置选项，同时仍然利用 SDK 的核心配置系统。

53.3.2 3.2 配置处理流程

1. **配置生成**：通过 `kconfig.py` 合并配置文件
2. **交互式配置**：使用 `menuconfig.py` 提供菜单界面
3. **配置预处理**：`preprocess_kconfig.py` 处理配置宏和变量
4. **配置应用**：CMake 系统使用生成的配置文件进行构建配置

53.3.3 3.3 关键配置脚本

3.3.1 kconfig.py

主要功能：

- 合并多个配置文件
- 验证配置有效性
- 生成最终的 `.config` 文件
- 执行配置检查和警告

3.3.2 menuconfig.py

交互式配置界面实现：

- 基于 Curses 的文本界面
- 支持菜单导航和选项设置
- 提供帮助信息显示
- 支持配置保存和加载

3.3.3 preprocess_kconfig.py

配置预处理工具：

- 处理 YAML 配置变量
- 支持配置宏展开
- 处理嵌套配置结构
- 支持数组和特殊格式值

53.4 4. West Compilation Support

53.4.1 4.1 West 命令扩展

UniSDK 扩展了 West 命令集，通过 `west-commands.yml` 注册自定义命令：

```
west-commands:
  - file: scripts/west_commands/build.py
    commands:
      - name: tl-build
        class: Build
        help: compile a Telink application
```

(续下页)

(接上页)

```
  - file: scripts/west_commands/boards.py
    commands:
      - name: tl-boards
        class: Boards
        help: display information about supported Telink boards
  - file: scripts/west_commands/socs.py
    commands:
      - name: tl-socs
        class: Socs
        help: display information about supported Telink socs
  - file: scripts/west_commands/config.py
    commands:
      - name: tl-config
        class: Config
        help: configure a Telink application
```

53.4.2 4.2 west tl-build 命令详解

build.py 实现了主要的构建功能：

4.2.1 命令行选项

位置参数：

- source_dir：应用源目录（默认：当前目录）

可选参数：

- -d, --build-dir：指定构建目录
- -c, --cmake-only：仅运行 CMake 配置，不构建
- -k, --kconfig：构建前运行交互式配置
- -t, --target：构建目标（例如 flash、debug 等）
- -p, --pristine：控制是否使用干净的构建目录（选项：auto、always、never）
- --clean：构建前清理构建目录
- --flash：构建后烧录固件
- -o, --build-option：传递给构建系统的额外选项（可多次使用）
- --soc SOC：指定配置的 SOC
- --board BOARD：指定配置的 BOARD
- cmake_opt：传递给 CMake 的额外选项（必须跟在"--" 分隔符之后）

使用示例：

```
# Basic build
west tl-build

# Build with specific directory
west tl-build samples/gpio_demo

# Build with BOARD
west tl-build --board TLSR9528A_EVK
```

(续下页)

(接上页)

```
# Configuration only
west tl-build --cmake-only

# Interactive configuration followed by build
west tl-build --kconfig

# Clean build
west tl-build --clean

# Pristine build
west tl-build --pristine always

# Pass extra CMake options
west tl-build -- -DCONFIG_DEBUG=y
```

4.2.2 执行流程

1. **环境检查**：验证 TELINK_TOOLCHAIN_PATH 环境变量已设置
2. **SOC/BOARD 验证**：如果同时提供了 SOC 和 BOARD，则验证其组合
3. **路径设置**：配置源目录和构建目录
4. **TELINK_BASE 设置**：将 TELINK_BASE 环境变量设置为 SDK 根目录
5. **清理处理**：如果使用了 --clean 或 --pristine 选项，则清理构建目录
6. **构建目录创建**：如果构建目录不存在，则创建它
7. **交互式配置**：如果指定了 --kconfig 选项，则运行 menuconfig
8. **CMake 配置**：使用 SOC/BOARD 参数调用 CMake 并生成构建系统
9. **项目构建**：执行编译和链接（如果指定了 --cmake-only 则跳过）
10. **结果输出**：显示构建状态和结果

主要特性：

- **SOC/BOARD 支持**：通过命令行选项直接指定 SOC 和 BOARD
- **依赖解析**：CMake 配置期间自动处理 SOC/BOARD 依赖
- **验证**：构建前实时验证 SOC/BOARD 组合
- **灵活配置**：支持交互式和非交互式配置
- **干净构建**：提供干净和全新构建环境的选项

53.4.3 4.3 west tl-config 命令详解

`config.py` 实现了 Telink 应用配置功能，提供统一的配置接口：

4.3.1 命令行选项

位置参数：

- `source_dir`：应用源目录（可选，默认为当前目录）

可选参数：

- `-d, --build-dir`: 指定构建目录（可选，默认为 `source_dir/build`）
- `--soc SOC`: 指定配置的 SOC
- `--board BOARD`: 指定配置的 BOARD
- `--list-socs`: 列出所有可用的 SOC 及其依赖
- `--list-boards`: 列出所有可用的板卡及其依赖
- `--validate SOC BOARD`: 验证 SOC 和 BOARD 组合

使用示例:

```
# Interactive configuration for current directory
west tl-config

# Configure specific project
west tl-config samples/gpio_demo

# Configure with specific build directory
west tl-config -d build_debug

# List available SOCs
west tl-config --list-socs

# List available boards
west tl-config --list-boards

# Validate SOC/BOARD combination
west tl-config --validate TLSR9528A TLSR9528A_EVK
```

4.3.2 配置过程

`tl-config` 命令支持多种操作模式:

列表操作:

- `--list-socs`: 显示所有可用的 SOC 及其依赖
- `--list-boards`: 显示所有可用的板卡及其依赖
- `--validate`: 验证 SOC/BOARD 组合而不进行配置

配置流程:

1. **SOC/BOARD 验证**:
   - 如果同时提供了 SOC 和 BOARD，则验证其组合
   - 使用 KconfigParser 检查依赖兼容性
   - 为无效组合提供清晰的错误信息
2. **环境设置**:
   - 将 `TELINK_BASE` 环境变量设置为 SDK 根目录
   - 解析源码和构建目录路径
   - 如果构建目录不存在，则创建它
3. **CMake 配置**（如需要）:
   - 检查现有的 `CMakeCache.txt` 以确定是否需要配置

- 使用 Ninja 生成器和 Telink 包路径运行初始 CMake 配置
- 如果在命令行中指定，则应用 SOC 和 BOARD 设置

4. **配置目标执行**:
   - 执行 `cmake --build <build_dir> --target config`
   - 这将触发完整的配置序列:
     - 清理现有配置文件
     - 从 `Kconfig.chip` 生成 `chip.config`
     - 从芯片配置生成 `capabilities.h`
     - 从 `Kconfig.build` 生成 `build.config`
     - 将配置合并为最终的 `.config`
     - 生成 pinmux 配置文件
5. **结果处理**:
   - 返回适当的退出码（0 表示成功，非零表示错误）
   - 在整个过程中提供信息性状态消息

主要特性:

- **SOC/BOARD 发现**：列出可用的 SOC 和板卡及其依赖信息
- **验证**：构建前验证 SOC/BOARD 组合
- **依赖解析**：自动处理 SOC/BOARD 依赖
- **交互式配置**：基于 Menuconfig 的配置界面

4.3.3 代码复用集成

`tl-config` 命令通过 `run_config()` 函数与 `tl-build` 命令共享其核心配置逻辑。此函数被提取到可复用的模块中，该模块:

- 在两个命令之间提供一致的配置行为
- 处理所有配置文件的生成和 CMake 目标执行
- 确保正确的错误处理和状态报告
- 保持与现有构建工作流的兼容性

`tl-build` 命令的 `-k/--kconfig` 选项内部调用相同的配置逻辑，确保独立配置和构建时配置之间的一致性。

53.4.4 4.4 west tl-boards 命令详解

`boards.py` 实现了板卡信息显示功能:

4.4.1 命令行选项

- `-v, --verbose`：显示每个板卡的详细信息
- `--soc SOC`：按支持的 SOC 过滤板卡（例如 TL3218X）

4.4.2 命令特性

1. **板卡枚举**：从 Kconfig.chip 列出所有支持的开发板
2. **SOC 过滤**：允许按特定 SOC 兼容性过滤板卡
3. **依赖显示**：显示板卡依赖和要求
4. **详细信息**：提供详细的板卡规格，包括：
   - 完整的板卡名称和提示
   - 类型和依赖
   - 每个板卡支持的 SOC
   - 帮助文档

4.4.3 使用示例

```
# List all supported boards
west tl-boards

# Show detailed board information
west tl-boards -v

# Filter boards for specific SOC
west tl-boards --soc TLSR9528A

# Detailed information for specific SOC boards
west tl-boards -v --soc TLSR9528A
```

53.4.5 4.5 west tl-socs 命令详解

`socs.py` 实现了 SOC 信息显示功能：

4.5.1 命令行选项

- `-v, --verbose`：显示每个 SOC 的详细信息

4.5.2 命令特性

1. **SOC 枚举**：从 Kconfig.chip 列出所有支持的片上系统变体
2. **依赖显示**：显示 SOC 依赖和要求
3. **详细信息**：提供详细的 SOC 规格，包括：
   - 完整的 SOC 名称和提示
   - 类型和依赖
   - 帮助文档

4.5.3 使用示例

```
# List all supported SOCs
west tl-socs

# Show detailed SOC information
west tl-socs -v
```

53.4.6 4.6 BDT 命令扩展

`west tl-bdt` 命令是 West 扩展，提供与 Telink 烧录调试工具（BDT）的接口，用于在 Telink 设备上执行 Flash 操作。此命令通过提供统一的跨平台接口简化了与 BDT 交互的过程。

4.6.1 命令语法

```
west tl-bdt [-h] [-d BUILD_DIR] [-c] [--usb] [--bdt-path BDT_PATH] [--chip CHIP]
            [--sws EVK_CLK EVK_RATE TARGET_CLK TARGET_RATE] [--bus BUS] [--devid DEVID]
            {download,read,write,erase,lock,unlock,reset} ...
```

4.6.2 支持的操作

操作	描述
download	下载固件到 Flash
read	从 Flash 读取数据
write	向 Flash 写入数据
erase	擦除 Flash
lock	锁定 Flash 区域
unlock	解锁 Flash 区域
reset	复位设备

4.6.3 BDT 工具路径配置

4.6.3.1 设置 BDT 路径

BDT 工具路径可以通过以下方式设置（按优先级顺序）：

1. **命令行选项**：`--bdt-path /path/to/bdt_executable`
2. **环境变量**：`BDT_PATH=/path/to/bdt_executable`
3. **系统 PATH**：命令将在系统 PATH 中搜索 BDT
4. **Telink 基础目录**：如果设置了 `TELINK_BASE`，将在 `${TELINK_BASE}/tools/` 中查找

4.6.3.2 重要说明

- **参数顺序**：从版本 5.9.0 开始，`--bdt-path` 参数可以在子命令（如 download、read、write）之前或之后指定。

  ```
  # Both are now acceptable
  west tl-bdt --bdt-path C:\path\to\bdt --chip B92 download -i firmware.bin
  west tl-bdt download --chip B92 -i firmware.bin --bdt-path C:\path\to\bdt
  ```

- **正确的路径格式**：`--bdt-path` 参数可以接受以下两种格式之一：

1. **可执行文件路径**：
   - **Windows**：`Cmd_download_tool.exe`
   - **Linux**：`bdt`
2. **包含可执行文件的目录路径**：
   - 工具将自动在目录中搜索适当的可执行文件

示例：

```
# Path to executable file
west tl-bdt --bdt-path C:\BDT\Cmd_download_tool.exe

# Path to directory containing executable
west tl-bdt --bdt-path C:\BDT
```

- **路径格式**：在 Windows 上，您可以在路径中使用正斜杠或反斜杠：

```
# Both are acceptable
west tl-bdt --bdt-path C:\BDT\Cmd_download_tool.exe
west tl-bdt --bdt-path C:/BDT/Cmd_download_tool.exe
```

4.6.4 全局参数

参数	描述	默认值
`-h、--help`	显示帮助信息并退出	-
`-d、--build-dir`	构建目录	当前目录下的 `build`
`-c、--core`	在核心上操作而非 Flash	False
`--usb`	使用 USB 模式（默认：EVK 模式）	False
`--bdt-path`	BDT 工具可执行文件路径	-
`--chip`	芯片类型（例如 B92、TL721X、TL321X）	从构建配置中检测
`--sws EVK_CLK EVK_RATE TARGET_CLK TARGET_RATE`	设置 SWire 时钟值	-
`--bus`	USB 总线 ID（存在多个 USB 设备时需要）	-
`--devid`	USB 设备 ID（存在多个 USB 设备时需要）	-

4.6.5 子命令

4.6.5.1 download

将固件下载到 Flash。

```
west tl-bdt download [-h] [-i INPUT] [-a ADDR]
```

参数	描述	默认值
`-i、--input`	固件文件路径	`build/${TLK_ARTIFACT_NAME}.bin`
`-a、--addr`	要下载到的 Flash 地址	`0x00`

4.6.5.2 read

从 Flash 读取数据。

```
west tl-bdt read [-h] [-a ADDR] -s SIZE [-o OUTPUT]
```

参数	描述	默认值
-a、--addr	要读取的 Flash 地址	0x00
-s、--size	要读取的字节数（例如 16、1k）	必需
-o、--output	输出文件路径	save<timestamp>.bin

4.6.5.3 write

向 Flash 写入数据。

```
west tl-bdt write [-h] [-a ADDR] [-s SIZE] [-i INPUT] [-e] [data ...]
```

参数	描述	默认值
-a、--addr	要写入的 Flash 地址	0x00
-s、--size	要写入的字节数（写入原始数据时需要）	-
-i、--input	输入文件路径（不写入原始数据时需要）	-
data	要写入的原始数据字节（例如 01 02 03 04）	-
-e、--erase	写入前擦除	False

4.6.5.4 erase

擦除 Flash。

```
west tl-bdt erase [-h] [-a ADDR] -s SIZE
```

参数	描述	默认值
-a、--addr	要擦除的 Flash 起始地址	0x00
-s、--size	要擦除的字节数（例如 1k、4k）	必需

4.6.5.5 lock

锁定 Flash 区域。

```
west tl-bdt lock [-h] [-a ADDR] [-s SIZE]
```

参数	描述	默认值
-a、--addr	要锁定的 Flash 地址	0x00
-s、--size	要锁定的区域大小（例如 512k）	必需

4.6.5.6 unlock

解锁 Flash 区域。

```
west tl-bdt unlock [-h]
```

4.6.5.7 reset

复位设备。

```
west tl-bdt reset [-h] [-c]
```

参数	描述	默认值
-c、--core	从核心而非 Flash 复位	False

4.6.6 示例

4.6.6.1 下载固件

```
# Basic download using default chip and firmware path
west tl-bdt download

# Download with specific chip and firmware path
west tl-bdt download --chip B92 -i build/${TLK_ARTIFACT_NAME}.bin

# Download to specific address
west tl-bdt download --chip B92 -i build/${TLK_ARTIFACT_NAME}.bin -a 0x1000

# Download using USB mode
west tl-bdt download --chip B92 --usb -i build/${TLK_ARTIFACT_NAME}.bin
```

4.6.6.2 读取 Flash

```
# Read 16 bytes from address 0x00
west tl-bdt read --chip B92 -a 0x00 -s 16

# Read 1KB to output file
west tl-bdt read --chip B92 -a 0x00 -s 1k -o flash_dump.bin
```

4.6.6.3 写入 Flash

```
# Write raw data
west tl-bdt write --chip B92 -a 0x1000 -s 4 01 02 03 04

# Write from file
west tl-bdt write --chip B92 -a 0x1000 -i data.bin

# Write from file with erase
west tl-bdt write --chip B92 -a 0x1000 -i data.bin -e
```

4.6.6.4 擦除 Flash

```
# Erase 4KB at address 0x00
west tl-bdt erase --chip B92 -a 0x00 -s 4k

# Erase 1 sector
west tl-bdt erase --chip B92 -a 0x4000 -s 4k
```

4.6.6.5 锁定/解锁 Flash

```
# Lock region
west tl-bdt lock --chip B92 -a 0x00 -s 512k

# Unlock flash (Windows)
west tl-bdt unlock --chip B92
```

4.6.6.6 复位设备

```
# Reset device
west tl-bdt reset --chip B92

# Reset from core
west tl-bdt reset --chip B92 -c
```

4.6.7 高级用法

4.6.7.1 使用特定 BDT 路径

```
west tl-bdt download --chip B92 --bdt-path /path/to/bdt -i build/${TLK_ARTIFACT_NAME}.
↪bin
```

4.6.7.2 使用 SWire 时钟设置

```
west tl-bdt download --chip B92 --sws 8000000 1 4000000 1 -i build/${TLK_ARTIFACT_
↪NAME}.bin
```

4.6.7.3 多个 USB 设备

```
west tl-bdt download --chip B92 --usb --bus 1 --devid 5 -i build/${TLK_ARTIFACT_NAME}.
↪bin
```

4.6.8 跨平台注意事项

`west tl-bdt` 命令自动处理 Windows 和 Linux 环境之间的差异：

- **Windows**：使用 `Cmd_download_tool.exe`，并根据需要自动添加设备 ID 和电压信息
- **Linux**：使用 `bdt` 和标准 Linux 命令语法
- **路径处理**：自动转换路径分隔符（例如在 Windows 上 `/` → `\`）

4.6.9 故障排除

4.6.9.1 未找到 BDT 工具

```
Error: BDT tool not found. Please set BDT_PATH environment variable or use --bdt-path␣
↪option.
```

解决方案：使用第 4 节中描述的方法之一设置 BDT 路径。

4.6.9.2 未指定芯片类型

```
Error: Chip type not specified. Please use --chip option.
```

解决方案：使用 `--chip` 选项指定芯片类型，或确保构建配置包含芯片类型信息。

4.6.9.3 BDT 命令失败

```
Error: BDT command failed with exit code X
```

解决方案：检查 BDT 输出以获取详细的错误信息。常见问题包括：

- 芯片类型不正确
- 无效的地址或大小
- 设备未连接

53.4.7 4.7 West 工作空间配置

`west.yml` 定义了 West 工作空间配置：

- 远程仓库设置
- 项目路径配置
- 命令扩展定义

53.5 5. Make Support

`Makefile` 提供传统的 make 命令接口作为 CMake 的前端，为开发者提供简化的工作流。这只能在项目的根目录中使用。

53.5.1 5.1 Makefile 结构

项目根目录中的 `Makefile` 提供传统的 make 命令接口作为 CMake 的前端，为开发者提供简化的工作流。它最近经过重构以提高可维护性和灵活性：

```
.PHONY: cmake config pinmux build clean cleanbuild

# Default application to build
APP ?= samples/gpio_demo

# SOC and BOARD parameters (empty by default)
SOC ?=
```

(续下页)

(接上页)

```
BOARD ?=

# CMake configuration
CMAKE := cmake
CMAKE_FLAGS := -B build -G Ninja -DTelink_DIR='./cmake'
BUILD_DIR := build

# Python scripts
PYTHON := python
PINMUX_SCRIPT := -m scripts.pinmux

# Add SOC and BOARD flags if they are set
ifneq ($(SOC),)
CMAKE_FLAGS += -DSOC=$(SOC)
endif

ifneq ($(BOARD),)
CMAKE_FLAGS += -DBOARD=$(BOARD)
endif

# Pre-build checks
PRE_BUILD_CHECKS := \
        @$(PYTHON) scripts/pre_build/build_folder_check.py || (echo "Build folder not␣
↪found — running config..." && "$(MAKE)" config) \
        @$(PYTHON) scripts/pre_build/config_check.py || (echo ".config not found —␣
↪running config..." && "$(MAKE)" config) \
        @$(PYTHON) scripts/pre_build/pinmux_check.py || (echo "pinmux.h not found —␣
↪running pinmux..." && $(PYTHON) $(PINMUX_SCRIPT)) \
        @$(PYTHON) scripts/pre_build/pre_build_check.py || (echo "pinmux.h needs␣
↪update — running pinmux..." && $(PYTHON) $(PINMUX_SCRIPT))

# Targets
cmake:
        $(CMAKE) $(CMAKE_FLAGS) -S $(APP)

config:
        $(CMAKE) --build $(BUILD_DIR) --target config

pinmux:
        $(PYTHON) $(PINMUX_SCRIPT)

build: cmake config
        $(PRE_BUILD_CHECKS)
        $(CMAKE) --build $(BUILD_DIR)

clean:
        $(CMAKE) --build $(BUILD_DIR) --target clean

cleanbuild: clean build
```

重构后的 Makefile 定义了以下关键组件：

- **伪目标**：所有目标（cmake、config、pinmux、build、clean、cleanbuild）都被标记为伪目标，以避免与实际文件冲突
- **配置变量**：使用变量进行集中配置，便于维护
- **默认应用**：设置默认构建的应用（gpio_demo），支持覆盖

- **CMake 集成**：使用 CMake 命令（`cmake -B`、`cmake --build`）进行所有构建操作
- **预构建检查**：实现自动验证和恢复机制
- **自动依赖解析**：确保所有必需的组件在构建前已就绪
- **Pinmux 目标**：添加了显式的 `pinmux` 目标，便于引脚复用配置

5.1.1 SOC 和 BOARD 参数

Makefile 提供 SOC 和 BOARD 参数，允许定位特定的硬件配置：

参数定义：

```
# SOC and BOARD parameters (empty by default)
SOC ?=
BOARD ?=
```

参数集成：

```
# Add SOC and BOARD flags if they are set
ifneq ($(SOC),)
CMAKE_FLAGS += -DSOC=$(SOC)
endif

ifneq ($(BOARD),)
CMAKE_FLAGS += -DBOARD=$(BOARD)
endif
```

使用示例：

```
# Build with specific SOC and BOARD
make build SOC=TLSR9528A BOARD=TLSR9528A_EVK

# Build with only SOC (board will be auto-selected or default)
make build SOC=TLSR9528A

# Build with only BOARD (SOC dependencies will be resolved automatically)
make build BOARD=TLSR9528A_EVK
```

主要特性：

- **可选参数**：SOC 和 BOARD 参数都是可选的，可以独立使用
- **CMake 集成**：参数自动作为 `-DSOC` 和 `-DBOARD` 标志传递给 CMake
- **依赖解析**：当指定 BOARD 时，所需的 SOC 依赖会自动解析
- **验证**：构建系统在配置期间验证 SOC/BOARD 组合

53.5.2 5.2 Make 命令流程

5.2.1 cmake 目标

```
cmake:
	$(CMAKE) $(CMAKE_FLAGS) -S $(APP)
```

- **用途**：为指定应用初始化 CMake 构建系统
- **参数**：

  - `$(CMAKE)`：CMake 可执行文件的路径
  - `$(CMAKE_FLAGS)`：通用 CMake 标志，包括构建目录和生成器
  - `-S $(APP)`：将源目录指定为所选应用（默认：`samples/gpio_demo`）

5.2.2 config 目标

```
config:
	$(CMAKE) --build $(BUILD_DIR) --target config
```

- **用途**：生成项目配置文件
- **操作**：调用 CMake 的 config 目标生成像 `.config` 这样的配置文件
- **参数**：
  - `$(CMAKE)`：CMake 可执行文件的路径
  - `--build $(BUILD_DIR)`：指定构建目录
  - `--target config`：用于生成配置文件的 CMake 目标

5.2.3 build 目标

```
build: cmake config
	$(PRE_BUILD_CHECKS)
	$(CMAKE) --build $(BUILD_DIR)
```

- **依赖**：如果尚未执行，则自动运行 `cmake` 和 `config` 目标
- **预构建检查**（通过 `$(PRE_BUILD_CHECKS)`）：
  1. **构建文件夹检查**：确保构建目录存在，如果不存在则运行 `config`
  2. **配置文件检查**：验证 `.config` 文件是否存在，如果不存在则运行 `config`
  3. **Pinmux 检查**：确认 `pinmux.h` 是否存在，如果不存在则运行 pinmux 工具
  4. **Pinmux 更新检查**：验证 `pinmux.h` 是否是最新的，如果不是则运行 pinmux 工具
- **构建执行**：调用 `cmake --build $(BUILD_DIR)` 使用 Ninja 执行实际构建

5.2.4 clean 目标

```
clean:
	$(CMAKE) --build $(BUILD_DIR) --target clean
```

- **用途**：移除构建产物，同时保留配置文件
- **操作**：调用 CMake 的 clean 目标移除生成的二进制文件
- **参数**：
  - `$(CMAKE)`：CMake 可执行文件的路径
  - `--build $(BUILD_DIR)`：指定构建目录
  - `--target clean`：用于清理构建产物的 CMake 目标

5.2.5 cleanbuild 目标

```
cleanbuild: clean build
```

- **用途**：清理现有构建产物并立即重新构建项目
- **操作**：按顺序组合 `clean` 和 `build` 操作
- **参数**：继承 `clean` 和 `build` 目标的所有参数

5.2.6 pinmux 目标

```
pinmux:
	$(PYTHON) $(PINMUX_SCRIPT)
```

- **用途**：配置引脚复用
- **操作**：直接运行 pinmux 配置工具
- **参数**：
  - `$(PYTHON)`：Python 可执行文件的路径
  - `$(PINMUX_SCRIPT)`：Pinmux 脚本模块

53.6 6. Compilation Generation Flow

53.6.1 6.1 完整构建流程

1. **环境验证**：检查工具链、Python 环境和依赖
2. **CMake 配置**：生成构建系统和 Makefile，加载 CMake 模块
3. **配置生成**：通过 Kconfig 作为 CMake 目标生成配置文件
4. **源码编译**：编译 C/C++ 源代码
5. **链接**：将目标文件链接为可执行文件
6. **固件生成**：生成最终的固件二进制文件

6.1.1 完整构建流程图

下图说明了从开始到结束的完整构建流程：

53.6.2 6.2 配置文件生成流程

1. **默认配置**：加载芯片默认配置
2. **用户配置**：应用用户定义的配置
3. **配置合并**：通过 `kconfig.py` 合并配置文件
4. **头文件生成**：生成 `autoconf.h` 和 `capabilities.h`

53.6.3 6.3 构建文件生成

- **编译命令**：为每个源文件生成编译命令
- **依赖信息**：生成源文件依赖关系
- **链接脚本**：处理和生成链接脚本
- **固件文件**：生成最终二进制和烧录文件

53.7 7. Files and Scripts Composition

53.7.1 7.1 核心配置文件

文件	位置	功能
CMakeLists.txt	根目录	CMake 主配置入口
Makefile	根目录	Make 命令接口
west.yml	根目录	West 工作空间配置
Kconfig.chip	根目录	定义可用的 SoC 选项及其配置
Kconfig.build	根目录	定义构建时的选项和功能
chip.config	构建目录	生成的 SoC 特定配置
build.config	构建目录	生成的构建特定配置
.config	构建目录	用于构建的合并配置文件
capabilities.h	构建目录	基于所选芯片生成的包含 SoC 能力的头文件
autoconf.h	构建目录	包含所有配置选项作为 C 定义的生成头文件
pinmux.h	构建目录	包含引脚复用配置的生成头文件

53.7.2 7.2 CMake 文件和模块

7.2.1 核心 CMake 文件

文件	位置	主要功能
CMakeLists.txt	根目录	CMake 主入口，设置项目结构并包含子目录
TelinkConfig.cmake	cmake/	Telink 包配置，提供 include_boilerplate 宏并加载默认模块
CMakeLists.txt	应用目录	应用特定 CMake 配置，创建可执行目标并添加源文件

7.2.2 CMake 模块文件

模块	位置	主要功能
compiler.cmake	cmake/modules/	编译器配置和编译选项
linker.cmake	cmake/modules/	链接器配置和链接脚本
kconfig.cmake	cmake/modules/	Kconfig 集成和配置处理
west.cmake	cmake/modules/	West 工具集成
pinmux.cmake	cmake/modules/	引脚复用配置
extensions.cmake	cmake/modules/	CMake 扩展函数
python.cmake	cmake/modules/	Python 环境配置
telink_default.cmake	cmake/modules/	模块加载和默认配置

53.7.3 7.3 构建脚本

脚本	位置	功能
build.py	scripts/west_commands/	实现 west tl-build 命令
config.py	scripts/west_commands/	实现 west tl-config 命令
boards.py	scripts/west_commands/	显示支持的开发板
socs.py	scripts/west_commands/	显示支持的芯片
kconfig.py	scripts/	处理 Kconfig 配置文件
menuconfig.py	scripts/	交互式配置界面
preprocess_kconfig.py	scripts/	Kconfig 预处理
kconfig_parser.py	scripts/	Kconfig 解析和验证工具
soc_board_setter.py	scripts/	SOC/BOARD 配置和验证
pinmux_gen.py	scripts/preprocess/	从.pinmux 文件生成 pinmux 头文件
config_parser.py	scripts/parsers/	配置文件解析工具
yaml_parser.py	scripts/parsers/	YAML 文件解析工具
pinmux_update_check.py	scripts/checks/	检查 pinmux 是否需要更新
tlk_check_fw.py	scripts/	固件验证脚本

7.3.1 SOC/BOARD 相关脚本

soc_board_setter.py

- **用途**：处理 SOC 和 BOARD 配置，具有正确的依赖解析
- **主要特性**：
  - 使用 Kconfig 依赖验证 SOC/BOARD 组合
  - 自动解析和设置所需的依赖
  - 支持单独的 SOC 或 BOARD 设置，或组合设置
  - 提供编程 API 和命令行接口
- **使用示例**：

```
# Set SOC and BOARD together
python scripts/soc_board_setter.py --kconfig Kconfig.chip --soc TLSR9528A --board␣
↪TLSR9528A_EVK

# Validate combination without setting
python scripts/soc_board_setter.py --kconfig Kconfig.chip --validate-only --soc␣
↪TLSR9528A --board TLSR9528A_EVK

# Set only BOARD (SOC dependencies resolved automatically)
python scripts/soc_board_setter.py --kconfig Kconfig.chip --board TLSR9528A_EVK
```

kconfig_parser.py

- **用途**：为 SOC/BOARD 管理提供 Kconfig 解析和验证工具
- **主要特性**：
  - 解析 Kconfig.chip 以提取可用的 SOC 和板卡
  - 基于依赖关系验证 SOC/BOARD 组合
  - 提供查询每个 SOC 支持的板卡的方法，反之亦然
  - 被 West 命令和 CMake 配置系统使用

- **集成点**:
  - 被 `boards.py` 和 `socs.py` West 命令使用
  - 被 `kconfig.cmake` 调用用于配置期间的验证
  - 支持依赖解析和兼容性检查

7.3.2 工具脚本

pinmux_gen.py

- **用途**: 从.pinmux 配置生成 pinmux 头文件
- **位置**: `scripts/preprocess/`
- **主要特性**:
  - 从构建目录读取.pinmux 文件
  - 将 CONFIG_ 格式定义转换为 C 预处理器宏
  - 生成 pinmux.h 头文件供源代码包含
- **集成**: 在 pinmux 配置生成期间由 CMake 构建系统使用

config_parser.py

- **用途**: 配置文件解析工具
- **位置**: `scripts/parsers/`
- **主要特性**:
  - 解析.config 文件以获取特定配置值
  - 从配置文件中提取 pinmux 配置
  - 为配置文件操作提供一致的接口
- **用途**: 被各种脚本用于配置文件访问和验证

yaml_parser.py

- **用途**: YAML 文件解析工具
- **位置**: `scripts/parsers/`
- **主要特性**:
  - 加载和解析 YAML 配置文件
  - 优雅地处理错误并显示信息性消息
  - 返回字典结构以供编程访问
- **用途**: 用于解析 pinmux YAML 文件和其他配置数据

7.3.3 验证脚本

pinmux_update_check.py

- **用途**: 检查 pinmux 配置是否需要更新
- **位置**: `scripts/checks/`
- **主要特性**:
  - 验证.config 文件的存在

  - 检查 GPIO 驱动启用状态
  - 比较.config 和.pinmux 中的 TLK_SOC_SERIES
  - 验证所需功能在.pinmux 中存在
  - 检查启用端口的一致性
- **集成**：由预构建检查使用以确保 pinmux 是最新的

| pre_build_check.py | scripts/ | 预构建环境检查 |

53.7.4 7.4 Kconfig 文件

文件	位置	功能
core/Kconfig	core/	核心功能配置
soc/Kconfig	soc/	芯片系列选择
boards/Kconfig	boards/	开发板选择
api/Kconfig	api/	API 功能配置

53.8 8. Build System Working Principles

53.8.1 8.1 配置与构建分离

UniSDK 构建系统明确将配置过程与构建过程分离：

1. **配置阶段**：通过 Kconfig 生成配置文件
2. **生成阶段**：CMake 使用配置文件生成构建系统
3. **构建阶段**：Ninja 执行实际的编译和链接任务

53.8.2 8.2 模块化设计

系统采用高度模块化设计，每个模块负责特定的功能：

- 模块通过清晰的接口进行交互
- 配置信息通过变量和属性传递
- 支持条件加载和替换模块

53.8.3 8.3 多种构建方式支持

系统同时支持多种构建方式以满足不同使用场景：

- **West 命令**：提供简化的命令行接口
- **直接 CMake 调用**：提供完整的配置控制
- **Make 命令**：兼容传统构建习惯

53.9 9. Extension and Customization

53.9.1 9.1 West 扩展命令行为

不同目录下的 West 命令行为

由于扩展命令机制，west 工具的输出因当前工作目录而异:

1. **扩展命令发现**: West 在当前目录及其父目录中搜索 `.west/config` 配置文件以定位工作空间。
2. **项目清单加载**: 找到工作空间后，west 加载 `west.yml` 清单文件，该文件通过 `west-commands` 字段定义项目特定的扩展命令。
3. **命令可用性**:
   - 在已配置的工作空间内，west 加载 unisdk 项目扩展并显示自定义命令，如 `tl-build`、`tl-boards` 和 `tl-socs`
   - 在任何工作空间外，west 仅显示内置命令，因为没有项目特定的扩展可用
4. **UniSDK 扩展命令**: unisdk 项目在 `scripts/west-commands.yml` 中定义了三个自定义扩展:
   - `tl-build`: 编译 Telink 应用
   - `tl-boards`: 显示支持的 Telink 板卡信息
   - `tl-socs`: 显示支持的 Telink SOC 信息

这种行为使 west 能够基于当前项目工作空间提供上下文感知的工具，同时在不同开发环境之间保持一致性。

53.9.2 9.2 添加新的构建模块

要添加新的构建模块:

1. 在 `cmake/modules/` 目录中创建一个新的 `.cmake` 文件
2. 将该模块添加到 `telink_default.cmake` 的加载列表中
3. 实现模块的配置和功能

53.9.3 9.3 扩展 West 命令

要添加新的 West 命令:

1. 在 `scripts/west_commands/` 目录中创建命令实现文件
2. Register the command in `scripts/west-commands.yml`
3. Implement the command's parameter parsing and execution logic

53.9.4 9.4 Adding Kconfig Configuration

To add new Kconfig configuration:

1. Create or modify Kconfig files in appropriate locations
2. Use the `rsource` directive to integrate new configurations into the configuration tree
3. Define configuration options, dependencies, and default values

53.9.5 9.5 Adding new application

To add a new application:

1. Create a new directory **anywhere** on your filesystem
2. Add the application source files to the new directory (must include a `main` function)
3. Create a `CMakeLists.txt` file in the new directory with the following structure:

```
cmake_minimum_required(VERSION 3.20.0)

# Load the Telink SDK package
find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})

# Set project name and supported languages
project(YourAppName LANGUAGES C CXX)

# Add C source files to the existing Telink target
file(GLOB C_SOURCES "*.c")
target_sources(Telink PRIVATE ${C_SOURCES})

# Add C++ support conditionally if CONFIG_TLK_ALLOW_CPP_WRAPPERS is enabled
if(CONFIG_TLK_ALLOW_CPP_WRAPPERS)
    # Add C++ sources using wildcard
    file(GLOB CPP_SOURCES "*.cpp")

    # Update the Telink target with C++ sources
    target_sources(Telink PRIVATE ${CPP_SOURCES})
endif()
```

This implementation fully realizes the application-level entry point design, providing a modular and flexible build system for Telink chip development.

This document provides a detailed introduction to the architecture, components, and working principles of the UniSDK build system, helping developers understand and use the build system. With modular design and support for multiple build methods, the UniSDK build system provides a flexible, powerful build toolchain for Telink chip development.

CHAPTER 54

扩展与定制

54.1 添加新的 CMake 模块

1. 在 cmake/modules/ 中创建一个 .cmake 文件
2. 将其添加到 telink_default.cmake 的模块加载列表中

```
# telink_default.cmake
list(APPEND telink_cmake_modules my_module)
```

54.2 扩展 West 命令

1. 在 scripts/west_commands/ 中创建命令实现文件
2. 在 scripts/west-commands.yml 中注册命令

```
# west-commands.yml
west-commands:
  - file: scripts/west_commands/my_command.py
    commands:
      - name: tl-mycmd
        class: MyCommand
        help: my custom command
```

3. 实现命令类:

```
from west.commands import WestCommand

class MyCommand(WestCommand):
    def __init__(self):
        super().__init__('tl-mycmd', 'my custom command', '...')

    def do_add_parser(self, parser_adder):
```

(续下页)

(接上页)

```
        parser = parser_adder.add_parser(self.name, help=self.help)
        return parser

    def do_run(self, args, unknown_args):
        print("Hello from my custom command!")
```

54.3 添加 Kconfig 配置

1. 在适当位置创建或修改 Kconfig 文件
2. 使用 `rsource` 指令将其集成到配置树中

```
# Add my_feature/Kconfig
menu "My Feature"

config MY_FEATURE_ENABLED
    bool "Enable my feature"
    default n
    help
        Enable custom feature.

endmenu
```

然后在父 Kconfig 中包含它:

```
# Kconfig.build
rsource "my_feature/Kconfig"
```

54.4 添加新的应用

1. 在任何位置创建应用目录
2. 添加 `CMakeLists.txt` 和源文件
3. 构建:

```
west tl-build /path/to/my_app --board TLSR9528A_EVK
```

54.4.1 CMakeLists.txt 模板

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})
project(MyApp LANGUAGES C CXX)

file(GLOB C_SOURCES "*.c")
target_sources(Telink PRIVATE ${C_SOURCES})

if(CONFIG_TLK_ALLOW_CPP_WRAPPERS)
    file(GLOB CPP_SOURCES "*.cpp")
    target_sources(Telink PRIVATE ${CPP_SOURCES})
endif()
```

54.5 West 命令行为说明

West 命令的可用性取决于当前工作目录:

- **在 West 工作空间内**：加载 UniSDK 扩展命令（`tl-build`、`tl-boards`、`tl-socs` 等）
- **在工作空间外**：仅显示 West 内置命令

West 通过以下机制发现扩展:

1. 从当前目录向上搜索 `.west/config`
2. 从 `west.yml` 中加载 `west-commands` 声明

CHAPTER 55

文件与脚本参考

55.1 核心配置文件

文件	位置	功能
`CMakeLists.txt`	仓库根目录	CMake 主入口
`Makefile`	仓库根目录	Make 命令接口
`west.yml`	仓库根目录	West 工作空间配置
`Kconfig.chip`	仓库根目录	芯片 Kconfig 入口
`Kconfig.build`	仓库根目录	构建 Kconfig 入口

55.2 构建产物（build/ 目录）

文件	描述
`.config`	最终合并配置
`chip.config`	芯片配置
`build.config`	构建配置
`autoconf.h`	C 配置宏定义
`capabilities.h`	芯片能力声明
`pinmux.h`	引脚复用配置

55.3 CMake 文件

55.3.1 核心文件

文件	位置	功能
CMakeLists.txt	仓库根目录	CMake 主入口
TelinkConfig.cmake	cmake/	Telink 包配置
telink_default.cmake	cmake/modules/	模块加载和默认配置

55.3.2 CMake 模块（cmake/modules/）

模块	功能
compiler.cmake	RISC-V 编译器配置
extensions.cmake	CMake 扩展函数
kconfig.cmake	Kconfig 集成
linker.cmake	链接器配置
pinmux.cmake	引脚复用配置
python.cmake	Python 环境配置
west.cmake	West 工具集成
library.cmake	库构建模块
headers_gen.cmake	头文件生成

55.4 构建脚本（scripts/）

脚本	功能
kconfig.py	非交互式 Kconfig 处理
menuconfig.py	交互式配置界面
preprocess_kconfig.py	Kconfig 预处理
kconfig_parser.py	Kconfig 解析与验证
soc_board_setter.py	SOC/BOARD 配置
tl_check_fw.sh	固件验证
set_telink_base.sh	设置 TELINK_BASE（Linux/Mac）
set_telink_base.cmd	设置 TELINK_BASE（Windows CMD）
set_telink_base.ps1	设置 TELINK_BASE（Windows PowerShell）
windows_setup.ps1	Windows 自动环境设置
sdk_exporter.sh	SDK 导出脚本

55.4.1 West 命令实现（scripts/west_commands/）

脚本	对应命令
build.py	west tl-build
config.py	west tl-config
bdt.py	west tl-bdt
boards.py	west tl-boards
socs.py	west tl-socs
zcmake.py	CMake 辅助工具

55.4.2 Pinmux 工具（scripts/pinmux/）

文件	功能
app.py	Pinmux 应用核心
__main__.py	CLI 入口
data/	数据处理模块
ui/	终端 UI 组件
styles.tcss	UI 样式表

55.4.3 CI 脚本（scripts/ci/）

脚本	功能
ci_build.sh	单个项目 CI 构建
ci_build_all.sh	所有项目 CI 构建
ci_prepare.sh	CI 准备工作
defconfig_to_dotconfig.py	defconfig 到.config 转换
dotconfig_to_defconfig.py	.config 到 defconfig 转换

55.5 Kconfig 文件

文件	位置	功能
Kconfig.chip	仓库根目录	芯片 Kconfig 入口
Kconfig.build	仓库根目录	构建 Kconfig 入口
core/Kconfig	core/	核心配置
core/configs/Kconfig	core/configs/	核心功能配置
soc/Kconfig	soc/	芯片系列配置
boards/Kconfig	boards/	开发板配置
api/Kconfig	api/	API 配置
samples/Kconfig	samples/	示例配置

CHAPTER 56

开发工具

UniSDK 提供了一系列开发工具，以简化芯片配置、烧录/调试和代码编辑工作流程。

工具	描述	文档
BDT	Telink 烧录和调试工具	*BDT 工具*
Pinmux	图形化引脚复用配置工具	*Pinmux 工具*
Menuconfig	终端交互式系统配置界面	*Menuconfig*
West	统一命令行入口	*West 命令*

56.1 工具概览

CHAPTER 57

BDT 烧录和调试工具

BDT（烧录和调试工具）是 Telink 芯片的烧录和调试工具，通过 `west tl-bdt` 命令提供统一的跨平台接口。

57.1 支持的操作

操作	描述
`download`	下载固件到 Flash
`read`	从 Flash 读取数据
`write`	向 Flash 写入数据
`erase`	擦除 Flash
`lock`	锁定 Flash 区域
`unlock`	解锁 Flash 区域
`reset`	复位设备

57.2 快速开始

```
# Download firmware (default from build/ directory)
west tl-bdt download --chip TLSR9528A

# Specify firmware file
west tl-bdt download --chip TLSR9528A -i build/Telink.bin

# Specify flash address
west tl-bdt download --chip TLSR9528A -i build/Telink.bin -a 0x1000

# Use USB mode
west tl-bdt download --chip TLSR9528A --usb -i build/Telink.bin
```

57.3 Flash 操作

57.3.1 读取 Flash

```
# Read 16 bytes from address 0x00
west tl-bdt read --chip TLSR9528A -a 0x00 -s 16

# Read 1KB to a file
west tl-bdt read --chip TLSR9528A -a 0x00 -s 1k -o flash_dump.bin
```

57.3.2 写入 Flash

```
# Write raw data
west tl-bdt write --chip TLSR9528A -a 0x1000 -s 4 01 02 03 04

# Write from a file
west tl-bdt write --chip TLSR9528A -a 0x1000 -i data.bin

# Erase then write
west tl-bdt write --chip TLSR9528A -a 0x1000 -i data.bin -e
```

57.3.3 擦除 Flash

```
# Erase 4KB
west tl-bdt erase --chip TLSR9528A -a 0x00 -s 4k
```

57.3.4 Flash 锁定操作

```
# Lock 512KB
west tl-bdt lock --chip TLSR9528A -a 0x00 -s 512k

# Unlock
west tl-bdt unlock --chip TLSR9528A
```

57.3.5 复位

```
# Reset device
west tl-bdt reset --chip TLSR9528A

# Reset from Core
west tl-bdt reset --chip TLSR9528A -c
```

57.4 BDT 路径配置

优先级从高到低：

1. 命令行：`--bdt-path /path/to/bdt`
2. 环境变量：`export BDT_PATH=/path/to/BDT`
3. 系统 PATH

4. SDK 目录：`${TELINK_BASE}/tools/`

```
# Method 1: Specify via command line
west tl-bdt download --bdt-path /path/to/BDT --chip TLSR9528A -i build/firmware.bin

# Method 2: Environment variable
export BDT_PATH=/path/to/BDT  # Linux/macOS
set BDT_PATH=C:\BDT           # Windows CMD
$env:BDT_PATH="C:\BDT"        # Windows PowerShell
```

57.5 跨平台说明

平台	可执行文件	说明
Windows	`Cmd_download_tool.exe`	自动处理路径分隔符转换
Linux	`bdt`	标准 Linux 命令

57.6 常见问题

57.6.1 未找到 BDT 工具

```
Error: BDT tool not found. Please set BDT_PATH or use --bdt-path.
```

解决方案：使用上述方法配置 BDT 路径。

57.6.2 未指定芯片类型

```
Error: Chip type not specified. Please use --chip option.
```

解决方案：使用 `--chip` 参数指定芯片，或确保构建配置中包含芯片信息。

57.6.3 BDT 命令失败

检查常见原因：

- 芯片类型不正确
- 地址或大小无效
- 设备未连接

CHAPTER 58

Pinmux 引脚配置工具

58.1 概述

Pinmux（引脚复用）工具是 UniSDK 提供的图形化引脚配置工具，帮助开发者在终端中可视化配置芯片引脚复用功能。

主要特性：

- 图形化终端界面（基于 Textual 框架）
- 通过交互式弹出菜单可视化选择引脚功能
- 引脚配置冲突检测
- 需求验证（确保所有必需功能已分配）
- 支持多种芯片封装变体
- 与 Kconfig 配置系统集成
- 自动生成 `pinmux.h` 头文件

58.2 启动方式

```
# Method 1: Via CMake target
make pinmux

# Method 2: Direct Python module invocation
python -m scripts.pinmux

# Method 3: Skip GUI, generate default configuration
python -m scripts.pinmux --skip-gui
```

58.3 主视图

打开后，您将看到主仪表板：

- **端口：**所有 GPIO 口的可滚动垂直列表（例如 PORT A、PORT B）
- **引脚：**在每个端口部分内，各个引脚显示为交互式按钮，显示引脚名称及其当前分配的功能

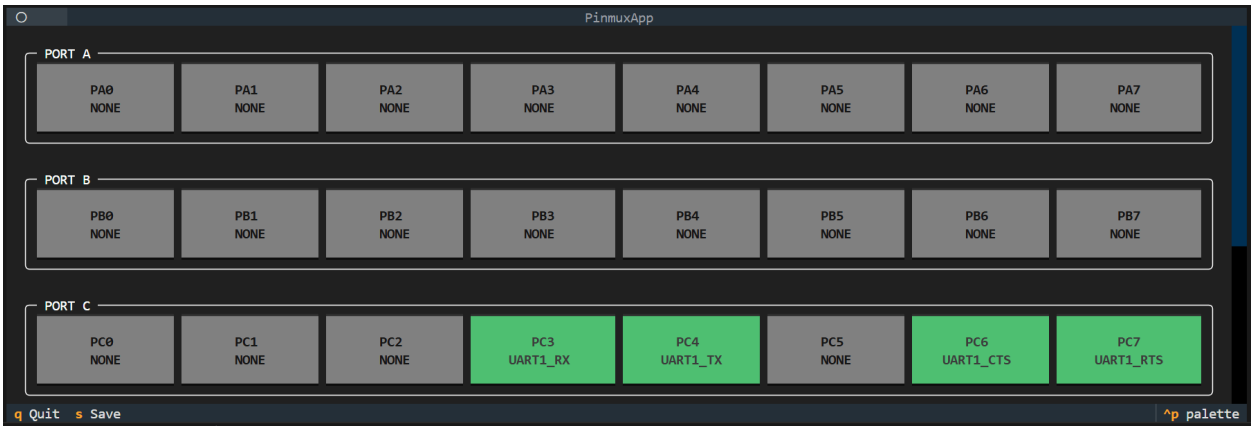


图 *1. Pinmux* 界面概览，显示 *GPIO* 端口、引脚和已分配的功能。

58.4 选择功能

1. 点击引脚（或导航到该引脚并按 **Enter**）
2. 将弹出一个标题为 **Function Select** 的窗口
3. 列表显示该特定引脚支持的所有硬件功能
4. 选择所需功能（例如 `UART0_RX`）或选择 **NONE** 禁用引脚
5. 按 **Enter** 确认

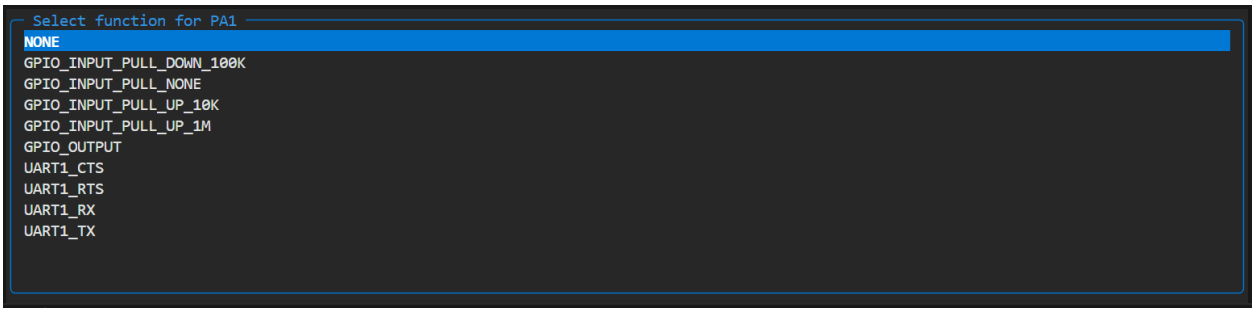


图 *2. GPIO* 引脚的功能选择界面。

58.5 快捷键

按键	操作
s（保存）	验证配置—检查冲突和缺失功能。如果有效，保存并退出。如果无效，显示错误通知
q（退出）	退出应用程序。如果有未保存的修改，将显示确认对话框

验证错误发生在以下情况:

- 必需功能未分配给引脚（例如 UART0 已启用但 UART0_TX 未分配）
- 存在引脚冲突（例如 UART0_TX 同时分配给 PC3 和 PC4）

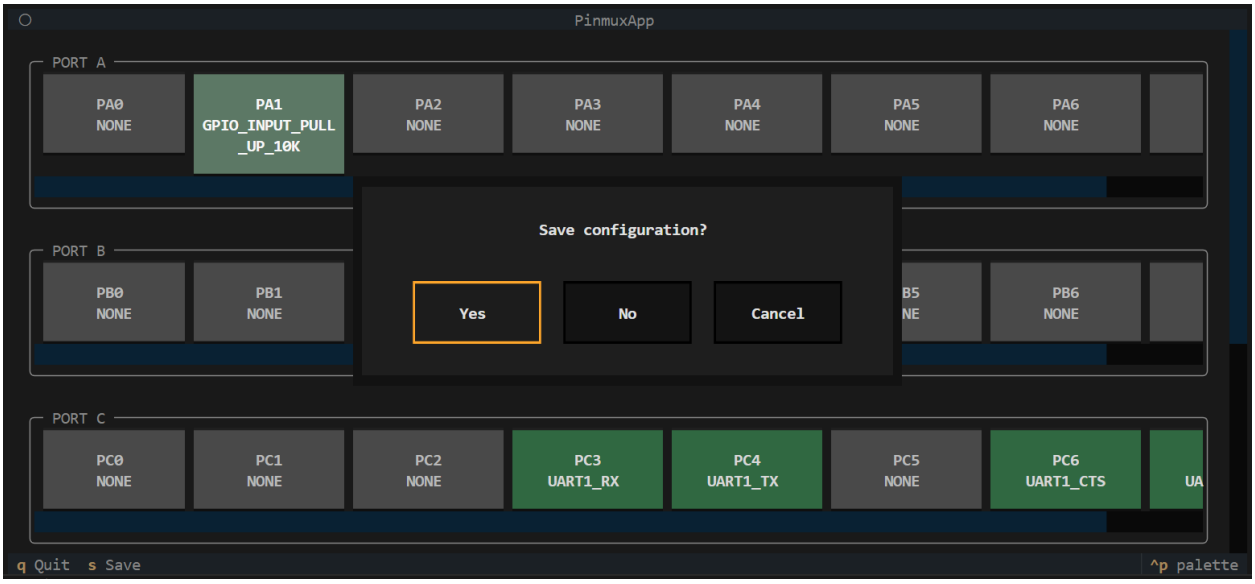


图 *3.* 退出界面。

58.6 保存

配置将保存到名为 `<SoC or board name>.<soc or board>.pinmux` 的文件中。例如，为 TL3218X_EVK 构建将生成 `TL3218X_EVK.board.pinmux`。

此 `.pinmux` 文件包含人类可读格式的引脚到功能映射，并在再次调用工具时用于恢复先前的引脚设置。

此外，根据保存的设置，在构建文件夹中生成 `pinmux.h`，其中包含固件代码可用的 C 预处理器定义。

`.pinmux` 文件会持续存在直到项目切换，允许在切换芯片时恢复先前的引脚设置。

58.7 文件格式

`.pinmux` 文件使用简单的键值格式:

```
PC4=UART0_TX
PB2=UART0_RX
PA0=GPIO_OUTPUT
```

- 每行代表一个引脚分配
- 所有字母均为大写
- 格式: `< 引脚名称>=< 功能名称>`
- 引脚名称由字母 **P**、端口字母和引脚编号组成

解析过程中，以下行将被忽略:

- 不符合格式的无效数据
- 引用不存在的引脚或功能的有效数据

- 分配给当前配置中禁用的端口上的引脚
- 分配给当前配置中未启用的功能

58.8 覆盖机制

每次调用 pinmux 时，数据按以下顺序加载（每个覆盖前一个）：

1. **SoC 默认值**—来自 `soc/<soc>/pinmux/function.yaml`（用于开发）
2. **项目 `.pinmux`** —在项目目录中（为任意芯片设置自定义默认值）
3. **`<identifier>.pinmux`** —在项目目录中（芯片特定覆盖）
4. **`<identifier>.pinmux`** —在构建目录中（上次实际用户引脚设置）

这使得项目可以拥有开箱即用的引脚设置，同时仍允许自定义。

58.9 内部执行流程

58.9.1 数据加载与同步

当应用程序启动时，它会合并多个数据源：

- `soc/<soc>/pinmux/function.yaml` —" 功能定义"：描述芯片的所有可用功能
- `soc/<soc>/pinmux/pinmux.yaml` —" 字典"：列出每个物理引脚及其支持的功能。每个端口可以有一个 `exclude` 属性，描述其引脚不支持哪些功能。单个引脚也可以使用 `exclude` 和 `include` 属性。限制按以下顺序应用：端口的 exclude → 引脚的 exclude → 引脚的 include。为简化起见，`all` 关键字表示所有功能。对于功能组，可以使用组名称（例如 `uart0` 代表 `uart0_tx`、`uart0_rx`、`uart0_cts` 和 `uart0_rts`；`uart` 涵盖 `uart0`、`uart1` 等）。如果某个引脚仅支持一个确切功能，请使用 `exclude: all` 配合 `include: your_function`。
- `soc/<soc>/pinmux/pinmux_<case>.yaml` —" 限制"：通过分配 `delete` 关键字覆盖 `pinmux.yaml`，以移除特定芯片封装的引脚或端口
- `.config` —" 需求"：告知应用程序哪些驱动已启用（例如"UART0 已启用，因此我们需要 TX 和 RX 的引脚"）
- `.pinmux` 文件—之前的用户配置（参见覆盖机制）

加载顺序：

1. `.config` 文件，包含当前配置并标识当前芯片。
2. 来自 `function.yaml` 的功能描述，跳过配置中禁用的功能。
3. 来自选定芯片的 `pinmux.yaml` 和 `pinmux_<case>.yaml` 的端口和引脚描述。
4. 按照覆盖机制部分描述的顺序加载 `.pinmux` 文件。

58.9.2 更改引脚功能

当一个引脚选择新功能时，将立即发生以下操作：

- **冲突检查：** 系统扫描所有其他引脚。如果您尝试将唯一功能（如 `UART0_TX`）分配给 PC0，但 PB2 已拥有该功能，系统会标记 **冲突**。
- **观察者更新：** 应用程序使用" 观察者模式"。
  - 引脚数据发生更改。

- UI 小部件收到通知并重新绘制自身（更新文本和颜色）。
- 下一次在 PinmuxManager 中访问 is_modified 属性将返回 True，直到触发保存操作。

58.9.3 保存过程

当您按下保存（s）时，应用程序在写入磁盘前执行严格验证。

第 1 步：验证

- **检查冲突：**是否有两个引脚分配到同一个不可重复的功能？如果是 → 报错。
- **检查需求：**配置所需的所有功能是否已分配给有效的引脚？如果否 → 报错。

第 2 步：写入

- 如果没有错误，引脚映射将使用简单的键值格式保存到 <identifier>.pinmux 文件中。

58.9.4 生成 pinmux.h

保存后，pinmux.h 将自动生成 C 预处理器定义：

```
#define PINMUX_UART0_TX_PORT TLK_GPIO_PORT_C
#define PINMUX_UART0_TX_PIN TLK_GPIO_PIN_4
#define PINMUX_UART0_RX_PORT TLK_GPIO_PORT_B
#define PINMUX_UART0_RX_PIN TLK_GPIO_PIN_2
#define PINMUX_GPIO_OUTPUT_0_PORT TLK_GPIO_PORT_A
#define PINMUX_GPIO_OUTPUT_0_PIN TLK_GPIO_PIN_0
#define PINMUX_GPIO_OUTPUT_COUNT 1
```

58.10 配置文件位置

Pinmux 配置文件

```
soc/<chip>/pinmux/
├── function.yaml          # Pin function definitions
├── pinmux.yaml            # Pin multiplexing configuration
├── pinmux_qfn48.yaml      # QFN48 package
├── pinmux_qfn88.yaml      # QFN88 package
└── ...
```

58.11 构建集成

Pinmux 配置深度集成到构建系统中：

- generate_pinmux CMake 目标在编译前自动执行
- 预构建检查自动检测 pinmux.h 是否需要更新
- 生成的 pinmux.h 放置在 build/ 目录中，并自动添加到包含路径

58.12 CI

CI 在所有示例中使用单一配置。这是可行的，因为未使用的分配会被跳过，因此您只需考虑此文件中所有示例的引脚使用情况。在构建之前，CI 将此文件复制到构建目录。由于构建设置会在覆盖过程中最后应用，因此最终配置与文件中描述的配置一致—所有先前的设置均被覆盖。

CHAPTER 59

Menuconfig 配置界面

59.1 概述

Menuconfig 是 UniSDK 的基于终端的交互式系统配置工具，提供可视化菜单导航以设置所有 Kconfig 配置选项。

主要特性：

- 双阶段配置：芯片配置和构建配置相互分离
- 终端图形界面，支持方向键导航
- 搜索功能（按 `/` 键）
- 帮助系统（按 `?` 键查看选项说明）
- 自动依赖管理（不可用的选项自动隐藏）
- 合并输出：自动将硬件和软件配置合并到 `.config`

59.2 启动方式

```
# Via Make
make config

# Via West (interactive)
west tl-config

# Via West (CMake only, skip interactive GUI)
west tl-config -c

# Direct invocation
python scripts/menuconfig.py Kconfig.chip     # Chip configuration
python scripts/menuconfig.py Kconfig.build    # Build configuration
```

备注

First-time Use `make config` will open the interface twice in sequence: first the chip configuration (Kconfig.chip), then the peripheral build configuration (Kconfig.build).

59.3 导航与操作

操作	按键
上/下移动	↑ ↓ 或 j k
进入子菜单	Enter 或 →
返回上级	ESC 或 ←
切换选项	Space
搜索	/
查看帮助	?
保存并退出	选择 < Save > 然后选择 < Exit >

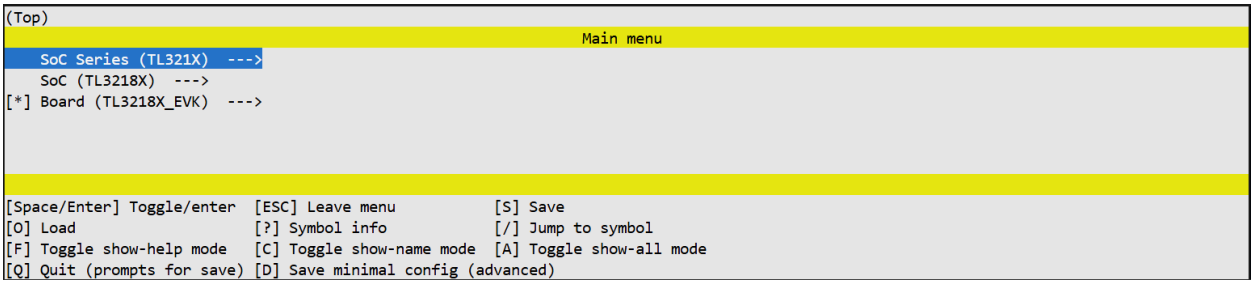


图 *1.* 芯片配置界面—选择目标 *SoC* 和开发板

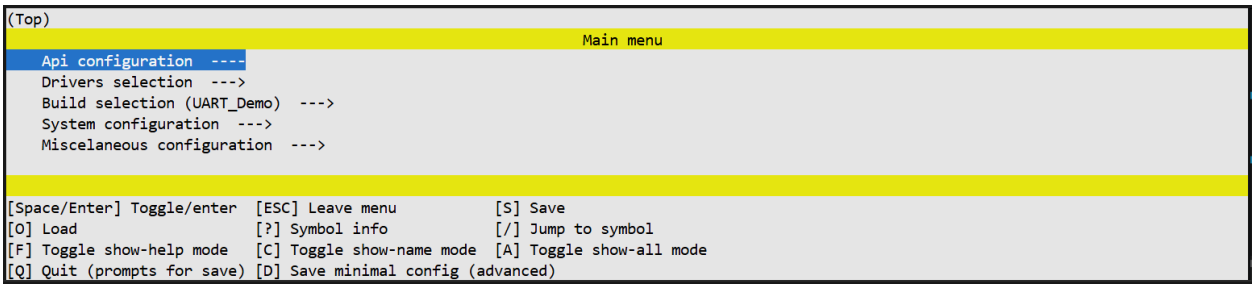


图 *2.* 构建配置界面—配置软件驱动和设置

59.4 双阶段配置

59.4.1 阶段 1：芯片配置（chip.config）

当工具首次打开时，您正在配置硬件。

- **您看到的内容：** 与 SoC 系列、SoC 和开发板相关的选项
- **操作：** 选择目标 SoC 并启用所需的芯片外设（GPIO 端口、UART 实例等）
- **退出：** 选择 < Save > 和 < Exit > 关闭窗口

59.4.2 阶段 2：构建配置（build.config）

第一个窗口关闭后，工具会再次打开以进行软件配置。

- **您看到的内容：**根据芯片能力（capabilities.h）筛选后的选项，包括 API 配置、驱动选择和功能开关
- **操作：**配置功能开关、参数、示例和调试选项
- **退出：**选择 < Save > 和 < Exit > 完成配置

59.5 结果

配置完成后，将生成以下文件：

生成的文件

```
build/
├── chip.config       # Chip configuration
├── build.config      # Build configuration
├── .config           # Final merged configuration
├── capabilities.h    # Chip capability macros
├── autoconf.h        # Configuration macro definitions
└── pinmux.h          # Pin multiplexing configuration
```

59.6 内部执行流程

59.6.1 阶段 1：芯片配置

1. 系统读取 Kconfig.chip —定义所有硬件相关选项
2. 用户在界面中选择设置
3. 输出保存到临时 chip.config 文件

59.6.2 阶段 2：预处理

在两个窗口之间，preprocess_kconfig.py 自动运行以准备构建配置阶段。

59.6.3 阶段 3：构建配置

1. 系统读取 Kconfig.build —定义软件和项目设置
2. 用户在界面中选择设置
3. 输出保存到临时 build.config 文件

59.6.4 阶段 4：合并

两个阶段合并为最终的 .config：

```
cat chip.config build.config > .config
```

此合并文件包含完整的系统配置（硬件和软件），供编译器和 Pinmux 工具使用。

59.7 常见问题

59.7.1 Windows PowerShell / VS Code 中方向键无法使用

- 改用字母键（j/k 上下移动）
- 使用 Windows 11
- 使用 CMD 替代 PowerShell

CHAPTER 60

安全

备注

Document Status: PLANNED —Content for this module has not yet been written

安全模块将涵盖 UniSDK 中与安全相关的功能，包括:

- 安全启动
- 固件签名和验证
- Flash 读写保护
- 调试接口安全
- 硬件加密引擎（AES / PKE / HASH / TRNG）

详细文档将在后续添加。

CHAPTER 61

生产指南

备注

Document Status: PLANNED —Content for this module has not yet been written

生产指南将帮助开发者将使用 UniSDK 开发的原型过渡到量产产品，涵盖以下内容：

- 量产烧录方案
- OTA（空中升级）固件更新
- 产线测试和校准
- 硬件设计参考
- 认证和合规

详细文档将在后续添加。

CHAPTER 62

示例与演示

UniSDK 在 `samples/` 目录中提供了丰富的即编即用示例程序。每个示例演示了一个或多个 SDK 功能的使用方法。

62.1 示例列表

62.1.1 基本外设

示例	描述	状态
GPIO 示例	GPIO 输入/输出、中断处理、睡眠 API	稳定
UART 示例	UART 回显、睡眠测试、发送/接收保护	稳定
PM 电源管理	低功耗模式、GPIO 唤醒、保持变量	稳定
PMP Sample	Physical Memory Protection driver demo	草稿
User Mode Sample	RISC-V User Mode / Machine Mode switching	草稿

62.1.2 通信接口

示例	描述	状态
ADC 示例	ADC 模数转换	草稿
DMA 示例	DMA 直接内存访问	草稿
I2C 示例	I2C 主从通信（DS1307 / Ping-Pong）	草稿
SPI 示例	SPI 主从通信	草稿
I2S Sample	I2S master/slave audio streaming	草稿

62.1.3 无线连接

示例	描述	状态
BLE 示例	Raw RF advertising and BLE Controller HCI demos	草稿
BLE Host Sample	BLE Host v2 peripheral with GATT services	草稿
RF Sample	RF driver ping-pong demo (BLE/Zigbee/Private/Hybee)	草稿

62.1.4 系统特性

示例	描述	状态
IRQ 嵌套	中断嵌套演示	草稿
定时器示例	STIMER 系统定时器演示	草稿
看门狗示例	看门狗定时器演示	草稿
USB CDC 示例	TinyUSB CDC 串口演示	草稿
Random Sample	Hardware random number generation	草稿
NV Storage Sample	Non-volatile storage device read/write/erase	草稿
Log Sample	Debug logging over the console/USB	草稿
Task Planner Sample	Cooperative loop, scheduler, and software timer execution models	草稿

62.1.5 Audio & Security

示例	描述	状态
Audio Sample	Audio input/output capture and playback via I2S/DMA	草稿
mbedTLS Sample	PSA Crypto random, ECDSA key generation, sign/verify	草稿
MCUboot Bootloader Sample	MCUboot bootloader with optional UART DFU recovery	草稿

62.2 构建示例

所有示例使用统一的构建方法:

```
# West command (recommended)
west tl-build samples/gpio_demo --board TLSR9528A_EVK
```

(续下页)

(接上页)

```
# Make command
make build APP=samples/gpio_demo BOARD=TLSR9528A_EVK
```

每个示例目录通常包含:

示例目录结构

```
samples/gpio_demo/
├── CMakeLists.txt    # Build entry
├── Kconfig           # Sample-specific configuration options
├── main.c            # Main program
├── main.cpp          # C++ version (optional)
└── README.md         # Sample description
```

CHAPTER 63

GPIO 示例

63.1 概述

gpio_demo 示例演示了 GPIO 驱动的基本用法，包括输入/输出操作、中断处理和睡眠 API。

63.2 功能演示

- 基本 GPIO 配置和引脚控制（tlk_gpio_configure、tlk_gpio_pin_write、tlk_gpio_pin_toggle）
- GPIO 中断使用和回调注册（需要启用 CONFIG_TLK_GPIO_DEMO_INTERRUPT）

63.3 工作原理

63.3.1 初始化

1. **LED0 配置**—配置为输出模式，在主循环中每 2 秒翻转一次
2. **中断配置（可选）**—当启用 CONFIG_TLK_GPIO_DEMO_INTERRUPT 时：
   - LED1 配置为输出（在中断回调中翻转）
   - KEY2 is configured as output and driven high, so it can supply a logic '1' to KEY0 when KEY0 is pressed
   - KEY0 配置为下拉输入
   - 注册 GPIO 中断回调（gpio_cb）
   - 设置上升沿触发
   - 启用全局中断

63.3.2 主循环

```
while (1)
{
    tlk_gpio_pin_toggle(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN);
    tlk_api_time_delay(TLK_SEC_TO_US(2));
    tlk_api_sleep(TLK_SEC_TO_MS(2));
}
```

备注

Board-Specific Macros The `PINMUX_LED_0_PORT` and `PINMUX_LED_0_PIN` macros (and their `LED_1`/`KEY_0`/`KEY_2` counterparts) are defined in the board-specific `tlk_board_pinout.h` header. These resolve to the correct port and pin for the target board's LEDs and keys.

63.4 配置选项

选项	描述
`CONFIG_TLK_GPIO_DEMO_INTERRUPT`	启用中断演示：KEY0 上升沿触发，LED1 在回调中翻转

63.5 构建与运行

```
west tl-build samples/gpio_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

源代码：`samples/gpio_demo/main.c`

CHAPTER 64

UART 示例

64.1 概述

uart_demo 示例演示了 UART 驱动的回显功能：接收数据并将其原样发送回去。

64.2 功能演示

- Manual UART configuration using tlk_uart_configure (when CONFIG_TLK_UART_DEMO_AUTO_CONFIG is disabled) —TX/RX pins are supplied directly in the struct tlk_uart_config passed to tlk_uart_configure
  - Manual UART flow control configuration using tlk_uart_flow_control_configure if CONFIG_TLK_UART0_FLOW_CONTROL is enabled
- 基于回调的数据接收（tlk_uart_receive_bytes）
- 数据传输（tlk_uart_send_bytes）
- 电源管理和睡眠防止（可选）

64.3 工作原理

64.3.1 初始化

1. **中断启用**—调用 tlk_core_interrupt_enable() 启用异步回调
2. **Manual Configuration** —If CONFIG_TLK_UART_DEMO_AUTO_CONFIG is disabled, manually configure the selected UART (CONFIG_TLK_UART_DEMO_UART) using tlk_uart_configure, with TX on port C pin 4 and RX on port C pin 5. If CONFIG_TLK_UART0_FLOW_CONTROL is enabled, also configure flow control using tlk_uart_flow_control_configure (RTS on port C pin 6, CTS on port C pin 7)

64.3.2 主循环（回显流程）

1. **电源管理测试**—如果启用了 CONFIG_TLK_UART_DEMO_SLEEP，先打印"Before sleep"，睡眠 2 秒，唤醒后打印"After sleep"
2. Call tlk_uart_receive_bytes to listen for data, using the rx_handler callback
3. Wait for rx_handler to set the rx_count variable to the number of bytes received
4. Send the received data back to the serial port (tlk_uart_send_bytes)
5. Reset rx_count and repeat

64.3.3 睡眠防止测试

When CONFIG_TLK_UART_DEMO_PREVENT is enabled, the sample attempts to sleep for 10 ms during reception (TLK_PM_SLEEP_MS(TLK_PM_SLEEP_MODE_SUSPEND, 10)) to verify that tlk_pm_sleep returns TLK_PM_SLEEP_DENIED.

64.4 配置选项

选项	描述
CONFIG_TLK_UART_DEMO_UART{i}	Selects which UART instance (UART0, UART1, ...) the sample uses
CONFIG_TLK_UART_DEMO_BUFFER_S	Size of the receive buffer (default 15)
CONFIG_TLK_UART_DEMO_SLEEP	测试唤醒后 UART 是否正常工作
CONFIG_TLK_UART_DEMO_PREVENT	测试接收期间是否正确防止了睡眠

备注

Auto-Configuration CONFIG_TLK_UART_DEMO_AUTO_CONFIG is not a user-selectable option in this sample's Kconfig —it is automatically set based on whether the selected UART's own TLK_UART{i}_AUTO_CONFIG option is enabled.

64.5 构建与运行

```
west tl-build samples/uart_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

CHAPTER 65

PM 电源管理示例

65.1 概述

pm_demo 示例演示了电源管理（PM）子系统的使用方法。它展示了如何进入低功耗模式、配置 GPIO 唤醒以及在低功耗转换过程中保持应用程序状态。

65.2 功能演示

- 使用 tlk_api_sleep 进入深度睡眠/挂起睡眠
- GPIO 唤醒配置（可选）
- 睡眠前/后钩子函数
- 深度睡眠保持模式下的变量保持

65.3 工作原理

65.3.1 初始化

1. **LED 状态初始化**—当启用 CONFIG_TLK_PM_DEMO_ENABLE_LED 时：
   - LED 状态变量存储在**保持区域**中
   - 初始状态：LED0 亮，LED1 灭
2. **钩子注册**—注册睡眠前和睡眠后回调：
   - flip_led() → 在睡眠前翻转 LED 状态
   - init_led() → 唤醒后重新初始化 GPIO

65.3.2 主循环

1. 延迟 1 秒
2. 可选配置 GPIO 唤醒源
3. 进入低功耗模式 1 秒

65.3.3 GPIO 唤醒（可选）

当启用 `CONFIG_TLK_PM_DEMO_ENABLE_GPIO_INPUT` 时：

- GPIO C4 被配置为唤醒源（高电平唤醒）
- 启用 GPIO 唤醒时使用挂起模式而非深度睡眠

65.4 配置选项

选项	描述
`CONFIG_TLK_PM_DEMO_ENABLE_LED`	LED 演示：状态保存在保持区域中
`CONFIG_TLK_PM_DEMO_ENABLE_GPIO_INPUT`	GPIO 唤醒演示
`CONFIG_TLK_PM_DEMO_DISABLE_LED_BEFORE_SLEEP`	睡眠前关闭 LED（可视的睡眠进入指示）

65.5 构建与运行

```
west tl-build samples/pm_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

CHAPTER 66

PMP 驱动示例

66.1 概述

一个示例应用程序，演示了 TLK PMP API 的使用。它将一个包含 16 个元素的数组的上半部分保护起来，禁止所有访问，然后尝试读写整个数组，从而在被保护区域触发 PMP 违规异常。

66.2 Features Demonstrated

- PMP region protection using TOR (Top-Of-Range) address matching (`tlk_pmp_configure_tor`)
- PMP region protection using NAPOT (Naturally Aligned Power-Of-Two) address matching (`tlk_pmp_configure_napot`)
- Locked PMP entries (`l = 1`) that deny all read/write/execute access
- PMP state retention across a sleep/wake cycle (optional)

66.3 工作原理

启动时，该示例：

1. *(*可选*)* 使用选定的地址匹配方案在 `demo_array[8..15]` 上配置 PMP 保护。所有权限都被禁用，并且条目被锁定（`l = 1`）。
2. *(*可选*)* 如果设置了 `CONFIG_TLK_PMP_DEMO_SLEEP`，则进入睡眠 500 毫秒并唤醒，演示 PMP 状态在睡眠过程中正确保存和恢复。
3. 遍历 `demo_array` 的所有 16 个元素：
   - 启用 `CONFIG_TLK_PMP_DEMO_READ` —读取并记录每个值。
   - 启用 `CONFIG_TLK_PMP_DEMO_WRITE` —写入并记录每个值。

元素 `[0..7]` 访问成功。当首次访问 `demo_array[8]` 时，会触发 PMP 违规异常。

66.4 配置

All options are exposed via **Kconfig** under *PMP Sample configuration*. The PMP mode and the access action are each modeled as a single-choice group (`choice`), so exactly one option in each group is active at a time; `TLK_PMP_DEMO_TOR` and `TLK_PMP_DEMO_READ` are the defaults.

符号	类型	默认值	描述
TLK_PMP	bool (choi	—	PMP mode: no protection is configured; all 16 elements are accessible without a fault.
TLK_PMP	bool (choi	y	PMP mode: protect `demo_array[8..15]` using TOR address matching. Entry 0 holds the lower bound; Entry 1 holds the upper bound.
TLK_PMP	bool (choi	n	PMP mode: protect `demo_array[8..15]` using NAPOT address matching.
TLK_PMP	bool (choi	y	Action mode: access the array by reading.
TLK_PMP	bool (choi	n	Action mode: access the array by writing.
TLK_PMP	bool	n	Enter sleep between PMP configuration and array access (only selectable when the PMP mode is not `OFF`). Demonstrates PMP state retention across sleep; selects `TLK_API_SLEEP` and implies `TLK_PMP_PM_DEVICE`.

> **Note:** With `TLK_PMP_DEMO_OFF` selected, no protection is configured and all 16 elements are accessible without a fault.

66.5 Build & Run

```
west tl-build samples/pmp_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/pmp_demo.bin
```

Source code: `samples/pmp_demo/main.c`

CHAPTER 67

U-mode 驱动示例

67.1 概述

A sample application demonstrating the TLK U-mode API. It switches the core from Machine mode (M-mode) to User mode (U-mode), executes a task under reduced privilege, optionally dispatches a service request back to M-mode via `ecall`, and optionally returns to M-mode permanently through `tlk_core_enable_mmode`.

67.2 Features Demonstrated

- Switching from M-mode to U-mode and back (`tlk_core_enable_umode`, `tlk_core_enable_mmode`)
- PMP-based privilege isolation: a NAPOT region granting full access to the 4 GB address space, and (optionally) a higher-priority region denying UART access from U-mode
- Environment calls (`ecall`) from U-mode to request a service performed in M-mode
- Illegal-instruction trap on a privileged CSR read attempted from U-mode

67.3 工作原理

启动时，该示例：

1. 通过 `tlk_trap_register_ecall_callback` 注册 `ecall_handler` 作为 ecall 异常回调。
2. 配置 PMP 条目 7，授予对整个 4 GB 地址空间的完全 RWX 访问权限—这是 U-mode 的基准权限。
3. *(*可选*)* 如果未设置 `CONFIG_TLK_UMODE_DEMO_ENABLE_UART`，则配置 PMP 条目 0 拒绝所有对 UART 外设区域的访问。由于 PMP 条目从最低索引开始检查，条目 0 在该范围内覆盖条目 7，阻止从 U-mode 的任何 UART 访问。
4. Calls `tlk_core_enable_umode(user_task, task_handler)` to switch to U-mode:

- **user_task** 在 U-mode 下执行，尝试通过 UART0 发送 "User task.\n"。仅在设置了 CONFIG_TLK_UMODE_DEMO_ENABLE_UART 时成功；否则，在首次访问 UART 寄存器时会触发 PMP 违规异常。如果设置了 CONFIG_TLK_UMODE_DEMO_ENABLE_SLEEP，user_task 还会在返回前设置 USER_REQUEST_SLEEP。
- user_task 返回后，**task_handler** 在 U-mode 下执行。该函数调用 ecall。
- ecall 导致环境调用异常，将控制权转移到 M-mode，在那里 **ecall_handler** 运行：
  - If USER_REQUEST_SLEEP is set, it logs "Ecall. Sleep request.\n", sleeps for 1 second via tlk_api_sleep, then clears the request.
  - If CONFIG_TLK_UMODE_DEMO_ENABLE_MMODE is set, it calls tlk_core_enable_mmode() to permanently restore M-mode privilege before returning.

5. After tlk_core_enable_umode returns, main performs a privilege check: it attempts to read the mstatus CSR. This read succeeds in M-mode and raises an illegal-instruction exception in U-mode. If CONFIG_TLK_UMODE_DEMO_ENABLE_MMODE is not set, ecall_handler never calls tlk_core_enable_mmode(), so execution returns to main still in U-mode and the read traps.

67.4 配置

All options are exposed via **Kconfig** under *UMODE Sample configuration*.

符号	类型	默认值	描述
CONFIG_T	bool	n	允许 U-mode 访问 UART 外设。禁用时，PMP 条目 0 被配置为拒绝所有对 UART 区域的访问，导致从 U-mode 尝试的任何 UART 访问触发 PMP 违规异常。
CONFIG_T	bool	n	启用睡眠服务请求。设置后，user_task 在返回前发出 USER_REQUEST_SLEEP 请求，使 ecall_handler 在 M-mode 下执行 1 秒睡眠。
CONFIG_T	bool	n	Return to M-mode after the ecall. When set, ecall_handler calls tlk_core_enable_mmode() so that execution continues in main with full M-mode privilege. When disabled, main remains in U-mode after tlk_core_enable_umode returns, and the subsequent mstatus CSR read raises an illegal-instruction exception.

注意：在所有选项都禁用的情况下，示例依次演示两种保护：当 user_task 访问被阻止的 UART 区域时触发 PMP 违规异常，以及当 main 尝试从 U-mode 读取特权 CSR 时触发非法指令异常。

67.5 Build & Run

```
west tl-build samples/umode_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/umode_demo.bin
```

Source code: samples/umode_demo/main.c

CHAPTER 68

ADC 示例

备注

Document Status: DRAFT —Content is being finalized

68.1 Overview

The `adc_demo` sample demonstrates the basic usage of the ADC driver: configuring a single-ended channel, sampling into a buffer, and logging the results in a loop. An optional configuration shows the ADC read loop running alongside the power-management sleep API.

68.2 Features Demonstrated

- ADC channel configuration (`tlk_adc_configure`) —channel, prescale, resolution, sample frequency, voltage reference, and sampling pin
- Buffered sampling (`tlk_adc_read`) with timeout/error handling via `enum tlk_adc_status`
- Optional DMA channel reservation for the ADC (`tlk_dma_chn_request`, guarded by `CONFIG_TLK_DMA`)
- Optional low-power sampling loop using `tlk_api_sleep` (requires `CONFIG_TLK_ADC_DEMO_PM`)

68.3 How It Works

68.3.1 Initialization

The sample builds a `struct tlk_adc_config` and passes it to `tlk_adc_configure`:

```
struct tlk_adc_config cfg = {
    .channel      = TLK_ADC_CHANNEL_0,
    .prescale     = TLK_ADC_PRESCALE_1F4,
    .resolution   = TLK_ADC_RESOLUTION_12,
    .sample_freq = TLK_ADC_SAMPLE_FREQ_96K,
    .vref         = TLK_ADC_VREF_1P2V,
    .sampling_port_pin =
        {
            .port = TLK_GPIO_PORT_B,
            .pin  = TLK_GPIO_PIN_0,
        },
    .is_differential = false,
    .differential_port_pin =
        {
            .port = TLK_GPIO_PORT_D,
            .pin  = TLK_GPIO_PIN_0,
        },
    .timeout_us = 10000,
#if TLK_IS_ENABLED(CONFIG_TLK_DMA)
    .dma_en       = false,
    .dma_channel = tlk_dma_chn_request(),
#endif
};

tlk_adc_configure(&cfg);
```

The channel samples on GPIO port B pin 0 in single-ended mode (`is_differential = false`); the differential pin is set but unused in that mode. When `CONFIG_TLK_DMA` is enabled, a DMA channel is reserved for the ADC configuration up front (though `dma_en` is left `false` in this sample).

68.3.2 Main Loop

```
while (1)
{
    enum tlk_adc_status status = tlk_adc_read(adc_values, ADC_BUFFER_SIZE);

    if (status == TLK_ADC_OK)
    {
        for (uint32_t i = 0; i < ADC_BUFFER_SIZE; i++)
        {
            TLK_LOG_INFO(adc_demo, "%u", adc_values[i]);
        }
    }
    else
    {
        TLK_LOG_ERROR(adc_demo, "Timeout");
    }

#if TLK_IS_ENABLED(CONFIG_TLK_ADC_DEMO_PM)
    tlk_api_sleep(TLK_SEC_TO_MS(1));
#else
    tlk_api_time_delay(TLK_SEC_TO_US(1));
#endif
}
```

Each iteration reads `ADC_BUFFER_SIZE` (8) samples into `adc_values` and logs every value on

success, or logs a timeout error otherwise. The loop then either sleeps for 1 second (PM build) or busy-delays for 1 second (default build).

68.4 Configuration Options

Option	Description
CONFIG_TLK_	Build the ADC demo with power management: uses tlk_api_sleep instead of a busy delay between reads, selects TLK_API_SLEEP and implies TLK_ADC_PM_DEVICE

The sample's base TLK_ADC_DEMO config selects TLK_ADC, TLK_DEBUG_LOG_CONSOLE, and TLK_API_TIME unconditionally.

68.5 构建与运行

```
west tl-build samples/adc_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

Source code: samples/adc_demo/main.c

CHAPTER 69

DMA 示例

备注

Document Status: DRAFT —Content is being finalized

69.1 Overview

The `dma_demo` sample demonstrates memory-to-memory transfers using the DMA driver: requesting and configuring a DMA channel, starting a transfer, aborting a transfer mid-flight, and completing a transfer —optionally driven by interrupt callbacks and/or combined with sleep/PM-prevent logic.

69.2 Features Demonstrated

- DMA channel request and configuration (`tlk_dma_chn_request`, `tlk_dma_chn_configure`)
- Memory-to-memory transfer setup, start, and abort (`tlk_dma_chn_transfer_configure`, `tlk_dma_chn_transfer_start`, `tlk_dma_chn_transfer_abort`)
- Optional DMA interrupt callback registration (`tlk_dma_chn_add_callback`, requires `CONFIG_TLK_DMA_DEMO_INTERRUPT`)
- Optional sleep between transfer cycles (requires `CONFIG_TLK_DMA_DEMO_SLEEP`)
- Optional PM-suspend-with-prevent-sleep pattern around an active transfer (requires `CONFIG_TLK_DMA_DEMO_PREVENT`)

69.3 How It Works

69.3.1 Initialization

```
demo_chn = tlk_dma_chn_request();
tlk_dma_chn_configure(demo_chn, &chn_config);

#if TLK_IS_ENABLED(CONFIG_TLK_DMA_DEMO_INTERRUPT)
tlk_dma_chn_add_callback(demo_chn, chn_handler);
#endif
```

`chn_config` is a `struct tlk_dma_config` configured for a normal-mode, byte-width, incrementing-address transfer on both source and destination, with a single-transfer burst size:

```
struct tlk_dma_config chn_config = {
    .dst_req_sel    = 0,
    .src_req_sel    = 0,
    .dst_addr_ctrl  = TLK_DMA_ADDR_INCREMENT,
    .src_addr_ctrl  = TLK_DMA_ADDR_INCREMENT,
    .dstmode        = TLK_DMA_NORMAL_MODE,
    .srcmode        = TLK_DMA_NORMAL_MODE,
    .dstwidth       = TLK_DMA_BYTE_WIDTH,
    .srcwidth       = TLK_DMA_BYTE_WIDTH,
    .src_burst_size = TLK_DMA_BURST_1_TRANSFER,
    .read_num_en    = 0,
    .priority       = 0,
    .write_num_en   = 0,
    .auto_en        = 0,
};
```

When `CONFIG_TLK_DMA_DEMO_INTERRUPT` is enabled, `chn_handler` sets a `dma_complete` or `dma_abort` flag depending on the IRQ type (`TLK_DMA_IRQ_TC` / `TLK_DMA_IRQ_ABT`), which the main loop then polls.

备注

Minimum Stack Size The sample enforces a minimum 2KB stack at compile time via `#if CONFIG_TLK_MEMORY_STACK_SIZE < 2` / `#error`.

69.3.2 Main Loop

Each iteration of the loop:

1. (Optional, `CONFIG_TLK_DMA_DEMO_PREVENT`) Starts a transfer of the full `src`/`dst` arrays (`ARRAY_LEN` = 300 bytes), then calls `TLK_PM_SLEEP_MS(TLK_PM_SLEEP_MODE_SUSPEND, 10)` —if the DMA transfer is still active, sleep entry is denied and the demo logs `"The sleep was prevented due to active receiving"`.
2. Fills `src` with incrementing values and clears `dst` for `TRANSFER_SIZE` (32) bytes, then logs both buffers.
3. Starts a `TRANSFER_SIZE`-byte transfer and immediately calls `tlk_dma_chn_transfer_abort`, demonstrating mid-transfer abort. With `CONFIG_TLK_DMA_DEMO_INTERRUPT` enabled, the loop spins on `dma_abort` before logging `"DMA IRQ: abort"`. Buffers are logged again afterward.

4. Clears `dst`, starts a fresh `TRANSFER_SIZE`-byte transfer and lets it run to completion. With interrupts enabled, the loop spins on `dma_complete` before logging `"DMA IRQ: complete"`. Buffers are logged again to show the completed copy.
5. Delays 5 seconds —via `tlk_api_sleep` (if `CONFIG_TLK_DMA_DEMO_SLEEP`) or `tlk_api_time_delay` otherwise —before repeating.

69.4 Configuration Options

Option	Description
`CONFIG_TLK_DMA_DE`	Use DMA interrupts (`TLK_PLIC`, `TLK_DMA_IRQ_HANDLER`) to detect transfer completion/abort instead of polling registers directly
`CONFIG_TLK_DMA_DE`	Use `tlk_api_sleep` for the end-of-loop delay; selects `TLK_API_SLEEP` and implies `TLK_DMA_PM_DEVICE`
`CONFIG_TLK_DMA_DE`	Demonstrate PM sleep prevention while a DMA transfer is active; selects `TLK_PM`, `TLK_PM_SUSPEND`, and `TLK_DMA_PREVENT_SLEEP`

The sample's base `TLK_DMA_DEMO` config selects `TLK_DMA`, `TLK_API_PRINT`, and `TLK_API_TIME` unconditionally.

69.5 构建与运行

```
west tl-build samples/dma_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

Source code: `samples/dma_demo/main.c`

CHAPTER 70

I2C 示例

备注

Document Status: DRAFT —Content is being finalized

70.1 Overview

The `i2c_demo` sample includes two independent sub-samples that both configure `tlk_i2c0` on GPIO port C pins 6/7 (SCL/SDA):

- **`ds1307`** —I2C master driving a DS1307 real-time-clock chip: periodically reads the current time and, on a button press, writes a new time.
- **`ping_pong`** —a minimal I2C master/slave loopback demo (role selected at build time) that exchanges a fixed 8-byte buffer and toggles LEDs on activity.

70.2 Features Demonstrated

- I2C pinmux and master configuration (`tlk_i2c_pinmux_configure`, `tlk_i2c_master_configure`)
- Single-buffer master writes/reads (`tlk_i2c_master_write`, `tlk_i2c_master_read`)
- Combined write+read transactions using `tlk_i2c_master_xfer` with a `struct tlk_i2c_message` array (DS1307 register-pointer + burst read pattern)
- I2C slave mode configuration and event-driven handler (`tlk_i2c_slave_configure`, `tlk_i2c_slave_set_handler`, `TLK_I2C_EVENT_WRITE_REQUEST` / `TLK_I2C_EVENT_READ_REQUEST` / `TLK_I2C_EVENT_STOP`)
- GPIO interrupt integration: the DS1307 sample sets the time from a button-press callback (`tlk_gpio_irq_add_callback`)

70.3 How It Works

70.3.1 ds1307 sub-sample

`main.c` configures the I2C pinmux and master mode, then arms a GPIO interrupt on `KEY_0` (rising edge) whose callback writes a new fixed time to the RTC via `ds1307_set_time`:

```
struct tlk_i2c_pinmux_config pinmux = {
    .scl_port_pin = {.port = TLK_GPIO_PORT_C, .pin = TLK_GPIO_PIN_6},
    .sda_port_pin = {.port = TLK_GPIO_PORT_C, .pin = TLK_GPIO_PIN_7},
};

tlk_i2c_pinmux_configure(dev, &pinmux);

struct tlk_i2c_master_config config = {
    .clock_speed = 100000,
    .stretch_en  = 1,
};

tlk_i2c_master_configure(dev, &config);
```

The main loop reads the current time every second via `ds1307_read_time` and logs it. The DS1307 driver (`ds1307.c`) implements the register protocol on top of the I2C API:

- `ds1307_set_time` writes an 8-byte buffer (start register `0x00` followed by BCD-encoded sec/min/hour/day/date/month/year) with a single `tlk_i2c_master_write`.
- `ds1307_read_time` issues a combined transaction with `tlk_i2c_master_xfer`: a 1-byte write of the register pointer followed by a 7-byte read, using two `struct tlk_i2c_message` entries (`is_read = 0` then `is_read = 1`).

70.3.2 ping_pong sub-sample

Depending on `CONFIG_TLK_I2C_DEMO_ROLE_MASTER` / `CONFIG_TLK_I2C_DEMO_ROLE_SLAVE`, `main.c` builds one of two roles against slave address `0x5a`:

- **Master**: repeatedly writes an 8-byte buffer (`"i2c_demo"`) with `tlk_i2c_master_write`, then reads it back with `tlk_i2c_master_read`, and logs whether the round-tripped data matches. Delays between operations via `tlk_api_time_delay`, or `tlk_api_sleep` if `CONFIG_TLK_I2C_DEMO_PM` is enabled.
- **Slave**: configures slave mode with `tlk_i2c_slave_configure`, registers an event handler with `tlk_i2c_slave_set_handler`, and sets its TX/RX buffers with `tlk_i2c_slave_set_tx` / `tlk_i2c_slave_set_rx`. The handler toggles LED0 on a write request, toggles LED1 and echoes the received buffer back into the TX buffer on a read request, and logs on a stop condition:

```
void handler(struct tlk_i2c_device* dev, enum tlk_i2c_event event)
{
    switch (event)
    {
    case TLK_I2C_EVENT_WRITE_REQUEST:
        tlk_gpio_pin_toggle(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN);
        break;
    case TLK_I2C_EVENT_READ_REQUEST:
        tlk_gpio_pin_toggle(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN);
        memcpy(tx_buffer, rx_buffer, BUFFER_SIZE);
```

(续下页)

(接上页)

```
        break;
    case TLK_I2C_EVENT_STOP:
        TLK_LOG_INFO(i2c_demo, "Stop received.");
        break;
    default:
        break;
    }
}
```

70.4 Configuration Options

70.4.1 ds1307

The TLK_I2C_DEMO config for this sub-sample selects TLK_I2C, TLK_I2C0, TLK_I2C0_MASTER, TLK_GPIO, and TLK_PLIC unconditionally (no sub-options exposed).

70.4.2 ping_pong

Option	Description
TLK_I2C_DEMO_RO	Build as I2C master (selects TLK_I2C0_MASTER) —choice option, default role
TLK_I2C_DEMO_RO	Build as I2C slave (selects TLK_I2C0_SLAVE) —choice option
CONFIG_TLK_I2C_	Master role only: use tlk_api_sleep for the end-of-loop delay; selects TLK_API_SLEEP and implies TLK_I2C0_PM_DEVICE

The base TLK_I2C_DEMO config selects TLK_I2C and TLK_I2C0 unconditionally; exactly one of the two role options must be selected (choice block).

70.5 构建与运行

```
# ds1307
west tl-build samples/i2c_demo/ds1307 --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin

# ping_pong
west tl-build samples/i2c_demo/ping_pong --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

Source code: samples/i2c_demo/ds1307/main.c, samples/i2c_demo/ds1307/ds1307.c, samples/i2c_demo/ping_pong/main.c

CHAPTER 71

SPI 示例

备注

Document Status: DRAFT —Content is being finalized

71.1 Overview

The `spi_demo` sample demonstrates SPI master/slave communication on `tlk_spi0`, using a command byte to select between a write and a read phase. The role (master or slave) is selected at build time.

71.2 Features Demonstrated

- SPI pinmux configuration (`tlk_spi_pinmux_configure`) for CS/SCK/MOSI/MISO
- SPI master configuration and command-based transfers (`tlk_spi_master_configure`, `tlk_spi_master_xfer` with `struct tlk_spi_xfer`)
- SPI slave configuration with a command handler (`tlk_spi_slave_configure`, `tlk_spi_slave_set_handler`, `tlk_spi_slave_read/tlk_spi_slave_write`)
- LED toggling to visualize transfer activity and success

71.3 How It Works

71.3.1 Initialization

Both roles share pin and mode setup:

```
struct tlk_spi_config config = {
    .mode    = TLK_SPI_MODE0,
    .io_mode = TLK_SPI_SINGLE_MODE,
    /* lower the frequency in case of errors on the slave side, if the master's div␣
↪allows it */
    .frequency = 2 * 1000 * 1000,
};

struct tlk_spi_pinmux pinmux = {
    .cs   = {.port = TLK_GPIO_PORT_E, .pin = TLK_GPIO_PIN_0},
    .sck  = {.port = TLK_GPIO_PORT_E, .pin = TLK_GPIO_PIN_1},
    .mosi = {.port = TLK_GPIO_PORT_E, .pin = TLK_GPIO_PIN_2},
    .miso = {.port = TLK_GPIO_PORT_E, .pin = TLK_GPIO_PIN_3},
};

tlk_gpio_configure(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN, TLK_GPIO_OUTPUT);
tlk_gpio_configure(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN, TLK_GPIO_OUTPUT);

tlk_core_interrupt_enable();

tlk_spi_pinmux_configure(dev, &pinmux);
```

SPI runs in mode 0, single-IO mode, at 2 MHz.

71.3.2 Master Role (CONFIG_TLK_SPI_DEMO_ROLE_MASTER)

The master issues two command-tagged transfers per loop using `struct tlk_spi_xfer` with `use_cmd = 1`:

```
xfer.type = TLK_SPI_DUMMY_TX;
xfer.cmd  = COMMAND_WRITE;
status = tlk_spi_master_xfer(dev, &xfer);
...
xfer.type = TLK_SPI_DUMMY_RX;
xfer.cmd  = COMMAND_READ;
status = tlk_spi_master_xfer(dev, &xfer);
```

`COMMAND_WRITE` (1) sends `tx_data` to the slave; `COMMAND_READ` (2) reads back into `rx_data`. LED0 toggles every cycle; LED1 toggles when the first byte of `tx_data` matches the first byte of `rx_data`, indicating a successful round trip. A 10 ms delay separates the write and read phases, and a 1 second delay separates loop iterations.

71.3.3 Slave Role (CONFIG_TLK_SPI_DEMO_ROLE_SLAVE)

The slave registers a command handler and then idles in an empty `while (1)` loop, relying entirely on the interrupt-driven handler:

```
void handler(struct tlk_spi_device* dev, tlk_spi_cmd_t cmd)
{
    tlk_gpio_pin_toggle(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN);

    switch (cmd)
    {
    case COMMAND_WRITE:
        tlk_spi_slave_read(dev, rx_data, BUFFER_SIZE);
        break;
```

(续下页)

(接上页)

```
    case COMMAND_READ:
        tlk_spi_slave_write(dev, tx_data, BUFFER_SIZE);
        break;
    default:
        break;
    }
}
```

On `COMMAND_WRITE` the slave reads the incoming bytes into `rx_data`; on `COMMAND_READ` it writes `tx_data` back to the master. LED0 toggles on every command received.

71.4 Configuration Options

Option	Description
`TLK_SPI_DEMO_ROLE_MASTER`	Build as SPI master —choice option
`TLK_SPI_DEMO_ROLE_SLAVE`	Build as SPI slave —choice option

The base `TLK_SPI_DEMO` config selects `TLK_SPI`, `TLK_SPI0`, and `TLK_API_TIME` unconditionally; exactly one role must be selected (`choice` block). `TLK_BLE_CONTROLLER` is explicitly disabled (`default n`) for this sample.

71.5 构建与运行

```
west tl-build samples/spi_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

Source code: `samples/spi_demo/main.c`

CHAPTER 72

I2S Sample

72.1 Overview

The `i2s_demo` sample demonstrates the I2S driver by continuously transmitting a fixed test pattern out over I2S while simultaneously receiving on the same bus, and using an LED to indicate whether the transmitted and received data match.

72.2 Features Demonstrated

- I2S pinmux configuration (`tlk_i2s_pinmux_configure`) for BCLK, ADC (RX) LR clock/data, and DAC (TX) LR clock/data pins
- I2S device configuration (`tlk_i2s_configure`) with 16-bit data width and I2S mode
- Master/Slave role selection (`CONFIG_TLK_I2S_DEMO_ROLE_MASTER` / `CONFIG_TLK_I2S_DEMO_ROLE_SLAVE`)
- Simultaneous DMA-driven output and input paths (`tlk_i2s_configure_output`, `tlk_i2s_input_configure`) using circular DMA linked-list nodes (`struct tlk_dma_ll_node`, self-referencing `next` for continuous looping)
- Starting the I2S device with `tlk_i2s_start`

72.3 How It Works

72.3.1 Initialization

1. **Pinmux** —BCLK, ADC LR clock/data, and DAC LR clock/data pins are assigned (`struct tlk_i2s_pinmux`).
2. **Role Selection** —At compile time, if `CONFIG_TLK_I2S_DEMO_ROLE_MASTER` is enabled, the device is configured as `TLK_I2S_MASTER` with an explicit sample-rate divider (`{8,`

625, 0, 64, 64}); if CONFIG_TLK_I2S_DEMO_ROLE_SLAVE is enabled, it is configured as TLK_I2S_SLAVE (sample rate is driven externally).

3. **Circular DMA Buffers** —rx_buff0 and tx_buff0 are each set up as a single-node circular linked list (next points back to itself), so RX and TX run continuously without needing new nodes queued.
4. **I2S Configuration** —tlk_i2s_configure, tlk_i2s_pinmux_configure, tlk_i2s_configure_output, and tlk_i2s_input_configure are called on the tlk_i2s0 device, wiring DMA channels from tlk_dma_chn_request() to FIFO0 on the left I2S channel for both TX and RX.
5. **Start** —tlk_i2s_start(dev) begins continuous transmit/receive.

72.3.2 Main Loop

```
while (1)
{
    // Mark the successful transmission with the enabled LED
    // I2S has a known issue that the rx buffer has 3 dummy elements at the beginning
    tlk_gpio_pin_write(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN, tx_buffer[0] == rx_
↪buffer[3]);
}
```

The loop continuously compares tx_buffer[0] (the first transmitted sample) against rx_buffer[3] (offset by 3 to account for a known driver quirk where the RX buffer starts with 3 dummy elements) and drives the board LED accordingly —the LED lights up once transmitted data is correctly looped back into the receive buffer.

备注

Loopback Wiring This sample transmits and receives at the same time on the same I2S device. To see the LED light up, the DAC output pins must be physically wired to the ADC input pins on the board (or an external I2S codec must loop the signal back).

72.4 Configuration Options

Option	Description
CONFIG_TLK_I2S_DEMO	Enables the sample; selects TLK_I2S and TLK_I2S0 (default y, not user-facing)
CONFIG_TLK_I2S_DEMO_R	Configure the I2S device as bus master, driving BCLK/LRCLK with an explicit sample-rate divider
CONFIG_TLK_I2S_DEMO_R	Configure the I2S device as bus slave, following an externally supplied clock

72.5 Build & Run

```
west tl-build samples/i2s_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/i2s_demo.bin
```

Source code: `samples/i2s_demo/main.c`

CHAPTER 73

BLE 示例

备注

Document Status: DRAFT —Content is being finalized

73.1 Overview

This page covers two low-level BLE radio/controller samples:

- **`samples/adv_demo`** —a minimal example that drives the RF driver directly to transmit raw BLE advertising packets, without using the BLE Controller stack.
- **`samples/ble_controller_demo`** —a full BLE Controller demo that exposes the standard HCI interface over a UART transport, for use with an external Host.

For the newer SAL-based BLE Host stack sample, see the *BLE Host Sample*.

73.2 Features Demonstrated

73.2.1 adv_demo

- Direct `tlk_rf_*` API usage to configure the radio for BLE (`tlk_rf_configure`)
- Manual construction of a raw BLE advertising PDU (flags, complete local name, appearance)
- Packet transmission via `tlk_rf_tx_prepare_packet` / `tlk_rf_start_stx`
- GPIO toggling to visually indicate each transmitted packet

73.2.2 ble_controller_demo

- BLE Controller stack initialization (`tlk_ble_controller_initial`)
- Standard HCI transport carried over UART1 (`tlk_uart_send_bytes` / `tlk_uart_receive_bytes`)
- HCI command/event dispatch via `tlk_ble_controller_hci_handler` and `tlk_ble_controller_hci_registerEventHandler`
- Kconfig-selectable BLE Controller capabilities: accept list, resolving list, legacy advertising/scanning, and ACL central/peripheral roles

73.3 How It Works

73.3.1 adv_demo

1. **RF Configuration** —`rf_configure()` sets up the radio in `TLK_RF_MODE_BLE` mode, 1M bitrate, channel 37, TX power 4, with the standard BLE advertising access code `0xd6be898e`.
2. **PDU Setup** —A fixed-content advertising PDU is prepared in `tx_buffer`: advertiser address `3c:cf:b4:03:e3:23`, AD flags, complete local name `"adv_demo"`, and an appearance field.
3. **Main Loop** —Repeatedly prepares and transmits the packet, toggles LED2 on each iteration, waits for the transmission to complete, then delays 100 ms before repeating:

```
while (1)
{
    tlk_rf_tx_prepare_packet(tx_packet, TX_PDU_SIZE);

    tlk_gpio_pin_toggle(PINMUX_LED_2_PORT, PINMUX_LED_2_PIN);

    tlk_rf_start_stx(tlk_rf_get_tick());

    while (!tlk_rf_instance.tx_end)
    {
    }

    tlk_sys_delay(TLK_MS_TO_US(TX_DELAY_MS));
}
```

73.3.2 ble_controller_demo

1. **UART Configuration** —Configures UART1 at 115200-8N1, with TX on GPIO port B pin 3 and RX on GPIO port B pin 2, used as the HCI transport.
2. **Controller Initialization** —Calls `tlk_ble_controller_initial()` and registers `app_ble_controller_hci_send_data` (via `tlk_ble_controller_hci_registerEventHandler`) so that outgoing HCI events are written back to the UART.
3. **Main Loop** —Receives HCI command bytes from the UART into `hci_rx_buffer` and, once a full receive completes, forwards them to `tlk_ble_controller_hci_handler`. The controller stack processes the command and emits HCI events through the registered callback, which sends them back out over the same UART.

This models a standard UART HCI transport for driving the Telink BLE Controller from an external Host implementation (e.g. a PC-based Host stack or a Host running on another MCU).

73.4 Configuration Options

`ble_controller_demo` exposes the BLE Controller stack sizing/feature options directly (there is no dedicated demo submenu):

Option	Description
`CONFIG_TLK_BLE_CONTROLLER_HCI_ACL_DATA`	Maximum HCI ACL data payload size (default `50`)
`CONFIG_TLK_BLE_CONTROLLER_HCI_ACL_BUFF`	Number of HCI ACL buffers (default `10`)
`CONFIG_TLK_BLE_CONTROLLER_ACCEPT_LIST_`	Enable the controller accept (white) list (default y)
`CONFIG_TLK_BLE_CONTROLLER_RESOLVING_LI`	Enable the resolving list for address resolution (default y)
`CONFIG_TLK_BLE_CONTROLLER_LEGACY_ADV_E`	Enable legacy advertising (default y)
`CONFIG_TLK_BLE_CONTROLLER_LEGACY_SCAN_`	Enable legacy scanning (default y)
`CONFIG_TLK_BLE_CONTROLLER_ACL_PERIPHEF`	Enable the ACL peripheral (slave) role (default y)
`CONFIG_TLK_BLE_CONTROLLER_ACL_PERIPHEF`	Maximum concurrent peripheral connections (default 1)
`CONFIG_TLK_BLE_CONTROLLER_ACL_CENTRAL_`	Enable the ACL central (master) role (default y)
`CONFIG_TLK_BLE_CONTROLLER_ACL_CENTRAL_`	Maximum concurrent central connections (default 1)
`CONFIG_TLK_BLE_CONTROLLER_ACL_RX_MAX_L`	Maximum ACL RX payload length (default 27)

`adv_demo` has no demo-specific Kconfig options; it only selects `TLK_RF` and `TLK_GPIO`.

73.5 构建与运行

```
# adv_demo — raw advertising over the RF driver
west tl-build samples/adv_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/adv_demo.bin
```

```
# ble_controller_demo — full BLE Controller with UART HCI transport
west tl-build samples/ble_controller_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/ble_controller_demo.bin
```

73.6 相关资源

- *BLE* 控制器文档
- *BLE Host Sample* —newer SAL-based BLE Host stack sample (`samples/ble_host_demo`)

Source code: `samples/adv_demo/main.c`, `samples/ble_controller_demo/main.c`

CHAPTER 74

BLE Host Sample

74.1 Overview

The `ble_host_demo` sample demonstrates the **BLE Host v2** stack: a newer host implementation built on a SAL (Service Abstraction Layer, `tlk_ble_host_sal.h`), distinct from the lower-level advertising and controller-only samples. It brings up a BLE peripheral that advertises, accepts multiple simultaneous ACL connections, registers the standard Core and Device Information Service (DIS) GATT groups, and re-starts advertising automatically as connections come and go.

See also: *BLE Sample* for the lower-level advertising/controller demos.

74.2 Features Demonstrated

- BLE Host v2 stack bring-up (`ble_host_v1_init`, `tlk_ble_host_loop`)
- Peripheral advertising setup with custom advertising and scan-response data (`ble_host_gap_adv_set_adv_ind_param`, `ble_host_gap_adv_start`)
- GATT service groups: Core group and Device Information Service (`blc_svc_addCoreGroup`, `blc_svc_addDisGroup`, `blc_svc_calculateDatabaseHash`, `blc_svc_setDeviceName`)
- ACL connection lifecycle callbacks (`ble_host_acl_conn_register_user_data` with `connected`/`disconnected` callbacks)
- Tracking up to 4 simultaneous peripheral-role ACL connections
- SMP (Security Manager Protocol) initialization for pairing (`ble_host_smp_initial` with `BLE_HOST_SMP_LEGACY_JUST_WORKS_INIT_PARAMS`, `ble_host_smp_store_init`)

74.3 How It Works

74.3.1 Advertising Data

Advertising and scan-response payloads are built from static LTV (length-type-value) structures:

```
static const struct ad_data_flags s_adv_flags = {
    .header.length                      = 0x02,
    .header.type                        = DT_FLAGS,
    .flags.le_limited_discoverable_mode = 1,
    .flags.br_edr_not_supported         = 1,
};

static const struct ad_data_complete_local_name_complete s_adv_complete_name = {
    .header.length = sizeof(BLE_PERIPHERAL_DEVICE_NAME),
    .header.type   = DT_COMPLETE_LOCAL_NAME,
    .name          = BLE_PERIPHERAL_DEVICE_NAME,
};
```

The device advertises as `"tlk_bluetooth_ble_per"`, including the flags and complete local name AD types in both the advertising data and the scan response.

74.3.2 Initialization (`ble_host_v2_peripheral_init`)

1. Sets the advertising interval to 150 (units per `ble_host_gap_adv_set_adv_ind_param`) along with the advertising/scan-response data.
2. Registers the Core GATT group and the Device Information Service group (`blc_svc_addCoreGroup`, `blc_svc_addDisGroup`), computes the GATT database hash, and sets the device name.
3. Resets the local peripheral-connection tracking table (`s_app_acl_peripheral_info`), up to `APP_BLE_ACL_PERIPHERAL_MAX_COUNT` (4) entries, all initialized to `BLE_CONN_HANDLE_INVALID`.
4. Registers `s_app_acl_callbacks` (connected/disconnected handlers) via `ble_host_acl_conn_register_user_data`.
5. Initializes SMP pairing with Legacy Just Works parameters and sets up SMP bond storage for up to 4 devices (`ble_host_smp_store_init(4, 0)`).

74.3.3 Connection Callbacks

- **`app_connected_callback`** —If the new connection is peripheral-role, records its connection handle in the first free slot and increments the connection count. As long as fewer than 4 peripheral connections are active, advertising is restarted (`ble_host_gap_adv_start`) so additional centrals can connect.
- **`app_disconnected_callback`** —If the disconnected connection was peripheral-role, frees its slot and decrements the count, then unconditionally restarts advertising.

74.3.4 Main Loop

```
int main(void)
{
    TLK_LOG_INFO(ble_host_v2_demo, "BLE HOST v2 starting...");

    ble_host_v1_init(NULL, NULL);
    tlk_core_interrupt_enable();

    ble_host_v2_peripheral_init();
    ble_host_v2_peripheral_start();

    // Main loop
    while (1)
    {
        tlk_ble_host_loop();
    }

    return 0;
}
```

`ble_host_v1_init` brings up the underlying host, interrupts are enabled, the peripheral role is initialized and advertising is started (`ble_host_v2_peripheral_start` calls `ble_host_gap_adv_start`), and the main loop repeatedly pumps the host's event loop via `tlk_ble_host_loop()`.

备注

SAL-Based Host Stack This sample's naming (`ble_host_v1_init`, log tag `ble_host_v2_demo`) reflects that it exercises the newer BLE Host v2 architecture through the SAL header `tlk_ble_host_sal.h`, layered on top of the v1 host init call. This is architecturally different from `samples/adv_demo` and `samples/ble_controller_demo`, which work directly against the advertising/controller APIs —see *BLE Sample*.

74.4 Configuration Options

Option	Description
`CONFIG_TLK_BLE_HOST`	Enables the sample; selects `TLK_BLE_HOST_V2` and `TLK_BLE_CONTROLLER` (default `y`, not user-facing)

备注

This sample has no user-selectable Kconfig choices beyond the auto-selected stack dependencies above —the peripheral behavior (advertising params, service groups, max connections) is fixed in `main.c`.

74.5 Build & Run

```
west tl-build samples/ble_host_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/ble_host_demo.bin
```

Source code: `samples/ble_host_demo/main.c`

CHAPTER 75

RF Sample

75.1 Overview

The `rf_demo` sample demonstrates the RF driver capabilities by implementing a ping-pong application. It requires two devices for communication:

- **Transmitter** —sends a **ping** packet to the **Receiver**, then listens for the **pong** response. To avoid deadlocking, it has a timeout to resend the **ping** if no **pong** is received.
- **Receiver** —waits indefinitely for a **ping** packet, then sends a **pong** response once received.

LED1 indicates whether the chip is on or off (mainly useful with PM enabled). LED2 is toggled each time the Receiver sends a **pong**, or once the Transmitter's receive attempt ends (regardless of the result).

75.2 Features Demonstrated

- RF configuration for BLE, Zigbee, Private, and Hybee modes (`tlk_rf_configure`)
- Multiple exchange APIs: Manual, STX/SRX, STR/SRT, BTX/BRX, PTX/PRX
- Transmitter and receiver roles (`CONFIG_TLK_RF_DEMO_ROLE_TRANSMITTER` / `_RECEIVER`)
- IRQ-driven and polling-mode packet handling
- Optional pipe switching and SB packet format (Private mode, PTX/PRX only)
- Optional power management integration (`CONFIG_TLK_RF_DEMO_PM`)

75.3 How It Works

75.3.1 Initialization

```
int main(void)
{
    tlk_core_interrupt_enable();

    tlk_gpio_configure(PINMUX_LED_2_PORT, PINMUX_LED_2_PIN, TLK_GPIO_OUTPUT);
    tlk_gpio_configure(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN, TLK_GPIO_OUTPUT);

    tlk_gpio_pin_write(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN, 1);

    rf_configure(CONFIG_TLK_RF_DEMO_MODE, CONFIG_TLK_RF_DEMO_BITRATE);

#if TLK_IS_ENABLED(CONFIG_TLK_RF_DEMO_ROLE_TRANSMITTER)
    transmitter_loop();
#elif TLK_IS_ENABLED(CONFIG_TLK_RF_DEMO_ROLE_RECEIVER)
    receiver_loop();
#endif

    return 0;
}
```

`rf_configure()` builds a `struct tlk_rf_config` based on the selected mode (BLE, Zigbee, Private, Hybee), setting the RF channel, handling mode (IRQ or polling, per `CONFIG_TLK_RF_DEMO_POLLING_MODE`), and mode-specific parameters such as the BLE/Private access code and, for Private mode, the preamble length (derived from the selected bitrate) and optional SB packet format/length.

75.3.2 Exchange APIs

`main.h/main.c` are shared across five `demo_*.c` files, each compiled only when its matching `CONFIG_TLK_RF_DEMO_EXCHANGE_*` option is selected, and each implementing `transmitter_loop()/receiver_loop()`:

- **Manual** (`demo_manual.c`) —Drives the RF state machine directly with `tlk_rf_set_tx_state/tlk_rf_start_tx_state`, polling `tlk_rf_instance.tx_end/rx_end`. Also supports optional mode switching between two configured modes each iteration, and BLE channel switching with fast-settle calibration.
- **STX & SRX** (`demo_stx_srx.c`) —Uses `tlk_rf_start_stx()` to transmit and `tlk_rf_start_srx()` to receive, as two separate blocking calls.
- **STR & SRT** (`demo_str_srt.c`) —Uses `tlk_rf_start_stx2rx()` on the transmitter (send then auto-switch to receive) and `tlk_rf_start_srx2tx()` on the receiver (receive then auto-switch to send).
- **BTX & BRX** (`demo_btx_brx.c`) —Uses `tlk_rf_start_btx()` / `tlk_rf_start_brx()`, the "block" variant of send-then-receive / receive-then-send.
- **PTX & PRX** (`demo_ptx_prx.c`) —Uses `tlk_rf_start_ptx()` / (implicit PRX receive), and is the only mode that supports pipe switching (`CONFIG_TLK_RF_DEMO_PIPE_SWITCHING`) across `UNISDK_RF_PIPE_COUNT` logical pipes, each with its own access code, and the SB packet format (`CONFIG_TLK_RF_DEMO_SB_FORMAT`).

Each variant times the round trip via `tlk_api_time_get_micros()` and reports it, together with RSSI, through `on_pong_received()`.

75.3.3 Callbacks

```
void on_pong_received(void)
{
    ...
    if (ping >= TLK_MS_TO_US(PING_TIMEOUT_MS) || tlk_rf_instance.rx_timeout_error)
    {
        TLK_LOG_INFO(rf_demo, "Error: Ping timeout.");
    }
    else if (tx_payload[0] == rx_payload[0])
    {
        TLK_LOG_INFO(rf_demo, "[%u] Received!", rx_payload[0]);
        TLK_LOG_INFO(rf_demo, "Time: %-4u us | RSSI: %i", ping, tlk_rf_instance.rx_
↪rssi);
        success_packets++;
    }
    else
    {
        TLK_LOG_INFO(rf_demo, "Error: Diff in packets. Expected [%u], but got [%u].",␣
↪tx_payload[0], rx_payload[0]);
    }

    TLK_LOG_INFO(rf_demo, "Count: %-6u | Success count: %u", sent_packets, success_
↪packets);
    tlk_gpio_pin_toggle(PINMUX_LED_2_PORT, PINMUX_LED_2_PIN);
}
```

`on_ping_sent()`, `on_ping_received()`, `on_pong_sent()`, and `on_pong_received()` log each stage of the exchange; `on_pong_sent()` and `on_pong_received()` also toggle LED2.

备注

Power management When `CONFIG_TLK_RF_DEMO_PM` is enabled, most exchange variants replace the fixed post-exchange delay with `tlk_api_sleep(1000 + 500)` (a 1.5 s low-power sleep, preceded by a short visible delay), and the sample implies `TLK_RF_PM_DEVICE`.

75.4 Configuration Options

Option	Description
CONFIG_TLK_RF_DEMO_PM	Enable PM integration: sleeps between exchange iterations instead of delaying; selects TLK_API_SLEEP and implies TLK_RF_PM_DEVICE.
CONFIG_TLK_RF_DEMO_MODE_{BI ZIGBEE,PRIVATE,HYBEE}	Selects the RF protocol mode (choice).
CONFIG_TLK_RF_DEMO_BITRATE_ 250K,500K,1M,2M}	Selects the bitrate (choice); available options depend on the selected mode.
CONFIG_TLK_RF_DEMO_ROLE_{TF RECEIVER}	Selects the device role (choice).
CONFIG_TLK_RF_DEMO_EXCHANGI STX_SRX,STR_SRT,BTX_BRX, PTX_PRX}	Selects the exchange API used for the ping-pong loop (choice).
CONFIG_TLK_RF_DEMO_POLLING_	(Manual exchange only) Use polling mode instead of IRQ-driven handling.
CONFIG_TLK_RF_DEMO_MODE_SWI	(Manual exchange only) Switch between the primary mode and a second configured mode each iteration.
CONFIG_TLK_RF_DEMO_SECOND_M / CONFIG_TLK_RF_DEMO_SECOND_	(Manual + mode switching only) Mode/bitrate of the secondary configuration used when switching.
CONFIG_TLK_RF_DEMO_CHANNEL_	(Manual exchange, BLE mode, not combined with mode switching) Alternate between BLE channels 17 and 25 each iteration, using fast-settle calibration.
CONFIG_TLK_RF_DEMO_SB_FORMA	(Private mode, PTX/PRX exchange) Use the SB packet format.
CONFIG_TLK_RF_DEMO_PIPE_SWI	(Private mode, PTX/PRX exchange) Cycle transmissions across the device's logical pipes, each with a distinct access code.

75.5 Build & Run

```
west tl-build samples/rf_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/rf_demo.bin
```

备注

Two boards are required to observe the ping-pong exchange: flash one with CONFIG_TLK_RF_DEMO_ROLE_TRANSMITTER and the other with CONFIG_TLK_RF_DEMO_ROLE_RECEIVER, using matching mode/bitrate/exchange-API configuration.

Source code: samples/rf_demo/main.c

CHAPTER 76

IRQ 嵌套示例

备注

Document Status: DRAFT —Content is being finalized

76.1 Overview

The `irq_demo` sample (source directory `samples/irq_demo`) demonstrates interrupt management, prioritization, and nesting using the PLIC (Platform-Level Interrupt Controller), Machine Software Interrupts (PLIC SW / MSI), and Machine Timer Interrupts (MTIMER / MTI).

备注

Source directory name The sample's source directory is `samples/irq_demo` (project name `IRQ_Demo`), not `irq_nesting_demo`. Build and download paths below use the actual directory name.

76.2 Features Demonstrated

- PLIC software interrupt (MSI) setup and callback registration (`tlk_plic_sw_register_callback`)
- Machine timer interrupt (MTI) setup and callback registration (`tlk_mtimer_irq_register_callback`)
- PLIC external interrupt (MEI) priority configuration across three tiers (`tlk_plic_set_priority`)
- Optional nested/preemptive interrupt handling via `tlk_plic_enable_preempt()`
- Cross-triggering of interrupts from within handlers to visualize nesting depth

76.3 How It Works

76.3.1 Initialization

1. **MSI Configuration** —Enables the PLIC_SW subsystem and registers `tlk_sample_msi_handler` as its callback.
2. **MTI Configuration** —Registers `tlk_sample_mti_handler` as the machine timer interrupt callback.
3. **MEI Configuration** —Sets PLIC priorities for three sources:
   - `TLK_SAMPLE_PLIC_SRC_LOW` (vector 12) → `TLK_IRQ_PRI_LEV1`
   - `TLK_SAMPLE_PLIC_SRC_HIGH` (vector 13) → `TLK_IRQ_PRI_LEV2`
   - `UNISDK_PLIC_IRQ_NUM_UART0` → `TLK_IRQ_PRI_LEV3` (used for debug logging output)
4. **Preemption** —If both `CONFIG_TLK_IRQ_DEMO_IRQ_BASE_PLIC` and `CONFIG_TLK_IRQ_DEMO_PLIC_PREEMPT` are enabled, `tlk_plic_enable_preempt()` is called to allow nested PLIC interrupt execution.
5. **Global Enable** —Enables the MSIE, MTIE and MEIE bits in `mie`, then calls `tlk_core_interrupt_enable()`.

76.3.2 Cascading Nesting Cycle

Based on the `TLK_IRQ_DEMO_IRQ_BASE` choice, `main()` triggers one of the primary interrupt sources to kick off the cascade:

```
#if TLK_IS_ENABLED(CONFIG_TLK_IRQ_DEMO_IRQ_BASE_PLIC_SW)
    tlk_plic_sw_set_pending();
#elif TLK_IS_ENABLED(CONFIG_TLK_IRQ_DEMO_IRQ_BASE_MTIMER)
    tlk_mtimer_set_mtime_compare(tlk_mtimer_get_mtime());
#elif TLK_IS_ENABLED(CONFIG_TLK_IRQ_DEMO_IRQ_BASE_PLIC)
    tlk_plic_set_pending(TLK_SAMPLE_PLIC_SRC_LOW);
#endif
```

Once inside a handler, it purposely cross-triggers the other interrupt sources by setting their pending registers (`tlk_plic_set_pending`, `tlk_plic_sw_set_pending`, or writing `mtimecmp`). To make the nesting visible over UART, each handler:

- increments a shared `tlk_log_depth` counter and indents its log output accordingly,
- logs the current `mie` bits via `tlk_sample_log_mie()`,
- uses a local `static volatile uint8_t processed` guard (in the PLIC handlers) to avoid re-triggering itself indefinitely, and
- decrements `tlk_log_depth` before returning.

The PLIC low/high handlers additionally call `tlk_api_time_delay(100)` before returning, giving the newly-pended higher/lower-priority interrupt time to become pending before the current handler exits.

76.4 Configuration Options

All options are exposed via **Kconfig** under *IRQ Sample configuration*.

Option	Description
CONFIG_TLK_IRQ_DEMO_	Use the PLIC software interrupt (MSI) as the cascade's starting trigger (default choice)
CONFIG_TLK_IRQ_DEMO_	Use the machine timer interrupt (MTI) as the cascade's starting trigger
CONFIG_TLK_IRQ_DEMO_	Use the PLIC external low-priority interrupt (MEI) as the cascade's starting trigger
CONFIG_TLK_IRQ_DEMO_	Enable PLIC preemption (nested execution) of external interrupts; only selectable when TLK_IRQ_DEMO_IRQ_BASE_PLIC is chosen

TLK_IRQ_DEMO_IRQ_BASE_PLIC_SW, _MTIMER, and _PLIC form a single-choice group (exactly one is active at a time); PLIC_SW is the default.

76.5 构建与运行

```
west tl-build samples/irq_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/irq_demo.bin
```

Source code: samples/irq_demo/main.c

CHAPTER 77

STIMER 示例

备注

Document Status: DRAFT —Content is being finalized

77.1 Overview

The `stimer_demo` sample demonstrates the usage of the System Timer (STIMER) driver on Telink RISC-V MCUs: timer initialization, interrupt configuration, periodic callbacks, and integration with GPIO and the sleep API.

77.2 Features Demonstrated

- System timer configuration and start (`tlk_stimer_enable`, `tlk_stimer_set_tick`, `tlk_stimer_set_auto_mode`)
- Timer interrupt setup and callback registration (`tlk_stimer_register_callback`)
- Periodic interrupt generation using capture-compare (`tlk_stimer_irq_set_capture`)
- GPIO control from both the main loop and interrupt context
- Low-power sleep usage in the main loop while the timer interrupt continues in the background

77.3 How It Works

77.3.1 Initialization

1. **GPIO Configuration** —LED0 and LED1 are both configured as outputs. LED0 is toggled in the main loop; LED1 is toggled inside the timer interrupt callback.

2. **System Timer Setup** —The timer is enabled (`tlk_stimer_enable()`), its tick counter reset to 0 (`tlk_stimer_set_tick(0)`), and auto mode enabled (`tlk_stimer_set_auto_mode(true)`) for continuous free-running operation.
3. **Interrupt Configuration** —A callback (`timer_cb`) is registered via `tlk_stimer_register_callback()`. The first capture-compare event is scheduled 500 ms out, the STIMER IRQ mask is enabled, the STIMER PLIC interrupt is enabled, and global interrupts are enabled with `tlk_core_interrupt_enable()`.

```
tlk_stimer_enable();
tlk_stimer_set_tick(0);
tlk_stimer_set_auto_mode(true);
tlk_stimer_register_callback(timer_cb);

tlk_stimer_irq_set_capture(tlk_stimer_get_tick() + TLK_SYSTEM_TIMER_TICK_1MS * 500);
tlk_stimer_irq_set_mask(TLK_STIMER_IRQ_MASK_TIMER_IRQ_EN);
tlk_plic_interrupt_enable(UNISDK_PLIC_IRQ_NUM_STIMER);

tlk_core_interrupt_enable();
```

77.3.2 Main Loop

The main loop toggles LED0 and sleeps for 1 second between toggles:

```
while (1)
{
    tlk_gpio_pin_toggle(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN);
    tlk_api_sleep(TLK_SEC_TO_MS(1));
}
```

77.3.3 Timer Callback

`timer_cb()` runs on every STIMER interrupt (every 500 ms): it reschedules the next capture compare event, clears the interrupt status, and toggles LED1.

```
void timer_cb(void)
{
    tlk_stimer_irq_set_capture(tlk_stimer_get_tick() + TLK_SYSTEM_TIMER_TICK_1MS *␣
↪500);
    tlk_stimer_irq_clr_status(TLK_STIMER_IRQ_STATUS_TIMER_IRQ);
    tlk_gpio_pin_toggle(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN);
}
```

Because the main loop toggles LED0 once per second while the timer interrupt toggles LED1 twice per second (500 ms period), LED0 and LED1 blink at visibly different rates, demonstrating that the STIMER interrupt keeps firing independently of the main loop's sleep/wake cycle.

77.4 Configuration Options

Option	Description
CONFIG_TLK_	Enable the STIMER interrupt demo: registers `timer_cb`, schedules periodic capture-compare events, and toggles LED1 from interrupt context. Enabled by default (`def_bool y`) and selects `TLK_SYSTEM_TIMER_IRQ_ENABLE`.

77.5 构建与运行

```
west tl-build samples/stimer_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/stimer_demo.bin
```

Source code: `samples/stimer_demo/main.c`

CHAPTER 78

看门狗示例

备注

Document Status: DRAFT —Content is being finalized

78.1 Overview

The `wdt_demo` sample demonstrates the usage of the Watchdog Timer (WDT) driver, including detecting a watchdog-triggered reboot, feeding the watchdog to pass its window, and letting it expire to force a reset.

78.2 Features Demonstrated

- Detecting the previous reboot's cause via `tlk_sys_get_reboot_reason` (`TLK_SYS_REBOOT_REASON_WDT`)
- Starting the watchdog with a timeout (`tlk_wdt_start`) and feeding it (`tlk_wdt_feed`)
- Optional GPIO indication of demo progress (LED0/LED1)
- Optional low-power sleep while the watchdog is running (`CONFIG_TLK_WDT_DEMO_PM`)
- Optional watchdog auto-start from startup code (`CONFIG_TLK_WDT_DEMO_STARTUP`)

78.3 How It Works

On each boot:

1. **Reboot Reason Check** —Reads `tlk_sys_get_reboot_reason()` and logs whether the reset was caused by the watchdog or by another source.

2. **GPIO Indication (optional)** —If `CONFIG_TLK_WDT_DEMO_GPIO_INDICATION` is enabled, LED0 is configured as output and driven high, and LED1 is configured as output and driven low.
3. **Initial Delay** —Blocks for 2 seconds (`tlk_api_time_delay`).
4. **Watchdog Start** —If `CONFIG_TLK_WDT_DEMO_STARTUP` is enabled, the watchdog is assumed to already be running (started from startup code via `TLK_WDT_STARTUP_ENABLE`), so the sample just feeds it (`tlk_wdt_feed()`). Otherwise it starts the watchdog itself with a 5-second timeout (`tlk_wdt_start(TLK_SEC_TO_MS(5))`).
5. **Pass the WDT Window** —Waits 2 seconds, either via blocking delay (`tlk_api_time_delay`) or, if `CONFIG_TLK_WDT_DEMO_PM` is enabled, via low-power sleep (`tlk_api_sleep`) —demonstrating that the watchdog configuration survives a sleep cycle.
6. **LED1 On (optional)** —If GPIO indication is enabled, LED1 is driven high to show that the WDT window was passed successfully without a reset.
7. **Final Feed & Expire** —Feeds the watchdog one last time, then enters an infinite empty loop. Because nothing feeds the watchdog again, it eventually expires and resets the chip, and on the next boot `tlk_sys_get_reboot_reason()` reports `TLK_SYS_REBOOT_REASON_WDT`.

```
// Feed the WDT and wait until it fires.
tlk_wdt_feed();

while (1)
{
}
```

78.4 Configuration Options

All options are exposed via **Kconfig** under *WDT Sample configuration*.

Option	Description
`CONFIG_TLK_WDT_DEMO_`	Enable LED0/LED1 GPIO indication of demo progress (selects `TLK_GPIO`)
`CONFIG_TLK_WDT_DEMO_`	Use `tlk_api_sleep` instead of a blocking delay to pass the WDT window, exercising PM (selects `TLK_API_SLEEP`, `TLK_PM`)
`CONFIG_TLK_WDT_DEMO_`	Start the watchdog from startup code instead of from `main()` (selects `TLK_WDT_STARTUP_ENABLE`)

78.5 构建与运行

```
west tl-build samples/wdt_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/wdt_demo.bin
```

Source code: `samples/wdt_demo/main.c`

CHAPTER 79

USB CDC 示例

备注

Document Status: DRAFT —Content is being finalized

79.1 Overview

The USB CDC demo, built on the TinyUSB framework, demonstrates a USB CDC (virtual serial port) device that echoes back any data received from the host.

备注

Source directory name The sample currently lives at `samples/tinyusb_demos/usb_device/cdc_demo` (project name `USB_Demo`), not `samples/tinyusb_demos/usb_cdc_demo`. Build and download paths below use the actual directory.

79.2 Features Demonstrated

- TinyUSB device stack initialization (`tud_init`) and task servicing (`tud_task`)
- USB device/configuration/string descriptor callbacks for a single CDC interface (`tud_descriptor_device_cb`, `tud_descriptor_configuration_cb`, `tud_descriptor_string_cb`)
- CDC data echo using `tud_cdc_available`, `tud_cdc_read`, `tud_cdc_write`, `tud_cdc_write_flush`
- Device lifecycle callbacks: `tud_mount_cb`, `tud_umount_cb`, `tud_suspend_cb`, `tud_resume_cb`
- CDC line coding/state callbacks: `tud_cdc_line_coding_cb`, `tud_cdc_line_state_cb`

79.3 How It Works

79.3.1 Descriptors

The device descriptor uses the Miscellaneous class (`TUSB_CLASS_MISC`) with the Interface Association Descriptor subclass/protocol, which is the recommended setup for CDC so host drivers (Windows/Linux) enumerate it correctly. Vendor ID, product ID, device version, manufacturer, product, serial number, and interface name strings are all pulled from Kconfig (`CONFIG_TLK_USB_VENDOR_ID`, `CONFIG_TLK_USB_PRODUCT_ID`, `CONFIG_TLK_USB_DEVICE_VERSION`, `CONFIG_TLK_USB_DEVICE_MANUFACTURER`, `CONFIG_TLK_USB_DEVICE_PRODUCT_NAME`, `CONFIG_TLK_USB_DEVICE_SERIAL_NUMBER`, `CONFIG_TLK_USB_DEVICE_INTERFACE_NAME`). The single CDC interface's notification/OUT/IN endpoint numbers come from `CONFIG_TLK_USB_CDC_NOTIF_EP`, `CONFIG_TLK_USB_CDC_OUT_EP`, and `CONFIG_TLK_USB_CDC_IN_EP`.

79.3.2 Main Loop

```
int main(void)
{
    tud_init(BOARD_TUD_RHPORT); // Initialize USB Device Stack

    while (1)
    {
        tud_task(); // Device task: Handle USB events (MUST be called frequently)
        cdc_task(); // Application task: Handle Serial data
    }

    return 0;
}
```

79.3.3 Echo Task

`cdc_task()` checks that a host is connected and that RX data is available, reads up to 64 bytes from the CDC RX FIFO, and immediately writes the same bytes back out, flushing to force transmission without waiting for the buffer to fill:

```
void cdc_task(void)
{
    if (tud_cdc_connected())
    {
        if (tud_cdc_available())
        {
            uint8_t buf[64];
            uint32_t count = tud_cdc_read(buf, sizeof(buf));

            tud_cdc_write(buf, count);
            tud_cdc_write_flush();
        }
    }
}
```

79.4 Configuration Options

Option	Description
CONFIG_TLK_USB_CDC_DEMO	Enables the sample (selects TLK_USB); enabled by default

The device/interface identity (vendor/product IDs, strings) and endpoint numbers are configured via the shared TLK_USB Kconfig options referenced above rather than demo-specific symbols.

79.5 构建与运行

```
west tl-build samples/tinyusb_demos/usb_device/cdc_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/cdc_demo.bin
```

Source code: samples/tinyusb_demos/usb_device/cdc_demo/main.c

CHAPTER 80

Random Sample

80.1 Overview

The `random_demo` sample demonstrates the usage of the TLK Random API. It generates a configurable sequence of random numbers and logs them to the console in a loop.

80.2 Features Demonstrated

- Software random number generation (`tlk_random()`)
- Optional hardware TERO-based random source (`tlk_random_tero()`, TL721X only)
- Delay vs. sleep between iterations (`tlk_api_delay` / `tlk_api_sleep`)

80.3 How It Works

80.3.1 Main Loop

On every loop iteration the sample generates `CONFIG_TLK_RANDOM_DEMO_NUMBERS_COUNT` random values in the range `[0, CONFIG_TLK_RANDOM_DEMO_MAX_NUMBER]` using `tlk_random()` and logs each one:

```
int main(void)
{
    while (1)
    {
        TLK_LOG_INFO(random_demo, "Random numbers:\n");

        for (uint8_t i = 0; i < CONFIG_TLK_RANDOM_DEMO_NUMBERS_COUNT; i++)
        {
            TLK_LOG_INFO(random_demo,
                         "\t%u: %u\n",
```

(续下页)

(接上页)

```
                        (i + 1),
                        tlk_random() % (CONFIG_TLK_RANDOM_DEMO_MAX_NUMBER + 1));
        }

        TLK_LOG_INFO(random_demo, "\n");

#ifndef CONFIG_TLK_RANDOM_DEMO_SLEEP
        tlk_api_delay(TLK_SEC_TO_US(4));
#else
        tlk_api_sleep(TLK_SEC_TO_MS(4));
#endif
    }
}
```

Between iterations, the sample either busy-delays for 4 seconds via `tlk_api_delay`, or — if `CONFIG_TLK_RANDOM_DEMO_SLEEP` is enabled —enters low-power sleep for 4 seconds via `tlk_api_sleep`.

备注

TERO Module (TL721X only) The sample can optionally repeat the same sequence using `tlk_random_tero()`, a hardware TERO-based random source available only on **TL721X**. It is disabled by default; to enable it, edit `main.c` and set `#define USE_TERO_MODULE 1`. This is a source-level toggle, not a Kconfig option.

80.4 Configuration Options

Option	Type	Default	Description
CONFIG_TLK_RANDOM_[	int	10	How many numbers to generate per iteration. Range: 1–100.
CONFIG_TLK_RANDOM_[	int	10	Upper bound (inclusive) of the generated values.
CONFIG_TLK_RANDOM_[	bool	n	Use sleep instead of delay between iterations. Automatically selects TLK_API_SLEEP and implies TLK_RANDOM_PM.

`TLK_RANDOM_DEMO` itself auto-selects `TLK_RANDOM` and `TLK_API_TIME`, so no manual selection of those is required.

80.5 Build & Run

```
west tl-build samples/random_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/random_demo.bin
```

Source code: `samples/random_demo/main.c`

CHAPTER 81

NV Storage Sample

81.1 Overview

The `nv_storage_demo` sample demonstrates the non-volatile storage abstraction layer: enumerating the available storage devices and performing a read/erase/write/read round-trip against flash.

81.2 Features Demonstrated

- Storage device enumeration (`tlk_storage_get_next_device`)
- Locating the storage device backing a given address range (`tlk_storage_find_device`)
- Reading and writing raw flash data (`tlk_storage_read`, `tlk_storage_write`)
- Log output via `TLK_LOG_INFO` / `TLK_LOG_HEXDUMP_INFO`

81.3 How It Works

81.3.1 Device Enumeration

At startup, the sample walks the list of registered storage devices and logs each one's name, base address, size, page size, clean (erased) byte pattern, and erase/read/write capability flags:

```
for (struct tlk_storage_device* dev = tlk_storage_get_next_device(NULL); dev != NULL;
     dev                            = tlk_storage_get_next_device(dev))
{
    TLK_LOG_INFO(nv_storage_demo, "device: %s", dev->name);
    TLK_LOG_INFO(nv_storage_demo, "\tbase address: 0x%zx", dev->base);
    TLK_LOG_INFO(nv_storage_demo, "\tsize: %zu (0x%zx)", dev->size, dev->size);
    TLK_LOG_INFO(nv_storage_demo, "\tpage: %zu", dev->page);
```

(续下页)

(接上页)

```
    TLK_LOG_INFO(nv_storage_demo, "\tclean pattern: 0x%02x", dev->clean_byte);
    TLK_LOG_INFO(nv_storage_demo,
                 "\tcapable: %c%c%c",
                 dev->erase ? 'E' : '_',
                 dev->read ? 'R' : '_',
                 dev->write ? 'W' : '_');
}
```

81.3.2 Read / Write Round-Trip

The sample picks a 1024-byte test region near the top of chip ROM (`UNISDK_CHIP_MEMORY_ROM_STARTADDR + UNISDK_CHIP_MEMORY_ROM_SIZE * 1024 - 5000`) and:

1. Locates the storage device covering that region with `tlk_storage_find_device`.
2. Reads the current contents with `tlk_storage_read` and hex-dumps them.
3. Checks whether the region is in its "clean" (erased) state by comparing every byte to `s_dev->clean_byte`.
4. Builds new test data —an incrementing byte pattern if the region was clean, or each byte incremented by one otherwise.
5. Writes the new data with `tlk_storage_write`.
6. Reads the region back and hex-dumps it again to confirm the write.

Any failure at each step (`tlk_storage_find_device` returning `NULL`, or a non-zero result from `tlk_storage_read`/`tlk_storage_write`) is logged and the sample stops early.

备注

Device-agnostic API The sample never assumes a specific flash chip —it discovers devices via `tlk_storage_get_next_device`/`tlk_storage_find_device`, so the same code works across boards with different storage layouts.

81.4 Configuration Options

The sample itself exposes no user-facing Kconfig options —its `Kconfig` only auto-selects the storage and logging subsystems:

Option	Description
`TLK_NV_STORAGE`	Always enabled (`def_bool y`) when this sample is built; selects `TLK_NV_STORAGE` and `TLK_DEBUG_LOG_CONSOLE`.

81.5 Build & Run

```
west tl-build samples/nv_storage_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/nv_storage_demo.bin
```

Source code: samples/nv_storage_demo/main.c

CHAPTER 82

System Log Sample

82.1 Overview

The `log_demo` sample (under `samples/system_demo/`) demonstrates the debug logging subsystem, emitting a log message roughly once per second. Its behavior adapts to whether the Task Planner is enabled and, if so, which execution mode it runs in.

备注

README/sample mismatch The `README.md` shipped alongside this sample currently describes the **Task Planner** demo (GPIO/LED toggling, software timers), not this log sample. This page is based directly on `main.c` and `Kconfig` instead —see *Task Planner Sample* for the sample that README actually documents.

82.2 Features Demonstrated

- Logging via `TLK_LOG_CREATE` / `TLK_LOG_INFO` / `tlk_log`
- Log rotation and the log interface loop (`tlk_logrotate`, `tlk_log_iface_loop`) when running without the Task Planner
- Integration with the Task Planner's Loop mode (`user_setup` / `user_loop`) and Scheduler mode

82.3 How It Works

82.3.1 Without Task Planner

If `CONFIG_TLK_TASK_PLANNER` is disabled, the sample owns its own `main()` and drives the logging pump directly:

```
#if TLK_IS_DISABLED(CONFIG_TLK_TASK_PLANNER)

static uint32_t last = 0;
int             main(void)
{
    while (1)
    {
        tlk_log_iface_loop();
        tlk_logrotate();
        uint32_t now = tlk_api_time_get_micros();
        if (now - last >= 1000000)
        {
            last = now;
            tlk_log(demo_log, TLK_LOG_LEVEL_ERROR, "Test log USB");
        }
    }
}

#endif
```

`tlk_log_iface_loop()` and `tlk_logrotate()` are called on every iteration to service the log transport and rotate log storage; the actual message is only emitted once a second.

82.3.2 With Task Planner —Loop Mode

When `CONFIG_TLK_TASK_PLANNER_MODE_LOOP` is enabled, the sample instead defines `user_setup()`/`user_loop()`, which the platform dispatcher calls repeatedly:

```
#elif TLK_IS_ENABLED(CONFIG_TLK_TASK_PLANNER_MODE_LOOP)
void user_setup(void) {}

void user_loop(void)
{
    static uint32_t last = 0;
    uint32_t        now  = tlk_api_time_get_micros();
    if (now - last >= 1000000)
    {
        last = now;
        tlk_log(demo_log, TLK_LOG_LEVEL_ERROR, "Test log USB from Telink Loop");
    }
}
```

82.3.3 With Task Planner —Scheduler Mode

When `CONFIG_TLK_TASK_PLANNER_MODE_SCHEDULER` is enabled, `user_loop()` is invoked periodically by the default task and simply logs on every call (the periodic interval is controlled by the Task Planner's own configuration, not by the sample):

```
#elif TLK_IS_ENABLED(CONFIG_TLK_TASK_PLANNER_MODE_SCHEDULER)
void user_setup(void) {}

void user_loop(void)
{
    tlk_log(demo_log, TLK_LOG_LEVEL_ERROR, "Test log USB with Telink Scheduler");
}
#endif
```

A C++ variant of the same logic (`main.cpp`) is compiled instead of `main.c` when `CONFIG_TLK_ALLOW_CPP_WRAPPERS` is enabled.

82.4 Configuration Options

The sample's own `Kconfig` only pulls in the modules it needs; the execution-mode behavior above is controlled by the Task Planner subsystem's own options.

Option	Description
`CONFIG_TLK_GPIO`	Always enabled (`def_bool y`) —required by the shared board pinout headers.
`CONFIG_TLK_DEBUG_LO`	Always enabled (`def_bool y`) —routes log output to the console/log interface.
`CONFIG_TLK_BLE_CONT`	Defaults to `n` for this sample.
`CONFIG_TLK_TASK_PLA`	(Task Planner subsystem) When disabled, the sample runs its own `main()` loop; when enabled, behavior depends on the mode below.
`CONFIG_TLK_TASK_PLA`	(Task Planner subsystem) Cooperative loop execution —logs once per second, gated in `user_loop()`.
`CONFIG_TLK_TASK_PLA`	(Task Planner subsystem) Task scheduler execution —logs unconditionally on each `user_loop()` invocation, at the scheduler's configured period.

82.5 Build & Run

```
west tl-build samples/system_demo/log_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/log_demo.bin
```

Source code: `samples/system_demo/log_demo/main.c`

CHAPTER 83

Task Planner Sample

83.1 Overview

The `task_planner_demo` sample (under `samples/system_demo/`) demonstrates the Task Planner subsystem on the Telink RISC-V MCU platform: basic GPIO output driven through cooperative loop execution, task-scheduler execution, and an optional software timer, depending on which Task Planner mode is enabled.

83.2 Features Demonstrated

- GPIO output configuration and control (`tlk_gpio_configure`, `tlk_gpio_pin_toggle`)
- Cooperative loop execution model (`user_loop()`, `TLK_REGISTER_LOOP`)
- Scheduler-based task execution (`TASK_SPAWN`, `TASK_DELAY`)
- Periodic execution using software timers (`TLK_SW_TIMER_REGISTER`, `tlk_task_sw_timer_start`), when enabled

83.3 How It Works

83.3.1 Initialization (`user_setup()`)

Called once during system startup. LED0, LED1, and LED2 are configured as GPIO outputs; if `CONFIG_TLK_SOFTWARE_TIMERS` is enabled, the periodic software timer `test_timer1` is also started:

```
void user_setup(void)
{
    tlk_gpio_configure(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN, TLK_GPIO_OUTPUT);
    tlk_gpio_configure(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN, TLK_GPIO_OUTPUT);
    tlk_gpio_configure(PINMUX_LED_2_PORT, PINMUX_LED_2_PIN, TLK_GPIO_OUTPUT);
```

(续下页)

(接上页)

```
#if TLK_IS_ENABLED(CONFIG_TLK_SOFTWARE_TIMERS)
    tlk_task_sw_timer_start(&test_timer1);
#endif
}
```

83.3.2 Mode 1: Loop Mode (CONFIG_TLK_TASK_PLANNER_MODE_LOOP)

`user_loop()` is executed repeatedly by the platform dispatcher (`while (1) { tlk_loop(); }`) and toggles LED0 every second:

```
void user_loop(void)
{
    tlk_gpio_pin_toggle(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN);
    tlk_api_time_delay(TLK_SEC_TO_US(1));
}

static void loop1(void)
{
    tlk_gpio_pin_toggle(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN);
    tlk_api_time_delay(TLK_SEC_TO_US(2));
}

TLK_REGISTER_LOOP(loop1)
```

`loop1()` is registered as an additional loop handler via `TLK_REGISTER_LOOP` and toggles LED1 every 2 seconds, resembling the Arduino-style `loop()` model.

83.3.3 Mode 2: Scheduler Mode (CONFIG_TLK_TASK_PLANNER_MODE_SCHEDULER)

`user_loop()` is called from the default task (its period set by `CONFIG_TLK_TASK_PLANNER_DEFAULT_TASK_LOOP_PERIOD_MS`) and just toggles LED0. `loop1()` is instead defined as a cooperatively-scheduled task via `TASK_SPAWN`, toggling LED1 and rescheduling itself every 2 seconds:

```
void user_loop(void)
{
    tlk_gpio_pin_toggle(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN);
}

static void loop1(tTask* t)
{
    tlk_gpio_pin_toggle(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN);
    TASK_DELAY(t, TASK_MS(2000));
}

TASK_SPAWN(test_task1, loop1);
```

83.3.4 Optional: Software Timer

When `CONFIG_TLK_SOFTWARE_TIMERS` is enabled, a periodic software timer (`test_timer1`) is statically registered via `TLK_SW_TIMER_REGISTER` and toggles LED2 every second, independent of the loop/scheduler mode:

```
static void sw_timer_loop(void* arg)
{
    (void) arg;
    tlk_gpio_pin_toggle(PINMUX_LED_2_PORT, PINMUX_LED_2_PIN);
}

TLK_SW_TIMER_REGISTER(test_timer1, sw_timer_loop, SW_TIMER_MODE_PERIODIC, TASK_
↪MS(1000));
```

A C++ variant of the same logic (`main.cpp`) is compiled instead of `main.c` when `CONFIG_TLK_ALLOW_CPP_WRAPPERS` is enabled.

备注

LED-to-mode mapping

LED	Controlled by	Mode dependency
LED0	`user_loop()`	Loop / Scheduler
LED1	`loop1()`	Loop / Scheduler
LED2	Software timer callback	`CONFIG_TLK_SOFTWARE_TIMERS`

83.4 Configuration Options

The sample's own `Kconfig` just enables the Task Planner subsystem; execution mode and timing are controlled by the Task Planner subsystem's own options.

Option	Description
`CONFIG_TLK_TASK_PLANNER_DEM`	Always enabled (`def_bool y`) when this sample is built; selects `TLK_TASK_PLANNER`.
`CONFIG_TLK_TASK_PLANNER_MOD`	(Task Planner subsystem) Enables cooperative loop-based execution.
`CONFIG_TLK_TASK_PLANNER_MOD`	(Task Planner subsystem) Enables task scheduler-based execution.
`CONFIG_TLK_TASK_PLANNER_DEF`	(Task Planner subsystem, scheduler mode only) Enables periodic execution of `user_loop()` from the default task.
`CONFIG_TLK_TASK_PLANNER_DEF`	(Task Planner subsystem) Execution period (ms) for `user_loop()` in scheduler mode; default `500`.
`CONFIG_TLK_SOFTWARE_TIMERS`	(Task Planner subsystem) Enables the periodic software timer that toggles LED2.

83.5 Build & Run

```
west tl-build samples/system_demo/task_planner_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/task_planner_demo.bin
```

Source code: `samples/system_demo/task_planner_demo/main.c`

CHAPTER 84

Audio Sample

84.1 Overview

The `audio_demo` sample demonstrates the Audio driver's record-and-playback pipeline: it records audio from a digital microphone (DMIC) into flash-backed storage using DMA linked-list buffers, then plays the recorded audio back out through the audio output path.

84.2 Features Demonstrated

- Audio input (recording) configuration via `tlk_audio_input_configure` / `tlk_audio_input_enable` / `tlk_audio_input_disable`
- Audio output (playback) configuration via `tlk_audio_output_configure` / `tlk_audio_output_enable` / `tlk_audio_output_disable`
- Double-buffered DMA linked-list transfers (`struct tlk_dma_ll_node`) for both RX and TX
- DMA interrupt callbacks (`tlk_dma_chn_add_callback`) that feed/drain the buffers to/from storage on every DMA transfer-complete interrupt
- Chip-specific audio pinmux configuration for AMIC bias, DMIC data/clock, and SDM (sigma-delta modulator) pins, selected per SoC (`CONFIG_TLK_CORE_TL321X`, `CONFIG_TLK_CORE_TL721X`, `CONFIG_TLK_CORE_B92`)
- Non-volatile storage read/write/erase (`tlk_storage_write`, `tlk_storage_read`, `tlk_storage_erase`) used as the recording buffer
- Benchmarking of erase/record/playback timing via `tlk_benchmark_begin` / `tlk_benchmark_end` / `tlk_benchmark_elapsed`

84.3 How It Works

84.3.1 Initialization

1. **Storage Setup** —A storage region is reserved at the top of ROM (`storage_addr`, sized `BUFFER_COUNT * BUFFER_SIZE` = 256 KiB) and located with `tlk_storage_find_device`.
2. **Pinmux Configuration** —Depending on the target core, AMIC bias, DMIC data/clock pins, and SDM P/N pins are set up (e.g. for `CONFIG_TLK_CORE_B92`, only DMIC pins are configured; the AMIC/SDM structs are left empty).
3. **DMA Linked Lists** —Two double-buffered linked lists are built: `tx_buffer0`/`tx_buffer1` for playback and `rx_buffer0`/`rx_buffer1` for recording, each node covering one 1024-byte `audio_buffer` slot.
4. **Audio Input/Output Configuration** —`tlk_audio_input_configure` and `tlk_audio_output_configure` are called with `TLK_AUDIO_48K` sample rate, 16-bit data width, and the left I2S channel, wired to DMA channels obtained from `tlk_dma_chn_request()`.
5. **DMA Callbacks** —`on_rx_buffer_fill` and `on_tx_buffer_sent` are registered as DMA transfer-complete callbacks; they call `save_buffer()` / `fill_buffer()` to move data between the DMA buffer and flash storage on each completed block.

84.3.2 Main Loop

```
while (1)
{
    audio_tx_buffer_index = 0;
    audio_rx_buffer_index = 0;

    is_rx_done = 0;
    is_tx_done = 0;

    tlk_benchmark_begin(&span);
    tlk_storage_erase(storage_addr, STORAGE_SIZE);
    tlk_benchmark_end(&span);

    tlk_gpio_pin_toggle(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN);
    tlk_benchmark_begin(&span);

    tlk_audio_input_enable();
    while (!is_rx_done) { }
    tlk_benchmark_end(&span);

    tlk_gpio_pin_toggle(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN);
    tlk_benchmark_begin(&span);

    fill_buffer();
    tlk_audio_output_enable();
    while (!is_tx_done) { }
    tlk_benchmark_end(&span);
}
```

Each cycle: erase the storage region, record `BUFFER_COUNT` (256) blocks of `BUFFER_SIZE` (1024) bytes from the DMIC into storage (recording stops automatically once `audio_rx_buffer_index == BUFFER_COUNT`, disabling audio input), then play the recorded

blocks back out (stopping once `audio_tx_buffer_index == BUFFER_COUNT`, disabling audio output). LED1 is toggled between the record and playback phases as a visual marker, and elapsed erase/record/playback time is logged via the benchmark span.

备注

Per-Core Pinmux The sample selects its AMIC/DMIC/SDM pin assignments at compile time based on which core config (`CONFIG_TLK_CORE_TL321X`, `CONFIG_TLK_CORE_TL721X`, or `CONFIG_TLK_CORE_B92`) is active, since the audio pin layout differs between chips.

84.4 Configuration Options

Option	Description
`CONFIG_TLK_AUD`	Enables the sample; selects `TLK_AUDIO`, `TLK_NV_STORAGE`, and `TLK_DMA_IRQ_HANDLER` (default `y`, not user-facing)

备注

This sample has no user-selectable Kconfig choices beyond the auto-selected dependencies above —behavior is fixed at record-then-playback, with pin assignments chosen automatically based on the target core.

84.5 Build & Run

```
west tl-build samples/audio_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/audio_demo.bin
```

Source code: `samples/audio_demo/main.c`

CHAPTER 85

mbedTLS Sample

85.1 Overview

The `mbedtls_demo` sample demonstrates using mbedTLS's PSA Crypto API on-device to generate random data and perform ECDSA (elliptic-curve) key generation, signing, and verification. The sample is built twice from the same logic: `main.c` (plain C) and `main.cpp` (a C++ variant that wraps the same headers in `extern "C"`), selected at build time by whether C++ wrappers are enabled.

85.2 Features Demonstrated

- PSA Crypto initialization (`psa_crypto_init`)
- Random number generation (`psa_generate_random`)
- EC key pair generation on the SECP256R1 curve (`psa_generate_key` with `PSA_KEY_TYPE_ECC_KEY_PAIR(PSA_ECC_FAMILY_SECP_R1)`, 256-bit)
- Exporting private and public key material (`psa_export_key`, `psa_export_public_key`)
- ECDSA signing and verification over SHA-256 (`psa_sign_hash`, `psa_verify_hash` with `PSA_ALG_ECDSA(PSA_ALG_SHA_256)`)
- Key cleanup (`psa_destroy_key`)
- Hex-dump logging helper (`tlk_str_utils_convert_buf_to_hex` in `main.c`, `tlk_str_utils_buf_to_hex` in `main.cpp`)

85.3 How It Works

85.3.1 Initialization

1. `tlk_core_interrupt_enable()` is called first to enable interrupts.

2. The mbedTLS version string (`MBEDTLS_VERSION_STRING_FULL`) is logged.
3. `psa_crypto_init()` is called; if it fails, an error is logged and no tests run.

85.3.2 Test Sequence

Once PSA Crypto is initialized, two tests run in sequence, aborting early (via `break` out of a `do { } while(0)` block) if any step fails:

1. **Random Testing** —Generates 31 random bytes with `psa_generate_random` and logs them as hex.
2. **EC DSA Testing** —
   - Builds key attributes for an exportable SECP256R1 key pair usable for signing and verifying hashes (`PSA_KEY_USAGE_SIGN_HASH | PSA_KEY_USAGE_VERIFY_HASH | PSA_KEY_USAGE_EXPORT`)
   - Generates the key pair with `psa_generate_key`
   - Exports and logs both the private key (`psa_export_key`) and public key (`psa_export_public_key`)
   - Signs a zeroed 32-byte hash with `psa_sign_hash` and logs the signature
   - Verifies the signature with `psa_verify_hash`
   - Destroys the key with `psa_destroy_key`

```
psa_set_key_type(&attr, PSA_KEY_TYPE_ECC_KEY_PAIR(PSA_ECC_FAMILY_SECP_R1));
psa_set_key_bits(&attr, 256);
psa_set_key_usage_flags(&attr,
                        PSA_KEY_USAGE_SIGN_HASH | PSA_KEY_USAGE_VERIFY_HASH |
                            PSA_KEY_USAGE_EXPORT);
psa_set_key_algorithm(&attr, PSA_ALG_ECDSA(PSA_ALG_SHA_256));
```

A final `"All tests success!"` or `"Some tests failed!"` message is logged, and the program then spins forever in an empty `for (;;) {}` loop.

备注

C vs C++ Build `CMakeLists.txt` globs all `*.c` files by default. If `CONFIG_TLK_ALLOW_CPP_WRAPPERS` is enabled, `main.c` is removed from the C sources and `main.cpp` (which wraps the same mbedTLS/PSA calls in `extern "C"` blocks) is built instead. Functionally the two files are equivalent.

85.4 Configuration Options

Option	Description
`CONFIG_TLK_MBEDTLS_I`	Enables the sample; selects `TLK_MBEDTLS` and `TLK_API_PRINT` (default `y`, not user-facing)
`CONFIG_TLK_BLE_CONTI`	Explicitly disabled (`default n`) for this sample, since it does not use BLE

85.5 Build & Run

```
west tl-build samples/mbedtls_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/mbedtls_demo.bin
```

Source code: `samples/mbedtls_demo/main.c` (or `samples/mbedtls_demo/main.cpp` when built with `CONFIG_TLK_ALLOW_CPP_WRAPPERS`)

CHAPTER 86

MCUboot Bootloader Sample

86.1 Overview

The `mcuboot_bootloader_demo` sample builds MCUboot as the bootloader stage: it validates and jumps to the signed application in Slot 0, and can optionally expose a UART serial recovery (DFU) console for uploading new images without a debugger.

86.2 Features Demonstrated

- Bootloader startup and handoff to the application via `tlk_mcuboot_run()`
- Board-specific DFU-entry detection and recovery UART setup
- UART serial recovery transport configuration for `boot_serial`

86.3 How It Works

86.3.1 Initialization

1. **Global interrupts** are enabled via `tlk_core_interrupt_enable()`.
2. **Debug UART0** is configured if `CONFIG_TLK_DEBUG_LOG_INTERFACE_UART` is enabled, so bootloader log messages are visible.
3. `tlk_mcuboot_run()` is called, handing control to the bootloader core.

```
int main(void)
{
    tlk_core_interrupt_enable();

#if TLK_IS_ENABLED(CONFIG_TLK_DEBUG_LOG_INTERFACE_UART)
    configure_uart(TLK_UART0, CONFIG_TLK_UART0_BAUDRATE, TLK_UART_RX_TIMEOUT_MUL_2);
```

(续下页)

(接上页)

```
#endif

    tlk_mcuboot_run();

    while (1)
    {
    }
}
```

86.3.2 Boot Sequence (`tlk_mcuboot_run`)

1. Logs startup and warns if the image is verified against the SDK's development signing key (see `subsystem/mcuboot/port/keys.c`).
2. Initializes the crypto backend used for signature verification.
3. Calls `tlk_mcuboot_recovery_requested()`. If it returns `true`, the bootloader enters serial recovery instead of booting:
   - `tlk_mcuboot_prepare_recovery()` configures the recovery UART and DFU indicator LED.
   - `boot_serial_start()` runs the `boot_serial` console until reset.
4. Otherwise, `boot_go()` locates and validates a bootable image in Slot 0/Slot 1 and the sample jumps to it.
5. If no valid image is found, the bootloader logs an error and halts.

86.3.3 DFU-Entry Detection

`tlk_mcuboot_recovery_requested()` is checked once, at boot, before `boot_go()` runs —the button must be held through power-up/reset, not pressed after the app has already booted. A 30 ms debounce confirms the press:

```
bool tlk_mcuboot_recovery_requested(void)
{
    tlk_gpio_configure(PINMUX_KEY_2_PORT, PINMUX_KEY_2_PIN, TLK_GPIO_OUTPUT);
    tlk_gpio_pin_write(PINMUX_KEY_2_PORT, PINMUX_KEY_2_PIN, 1);

    tlk_gpio_configure(PINMUX_KEY_0_PORT, PINMUX_KEY_0_PIN, TLK_GPIO_INPUT_PULL_DOWN);

    tlk_api_time_delay(TLK_MS_TO_US(1));

    if (!tlk_gpio_pin_read(PINMUX_KEY_0_PORT, PINMUX_KEY_0_PIN))
    {
        return false;
    }

    tlk_api_time_delay(TLK_MS_TO_US(DFU_BUTTON_DEBOUNCE_MS));

    return tlk_gpio_pin_read(PINMUX_KEY_0_PORT, PINMUX_KEY_0_PIN) != 0;
}
```

When recovery is entered, `tlk_mcuboot_prepare_recovery()` turns on `LED1` as a visual indicator and configures the recovery UART (`RECOVERY_UART_TX_PORT/PIN` = PC4,

RECOVERY_UART_RX_PORT/PIN = PC5) at 115200 baud on the tlk_uart_module selected by CONFIG_TLK_MCUBOOT_SERIAL_UART.

备注

Bootloader vs. Application build This sample only builds the **bootloader** stage (TLK_MCUBOOT_MODE_BOOTLOADER). A separate application target built with CONFIG_TLK_MCUBOOT_MODE_APP runs under it, linked with space reserved for the MCUboot header. That application also links bootutil_public.c, so it can call boot_set_confirmed() to mark itself confirmed and avoid a revert on the next reset.

86.4 Configuration Options

Option	Description
CONFIG_TLK_MCUE	Compiles in boot_serial and enables the DFU-entry button check. Disabled by default —without it, the sample only boots the application.
CONFIG_TLK_MCUE	Selects which tlk_uart_module (integer instance) carries the recovery console (default 0). The corresponding TLK_UARTx must be enabled.
CONFIG_TLK_MCUE	Silences the development-key warning once the public key in subsystem/mcuboot/port/keys.c has been replaced with a real release key.
CONFIG_TLK_MCUE	Flash size allocated for the bootloader binary (default 0x00090000, 576KB).
CONFIG_TLK_MCUE	Space reserved at the start of each application slot for the MCUboot image header (default 0x00000200). Must exactly match the --header-size passed to imgtool when signing.
CONFIG_TLK_MCUE	Size allocated for each application slot (default 0x000B0000, 704KB).
CONFIG_TLK_MCUE	Size allocated for swap operations (default 0x00004000, 16KB).

Signing an Application Image

An application built with CONFIG_TLK_MCUBOOT_MODE_APP is linked with space reserved for the MCUboot header, but still needs to be signed with imgtool before the bootloader will accept it:

```
imgtool sign \
    --header-size   <CONFIG_TLK_MCUBOOT_HEADER_SIZE> \
    --pad-header \
    --slot-size     <CONFIG_TLK_MCUBOOT_SLOT_SIZE> \
    --version       1.0.0 \
    --key           <path-to-signing-key>.pem \
    app.bin app_signed.bin
```

Use the corresponding development signing key for local testing, or generate a real key pair and set CONFIG_TLK_MCUBOOT_PRODUCTION_KEY.

Testing Serial Recovery (DFU)

1. Build and flash this sample with `CONFIG_TLK_MCUBOOT_SERIAL_RECOVERY=y`.
2. Hold `KEY_0` down **through power-up or reset** —`LED1` turns on once recovery mode is entered.
3. Connect to the recovery UART pins at 115200 baud and use an `mcumgr`-compatible client to interact with `boot_serial`, e.g.:
   - `image list` —list images currently in the slots.
   - `image upload` —upload a signed image. By default this targets the **primary** slot; pass the client's image-index flag (e.g. `-n 2`) to target the secondary slot for a normal test/confirm upgrade flow.
   - `image test` / `reset` / `image confirm` —mark the uploaded image for one-time boot or permanent confirmation.

86.5 Build & Run

```
west tl-build samples/mcuboot_bootloader_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/mcuboot_bootloader_demo.bin
```

Source code: `samples/mcuboot_bootloader_demo/main.c`

CHAPTER 87

API 参考

UniSDK 的完整 API 文档，通过 Doxygen + Breathe 从 C 头文件中的 Doxygen 注释自动生成。

小技巧

以下 API 参考由 C 头文件构建。使用搜索框可快速查找函数、宏或类型。

87.1 自动生成说明

此 API 参考由 **Doxygen + Breathe** 从以下目录中的头文件自动生成：

- 公共 API：`api/include/`
- 核心驱动 API：`core/include/`、`core/common/include/`
- SoC 驱动 API：`soc/*/drivers/include/`

重新生成方式：

```
python scripts/gen_api_rst.py
cd docs && sphinx-build -M html . _build
```

CHAPTER 88

Kconfig 参考

备注

Document Status: PLANNED —Automatic generation system under development

Kconfig 参考将从 `Kconfig.build`、`Kconfig.chip` 以及所有 `rsource` 子文件中自动提取，提供完整的 Kconfig 选项列表。

88.1 计划内容

- 每个配置选项的名称、类型和默认值
- 依赖关系（`depends on`、`select`）
- 取值范围（`range`、`choice`）
- 帮助文本
- 按功能模块组织的导航

88.2 实现计划

基于扩展现有的 `kconfig_parser.py`，将从 Kconfig 源文件中自动提取 Markdown 格式的参考文档。自动生成系统将在后续添加。

CHAPTER 89

YAML 描述参考

备注

Document Status: PLANNED —Automatic generation system under development

YAML 参考将从 `core/*/properties/`、`core/*/registers/`、`soc/*/pinmux/` 等目录中的 YAML 文件中自动提取，提供设备属性、寄存器定义和引脚复用配置的完整参考。

89.1 计划内容

89.1.1 设备属性

- 每个外设的配置属性和能力描述（`core/*/properties/*.yaml`）

89.1.2 寄存器定义

- 每个外设的寄存器布局和位域描述（`core/*/registers/*.yaml`）

89.1.3 引脚复用（Pinmux）

- 每个芯片封装的引脚功能分配（`soc/*/pinmux/*.yaml`）

89.2 实现计划

基于扩展现有的 `yaml_reader.py`、`properties_gen.py` 和 `registers_gen.py`，将自动生成结构化的参考文档。

自动生成系统将在后续添加。

CHAPTER 90

发布版本

90.1 版本命名规范

UniSDK 遵循 **MAJOR.MINOR.PATCH** 格式（例如 `1.2.3`）：

- **MAJOR**：不兼容的 API 变更
- **MINOR**：向后兼容的功能新增
- **PATCH**：向后兼容的缺陷修复

90.2 当前版本

当前版本信息定义在 `common/include/tlk_sdk_version.h` 中。

90.3 版本兼容性

90.3.1 芯片支持矩阵

SDK 版本	TL321〉	TL721〉	TLSR922	TLSR952	说明
0.2.0	□	□	□	□	初始版本
0.3.0	□	□	□	□	Adds BLE Host v2, MCUboot, TL321X i2s
0.4.0	□	□	□	□	Modular SDK: TLKPKG packages, registry server, git fallback

90.3.2 工具链版本

组件	版本要求
CMake	>= 3.20.0
Python	>= 3.10
Ninja	>= 1.10
West	>= 0.14.0
RISC-V GCC	V5.4.1+

90.4 版本历史

90.4.1 v0.4.0

- *v0.4.0 Release Notes*

90.4.2 v0.3.0

- *v0.3.0 Release Notes*

90.4.3 v0.2.0

- *v0.2.0* 发布说明

90.5 升级指南

详细的升级指南和迁移说明将随着未来版本的发布提供。

CHAPTER 91

V0.2.0(FR)

91.1 版本

- SDK 版本：unified_sdk V0.2.0
- 芯片版本
  - TLSR952x
  - TL721x A2
  - TL321x A0/A1
- 硬件 EVK 版本
  - TLSR952x: C1T266A20
  - TL721x: C1T314A20
  - TL321x: C1T331A20/C1T335A20
- 工具链版本
  - TLSR952x/TL721x: TL32 ELF MCULIB V5F GCC14.2
  - TL321x: TL32 ELF MCULIB V5 GCC14.2

91.1.1 功能特性

- PMP 驱动 - 用于 RISC-V 权限强制的物理内存保护驱动（`feat(pmp)`）
- 用户态框架 - 在 RISC-V 用户模式下运行不可信应用程序代码的基础设施（`feat/user mode`）
- 加密—硬件加密加速：hash、PKE、SKE、TRNG（`feat(cryptography)`）
- RF 快速稳定 - 快速 RF 稳定模式，改善 RF 唤醒延迟（`feat(rf): Add the fast settle feature`）
- RF 复位函数 - 新增 `tlk_rf_reset()` 函数系列（`feat(rf): Add the rf reset functions`）

- RF 模式切换示例 - 演示动态 RF 模式切换的示例（`feat(rf): Add the mode switching demo`）
- 旧版工具链环境变量 - 用于向后兼容的 `TELINK_BASE_SDK` 环境变量别名（`feat(toolchain)`）
- IRQ 嵌套 - 更新的中断控制器代码，支持中断抢占（`feat(IRQ code)`）
- USB 通用 API - 全面的 USB 驱动，具有统一访问接口和 TinyUSB 集成层
- I2C 通用 API - 完整的 I2C 驱动，为所有 MCU 提供统一的驱动结构
- SPI 通用 API - 支持主模式并为所有支持的 SoC 提供统一 API 的 SPI 驱动
- 电源 API - 电源轨控制 API（`tlk_power.h`）
- DMA API - 扩展的 DMA 驱动，支持统一动态配置
- PLIC API - 统一的中断控制器，为所有支持的 SoC 提供通用 API
- ADC API - 为所有支持的 SoC 提供统一的 ADC API
- mbedTLS 模块 - PSA Crypto API 集成，支持硬件 TRNG 加速
- 日志子系统 - 调试日志，简化格式化与发送调试消息的用户体验
- 多级初始化系统 - 支持不同的系统初始化级别
- 任务规划器和循环 - 简单的多任务队列，支持多线程或串行执行
- 软件定时器 - 基于 Mtimer 的多实例软件定时器
- 存储系统 - 抽象存储设备层，支持 SoC Flash 驱动
- 调试系统 - 断言处理和 GPIO 调试支持
- 动态内存管理 - 可配置的堆分配，支持保持内存
- Pinmux 配置工具 - 用于可视化引脚复用配置的交互式 Python TUI
- 工具链自动发现 - 自动检测和验证 RISC-V GCC 工具链路径
- Zephyr 模块集成 - UniSDK 作为标准 Zephyr RTOS 模块（`module.yml`），带有 CMake/Kconfig 桥接
- Zephyr Kconfig 桥接 - 从 Zephyr 到 UniSDK 的 SoC/Core/Board 映射（`CONFIG_SOC_SERIES_TELINK_*` → `CONFIG_TLK_CORE/SOC/BOARD`）
- 子系统
  - BLE Controller - 完整的 BLE 链路层，支持 HCI、LL、ACL、广播和扫描
- 文档
  - 完整的英文文档集已添加至 Unified SDK Documentation，涵盖：
    - **入门指南**—Linux、Windows、macOS 环境搭建；首次构建指南；故障排除
    - **简介**—SDK 定位、架构概览、芯片系列表
    - **芯片与开发板**—各芯片规格（TL321X、TL721X、TLSR922X、TLSR952X）和 EVK 开发板文档
    - **外设驱动**—GPIO、UART、PM、PLIC、Analog、I2C、SPI、ADC、DMA、WDT、Timer、USB
    - **示例**—gpio、uart、pm、adc、dma、i2c、spi、BLE 广播、BLE HCI 控制器、IRQ 嵌套、定时器、看门狗、USB CDC、RF 多模、NV 存储、任务规划器、mbedTLS、系统日志
    - **构建系统**—CMake 流程、Kconfig 系统、West 工作空间、Zephyr 模块集成、Make 支持、扩展

* **工具**—Menuconfig、Pinmux 可视化工具、BDT Flash/调试工具
* **开发指南**—目录结构、公共 API 设计、核心架构、IDE 集成、调试
* **API 参考、Kconfig 参考、YAML 参考**—含索引文件的框架
* **连接性**—BLE 控制器文档
* **术语**—SDK 全局术语表

91.2 代码大小

SoC	示例	配置	Flash	RAM
TL321X	gpio	min	7234	4173
TL321X	gpio	suspend	11280	4300
TL321X	gpio	retention	12140	4816
TL321X	gpio	max	11432	4348
TL321X	gpio	max_all	19812	5112
TL321X	pm	suspend	6858	4173
TL321X	pm	retention	6858	4173
TL321X	pm	max	6858	4173
TL321X	pm	max_all	19320	5112
TL321X	uart	min	12546	4199
TL321X	uart	suspend	17204	4427
TL321X	uart	retention	18688	4943
TL321X	uart	max	21604	4607
TL321X	uart	max_all	23732	5135
TL721X	gpio	min	10118	4173
TL721X	gpio	suspend	14202	4284
TL721X	gpio	retention	15362	4804
TL721X	gpio	max	14334	4364
TL721X	gpio	max_all	23106	5132
TL721X	pm	suspend	9762	4173
TL721X	pm	retention	9762	4173
TL721X	pm	max	9762	4173
TL721X	pm	max_all	22618	5132
TL721X	uart	min	15426	4199
TL721X	uart	suspend	20118	4443
TL721X	uart	retention	21906	4963
TL721X	uart	max	24490	4623
TL721X	uart	max_all	26906	5155
TLSR952X	gpio	min	7530	4180
TLSR952X	gpio	suspend	12116	4296
TLSR952X	gpio	retention	13400	4812
TLSR952X	gpio	max	12280	4376
TLSR952X	gpio	max_all	19160	5080
TLSR952X	pm	suspend	7174	4180
TLSR952X	pm	retention	7174	4180
TLSR952X	pm	max	7174	4180
TLSR952X	pm	max_all	18720	5080
TLSR952X	uart	min	11854	4203
TLSR952X	uart	suspend	17044	4455

续下页

表 1 – 接上页

SoC	示例	配置	Flash	RAM
TLSR952X	uart	retention	18688	4971
TLSR952X	uart	max	19848	4575
TLSR952X	uart	max_all	22140	5103

CHAPTER 92

V0.3.0(FR)

92.1 Version

- SDK Version: unisdk V0.3.0
- Chip Version
  - TLSR952x
  - TL721x A2
  - TL321x A0/A1
- Hardware EVK Version
  - TLSR952x: C1T266A20
  - TL721x: C1T314A20
  - TL321x: C1T331A20/C1T335A20
- Toolchain Version
  - TL321X: TL32 ELF MCULIB V5 GCC14.2 (andes-riscv32-v5-gcc14.2)
  - TL721X/TLSR952X/TLSR922X: TL32 ELF MCULIB V5F GCC14.2 (andes-riscv32-v5f-gcc14.2)
  - Zephyr build (all series): Zephyr RISC-V64 SDK v0.17.0

92.2 Features

- [BLE Host]
  - Added BLE Host v2 project, build configuration, and sample application (`ble_host_demo`), including the SAL (Service Abstraction Layer) timers implementation and an enabled ACL data path.

  - Moved the SAL layer into the BLE host and split the BLE host module out into its own repository, integrated via West.

- [MCUboot]
  - Introduced MCUboot support, including a bootloader sample and its README/API documentation.
- [Build System / Kconfig]
  - Improved the Kconfig configuration flow and fixed several follow-on issues in the chip/build configuration generation.
  - `chip.config` generation now honors `SOC/BOARD` values passed on the CMake command line (e.g. `make cmake BOARD=TL3218X_EVK`), so the board/SoC can be preselected non-interactively instead of only through the interactive `menuconfig` UI.
  - Propagated the toolchain type (Andes / Zephyr) into `build.config`, and the cached toolchain path is now invalidated when the selected SoC series changes.
- [Toolchain]
  - Updated the Zephyr toolchain to v0.17.0.
  - Added automatic winget-based toolchain installation with a fallback path.
- [Drivers]
  - Added missing RF driver functions and DCOC/LDO_PLL/HPMC calibration and compensation support.
  - Implemented the advanced random driver for TL721X.
  - Added DMA driver improvements.
  - Reworked the i2s demo and added i2s support for TL321X.
  - Added common driver source files.
- [Logging]
  - Added hexdump support to the logger.
- [Security]
  - Added Mbed TLS support.
- [Linker / Binary]
  - Linker script no longer forces the LMA address of FLASH sections.
  - Added binary identification values (SDK/app version signature) for the debugger.
- [Samples]
  - Updated existing samples to support C++.

92.3 Bug Fixes

- Critical Bugs:
  - [Build System]

        * Fixed the board selector so it works correctly when `SOC/BOARD` are passed on the command line —previously a first-time, non-interactive `chip.config` generation silently ignored these values and left the board unselected, causing configure-time Kconfig source failures.
            · Impact Scope: Any project first configuring a released/exported SDK tree (`DEV SDK mode`) with `-DBOARD=.../-DSOC=...` on the command line.
            · Update Recommendation: Mandatory update for anyone relying on command-line board/SoC selection instead of interactive `menuconfig`.
    - [Audio]
        * Fixed a linker issue affecting audio builds.
- General Bugs:
    - [Power Management]
        * Reduced power consumption and added tick resolution for sleep duration.
    - [SPI]
        * Increased SPI driver speed.
    - [USB]
        * Fixed typos left over from a prior renaming pass.
    - [System]
        * Fixed `sys_init` priority ordering.
    - [Logger]
        * Fixed an unused-parameter build error that occurred when the logger is disabled.
    - [mtimer]
        * Guarded mtimer source with the correct `#if` conditional.
    - [CI]
        * Fixed the Andes CI job build.
    - [Samples / BLE Host v2]
        * Removed an audio demo file that was added by mistake.
        * Fixed several coding-style issues in the BLE Host v2 demo.
        * Renamed `ble_host_v2_demo` to `ble_host_demo`.
        * Removed building applications from within the `ble_host_v2` module.
    - [Misc]
        * Fixed a typo in `core/common/src/tlk_plic.c`.
        * Removed excessive blank lines in `nv_storage_demo` logs.

92.4 BREAKING CHANGES

- Change Description: The board-properties file path layout was changed (`refactor(properties): Change board properties path`).
  - Impact Scope: Any out-of-tree code or configuration referencing the old board-properties path.
  - Update Recommendation: Re-check board-properties references after upgrading; see the updated path layout in `boards/`.
- Change Description: `ble_host_v2_demo` was renamed to `ble_host_demo`, and the BLE host module now lives in a separate repository pulled in via West.
  - Impact Scope: Any project referencing the old `ble_host_v2_demo` sample name, or directly vendoring/pinning the BLE host sources in-tree.
  - Update Recommendation: Update sample references to `ble_host_demo` and update West manifest pins as needed.

92.5 Note

- Development Note: The `SOC/BOARD` CMake cache variables only take effect the first time `chip.config` is generated. If `chip.config` already exists from a previous configure, delete it (or edit it directly) before re-running with new `SOC/BOARD` values.

92.6 Royalty fee for certain Audio Codec:

- This SDK may include options for multiple audio codecs, it should be noted that use of certain Codecs may incur Royalty fees. It is the end product manufacturer's responsibility to sign license agreement with the license onwers and pay royalty fees. Telink as an IC provider cannot cover these charges.
- Use of LC3+ codec: If you choose to use LC3+ codec, please contact Fraunhofer/Ericsson (Fraunhofer IIS: lc3-licensing@iis.fraunhofer.de and Ericsson: lc3.licensing@ericsson.com) for proper license agreement and royalty fee information. A flat fee is charged by these License owners for product incorporating LC3+ codec. The royalty fee is open and transparent and charged per device (e.g. Headset, TV ,Box, ⋯).
- Use of LC3 codec: LC3 usage is only free for product qualified as a Bluetooth product by Bluetooth SIG. If your product is not Bluetooth qualified and you choose to use LC3 codec, please contact Fraunhofer/Ericsson (Fraunhofer IIS: lc3-licensing@iis.fraunhofer.de and Ericsson: lc3.licensing@ericsson.com) for proper license agreement and royalty fee information. A flat fee is charged by these License owners for non-Bluetooth product incorporating LC3 codec. The royalty fee is open and transparent and charged per device (e.g. Headset, TV ,Box, ⋯).

92.7 CodeSize

`irq` is a new sample added in this release and has no v0.2.0 baseline to diff against (shown as "New"). `max_all` configs from v0.2.0 are no longer produced by the current sample set and are omitted below. Deltas are `(V0.3.0 - V0.2.0) / V0.2.0 * 100`.

SOC	Sample	Config	Flash	Flash Δ%	RAM	RAM Δ%
TL321X	gpio	min	8562	+18.4%	4173	+0.0%
TL321X	gpio	suspend	13108	+16.2%	4300	+0.0%
TL321X	gpio	retention	13952	+14.9%	4816	+0.0%
TL321X	gpio	max	13244	+15.9%	4348	+0.0%
TL321X	irq	min	17602	New	5356	New
TL321X	irq	suspend	21674	New	5508	New
TL321X	irq	retention	22518	New	6028	New
TL321X	irq	max	17602	New	5356	New
TL321X	pm	suspend	14912	+117.4%	4869	+16.7%
TL321X	pm	retention	14240	+107.6%	4869	+16.7%
TL321X	pm	max	14984	+118.5%	4869	+16.7%
TL321X	uart	min	13545	+8.0%	4355	+3.7%
TL321X	uart	suspend	17615	+2.4%	4507	+1.8%
TL321X	uart	retention	19131	+2.4%	5023	+1.6%
TL321X	uart	max	21115	-2.3%	4747	+3.0%
TL721X	gpio	min	11534	+14.0%	4173	+0.0%
TL721X	gpio	suspend	16032	+12.9%	4284	+0.0%
TL721X	gpio	retention	17240	+12.2%	4804	+0.0%
TL721X	gpio	max	16160	+12.7%	4364	+0.0%
TL721X	irq	min	20574	New	5368	New
TL721X	irq	suspend	24514	New	5524	New
TL721X	irq	retention	25722	New	6048	New
TL721X	irq	max	20574	New	5368	New
TL721X	pm	suspend	18136	+85.8%	4889	+17.2%
TL721X	pm	retention	17520	+79.5%	4889	+17.2%
TL721X	pm	max	18208	+86.5%	4889	+17.2%
TL721X	uart	min	16753	+8.6%	4367	+4.0%
TL721X	uart	suspend	20691	+2.8%	4523	+1.8%
TL721X	uart	retention	22515	+2.8%	5043	+1.6%
TL721X	uart	max	24347	-0.6%	4763	+3.0%
TLSR952X	gpio	min	8882	+18.0%	4180	+0.0%
TLSR952X	gpio	suspend	13932	+15.0%	4296	+0.0%
TLSR952X	gpio	retention	15224	+13.6%	4812	+0.0%
TLSR952X	gpio	max	14088	+14.7%	4376	+0.0%
TLSR952X	irq	min	17802	New	5372	New
TLSR952X	irq	suspend	22338	New	5536	New
TLSR952X	irq	retention	23630	New	6056	New
TLSR952X	irq	max	17802	New	5372	New
TLSR952X	pm	suspend	15920	+121.9%	4897	+17.2%
TLSR952X	pm	retention	15520	+116.3%	4897	+17.2%
TLSR952X	pm	max	15992	+122.9%	4897	+17.2%
TLSR952X	uart	min	14037	+18.4%	4371	+4.0%
TLSR952X	uart	suspend	18571	+9.0%	4535	+1.8%
TLSR952X	uart	retention	20263	+8.4%	5051	+1.6%
TLSR952X	uart	max	20839	+5.0%	4679	+2.3%

CHAPTER 93

V0.4.0(FR)

93.1 Version

- SDK Version: unisdk V0.4.0
- Package Versions
  - MCU packages: B92 (TLSR922X/TLSR952X) 0.1.1, TL321X 0.1.1, TL721X 0.1.1
  - Modules: flashdb 0.1.1, mbedtls 0.1.1, tinyusb 0.1.1
  - All packages declare `sdk: "^0.4.0"`
- Chip Version
  - TLSR952x
  - TL721x A2
  - TL321x A0/A1
- Hardware EVK Version
  - TLSR952x: C1T266A20
  - TL721x: C1T314A20
  - TL321x: C1T331A20/C1T335A20
- Toolchain Version
  - TL321X: TL32 ELF MCULIB V5 GCC14.2 (andes-riscv32-v5-gcc14.2)
  - TL721X/TLSR952X/TLSR922X: TL32 ELF MCULIB V5F GCC14.2 (andes-riscv32-v5f-gcc14.2)
  - Zephyr build (all series): Zephyr RISC-V64 SDK v0.17.0

93.2 Features

- [Modular SDK / TLKPKG]
  - Introduced TLKPKG, the UniSDK package manager. The SDK is now split into a small core plus independently versioned packages: MCU packages (`packages/B92`, `packages/TL321X`, `packages/TL721X`) and modules (`modules/FlashDB`, `modules/mbedtls`, `modules/tinyUSB`).
  - MCU sources and the tinyUSB, Mbed TLS and FlashDB modules moved to their own repositories and are pulled in as submodules.
  - Added package versioning: the SDK version is defined in `version.yaml`, and each package declares its own version and the SDK range it is compatible with in `tlkpkg.yaml`.
  - Added package integrity checks on installation.
  - Added module autodiscovery: installed modules are found and added to the build automatically.
  - Module dependencies are now declared in each module's `dependency.yaml` instead of the West manifest.
  - Added per-package exporters (`export-all` command), packing of the Python scripts into the SDK, and the SDK/SRC split of exported trees.
- [Package Registry Server]
  - Added the package registry server with a sample registry, used to publish, list and download packages.
- [Git Publishing and Fallback]
  - Added publishing and fetching of packages through git: each package can be released as a tag in its own git repository.
  - Added a git fallback: when the TLKPKG server is unavailable, packages are installed directly from their git repositories. The VS Code extension uses the same fallback.
- [VS Code Extension]
  - Updated the UniSDK VS Code extension and prepared it for publishing: SDK management (several SDK installations, stored in a file instead of VS Code storage), separate SDK and SRC modes, package install and refresh from the sidebar, and a status refresh button.
  - Added the new project wizard: a new project is created outside the SDK from a sample, with its own Makefile and `.clangd` file.
- [Pinmux]
  - Added the pinmux GUI, including the board view and pin generation for a bare SoC when no board is selected.
- [Build System / Kconfig]
  - Added `prj.conf` support: each sample and project sets its Kconfig options in its own `prj.conf`.
  - External projects can now be built with `make`, outside the SDK tree.
  - Added the `GEN_ALL_BOARD_PROPERTIES` CMake option, which generates the properties of all boards of the selected SoC at build time.

  - Headers of disabled drivers are now checked: including a header whose driver is disabled in Kconfig reports an error.
  - On Windows, the Makefile now runs all configuration steps instead of CMake.
- [Toolchain]
  - Toolchain discovery now verifies the installed GCC version against the version required for the SoC series in `soc.yaml`.
- [MCUboot]
  - Added a recovery request through a RAM flag, so the application can ask the bootloader to enter recovery mode.
  - Updated the MCUboot bootloader sample and its README.
- [FlashDB]
  - Introduced the FlashDB module and the `flashdb_demo` sample.
- [Audio / USB]
  - Added a USB audio sample with microphone and speaker.
- [TLSR922X]
  - Updated the TLSR922X support in the B92 package.
- [CI]
  - The CI script now discovers all chips, samples and configs, and can be limited to specific SoC series or samples.
  - Added CI configs for most of the samples and refactored the memory report generator.

93.3 Bug Fixes

- Critical Bugs:
  - [Build System]
    - Fixed the configure flow (`kconfig.cmake`) and the SDK exporter.
      - Impact Scope: Projects configured from an exported SDK tree.
      - Update Recommendation: Mandatory update for anyone exporting or building from an exported SDK.
    - Fixed a race condition in CMake that made CI builds fail.
- General Bugs:
  - [Kconfig]
    - Fixed the board selection, and the case where an empty board is selected.
    - Added Kconfig files missing from the default core installation.
  - [Registers]
    - Fixed the generation of the enum arrays in the register generator.
  - [Drivers]

    - Global variables that were exposed by the drivers are no longer visible outside their source files.
  - [Pinmux]
    - Fixed the pin selector, the pin layout and the pinmux dependencies.
  - [TLKPKG]
    - Application modules are now installed exactly into the `modules` folder.
    - The `ci_config` folder is no longer included when packing a package.
    - Fixed the package installers, the publish code and the missing installation directive for MCU packages.
    - Fixed the display of installed modules in SDK and SRC trees.
  - [Export]
    - Fixed the packing of the sources and the crypto library, the MCU export and the module export flow.
  - [New Project]
    - Fixed the Makefile of new projects.
  - [VS Code Extension]
    - Fixed the MCU package path, the Kconfig UI flow and the message shown when no MCU package is installed.
    - The sidebar is now refreshed after a package install or an SDK switch.
  - [CI]
    - Fixed the CI scripts for the new submodules and the samples build, and removed a broken pre-commit cache.

93.4 BREAKING CHANGES

- Change Description: The SDK is now modular. MCU sources (`core/<CHIP>`) are no longer part of the main repository; they are installed as MCU packages under `packages/`. tinyUSB, Mbed TLS and FlashDB are installed as modules under `modules/`.
  - Impact Scope: Any project or script that references the old in-tree MCU or module paths, or that expects every chip to be present in the SDK.
  - Update Recommendation: Install the MCU packages and modules your project needs (VS Code extension or TLKPKG), and update path references to `packages/<CHIP>` and `modules/<MODULE>`.
- Change Description: Kconfig option names were standardized (`review(configs): Standardize config names`). Examples:
  - `TLK_UART0_ENABLED` → `TLK_UART0`, `TLK_GPIO_PORT_A_ENABLED` → `TLK_GPIO_PORTA`
  - `TLK_SYSTEM_TIMER_*` → `TLK_STIMER_*` (e.g. `TLK_SYSTEM_TIMER_IRQ_ENABLE` → `TLK_STIMER_IRQ`)
  - `TLK_CRITICAL_PRINTOUT` → `TLK_CRITICAL_PRINT`, `TLK_CRITICAL_UART*` → `TLK_CRITICAL_PRINT_UART*`

  - TLK_DMA_IRQ_HANDLER → TLK_DMA_IRQ, TLK_CRYPTO_DMA_SUPPORT → TLK_CRYPTO_DMA
  - *_HAS_ENABLED_DEVICE → *_USE_DEVICE, *_AUTO_CONFIG → *_USE_AUTO_CONFIG, TLK_I2C_DMA → TLK_I2C_USE_DMA
  - *_NEST_SELECTED → *_USE_NEST, TLK_RANDOM_USED_PRNG/TRNG → TLK_RANDOM_USE_PRNG/TRNG, TLK_TRAP_ECALL_NESTING → TLK_TRAP_ECALL_NEST
  - Impact Scope: Any `prj.conf`, defconfig, `build.config` or source code that uses the old option names.
  - Update Recommendation: Rename the options in your configuration files and in `#if CONFIG_...` checks.
- Change Description: Sample and project Kconfig options moved to `prj.conf`.
  - Impact Scope: Projects that set their options in other config files.
  - Update Recommendation: Move project options into the project's `prj.conf`.
- Change Description: The West manifest is no longer used for module dependencies; dependencies are declared in each module's `dependency.yaml`.
  - Impact Scope: Projects that pin module versions through the West manifest.
  - Update Recommendation: Use the module versions installed through TLKPKG.
- Change Description: Samples can no longer be built directly inside an installed SDK.
  - Impact Scope: Workflows that build samples in the SDK folder.
  - Update Recommendation: Create a project from the sample with the new project wizard and build it outside the SDK.

93.5 Note

- Development Note: A package can only be installed into an SDK whose version matches the `sdk` range in the package's `tlkpkg.yaml` (`^0.4.0` for all packages in this release).
- Development Note: When the TLKPKG server is not reachable, packages are fetched from their git repositories. Git access to the package repositories is then required.

93.6 CodeSize

Full CI memory report for all samples and configs built in this release (sizes in bytes): `memory_report.log`

```
SOC          Sample                                      Config              Flash
↪ RAM          Text         Code       Rodata         Data          BSS          Dec
↪ Hex
----------  ----------------------------------------  ------------  ----------  ------
↪----  ----------  ----------  ----------  ----------  ----------  ----------  ------
↪----
TL321X       adc_demo                                      def                 19331
↪4625         19242        18118         1124           89         4536        23867
↪5d3b
TL321X       adc_demo                                      max                 20291
↪4653         20198        19110         1088           93         4560        24851
```

(续下页)

(接上页)

```
↪6113
TL321X      adv_demo                                  def                 18409
↪16937        18236       18072         164         173        16764       35173
↪ 8965
TL321X      audio_demo                                def                 26513
↪10913        26264       24876        1388         249        10664       37177
↪ 9139
TL321X      ble_controller_demo                       def                 91428
↪25236        90536       90128         408         892        24344      115772
↪1c43c
TL321X      ble_host_demo                             def                160280
↪24180       158976      143180       15796        1304        22876      183156
↪2cb74
TL321X      dma_demo                                  def                 18979
↪5173         18890       17646        1244          89         5084       24063
↪5dff
TL321X      dma_demo                                  max                 24789
↪5413         24692       23300        1392          97         5316       30105
↪7599
TL321X      flashdb_demo                              def                 30356
↪9773         30140       27724        2416         216         9557       39913
↪9be9
TL321X      gpio_demo                                 def                  7622
↪4181          7594        7430         164          28         4153       11775
↪2dff
TL321X      gpio_demo                                 max                  9102
↪4265          9054        8890         164          48         4217       13319
↪3407
TL321X      i2c_demo/ds1307                           def                 20538
↪4588         20422       19274        1148         116         4472       25010
↪61b2
TL321X      i2c_demo/ping_pong                        master              19574
↪4588         19466       18250        1216         108         4480       24054
↪5df6
TL321X      i2c_demo/ping_pong                        max                 19858
↪4600         19750       18530        1220         108         4492       24350
↪5f1e
TL321X      i2c_demo/ping_pong                        slave               19034
↪4612         18926       17742        1184         108         4504       23538
↪5bf2
TL321X      i2s_demo                                  master              21798
↪8696         19626       18530        1096        2172         6524       28322
↪6ea2
TL321X      i2s_demo                                  slave               21770
↪8696         19598       18506        1092        2172         6524       28294
↪6e86
TL321X      irq_demo                                  def                 17576
↪5320         17488       16504         984          88         5232       22808
↪5918
TL321X      irq_demo                                  max                 18084
↪5328         17996       16928        1068          88         5240       23324
↪5b1c
TL321X      mbedtls_demo                              def                195606
↪5460        195314      179582       15732         292         5168      200774
↪31046
TL321X      mcuboot_bootloader_demo                   def                233550
```

(续下页)

(接上页)

```
↪18248      232806      215374      17432       744       17504      251054      ␣
↪3d4ae
TL321X      mcuboot_bootloader_demo                 max               281772      ␣
↪20972      281004      260812      20192       768       20204      301976      ␣
↪49b98
TL321X      nv_storage_demo                         def               21149       ␣
↪9737       21004       19548       1456        145       9592       30741       ␣
↪7815
TL321X      pm_demo                                 def               14884       ␣
↪4877       14824       14360       464         60        4817       19701       ␣
↪4cf5
TL321X      pm_demo                                 max               15100       ␣
↪4877       15040       14576       464         60        4817       19917       ␣
↪4dcd
TL321X      pmp_demo                                def               18142       ␣
↪4584       18030       16886       1144        112       4472       22614       ␣
↪5856
TL321X      pmp_demo                                max               18390       ␣
↪4596       18278       17098       1180        112       4484       22874       ␣
↪595a
TL321X      random_demo                             def               17867       ␣
↪4569       17770       16666       1104        97        4472       22339       ␣
↪5743
TL321X      random_demo                             max               18127       ␣
↪4581       18030       16922       1108        97        4484       22611       ␣
↪5853
TL321X      rf_demo                                 brx               28828       ␣
↪17570      28602       27458       1144        226       17344      46172       ␣
↪ b45c
TL321X      rf_demo                                 btx               29272       ␣
↪17582      29046       27778       1268        226       17356      46628       ␣
↪ b624
TL321X      rf_demo                                 mrx               28900       ␣
↪17574      28674       27530       1144        226       17348      46248       ␣
↪ b4a8
TL321X      rf_demo                                 mrx_fs            30892       ␣
↪17574      30662       29502       1160        230       17344      48236       ␣
↪ bc6c
TL321X      rf_demo                                 mtx               29340       ␣
↪17586      29114       27846       1268        226       17360      46700       ␣
↪ b66c
TL321X      rf_demo                                 mtx_fs            31328       ␣
↪17586      31098       29814       1284        230       17356      48684       ␣
↪ be2c
TL321X      rf_demo                                 prx               29320       ␣
↪17570      29094       27938       1156        226       17344      46664       ␣
↪ b648
TL321X      rf_demo                                 ptx               29796       ␣
↪17582      29570       28290       1280        226       17356      47152       ␣
↪ b830
TL321X      rf_demo                                 srt               28836       ␣
↪17570      28610       27466       1144        226       17344      46180       ␣
↪ b464
TL321X      rf_demo                                 srx               28884       ␣
↪17570      28658       27514       1144        226       17344      46228       ␣
↪ b494
```

(续下页)

(接上页)

```
TL321X      rf_demo                                 str              29272
↪17582        29046       27778        1268         226       17356       46628
↪ b624
TL321X      rf_demo                                 stx              29312
↪17582        29086       27818        1268         226       17356       46668
↪ b64c
TL321X      spi_demo                                master           20526
↪4588         20426       19342        1084         100       4488        25014
↪61b6
TL321X      spi_demo                                slave            19558
↪4588         19458       18406        1052         100       4488        24046
↪5dee
TL321X      stimer_demo                             def               8878
↪4269         8826         8662         164          52       4217        13095
↪3327
TL321X      system_demo/log_demo                    def              12258
↪4440         12230       11150        1080         28        4412        16670
↪411e
TL321X      system_demo/log_demo                    loop             12338
↪4440         12310       11210        1100         28        4412        16750
↪416e
TL321X      system_demo/log_demo                    scheduler        12634
↪4460         12586       11534        1052         48        4412        17046
↪4296
TL321X      system_demo/task_planner_demo           loop             17775
↪4561         17686       16626        1060         89        4472        22247
↪56e7
TL321X      system_demo/task_planner_demo           scheduler        18206
↪4612         18070       17014        1056         136       4476        22682
↪589a
TL321X      system_demo/task_planner_demo           timers           18462
↪4620         18318       17254        1064         144       4476        22938
↪599a
TL321X      tinyusb_demos/usb_device/cdc_demo       def              31650
↪5440         31514       29814        1700         136       5304        36954
↪905a
TL321X      tinyusb_demos/usb_device/mic_demo       def              35768
↪6000         35624       33784        1840         144       5856        41624
↪a298
TL321X      tinyusb_demos/usb_device/spk_demo       def              34124
↪6060         33900       32060        1840         224       5836        39960
↪9c18
TL321X      uart_demo                               def              13025
↪4315         12937       12700         237         88        4227        17252
↪4364
TL321X      uart_demo                               max              18658
↪4451         18562       17934         628         96        4355        23013
↪59e5
TL321X      umode_demo                              def              13418
↪4316         13330       13056         274         88        4228        17646
↪44ee
TL321X      umode_demo                              max              13482
↪4316         13394       13092         302         88        4228        17710
↪452e
TL321X      wdt_demo                                def              18241
↪4569         18148       17020        1128         93        4476        22717
```

(续下页)

(接上页)

```
↪58bd
TL321X      wdt_demo                                max              22549
↪4697       22452       21280       1172          97       4600       27149
↪6a0d
TL721X      adc_demo                                def              22339
↪4637       22246       21114       1132          93       4544       26883
↪6903
TL721X      adc_demo                                max              23323
↪4665       23226       22130       1096          97       4568       27891
↪6cf3
TL721X      adv_demo                                def              20883
↪4621       20714       20542        172         169       4452       25335
↪62f7
TL721X      audio_demo                              def              29601
↪10925       29348       27956       1392         253      10672       40273
↪ 9d51
TL721X      ble_controller_demo                     def              94078
↪12744       93190       92806        384         888      11856      105934
↪19dce
TL721X      ble_host_demo                           def             162948
↪11688      161648      145880      15768        1300      10388      173336
↪2a518
TL721X      dma_demo                                def              22035
↪5185       21942       20690       1252          93       5092       27127
↪69f7
TL721X      dma_demo                                max              27785
↪5429       27684       26284       1400         101       5328       33113
↪8159
TL721X      flashdb_demo                            def              33404
↪9785       33184       30760       2424         220       9565       42969
↪a7d9
TL721X      gpio_demo                               def              10490
↪4181       10462       10290        172          28       4153       14643
↪3933
TL721X      gpio_demo                               max              12066
↪4277       12014       11842        172          52       4225       16291
↪3fa3
TL721X      i2c_demo/ds1307                         def              23682
↪4600       23562       22406       1156         120       4480       28162
↪6e02
TL721X      i2c_demo/ping_pong                      master           22718
↪4600       22606       21382       1224         112       4488       27206
↪6a46
TL721X      i2c_demo/ping_pong                      max              23002
↪4612       22890       21662       1228         112       4500       27502
↪6b6e
TL721X      i2c_demo/ping_pong                      slave            22182
↪4624       22070       20878       1192         112       4512       26694
↪6846
TL721X      i2s_demo                                master           25234
↪8728       23038       21934       1104        2196       6532       31766
↪7c16
TL721X      i2s_demo                                slave            25206
↪8728       23010       21910       1100        2196       6532       31738
↪7bfa
TL721X      irq_demo                                def              20814
```

(续下页)

(接上页)

```
↪5332       20722       19730         992          92        5240       26054
↪65c6
TL721X      irq_demo                                  max                 21322
↪5340       21230       20154        1076          92        5248       26570
↪67ca
TL721X      mbedtls_demo                              def                199030
↪5476      198734      182994       15740         296        5180      204210
↪31db2
TL721X      mcuboot_bootloader_demo                   def                237774
↪110296      237026      219574       17452         748      109548      347322
↪ 54cba
TL721X      mcuboot_bootloader_demo                   max                286024
↪113020      285252      265040       20212         772      112248      398272
↪ 613c0
TL721X      nv_storage_demo                           def                 24201
↪9749       24052       22588        1464         149        9600       33801
↪8409
TL721X      pm_demo                                   def                 18040
↪4897       17976       17504         472          64        4833       22873
↪5959
TL721X      pm_demo                                   max                 18256
↪4897       18192       17720         472          64        4833       23089
↪5a31
TL721X      pmp_demo                                  def                 21190
↪4592       21078       19926        1152         112        4480       25670
↪6446
TL721X      pmp_demo                                  max                 21434
↪4604       21322       20134        1188         112        4492       25926
↪6546
TL721X      random_demo                               def                 20919
↪4581       20818       19706        1112         101        4480       25399
↪6337
TL721X      random_demo                               max                 21179
↪4593       21078       19962        1116         101        4492       25671
↪6447
TL721X      rf_demo                                   brx                 31388
↪5254       31166       30014        1152         222        5032       36420
↪8e44
TL721X      rf_demo                                   btx                 31828
↪5266       31606       30330        1276         222        5044       36872
↪9008
TL721X      rf_demo                                   mrx                 31460
↪5258       31238       30086        1152         222        5036       36496
↪8e90
TL721X      rf_demo                                   mrx_fs              33400
↪5258       33174       32006        1168         226        5032       38432
↪9620
TL721X      rf_demo                                   mtx                 31900
↪5270       31678       30402        1276         222        5048       36948
↪9054
TL721X      rf_demo                                   mtx_fs              33836
↪5270       33610       32318        1292         226        5044       38880
↪97e0
TL721X      rf_demo                                   prx                 31884
↪5254       31662       30498        1164         222        5032       36916
↪9034
```

(续下页)

(接上页)

```
TL721X      rf_demo                                ptx             32344
↪5266       32122       30834       1288        222         5044        37388
↪920c
TL721X      rf_demo                                srt             31396
↪5254       31174       30022       1152        222         5032        36428
↪8e4c
TL721X      rf_demo                                srx             31448
↪5254       31226       30074       1152        222         5032        36480
↪8e80
TL721X      rf_demo                                str             31828
↪5266       31606       30330       1276        222         5044        36872
↪9008
TL721X      rf_demo                                stx             31876
↪5266       31654       30378       1276        222         5044        36920
↪9038
TL721X      spi_demo                               master          24786
↪4608       24674       23582       1092        112         4496        29282
↪7262
TL721X      spi_demo                               slave           23918
↪4608       23806       22746       1060        112         4496        28414
↪6efe
TL721X      stimer_demo                            def             11938
↪4281       11882       11710        172         56         4225        16163
↪3f23
TL721X      system_demo/log_demo                   def             15154
↪4440       15126       14038       1088         28         4412        19566
↪4c6e
TL721X      system_demo/log_demo                   loop            15234
↪4440       15206       14098       1108         28         4412        19646
↪4cbe
TL721X      system_demo/log_demo                   scheduler       15530
↪4460       15482       14422       1060         48         4412        19942
↪4de6
TL721X      system_demo/task_planner_demo          loop            20827
↪4573       20734       19666       1068         93         4480        25307
↪62db
TL721X      system_demo/task_planner_demo          scheduler       21254
↪4620       21118       20054       1064        136         4484        25738
↪648a
TL721X      system_demo/task_planner_demo          timers          21510
↪4628       21366       20294       1072        144         4484        25994
↪658a
TL721X      tinyusb_demos/usb_device/cdc_demo      def             35282
↪5452       35142       33434       1708        140         5312        40594
↪9e92
TL721X      tinyusb_demos/usb_device/mic_demo      def             39432
↪6012       39284       37436       1848        148         5864        45296
↪b0f0
TL721X      tinyusb_demos/usb_device/spk_demo      def             37820
↪6072       37592       35744       1848        228         5844        43664
↪aa90
TL721X      uart_demo                              def             16077
↪4327       15985       15740        245         92         4235        20312
↪4f58
TL721X      uart_demo                              max             21582
↪4467       21482       20846        636        100         4367        25949
```

(续下页)

(接上页)

```
↪655d
TL721X      umode_demo                                  def                 16466
↪4328        16374       16092         282          92        4236        20702
↪50de
TL721X      umode_demo                                  max                 16530
↪4328        16438       16128         310          92        4236        20766
↪511e
TL721X      wdt_demo                                    def                 21293
↪4581        21196       20060        1136          97        4484        25777
↪64b1
TL721X      wdt_demo                                    max                 25473
↪4713        25372       24192        1180         101        4612        30085
↪7585
TLSR952X    adc_demo                                    def                 19159
↪4637        19058       17930        1128         101        4536        23695
↪5c8f
TLSR952X    adc_demo                                    max                 20207
↪4665        20102       19010        1092         105        4560        24767
↪60bf
TLSR952X    adv_demo                                    def                 16263
↪4555        16134       15966         168         129        4426        20689
↪50d1
TLSR952X    audio_demo                                  def                 26805
↪10841         26632        25280        1352         173       10668         37473
↪  9261
TLSR952X    ble_controller_demo                         def                 86646
↪12868         85802        85438         364         844       12024         98670
↪1816e
TLSR952X    ble_host_demo                               def                155488
↪11812       154232      138480       15752        1256       10556       166044
↪2889c
TLSR952X    dma_demo                                    def                 19127
↪5189        19026       17778        1248         101        5088        24215
↪5e97
TLSR952X    dma_demo                                    max                 25453
↪5441        25340       23944        1396         113        5328        30781
↪783d
TLSR952X    flashdb_demo                                def                 30352
↪9789        30124       27704        2420         228        9561        39913
↪9be9
TLSR952X    gpio_demo                                   def                  7974
↪4188          7938         7770         168          36        4152        12126
↪2f5e
TLSR952X    gpio_demo                                   max                  9566
↪4284          9506         9338         168          60        4224        13790
↪35de
TLSR952X    i2c_demo/ds1307                             def                 20726
↪4604        20598       19446        1152         128        4476        25202
↪6272
TLSR952X    i2c_demo/ping_pong                          master              19750
↪4604        19630       18410        1220         120        4484        24234
↪5eaa
TLSR952X    i2c_demo/ping_pong                          max                 20034
↪4616        19914       18690        1224         120        4496        24530
↪5fd2
TLSR952X    i2c_demo/ping_pong                          slave               19218
```

(续下页)

(接上页)

```
↪4628       19098       17910       1188        120        4508       23726
↪5cae
TLSR952X    i2s_demo                                master             21826
↪8724       19630       18530       1100       2196        6528       28354
↪6ec2
TLSR952X    i2s_demo                                slave              21798
↪8724       19602       18506       1096       2196        6528       28326
↪6ea6
TLSR952X    irq_demo                                def                17846
↪5336       17746       16758        988        100        5236       23082
↪5a2a
TLSR952X    irq_demo                                max                18354
↪5344       18254       17182       1072        100        5244       23598
↪5c2e
TLSR952X    mbedtls_demo                            def               195958
↪5768      195654      179918      15736        304        5464      201422
↪312ce
TLSR952X    mcuboot_bootloader_demo                 def               235070
↪110588      234314      216870      17444        756      109832      344902
↪ 54346
TLSR952X    mcuboot_bootloader_demo                 max               282976
↪113256      282196      261992      20204        780      112476      395452
↪ 608bc
TLSR952X    nv_storage_demo                         def                21149
↪9753       20992       19532       1460        157        9596       30745
↪7819
TLSR952X    pm_demo                                 def                15892
↪4905       15816       15348        468         76        4829       20721
↪50f1
TLSR952X    pm_demo                                 max                16108
↪4905       16032       15564        468         76        4829       20937
↪51c9
TLSR952X    pmp_demo                                def                18222
↪4596       18102       16954       1148        120        4476       22698
↪58aa
TLSR952X    pmp_demo                                max                18466
↪4608       18346       17162       1184        120        4488       22954
↪59aa
TLSR952X    random_demo                             def                17951
↪4585       17842       16734       1108        109        4476       22427
↪579b
TLSR952X    random_demo                             max                18211
↪4597       18102       16990       1112        109        4488       22699
↪58ab
TLSR952X    rf_demo                                 brx                26336
↪5190       26154       25006       1148        182        5008       31344
↪7a70
TLSR952X    rf_demo                                 btx                26780
↪5202       26598       25326       1272        182        5020       31800
↪7c38
TLSR952X    rf_demo                                 mrx                26476
↪5194       26294       25146       1148        182        5012       31488
↪7b00
TLSR952X    rf_demo                                 mrx_fs             27464
↪5194       27278       26130       1148        186        5008       32472
↪7ed8
```

(续下页)

(接上页)

```
TLSR952X    rf_demo                                 mtx                 26920
↪5206        26738       25466       1272         182         5024        31944
↪7cc8
TLSR952X    rf_demo                                 mtx_fs              27904
↪5206        27718       26446       1272         186         5020        32924
↪809c
TLSR952X    rf_demo                                 prx                 26804
↪5190        26622       25462       1160         182         5008        31812
↪7c44
TLSR952X    rf_demo                                 ptx                 27288
↪5202        27106       25822       1284         182         5020        32308
↪7e34
TLSR952X    rf_demo                                 srt                 26344
↪5190        26162       25014       1148         182         5008        31352
↪7a78
TLSR952X    rf_demo                                 srx                 26396
↪5190        26214       25066       1148         182         5008        31404
↪7aac
TLSR952X    rf_demo                                 str                 26780
↪5202        26598       25326       1272         182         5020        31800
↪7c38
TLSR952X    rf_demo                                 stx                 26828
↪5202        26646       25374       1272         182         5020        31848
↪7c68
TLSR952X    spi_demo                                master              21886
↪4612        21766       20678       1088         120         4492        26378
↪670a
TLSR952X    spi_demo                                slave               20826
↪4612        20706       19650       1056         120         4492        25318
↪62e6
TLSR952X    stimer_demo                             def                  9422
↪4288         9358        9190        168          64         4224        13646
↪354e
TLSR952X    system_demo/log_demo                    def                 12618
↪4444        12582       11498       1084          36         4408        17026
↪4282
TLSR952X    system_demo/log_demo                    loop                12698
↪4444        12662       11558       1104          36         4408        17106
↪42d2
TLSR952X    system_demo/log_demo                    scheduler           12994
↪4464        12938       11882       1056          56         4408        17402
↪43fa
TLSR952X    system_demo/task_planner_demo           loop                17863
↪4577        17762       16698       1064         101         4476        22339
↪5743
TLSR952X    system_demo/task_planner_demo           scheduler           18290
↪4624        18146       17086       1060         144         4480        22770
↪58f2
TLSR952X    system_demo/task_planner_demo           timers              18550
↪4632        18398       17330       1068         152         4480        23030
↪59f6
TLSR952X    tinyusb_demos/usb_device/cdc_demo       def                 31858
↪5400        31710       30094       1616         148         5252        37110
↪90f6
TLSR952X    tinyusb_demos/usb_device/mic_demo       def                 36504
↪5956        36352       34600       1752         152         5804        42308
```

(续下页)

(接上页)

```
↪a544
TLSR952X    tinyusb_demos/usb_device/spk_demo         def                34296
↪5936       34144       32408        1736         152        5784       40080
↪9c90
TLSR952X    uart_demo                                 def                13125
↪4331       13025       12784         241         100        4231       17356
↪43cc
TLSR952X    uart_demo                                 max                19182
↪4479       19070       18438         632         112        4367       23549
↪5bfd
TLSR952X    umode_demo                                def                13498
↪4332       13398       13120         278         100        4232       17730
↪4542
TLSR952X    umode_demo                                max                13562
↪4332       13462       13156         306         100        4232       17794
↪4582
TLSR952X    wdt_demo                                  def                18325
↪4585       18220       17088        1132         105        4480       22805
↪5915
TLSR952X    wdt_demo                                  max                23029
↪4725       22916       21740        1176         113        4612       27641
↪6bf9
```

CHAPTER 94

贡献指南

欢迎社区开发者和贡献者为 UniSDK 项目贡献代码、文档、示例和问题报告。

> **关于 SDK 代码贡献的说明**：本指南主要针对 **unisdk-doc** 文档仓库。有关 SDK 代码贡献（unisdk），请参考 unisdk 仓库。

94.1 文档仓库：unisdk-doc

文档仓库（unisdk-doc）采用与 SDK 代码仓库相同的分支模型。

94.1.1 分支模型

```
main                                          <- Stable (latest release)
  ^ merge
release-v*.*.*                                <- Release branches
  ^ merge
develop                                       <- Active development
  ^ merge
doc/<topic>  fix/<topic>  feat/<topic>        <- Feature / fix branches
```

分支	用途	CI/CD 部署
`main`	稳定分支。仅在发布定稿时更新。	推送部署到 `/en/latest/`
`develop`	活跃开发分支。所有文档变更首先合并到此分支。	推送部署到 `/en/dev/`
`release-v*.*.*`	发布准备分支。在打标签前从 `develop` 创建。	标签推送部署到 `/en/v*.*.*/`
`doc/*`	用于文档变更的特性分支。	无部署
`fix/*`	用于文档修复的特性分支。	无部署

94.1.2 生命周期

1. **开发阶段**：从 `develop` 创建特性分支 -> 进行修改 -> 提交 PR -> 合并到 `develop`
2. **发布阶段**：准备就绪后，从 `develop` 创建 `release-v*.*.*` -> 定稿 -> 打标签 -> CI 构建标签
3. **稳定阶段**：将发布分支合并到 `main`，更新稳定版文档

94.2 文档版本与 SDK 版本

文档版本**与 SDK 版本保持一致**，纯文档修复不额外打标签：

场景	文档标签	说明
SDK 发布	v<SDK 版本>（例如 `v0.2.0`）	一对一映射，文档与 SDK 一同发布
纯文档修复	**不打标签**	直接合并到 `main`，`latest` 自动更新
开发预览	无标签（develop -> `dev`）	CI 部署到 `/en/dev/`
稳定版占位	无标签（main -> `latest`）	CI 部署到 `/en/latest/`

核心原则：

1. 文档标签与 SDK 标签一一对应：每个 SDK 发布最多只有一个文档标签
2. 纯文档修复不创建新标签；合并到 `main` 即更新 `latest`
3. 已发布的历史版本（例如 `v0.2.0`）不会追溯更新

94.3 发布流程

```
graph LR
    A[develop] -->|PR reviewed| B[release-v*.*.*]
    B -->|QA testing| C{Pass?}
    C -->|Yes| D[Tag & release]
    C -->|No| B
    D --> E[Merge to main]
    D --> F[Deploy to /en/v*.*.*/]
    E --> G[Deploy to /en/latest/]
```

1. **从 develop 创建发布分支**：

   ```
   git checkout develop
   git checkout -b release-v0.x.x
   ```

2. **定稿**：在发布分支上进行最后的修复，运行 QA。

3. **为发布打标签**：

   ```
   git tag v0.x.x
   git push origin v0.x.x
   ```

   CI 构建并将标签部署到 `/en/v0.x.x/`。

4. **合并到 main**：

```
git checkout main
git merge release-v0.x.x
git push origin main
```

CI 构建并部署到 `/en/latest/`。

94.4 多版本维护

```
graph TB
    A[develop] -->|create| B[release-v0.3.0]
    B -->|fix| B
    B -->|tag| C[v0.3.0]
    B -->|merge to| D[main]
    C -->|deploy| E[/en/v0.3.0/]

    A -->|cherry-pick critical fixes| F[v0.2.x maintenance]
    F -->|tag| G[v0.2.1]
    G -->|deploy| H[/en/v0.2.1/]
```

- 只有**最新的次版本**接收常规维护
- **上一个次版本**只接受安全修复和关键错误修复（通过 cherry-pick 方式）
- **更早的版本**不再维护；在文档中标记为 DEPRECATED
- 每个次版本都有一个 `release-v*.*.x` 维护分支

94.5 CI/CD 行为

触发条件	检出源	部署路径
推送到 `develop`	`develop` 分支	`/en/dev/`
推送到 `main`	`develop` 分支（快照）	`/en/latest/`
推送标签 `v*.*.*`	已打标签的提交	`/en/v*.*.*/`

所有版本均构建并部署到 `gh-pages` 分支，通过 GitHub Pages 提供文档服务。

94.6 文档结构标准

94.6.1 目录命名

规则	说明	示例
蛇形命名法	小写字母 + 下划线	`getting_started/`、`chips_boards/`
单数形式	目录名称使用单数形式	`peripherals/`（本身已是单数）
索引入口	每个目录必须包含 `index.md` 或 `index.rst`	—
图片	将图片放在 `_images/` 子目录中	`peripherals/_images/gpio_1.png`

当前位于 pics/ 中的图片应逐步迁移到 _images/。

94.6.2 前置元数据

每个文档页面必须包含一个前置元数据块：

```
---
title: Document Title        # Required: display title
status: DRAFT                # Required: STABLE/DRAFT/DEPRECATED/PLANNED
since: v0.1.0                # Optional: version first introduced
updated: v0.2.0              # Optional: version last updated
description: Short summary   # Optional: for search and indexing
tags: [gpio, peripheral]     # Optional: categorization tags
---
```

状态值：

状态	含义
STABLE	内容完整、已审阅，可供生产环境使用
DRAFT	内容正在开发中，可能不完整或未经审阅
DEPRECATED	内容已过时，不应用于新设计
PLANNED	内容已规划但尚未编写

94.6.3 版本差异标记

使用前置元数据字段追踪版本历史：

```
---
title: GPIO Driver Guide
status: STABLE
since: v0.1.0
updated: v0.2.0
---
```

在文档正文中，使用 Sphinx 指令进行逐元素的版本追踪：

```
.. versionadded:: v0.2.0
   Added DMA transfer mode support

.. deprecated:: v0.1.0
   ``legacy_api()`` is deprecated, use ``new_api()`` instead
```

94.6.4 文件格式指南

场景	推荐格式	原因
目录文件	.rst	需要使用 toctree 指令
内容页面	.md（MyST）	更易于读写
API 参考	.rst（自动生成）	由 gen_api_rst.py 生成
表格较多的页面	.rst	RST 表格语法更强大

Markdown 约定：

- 标题层级：# -> ## -> ### -> ####（最多 4 级）
- 代码块：标注语言（```c、```bash）
- 图片：使用 ![alt](path)，可选 :width: 指令
- 提示块：使用 MyST 指令，例如 {note}`、“‘{tip}‘
- 链接：使用 MyST {ref} 和 {doc} 进行交叉引用

RST 约定：

- 使用 .. code-block:: c 代替 :: + 缩进
- 使用 :ref: 和 :doc: 进行交叉引用
- 使用 .. versionadded:: / .. deprecated:: 进行版本追踪

94.6.5 交叉引用约定

引用类型	语法	示例
文档链接	{doc} / :doc:	{doc}`getting_started/index`
章节链接	{ref} / :ref:	{ref}`my-section-label`
外部链接	Markdown 链接	[text](url)
API 函数	{c:func} / :c:func:	{c:func}`tlk_gpio_init`
API 类型	{c:type} / :c:type:	{c:type}`tlk_gpio_config_t`
术语引用	{term} / :term:	{term}`GPIO`

标签命名约定：

```
<module>-<section>    Examples: gpio-introduction, uart-dma-mode
```

94.7 提交 Issue

- 使用 GitHub Issues 报告文档错误或提出改进建议
- 提供页面 URL 并描述哪些内容不正确或缺少
- 如适用，请附上截图或错误日志

94.8 代码规范（SDK 贡献）

对于 SDK 代码库（unisdk）的贡献：

- C 代码应遵循 .clang-format 中定义的格式
- 提交前运行 pre-commit 检查
- 详细指南请参见 unisdk 仓库

CHAPTER 95

技术支持

95.1 联系我们

- **销售咨询**：telinksales@telink-semi.com
- **技术论坛**：https://forum.telink-semi.cn/
- **官方网站**：https://www.telink-semi.cn/
- **文档中心**：https://doc.telink-semi.cn/

95.2 提交问题

提交技术问题时，请提供以下信息：

1. UniSDK 版本
2. 目标芯片型号和开发板
3. 操作系统版本
4. 工具链版本
5. 复现步骤
6. 错误日志或截图

95.3 社区资源

- **GitHub Issues**：https://github.com/telink-semi/unisdk-doc/issues
- **Gitee 仓库**：https://gitee.com/telink-semi
- **产品选型**：https://products.telink-semi.cn/
- **开发工具**：https://www.telink-semi.cn/development-tools

95.4 常见问题

请参阅故障排除。

95.5 资源链接

- 入门指南
- 开发指南
- 示例与演示
- 关于 *Telink*

CHAPTER 96

UniSDK 术语表

本文档定义了 UniSDK 中使用的核心术语，以确保文档和社区交流的一致性。

96.1 构建系统

术语	描述
构建系统	基于 CMake + Ninja 的编译框架
配置系统	基于 Kconfig 的编译时配置管理
目标	CMake 构建目标，可以是可执行文件或库
生成器	CMake 构建后端；UniSDK 使用 Ninja
预构建检查	在编译前自动运行的验证脚本

96.2 West 工具

术语	描述
West	Zephyr 项目管理工具，UniSDK 的统一构建入口
工作空间	由 West 管理的项目目录，包含 `.west` 配置
tl-build	UniSDK 自定义 West 命令，用于构建项目
tl-config	UniSDK 自定义 West 命令，用于配置项目
tl-bdt	UniSDK 自定义 West 命令，用于烧录和调试

96.3 硬件抽象

术语	描述
芯片 / SoC	Telink 片上系统
开发板 / EVK	Telink 官方评估套件
Pinmux	可编程芯片引脚功能配置
Core 层	芯片核心驱动代码（`core/`）
SoC 层	芯片特定功能代码（`soc/`）

96.4 Kconfig 配置

术语	描述
Kconfig	Linux 内核配置系统
chip.config	芯片特定配置
build.config	构建选项配置
.config	最终合并配置
autoconf.h	C 语言配置宏定义头文件
capabilities.h	芯片能力声明头文件
Menuconfig	基于终端的交互式配置界面

96.5 外设与驱动

术语	描述
GPIO	通用输入/输出
UART	通用异步收发器
I2C	集成电路互连总线
SPI	串行外设接口
ADC	模数转换器
DMA	直接存储器访问
PM	电源管理
PLIC	平台级中断控制器

96.6 文档状态标记

标记	描述
STABLE	内容已完成，可直接使用
DRAFT	内容正在编写中，可能不完整
DEPRECATED	即将弃用；建议迁移到新的解决方案
PLANNED	已计划但尚未开始

CHAPTER 97

关于 Telink Semiconductor

Telink Semiconductor 是一家领先的无晶圆半导体公司，致力于设计和开发低功耗、高性能的无线连接片上系统（SoC）解决方案。泰凌微电子成立于 2010 年，总部位于中国上海，已发展成为物联网连接领域的全球参与者。

97.1 我们的使命

提供一流的无线连接解决方案，赋能下一代物联网设备——在消费、工业和汽车市场中实现更智能、更互联、更节能的产品。

97.2 技术领先

Telink 的 SoC 集成了丰富的无线协议，在单芯片上提供真正的多协议支持：

协议	认证	主要应用
Bluetooth LE	Bluetooth SIG 董事会成员	物联网、可穿戴设备、信标
Bluetooth Mesh	Bluetooth SIG	智能照明、楼宇自动化
Zigbee	CSA 成员	智能家居、工业控制
Thread	Thread Group 贡献者	Matter over Thread
Matter	CSA 成员	智能家居互操作性
Wi-Fi	-	高带宽物联网
2.4GHz 专有协议	-	游戏、HID 设备

97.3 全球布局

- **总部：**中国上海（张江高科技园区）
- **研发中心：**上海、台北、宁波、深圳
- **销售与支持：**中国大陆、中国台湾、北美、欧洲、乌克兰
- **员工：**全球 500 余人

97.4 为什么选择 Telink?

- **成熟技术：**全球数十亿台设备采用 Telink SoC
- **全面的 SDK：** UniSDK 在所有芯片系列上提供统一的开发体验
- **低功耗领先：**针对电池受限设备的行业领先功耗
- **丰富的生态系统：**完整的工具链，包括 IDE、调试器和可视化工具
- **全球支持：**遍布全球的 FAE 团队，提供及时的技术支持

97.5 了解更多

- 官方网站
- 产品选型
- 技术论坛
- 文档中心
- GitHub